Serie Banco ZK

3 Volúmenes · 1000+ páginas · 20 Capítulos

Volumen III

Herramientas Avanzadas y Ecosistema

Capítulos 14–20: Identidad, Contratos Privados, Seguridad

Contenido del Volumen 3:

Parte I: Identidad y Regulación
Credenciales Anónimas • Compliance vs Privacidad

Parte II: Privacidad en Red
Metadata Protection • Network Privacy • Tor • Mixnets

Parte III: Contratos y Criptografía
Private Smart Contracts • MPC • FHE • TEEs

Parte IV: Seguridad y Herramientas
Ataques y Defensa • Auditoría • Ecosistema ZK

Walter Milstein

2025

Banco ZK: Criptografía y Privacidad en Blockchain
Volumen 3: Herramientas Avanzadas y Ecosistema

© 2025 Walter Milstein
Todos los derechos reservados.

Primera Edición: 2025

Serie completa:
Volumen 1: Fundamentos Teóricos (Caps. 1–5)
Volumen 2: Aplicaciones y Privacidad (Caps. 6–13)
Volumen 3: Herramientas Avanzadas y Ecosistema (Caps. 14–20)

El código incluido es de dominio público y puede ser utilizado libremente.
Este libro es para uso personal y educativo.

Banco ZK

Criptografía y Privacidad en Blockchain

Volumen 3

Aplicaciones y Privacidad

Serie Técnica de Criptomonedas con Privacidad

2025

In memoriam Hackeris Euleri, Iacobi Rossini,

ingenii cogitantis architecti,
veritatis technicae indefessi inquisitoris,
qui codicem non solum ut seriem praeceptorum intellexit,
sed ut formam ethicae, rigoris atque libertatis.

Aspectus tuus criticus, fortitudo intellectualis
et amor scientiae
vivunt adhuc
in omni systemate recte excogitato
et in omni errore tempestive detecto.

Walter Milstein, pater tuus

Este volumen es continuación de "Banco ZK: Volumen 1 - Fundamentos y Protocolos"

Bloques incluidos en este volumen:

Bloque 20: Herramientas y Ecosistema
(Bloques adicionales se agregarán en futuras ediciones)

Documento generado para fines educativos.
La información contenida es de carácter técnico y académico.

Índice general

Abstract **9**

Prefacio **10**

Introducción **13**

1. Identidad y Credenciales Anónimas **19**
1.1. Anonymous Credentials . . . 20
1.1.1. Concepto Fundamental . . . 20
1.1.2. Propiedades Matemáticas . . . 20
1.1.3. Implementaciones Históricas . . . 21
1.1.4. Comparación de Sistemas . . . 26
1.2. Selective Disclosure . . . 26
1.2.1. Concepto y Motivación . . . 26
1.2.2. Técnicas de Implementación . . . 27
1.3. ZK Identity Proofs . . . 33
1.3.1. Tipos de Pruebas de Identidad . . . 33
1.3.2. Semaphore Protocol . . . 34
1.3.3. Aplicaciones de Semaphore . . . 38
1.3.4. Non-membership Proofs . . . 39
1.4. Soulbound Tokens (SBTs) con Privacidad . . . 44
1.4.1. El Problema de los SBTs Públicos . . . 44
1.4.2. SBTs Privados con ZK . . . 45
1.5. DID + ZK . . . 50
1.5.1. Decentralized Identifiers (DIDs) . . . 50
1.5.2. Verifiable Credentials (VCs) . . . 51
1.5.3. Combinando DIDs + VCs + ZK . . . 51
1.6. Proof of Humanity sin Doxxing . . . 57
1.6.1. El Problema Sybil . . . 57
1.6.2. Proyectos de Proof of Humanity . . . 57
1.7. Age Verification ZK . . . 62
1.7.1. Motivación . . . 62
1.7.2. Arquitectura del Sistema . . . 62
1.7.3. Implementación . . . 62
1.8. KYC Preservando Privacidad . . . 70
1.8.1. Problemas del KYC Tradicional . . . 70
1.8.2. Modelo de KYC con ZK . . . 71

2. Compliance vs Privacidad **81**
2.1. Proof-of-Innocence . . . 82
2.1.1. Concepto y Motivación . . . 82
2.1.2. Tipos de Proof-of-Innocence . . . 82

2.1.3. Implementación de Proof-of-Innocence . . . 83
2.2. Selective De-anonymization . . . 94
2.2.1. Concepto y Necesidad . . . 94
2.2.2. Mecanismos de Selective De-anonymization . . . 95
2.3. Viewing Keys y Audit Keys . . . 103
2.3.1. Jerarquía de Claves de Privacidad . . . 103
2.3.2. Implementación de Viewing Keys . . . 104
2.3.3. Casos de Uso . . . 112
2.4. Regulatory-Compliant Mixers . . . 112
2.4.1. El Desafío . . . 112
2.4.2. Arquitectura de Mixer Compliant . . . 113
2.5. Travel Rule y Soluciones ZK . . . 118
2.5.1. Requisitos de la Travel Rule . . . 118
2.5.2. Soluciones ZK para Travel Rule . . . 118
2.6. Chainalysis y Contramedidas . . . 124
2.6.1. Técnicas de Blockchain Analytics . . . 124
2.6.2. Contramedidas . . . 131
2.7. Privacy Pools (Propuesta de Vitalik) . . . 131
2.7.1. Concepto y Motivación . . . 131
2.7.2. Arquitectura Técnica . . . 132
2.7.3. Trade-offs de Privacy Pools . . . 141
2.8. Opt-in Transparency . . . 142
2.8.1. Concepto Fundamental . . . 142
2.8.2. Casos de Uso . . . 143
2.8.3. Implementación . . . 144
2.8.4. Principios de Diseño . . . 152

3. Metadata y Network Privacy **156**
3.1. IP Leakage y Protección . . . 157
3.1.1. Anatomía del IP Leakage . . . 157
3.1.2. Vectores de IP Leakage . . . 158
3.1.3. Soluciones de Protección . . . 164
3.2. Timing Correlation Attacks . . . 166
3.2.1. Anatomía del Ataque . . . 166
3.2.2. Defensas contra Timing Correlation . . . 174
3.3. Traffic Analysis . . . 174
3.3.1. Fundamentos del Análisis de Tráfico . . . 174
3.3.2. Ataques de Traffic Analysis . . . 175
3.4. Mixnets vs Onion Routing . . . 182
3.4.1. Onion Routing (Tor) . . . 182
3.4.2. Mixnets . . . 183
3.4.3. Comparación Detallada . . . 183
3.4.4. Nym Mixnet . . . 183
3.5. VPN vs Tor vs I2P vs Nym . . . 189
3.5.1. Comparación Comprehensiva . . . 189
3.6. Packet Padding . . . 190
3.6.1. Concepto y Motivación . . . 190
3.7. Dummy Traffic (Cover Traffic) . . . 196
3.7.1. Concepto y Tipos . . . 196
3.8. P2P Network Privacy . . . 203
3.8.1. libp2p . . . 203
3.8.2. Waku . . . 207

4. Private Smart Contracts **211**
4.1. Secret Network . . . 212
4.1.1. Arquitectura y Diseño . . . 212
4.1.2. Intel SGX y Trusted Execution . . . 213
4.1.3. Desarrollo de Secret Contracts . . . 214
4.2. Oasis Network . . . 224
4.2.1. Arquitectura Separada . . . 224
4.2.2. Sapphire: EVM Confidencial . . . 224
4.2.3. Comparación Secret Network vs Oasis . . . 231
4.3. Aleo . . . 232
4.3.1. ZK-Native Blockchain . . . 232
4.3.2. Modelo de Records . . . 233
4.3.3. Lenguaje Leo . . . 233
4.4. Mina Protocol . . . 238
4.4.1. Blockchain Sucinta . . . 238
4.4.2. zkApps: Smart Contracts con ZK . . . 239
4.5. Fhenix y FHE . . . 244
4.5.1. Fully Homomorphic Encryption . . . 244
4.5.2. Comparación de Aproximaciones . . . 248
4.6. Private Function Calls . . . 249
4.6.1. Modelos de Ejecución Privada . . . 249
4.7. Encrypted Mempools . . . 256
4.7.1. El Problema del Mempool Transparente . . . 256
4.8. MEV Protection con Privacidad . . . 264
4.8.1. El Problema del MEV . . . 264
4.8.2. Técnicas de Protección . . . 264

5. Criptografía Avanzada para Privacidad **274**
5.1. FHE (Fully Homomorphic Encryption) . . . 274
5.1.1. Fundamentos Teóricos . . . 274
5.1.2. Matemáticas de FHE (Learning With Errors) . . . 276
5.1.3. Librerías y Aplicaciones . . . 283
5.2. MPC (Multi-Party Computation) . . . 283
5.2.1. Fundamentos . . . 283
5.2.2. Protocolos MPC Principales . . . 283
5.2.3. Protocolo SPDZ . . . 291
5.3. Secret Sharing (Shamir) . . . 292
5.3.1. Fundamentos Matemáticos . . . 292
5.4. Oblivious Transfer . . . 299
5.4.1. Definición y Variantes . . . 299
5.5. Private Set Intersection (PSI) . . . 305
5.5.1. Definición y Aplicaciones . . . 305
5.6. Secure Enclaves . . . 311
5.6.1. Trusted Execution Environments (TEE) . . . 311
5.6.2. Intel SGX . . . 312
5.6.3. Comparación de TEEs . . . 317
5.7. Threshold Signatures . . . 317
5.7.1. Fundamentos . . . 317
5.7.2. Threshold ECDSA . . . 317
5.7.3. FROST (Flexible Round-Optimized Schnorr Threshold) . . . 318
5.8. Distributed Key Generation (DKG) . . . 324
5.8.1. Generación sin Trusted Dealer . . . 324

5.8.2. Aplicaciones en Blockchain . . . 325

6. Ataques y Defensa **328**
6.1. Deanonymization Attacks . . . 328
6.2. Sybil Attacks en Mixers . . . 332
6.3. Eclipse Attacks . . . 334
6.3.1. Defensas contra Eclipse . . . 337
6.4. Dust Attacks . . . 337
6.5. Graph Analysis (Clustering Heuristics) . . . 343
6.6. Taint Analysis . . . 348
6.7. Intersection Attacks . . . 353
6.8. Long-term Correlation . . . 358

I Herramientas y Ecosistema **366**

7. Herramientas y Ecosistema **368**
7.1. Exploradores Privacy-Aware . . . 368
7.1.1. Arquitectura de Explorador Privado . . . 369
7.1.2. Análisis Comparativo de Exploradores . . . 369
7.1.3. Implementación de Explorador Privado . . . 369
7.2. Wallets con Soporte ZK . . . 375
7.2.1. Comparativa de Wallets ZK . . . 376
7.2.2. Arquitectura de Wallet ZK . . . 376
7.2.3. Railgun: Privacy para ERC-20 . . . 385
7.2.4. Umbra: Stealth Payments Simplificados . . . 385
7.2.5. Zcash Wallets: Privacidad Nativa . . . 389
7.3. RPC Privados . . . 390
7.3.1. Arquitectura de Transacción Privada . . . 390
7.3.2. Flashbots Protect . . . 390
7.4. Relayer Networks . . . 397
7.4.1. Arquitectura de Red de Relayers . . . 397
7.4.2. Implementación de Sistema de Relayers . . . 397
7.5. Note Management . . . 406
7.5.1. Ciclo de Vida de una Note . . . 406
7.5.2. Sistema de Gestión de Notes . . . 407
7.6. Proving Infrastructure . . . 416
7.6.1. Arquitecturas de Proving . . . 416
7.6.2. Sistema de Proving Completo . . . 416
7.7. Hardware Wallets y ZK . . . 427
7.7.1. Limitaciones Actuales . . . 427
7.7.2. Estrategias de Integración . . . 427

Conclusión **439**

Glosario **443**

Índice de Figuras **449**

Índice de Tablas **450**

Referencias y Recursos **451**

Este volumen es continuación de "Banco ZK: Volumen 1 - Fundamentos y Protocolos"

Bloques incluidos en este volumen:

Bloque 14: Identidad y Credenciales Anónimas
Bloque 15: Compliance vs Privacidad
Bloque 16: Metadata y Network Privacy
Bloque 17: Private Smart Contracts
Bloque 18: Criptografía Avanzada
Bloque 19: Ataques y Defensa
Bloque 20: Herramientas y Ecosistema

Documento generado para fines educativos.
La información contenida es de carácter técnico y académico.

Abstract

Banco ZK: Volumen 2

Aplicaciones Avanzadas y Ecosistema de Privacidad

Bloques 14–20 de la Serie Técnica

Este segundo volumen de la serie **Banco ZK** presenta las aplicaciones avanzadas y el ecosistema completo necesario para implementar sistemas de privacidad en blockchain. Mientras el Volumen 1 estableció los fundamentos criptográficos y los protocolos base (Bloques 1–13), este volumen se enfoca en llevar esos conceptos a implementaciones prácticas y sistemas de producción.

Alcance del volumen: Cubrimos siete bloques temáticos organizados en tres partes:

- **Parte I – Aplicaciones Avanzadas** (Bloques 14–17): Identidad anónima, balance entre compliance y privacidad, protección de metadatos, y contratos inteligentes privados.
- **Parte II – Criptografía y Seguridad** (Bloques 18–19): Primitivas criptográficas de frontera (FHE, MPC, witness encryption) y análisis exhaustivo de vectores de ataque con sus defensas.
- **Parte III – Ecosistema** (Bloque 20): Stack completo de herramientas incluyendo wallets, relayers, infraestructura de proving, y guías de implementación.

Metodología: Cada bloque combina teoría rigurosa con implementaciones prácticas en Python, Solidity y Circom. Se incluyen más de 70 diagramas TikZ, 95 definiciones formales, y casos de estudio de protocolos reales como Zcash, Tornado Cash, Railgun, y Aztec.

Audiencia: Desarrolladores blockchain, investigadores en criptografía aplicada, arquitectos de sistemas de privacidad, y profesionales de seguridad interesados en tecnologías de conocimiento cero.

Prerrequisitos: Se asume familiaridad con los conceptos del Volumen 1, particularmente: curvas elípticas, funciones hash criptográficas, árboles de Merkle, y fundamentos de SNARKs/STARKs.

Palabras clave: Zero-Knowledge Proofs, Privacidad Blockchain, Anonymous Credentials, zkEVM, Compliance, MEV Protection, Relayer Networks, FHE, MPC

Prefacio

La privacidad financiera es un derecho fundamental que la era digital ha erosionado sistemáticamente. Cada transacción bancaria, cada compra con tarjeta, cada transferencia deja un rastro permanente que puede ser analizado, vendido, o usado en nuestra contra. Las criptomonedas prometieron una alternativa, pero Bitcoin y Ethereum son, en realidad, los sistemas financieros más transparentes jamás creados: cada transacción es visible para siempre en un registro público global.

Este volumen representa la culminación práctica de la serie Banco ZK. Si el primer volumen construyó los cimientos teóricos—las matemáticas de curvas elípticas, los protocolos de conocimiento cero, las estructuras de datos criptográficas—este segundo volumen responde la pregunta: *¿cómo construimos sistemas de privacidad que funcionen en el mundo real?*

El Desafío de la Privacidad Práctica

La teoría criptográfica nos da herramientas poderosas: podemos probar que conocemos un secreto sin revelarlo, podemos demostrar que una transacción es válida sin mostrar los montos, podemos verificar identidad sin exponer datos personales. Pero convertir estas posibilidades teóricas en sistemas usables presenta desafíos únicos:

- **Usabilidad vs Seguridad:** Los usuarios no deberían necesitar un doctorado en criptografía para proteger su privacidad.
- **Eficiencia vs Privacidad:** Las pruebas de conocimiento cero son computacionalmente costosas; optimizarlas sin comprometer garantías es un arte.
- **Compliance vs Anonimato:** Los reguladores exigen visibilidad; los usuarios demandan privacidad. El balance es delicado y geográficamente variable.
- **Descentralización vs Rendimiento:** Los sistemas centralizados son más rápidos pero introducen puntos únicos de fallo y confianza.

Estructura del Volumen

Hemos organizado el material en una progresión lógica que va desde aplicaciones específicas hasta el ecosistema completo:

Bloques 14–17 abordan aplicaciones concretas de privacidad. Comenzamos con *identidad y credenciales anónimas*—cómo probar quiénes somos sin revelar más de lo necesario. Continuamos con el tenso equilibrio entre *compliance y privacidad*, un tema cada vez más relevante dado el escrutinio regulatorio. Exploramos la *privacidad de metadatos*—porque saber que Alice habló con Bob puede ser tan revelador como saber qué dijeron. Finalmente, examinamos los *contratos inteligentes privados*, la frontera de la computación confidencial en blockchain.

Bloques 18–19 profundizan en aspectos técnicos avanzados. La *criptografía avanzada* presenta primitivas como cifrado homomórfico y computación multipartita que complementan

las pruebas de conocimiento cero. El bloque de *ataques y defensa* es quizás el más importante: ningún sistema es seguro si no entendemos cómo puede ser comprometido.

Bloque 20 integra todo en un *ecosistema funcional*. Desde wallets que gestionan notas cifradas hasta redes de relayers descentralizados, desde exploradores que no rastrean hasta infraestructura de proving escalable.

Enfoque Práctico

A diferencia de textos puramente teóricos, cada concepto se acompaña de implementación. Incluimos:

- **Código Python** para prototipos y simulaciones
- **Contratos Solidity** listos para auditoría
- **Circuitos Circom** con optimizaciones documentadas
- **Diagramas de arquitectura** para sistemas completos
- **Análisis de trade-offs** con métricas concretas

Los ejemplos no son juguetes académicos sino extractos simplificados de sistemas en producción. Cuando discutimos Railgun, Tornado Cash, o Zcash, lo hacemos con conocimiento de sus implementaciones reales.

Nota sobre Responsabilidad

Las tecnologías de privacidad son herramientas neutrales. Un cuchillo puede preparar comida o causar daño; la responsabilidad recae en quien lo usa. Los sistemas que describimos pueden proteger disidentes políticos, preservar la privacidad financiera legítima, y habilitar nuevos modelos de negocio. También pueden ser mal utilizados.

Abogamos por privacidad *con accountability*: sistemas donde los usuarios honestos permanecen anónimos pero donde existen mecanismos para casos extremos (mediante viewing keys selectivas, compliance proofs, o auditorías consensuadas). El Bloque 15 dedica especial atención a este balance.

Cómo Usar Este Volumen

Lectura secuencial: Recomendada para comprensión completa. Cada bloque construye sobre conceptos anteriores.

Referencia selectiva: El glosario expandido y los índices permiten consultar temas específicos. Cada bloque es relativamente autocontenido para lectores con base sólida.

Implementación práctica: El código puede extraerse y adaptarse. Recomendamos comenzar con los ejemplos del Bloque 20 y retroceder según se necesiten conceptos específicos.

Profundización: Las referencias al final apuntan a papers originales, documentación de protocolos, y recursos comunitarios para estudio avanzado.

Agradecimientos

Este trabajo se construye sobre décadas de investigación en criptografía. Debemos reconocimiento especial a los equipos de Zcash (por pionear zk-SNARKs en producción), Tornado Cash

(por demostrar privacidad en Ethereum), y a la comunidad de investigadores que continúan avanzando el estado del arte.

La privacidad no es un lujo ni una preferencia: es la base de la autonomía individual en la era digital. Esperamos que este volumen contribuya a un ecosistema donde la privacidad sea accesible para todos.

Los autores
Enero 2025

Introducción

Contexto: De la Teoría a la Práctica

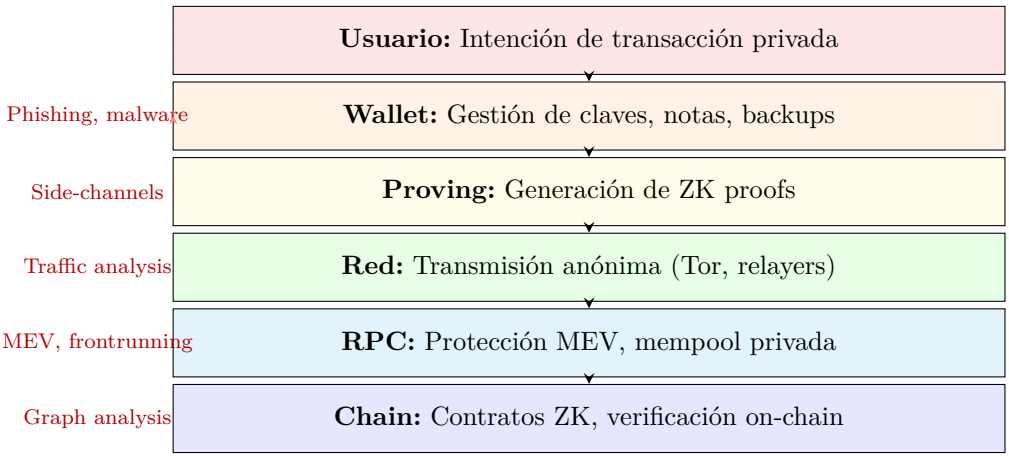
El Volumen 1 de esta serie estableció los fundamentos matemáticos y criptográficos necesarios para entender los sistemas de privacidad en blockchain:

- **Bloques 1–3:** Fundamentos de criptografía, teoría de grupos, curvas elípticas
- **Bloques 4–6:** Estructuras de datos (Merkle trees), compromisos criptográficos
- **Bloques 7–9:** Firmas digitales, ring signatures, stealth addresses
- **Bloques 10–13:** Zero-knowledge proofs: SNARKs, STARKs, circuitos aritméticos

Con esos cimientos, este volumen aborda la pregunta práctica: *¿cómo construimos sistemas completos de privacidad?*

El Stack de Privacidad

Un sistema de privacidad funcional no es solo un circuito ZK elegante o un contrato inteligente bien diseñado. Es un **stack completo** donde cada capa debe preservar las garantías de privacidad:



Una filtración en *cualquier* capa compromete la privacidad completa. Este volumen aborda cada capa sistemáticamente.

Mapa de Contenidos

Parte I: Aplicaciones Avanzadas de Privacidad

Bloque 14: Identidad y Credenciales Anónimas
La identidad es el caso de uso más poderoso y más delicado de ZK. Exploramos:

- Anonymous credentials: probar atributos sin revelar identidad
- Selective disclosure: mostrar solo lo necesario
- Group signatures: firmar como miembro de un grupo
- DIDs y Verifiable Credentials con privacidad
- Soulbound tokens que preservan anonimato
- Proof of humanity sin doxxing

Bloque 15: Compliance vs Privacidad
El elefante en la habitación: ¿cómo satisfacemos reguladores sin destruir la privacidad?

- El espectro de privacidad: de transparente a opaco
- Viewing keys y sus modelos de confianza
- Travel Rule compliance con ZK proofs
- Sanctions screening privacy-preserving
- Caso de estudio: Tornado Cash y OFAC
- Arquitecturas de compliance selectivo

Bloque 16: Metadata y Network Privacy
Los metadatos pueden ser más reveladores que el contenido:

- Timing analysis y correlación temporal
- Traffic analysis en redes P2P
- IP address leakage y protecciones
- Tor, I2P, mixnets: comparativa
- Dummy transactions y cover traffic
- Fingerprinting de wallets

Bloque 17: Private Smart Contracts
La frontera de privacidad programable:

- zkEVM: taxonomía (Type 1–4)
- Private state en contratos
- Aztec, Aleo, Mina: arquitecturas comparadas
- Composabilidad entre contratos privados
- Private DeFi: AMMs, lending, derivatives
- Oráculos con privacidad

Parte II: Criptografía Avanzada y Seguridad

Bloque 18: Criptografía Avanzada
Más allá de ZK, otras primitivas habilitan privacidad:

- Fully Homomorphic Encryption (FHE): cómputo sobre cifrado
- Multi-Party Computation (MPC): cómputo distribuido seguro
- Oblivious Transfer y Private Set Intersection
- Witness Encryption: cifrado condicional a pruebas
- Post-quantum considerations
- Combinando técnicas para defensa en profundidad

Bloque 19: Ataques y Defensa
Conocer al enemigo es esencial:

- Taxonomy de ataques a privacidad
- Deanonymization: heurísticas y machine learning
- Ataques al anonymity set: Sybil, poisoning
- Side-channels en proving: timing, power, cache
- Vulnerabilidades específicas de contratos privados
- Incident response y recovery
- Diseño de sistemas resilientes

Parte III: Herramientas y Ecosistema

Bloque 20: Herramientas y Ecosistema
El stack completo para implementación:

- Exploradores privacy-aware: sin tracking, con Tor
- Wallets ZK: Railgun, Umbra, Zcash wallets
- RPCs privados: Flashbots Protect, MEV Blocker
- Redes de relayers descentralizados
- Gestión de notas y backups seguros
- Infraestructura de proving: client vs server vs hybrid
- Hardware wallets y sus limitaciones
- Integración del stack Banco ZK completo

Convenciones del Documento

A lo largo del volumen utilizamos las siguientes convenciones:

Idea Clave

Las cajas **Idea Clave** resaltan conceptos fundamentales que el lector debe interiorizar.

Aplicación en Banco ZK

Las cajas **Aplicación en Banco ZK** muestran cómo cada concepto se integra en el sistema Banco ZK que construimos a lo largo de la serie.

Advertencia

Las cajas **Advertencia** señalan errores comunes, vulnerabilidades, o malentendidos peligrosos.

Nota

Las cajas **Nota** proporcionan información adicional, contexto histórico, o detalles de implementación.

Código: Los listados de código incluyen Python (prototipos), Solidity (contratos), y Circom (circuitos). Todo el código es funcional y extraíble.

Diagramas: Los diagramas TikZ ilustran arquitecturas, flujos de datos, y relaciones entre componentes.

Definiciones: Los términos técnicos se definen formalmente en su primera aparición y se compilan en el glosario final.

Prerrequisitos

Asumimos que el lector ha cubierto el Volumen 1 o tiene conocimiento equivalente de:

- Aritmética modular y teoría de grupos finitos
- Curvas elípticas: operaciones, pairings, curvas comunes (BN254, BLS12-381)
- Funciones hash: propiedades, Poseidon, Pedersen
- Árboles de Merkle y pruebas de inclusión
- Compromisos criptográficos: Pedersen, KZG
- Fundamentos de SNARKs: R1CS, trusted setup, Groth16
- Fundamentos de STARKs: FRI, AIR
- Modelo UTXO vs account-based
- Nullifiers y su rol en prevenir doble gasto

Lectores sin esta base pueden consultar el Volumen 1 o las referencias introductorias en la bibliografía.

Objetivos de Aprendizaje

Al completar este volumen, el lector será capaz de:

1. **Diseñar** sistemas de identidad anónima con selective disclosure
2. **Evaluar** trade-offs entre compliance y privacidad
3. **Implementar** protecciones contra análisis de metadatos
4. **Comprender** arquitecturas de contratos inteligentes privados
5. **Aplicar** criptografía avanzada (FHE, MPC) donde ZK no es suficiente
6. **Identificar** vectores de ataque y diseñar defensas apropiadas
7. **Construir** el stack completo de un sistema de privacidad funcional
8. **Operar** infraestructura de privacidad en producción

Comenzamos

La privacidad no es un feature que se agrega al final. Es una propiedad fundamental que debe diseñarse desde el inicio y preservarse en cada capa del sistema.

Este volumen proporciona las herramientas conceptuales y prácticas para lograrlo. Comencemos con el Bloque 14: cómo probar quiénes somos sin revelar más de lo estrictamente necesario.

Índice general

Capítulo 1

Identidad y Credenciales Anónimas

Bloque 14 de 20

Introducción

La identidad digital es una de las aplicaciones más prometedoras de la criptografía de conocimiento cero. Este bloque explora cómo demostrar propiedades sobre uno mismo sin revelar información innecesaria: desde credenciales anónimas hasta pruebas de humanidad, desde verificación de edad hasta KYC preservando privacidad.

Idea Clave

El dilema de la identidad digital:
Modelo tradicional:

- Cada servicio recolecta datos completos
- Datos replicados en múltiples bases de datos
- Riesgo de data breaches multiplicado
- Usuario no tiene control

Modelo con ZK:

- Datos permanecen con el usuario
- Solo se comparte lo necesario (selective disclosure)
- Pruebas verificables sin revelar fuente
- Usuario mantiene soberanía

[Timeline de Identidad Digital]

- **1985:** David Chaum propone anonymous credentials
- **2001:** IBM desarrolla Idemix (Identity Mixer)
- **2004:** Microsoft desarrolla U-Prove
- **2016:** Semaphore (primer protocolo ZK identity en blockchain)
- **2019:** W3C publica estándar de DIDs

- **2022:** Worldcoin lanza con ZK proofs
- **2023:** Soulbound Tokens + ZK emerge
- **2024:** EU eIDAS 2.0 incluye consideraciones ZK

1.1. Anonymous Credentials

1.1.1. Concepto Fundamental

Definición 1.1 (Anonymous Credential)**.** Sistema criptográfico que permite a un usuario obtener credenciales de un emisor y presentarlas a verificadores sin revelar su identidad ni permitir que emisor y verificador coludan para rastrear al usuario.

Participantes:

1. **Issuer (Emisor):** Entidad que emite credenciales (gobierno, universidad, etc.)
2. **Holder (Portador):** Usuario que posee la credencial
3. **Verifier (Verificador):** Entidad que verifica la credencial

Propiedades fundamentales:

- **Unforgeability:** No se pueden crear credenciales falsas
- **Anonymity:** Verificador no puede identificar al holder
- **Unlinkability:** Múltiples presentaciones no son vinculables
- **Issuer-Verifier unlinkability:** Emisor y verificador no pueden coludir

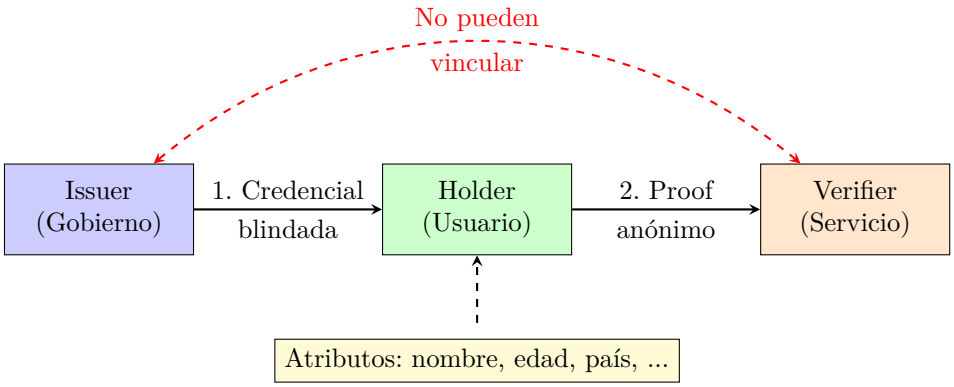


Figura 1.1: Modelo de anonymous credentials: el holder obtiene una credencial del issuer y la presenta al verifier sin que puedan coludir.

1.1.2. Propiedades Matemáticas

[Formalización de Anonymous Credentials] Un sistema de anonymous credentials consiste en:

Algoritmos:

1. $\text{Setup}(1^\lambda) \to (pp)$: Genera parámetros públicos
2. $\text{IssuerKeyGen}(pp) \to (isk, ipk)$: Genera claves del emisor

3. UserKeyGen(pp) $\to$ (usk, upk): Genera claves del usuario
4. Issue($isk, upk,$ attrs) $\to \sigma$: Emite credencial
5. Present($usk, \sigma,$ disclosed) $\to \pi$: Presenta prueba
6. Verify($ipk, \pi,$ disclosed) $\to \{0, 1\}$: Verifica prueba

Propiedades de seguridad:

- **Correctness:** Credenciales válidas siempre verifican
- **Unforgeability:** Pr[Adversario crea σ válido sin isk] $<$ negl(λ)
- **Anonymity:** $\forall\sigma_0, \sigma_1$: Present$(\cdot, \sigma_0, \cdot) \approx_c$ Present$(\cdot, \sigma_1, \cdot)$
- **Unlinkability:** Dos presentaciones de la misma credencial son indistinguibles de dos credenciales diferentes

1.1.3. Implementaciones Históricas

Idemix (IBM Identity Mixer)

Definición 1.2 (Idemix)**.** Sistema de anonymous credentials desarrollado por IBM Research:

- **Base:** CL signatures (Camenisch-Lysyanskaya)
- **Features:** Multi-show, selective disclosure, predicates
- **Usado en:** Hyperledger Indy/Aries

[CL Signatures (Simplificado)] **Setup:**

- RSA modulus $n = pq$
- Generadores $R_1, ..., R_L, S, Z \in QR_n$ (residuos cuadráticos)
- Public key: $(n, R_1, ..., R_L, S, Z)$
- Secret key: p (factor de n)

Firma de atributos $(m_1, ..., m_L)$**:**

$$A = \left(\frac{Z}{R_1^{m_1} \cdot ... \cdot R_L^{m_L} \cdot S^v} \right)^{1/e} \quad \text{mód } n$$

Donde e es primo, v es randomness.
Verificación:

$$Z \stackrel{?}{=} A^e \cdot R_1^{m_1} \cdot ... \cdot R_L^{m_L} \cdot S^v \quad \text{mód } n$$

ZK Presentation: El holder demuestra conocimiento de $(A, e, v, m_1, ..., m_L)$ sin revelarlos, usando protocolos Sigma.

U-Prove (Microsoft)

Definición 1.3 (U-Prove)**.** Sistema de anonymous credentials de Microsoft:

- **Base:** Blind signatures sobre curvas elípticas
- **Característica:** One-show tokens (uso único)
- **Ventaja:** Más eficiente que Idemix
- **Desventaja:** Tokens de un solo uso

BBS+ Signatures

Definición 1.4 (BBS+ Signatures)**.** Esquema de firma moderna para anonymous credentials:

- **Base:** Pairings sobre curvas elípticas
- **Ventaja:** Firmas cortas, ZK eficiente
- **Estándar:** IETF draft en progreso
- **Usado en:** Verifiable Credentials modernos

Listing 1.1: Implementación conceptual de BBS+ Signatures

```
from py_ecc.bls12_381 import (
    G1, G2, multiply, add, pairing,
    curve_order, neg
)
from hashlib import sha256
import secrets

class BBSPlusSignature:
    """
␣␣␣␣BBS+␣Signatures␣para␣anonymous␣credentials.
␣␣␣␣Basado␣en:␣https://identity.foundation/bbs-signature/
␣␣␣␣"""

    def __init__(self, num_messages: int):
        self.L = num_messages
        self.generators = self._generate_generators()

    def _generate_generators(self) -> list:
        """Generar␣L+2␣generadores␣independientes␣en␣G1."""
        generators = []
        for i in range(self.L + 2):
            # Hash-to-curve (simplificado)
            seed = sha256(f"BBS_GENERATOR_{i}".encode()).digest()
            scalar = int.from_bytes(seed, 'big') % curve_order
            generators.append(multiply(G1, scalar))
        return generators

    def keygen(self):
        """Generar␣par␣de␣claves."""
        # Secret key
        sk = secrets.randbelow(curve_order)

        # Public key: W = sk * G2
        pk = multiply(G2, sk)

        return (sk, pk)

    def sign(self, sk: int, messages: list) -> tuple:
        """
␣␣␣␣␣␣␣␣Firmar␣lista␣de␣mensajes.
```

```
        Args:
            sk: Secret key del issuer
            messages: Lista de L mensajes (como enteros)

        Returns:
            Signature (A, e, s)
        """
        assert len(messages) == self.L

        # Generar randomness
        e = secrets.randbelow(curve_order)
        s = secrets.randbelow(curve_order)

        # Calcular B = g1 + s*h0 + sum(m_i * h_i)
        # donde g1, h0, h1, ..., hL son los generadores
        g1, h0 = self.generators[0], self.generators[1]

        B = add(g1, multiply(h0, s))
        for i, m in enumerate(messages):
            hi = self.generators[i + 2]
            B = add(B, multiply(hi, m % curve_order))

        # A = B * (1 / (sk + e))
        inv = pow(sk + e, -1, curve_order)
        A = multiply(B, inv)

        return (A, e, s)

    def verify(self, pk, signature: tuple, messages: list) -> bool:
        """
        Verificar firma.

        Args:
            pk: Public key del issuer
            signature: (A, e, s)
            messages: Lista de mensajes

        Returns:
            True si v lida
        """
        A, e, s = signature

        # Reconstruir B
        g1, h0 = self.generators[0], self.generators[1]
        B = add(g1, multiply(h0, s))
        for i, m in enumerate(messages):
            hi = self.generators[i + 2]
            B = add(B, multiply(hi, m % curve_order))

        # Verificar: e(A, W + e*G2) = e(B, G2)
```

```
        # Equivalente: e(A, W) * e(A, e*G2) = e(B, G2)
        # Equivalente: e(A, W) * e(e*A, G2) = e(B, G2)

        W_plus_eG2 = add(pk, multiply(G2, e))

        lhs = pairing(W_plus_eG2, A)
        rhs = pairing(G2, B)

        return lhs == rhs

    def create_proof(
        self,
        signature: tuple,
        messages: list,
        disclosed_indices: list,
        nonce: bytes
    ) -> dict:
        """
␣␣␣␣␣␣␣␣Crear␣ZK␣proof␣de␣posesi n␣de␣credencial␣con␣selective␣disclosure.

␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣signature:␣(A,␣e,␣s)
␣␣␣␣␣␣␣␣␣␣␣␣messages:␣Todos␣los␣mensajes
␣␣␣␣␣␣␣␣␣␣␣␣disclosed_indices:␣ ndices ␣de␣mensajes␣a␣revelar
␣␣␣␣␣␣␣␣␣␣␣␣nonce:␣Challenge␣del␣verificador

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣Proof␣que␣revela␣solo␣mensajes␣en␣disclosed_indices
␣␣␣␣␣␣␣␣"""
        A, e, s = signature
        hidden_indices = [i for i in range(self.L) if i not in disclosed_indices

        # Randomizar A para unlinkability
        r1 = secrets.randbelow(curve_order)
        r2 = secrets.randbelow(curve_order)

        # A' = A * r1
        A_prime = multiply(A, r1)

        # A_bar = A' * (-e) + B * r1
        # (Esto oculta la firma original)

        # Generar commitments para protocolo Sigma
        # ... (protocolo completo omitido por brevedad)

        disclosed_messages = {i: messages[i] for i in disclosed_indices}

        return {
            'A_prime': A_prime,
            'disclosed': disclosed_messages,
            'proof': b'...',  # Prueba Sigma completa
```

```
        }

    def verify_proof(
        self,
        pk,
        proof: dict,
        disclosed_indices: list,
        nonce: bytes
    ) -> bool:
        """
␣␣␣␣␣␣␣␣Verificar␣ZK␣proof.

␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣pk:␣Public␣key␣del␣issuer
␣␣␣␣␣␣␣␣␣␣␣␣proof:␣Proof␣generado␣por␣create_proof
␣␣␣␣␣␣␣␣␣␣␣␣disclosed_indices:␣Qu␣mensajes␣se␣revelaron
␣␣␣␣␣␣␣␣␣␣␣␣nonce:␣Challenge␣usado

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣True␣si␣proof␣es␣v lido
␣␣␣␣␣␣␣␣"""
        # Verificar que proof es consistente
        # y que el prover conoce una firma v lida
        # sobre mensajes que incluyen los disclosed

        # ... (verificaci n completa omitida)

        return True  # Placeholder


def bbs_demo():
    print("===␣BBS+␣Signatures␣Demo␣===\n")

    # Setup para 4 atributos
    bbs = BBSPlusSignature(num_messages=4)

    # Issuer genera claves
    sk, pk = bbs.keygen()
    print("Issuer␣keys␣generated")

    # Atributos de una credencial de identidad
    # (En pr ctica, ser an hashes de strings)
    messages = [
        123456789,      # ID number
        19900115,       # Birth date (YYYYMMDD)
        1,              # Country code
        0,              # Additional flag
    ]

    # Issuer firma
    signature = bbs.sign(sk, messages)
```

```
    print(f"Credential signed")

    # Verificar firma completa
    is_valid = bbs.verify(pk, signature, messages)
    print(f"Full verification: {is_valid}")

    # Crear proof revelando solo pa s ( ndice  2)
    proof = bbs.create_proof(
        signature=signature,
        messages=messages,
        disclosed_indices=[2],  # Solo revelar pa s
        nonce=b"verifier_challenge_123"
    )

    print(f"\nSelective disclosure proof created")
    print(f"Revealed: country={proof['disclosed']}")
    print("Hidden: ID, birth date, flag")

    # Verificador comprueba
    is_proof_valid = bbs.verify_proof(
        pk=pk,
        proof=proof,
        disclosed_indices=[2],
        nonce=b"verifier_challenge_123"
    )
    print(f"Proof verification: {is_proof_valid}")

bbs_demo()
```

1.1.4. Comparación de Sistemas

Sistema	Multi-show	Selective	Predicates	Tamaño	Verificación
Idemix (CL)	Sí	Sí	Sí	∼4 KB	∼50 ms
U-Prove	No	Sí	Limitado	∼500 B	∼5 ms
BBS+	Sí	Sí	Con ZK	∼400 B	∼10 ms
SNARK-based	Sí	Sí	Arbitrario	∼200 B	∼5 ms

Tabla 1.1: Comparación de sistemas de anonymous credentials

1.2. Selective Disclosure

1.2.1. Concepto y Motivación

Definición 1.5 (Selective Disclosure)**.** Capacidad de revelar solo un subconjunto de atributos de una credencial, manteniendo el resto oculto pero demostrando que la credencial completa es válida.

Imagina tu licencia de conducir:

- Contiene: nombre, foto, dirección, fecha nacimiento, tipo de licencia, restricciones

- Para alquilar auto: Necesitan ver tipo de licencia
- Para comprar alcohol: Solo necesitan saber edad ≥ 21
- Para votar: Solo necesitan verificar que eres ciudadano

Con selective disclosure, muestras SOLO lo necesario para cada caso.

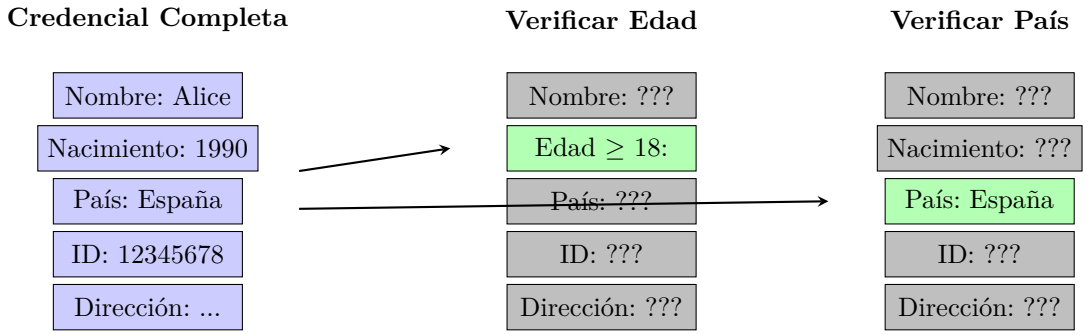


Figura 1.2: Selective disclosure: diferentes verificadores ven diferentes subconjuntos de atributos

1.2.2. Técnicas de Implementación

Con BBS+ Signatures

Listing 1.2: Selective disclosure con estructura Merkle

```
from hashlib import sha256
from typing import List, Dict, Optional
import json

class SelectiveDisclosureCredential:
    """
␣␣␣␣Credencial␣con␣selective␣disclosure␣usando␣Merkle␣tree.
␣␣␣␣Cada␣atributo␣es␣una␣hoja␣del␣ rbol .
␣␣␣␣"""

    def __init__(self, attributes: Dict[str, any]):
        self.attributes = attributes
        self.attribute_names = sorted(attributes.keys())
        self.tree = self._build_merkle_tree()
        self.root = self.tree[-1][0] if self.tree else None

    def _hash(self, data: bytes) -> bytes:
        return sha256(data).digest()

    def _hash_attribute(self, name: str, value: any) -> bytes:
        """Hash␣de␣un␣atributo␣individual."""
        data = json.dumps({name: value}, sort_keys=True).encode()
        return self._hash(data)

    def _build_merkle_tree(self) -> List[List[bytes]]:
        """Construir␣Merkle␣tree␣de␣atributos."""
        # Hojas: hash de cada atributo
```

```
        leaves = [
            self._hash_attribute(name, self.attributes[name])
            for name in self.attribute_names
        ]

        # Pad a potencia de 2
        while len(leaves) & (len(leaves) - 1) != 0:
            leaves.append(self._hash(b"PADDING"))

        tree = [leaves]

        # Construir niveles
        while len(tree[-1]) > 1:
            level = tree[-1]
            new_level = []
            for i in range(0, len(level), 2):
                combined = level[i] + level[i + 1]
                new_level.append(self._hash(combined))
            tree.append(new_level)

        return tree

    def get_merkle_proof(self, attribute_name: str) -> Dict:
        """
        Obtener proof de Merkle para un atributo espec fico.
        """
        try:
            idx = self.attribute_names.index(attribute_name)
        except ValueError:
            raise ValueError(f"Attribute {attribute_name} not found")

        proof_path = []
        current_idx = idx

        for level in self.tree[:-1]:
            # Hermano
            if current_idx % 2 == 0:
                sibling_idx = current_idx + 1
                position = 'right'
            else:
                sibling_idx = current_idx - 1
                position = 'left'

            if sibling_idx < len(level):
                proof_path.append({
                    'hash': level[sibling_idx].hex(),
                    'position': position
                })

            current_idx //= 2
```

```
        return {
            'attribute_name': attribute_name,
            'attribute_value': self.attributes[attribute_name],
            'leaf_hash': self.tree[0][idx].hex(),
            'proof_path': proof_path,
            'root': self.root.hex()
        }

    def create_selective_disclosure(
        self,
        disclosed_attributes: List[str]
    ) -> Dict:
        """
        Crear prueba de selective disclosure.
        Revela solo los atributos especificados.
        """
        proofs = {}
        for attr in disclosed_attributes:
            if attr in self.attributes:
                proofs[attr] = self.get_merkle_proof(attr)

        return {
            'root': self.root.hex(),
            'disclosed': proofs,
            'num_hidden': len(self.attributes) - len(disclosed_attributes)
        }

    @staticmethod
    def verify_selective_disclosure(disclosure: Dict) -> bool:
        """
        Verificar que los atributos revelados son parte de
        una credencial v lida (con el root dado).
        """
        expected_root = bytes.fromhex(disclosure['root'])

        for attr_name, proof in disclosure['disclosed'].items():
            # Verificar cada atributo revelado
            current_hash = bytes.fromhex(proof['leaf_hash'])

            # Recomputar hacia arriba
            for step in proof['proof_path']:
                sibling = bytes.fromhex(step['hash'])
                if step['position'] == 'right':
                    combined = current_hash + sibling
                else:
                    combined = sibling + current_hash
                current_hash = sha256(combined).digest()

            if current_hash != expected_root:
                return False
```

```
        return True


def selective_disclosure_demo():
    print("=== Selective Disclosure Demo ===\n")

    # Crear credencial con m ltiples atributos
    credential = SelectiveDisclosureCredential({
        'name': 'Alice Smith',
        'birth_date': '1990-05-15',
        'country': 'Spain',
        'id_number': '12345678X',
        'address': '123 Main St, Madrid',
        'email': 'alice@example.com'
    })

    print(f"Credential root: {credential.root.hex()[:16]}...")
    print(f"Total attributes: {len(credential.attributes)}")

    # Caso 1: Verificar solo edad (birth_date)
    print("\n--- Caso 1: Verificaci n de edad ---")
    disclosure1 = credential.create_selective_disclosure(['birth_date'])
    print(f"Revealed: birth_date = {disclosure1['disclosed']['birth_date']['attri
    print(f"Hidden: {disclosure1['num_hidden']} attributes")

    is_valid1 = SelectiveDisclosureCredential.verify_selective_disclosure(disclos
    print(f"Verification: {is_valid1}")

    # Caso 2: Verificar pa s y email
    print("\n--- Caso 2: Pa s y email ---")
    disclosure2 = credential.create_selective_disclosure(['country', 'email'])
    for attr, proof in disclosure2['disclosed'].items():
        print(f"Revealed: {attr} = {proof['attribute_value']}")
    print(f"Hidden: {disclosure2['num_hidden']} attributes")

    is_valid2 = SelectiveDisclosureCredential.verify_selective_disclosure(disclos
    print(f"Verification: {is_valid2}")

    # El verificador NO puede ver los atributos ocultos
    print("\n--- Informaci n oculta ---")
    print("Verifier cannot determine:")
    print("  - Name")
    print("  - ID number")
    print("  - Address")
    print("Even though they verify the credential is valid!")

selective_disclosure_demo()
```

Predicados sobre Atributos

Definición 1.6 (Predicate Proof)**.** Prueba de que un atributo oculto satisface cierta condición sin revelar el valor exacto:

- **Range:** $a \leq \text{attr} \leq b$
- **Comparison:** $\text{attr} > c$
- **Membership:** $\text{attr} \in \{v_1, v_2, ...\}$
- **Equality:** $\text{attr}_1 = \text{attr}_2$

Listing 1.3: Circuito para age verification con ZK

```
pragma circom 2.0.0;

include "circomlib/comparators.circom";
include "circomlib/poseidon.circom";

// Probar que una persona es mayor de edad sin revelar
// su fecha de nacimiento exacta

template AgeVerification() {
    // Inputs p blicos
    signal input credentialRoot;      // Root del Merkle tree de la credencial
    signal input currentDate;         // Fecha actual (YYYYMMDD como entero)
    signal input minimumAge;          // Edad m nima requerida (ej: 18)

    // Inputs privados
    signal input birthDate;           // Fecha de nacimiento (oculta)
    signal input birthDateSalt;       // Salt para el commitment
    signal input merkleProof[10];     // Prueba de Merkle
    signal input merkleIndices[10];   //   ndices   del path

    // Output
    signal output isAdult;

    // 1. Verificar que birthDate est   en la credencial
    component birthDateHash = Poseidon(2);
    birthDateHash.inputs[0] <== birthDate;
    birthDateHash.inputs[1] <== birthDateSalt;

    // Verificar Merkle proof (simplificado)
    // ... (similar a otros ejemplos de Merkle)

    // 2. Calcular edad actual
    // Simplificaci n: asumimos a os completos
    // birthDate y currentDate en formato YYYYMMDD

    signal birthYear;
    signal currentYear;

    birthYear <-- birthDate \ 10000;  // Extraer a o
```

```
    currentYear <-- currentDate \ 10000;

    // Constrains para la divisi n
    signal birthRemainder;
    birthRemainder <== birthDate - birthYear * 10000;

    signal currentRemainder;
    currentRemainder <== currentDate - currentYear * 10000;

    // Verificar que son divisiones correctas
    component birthYearRange = LessThan(16);
    birthYearRange.in[0] <== birthRemainder;
    birthYearRange.in[1] <== 10000;
    birthYearRange.out === 1;

    // Calcular edad aproximada
    signal age;
    age <== currentYear - birthYear;

    // Ajuste si a n no ha cumplido a os este a o
    signal monthDayBirth;
    signal monthDayCurrent;
    monthDayBirth <== birthRemainder;
    monthDayCurrent <== currentRemainder;

    component notYetBirthday = GreaterThan(16);
    notYetBirthday.in[0] <== monthDayBirth;
    notYetBirthday.in[1] <== monthDayCurrent;

    signal adjustedAge;
    adjustedAge <== age - notYetBirthday.out;

    // 3. Verificar edad >= minimumAge
    component ageCheck = GreaterEqThan(8);
    ageCheck.in[0] <== adjustedAge;
    ageCheck.in[1] <== minimumAge;

    isAdult <== ageCheck.out;

    // Forzar que sea adulto (si no, el proof falla)
    isAdult === 1;
}

component main {public [credentialRoot, currentDate, minimumAge]} = AgeVerificat
```

Aplicación en Banco ZK

Selective Disclosure en Banco ZK:
Banco ZK implementa selective disclosure para compliance:

1. **Credencial de identidad:**

- Usuario obtiene credencial ZK de proveedor KYC
- Credencial contiene: país, categoría de riesgo, fecha verificación
- NO contiene: nombre, documento, foto

2. **Uso en transacciones:**
   - Para operar en ciertos pools: probar "país no sancionado"
   - Para montos altos: probar "KYC completado"
   - Sin revelar identidad específica
3. **Privacy Pools integration:**
   - Probar membresía en "set de usuarios verificados"
   - Sin revelar cuál usuario específico
   - Compatible con regulación sin doxxing

```
// Prueba de compliance para Banco ZK
template BancoZKCompliance() {
    signal input kycCredentialRoot;  // Root de la credencial
    signal input userSecret;         // Secreto del usuario

    // Probar: "soy usuario verificado Y mi pa s no est  sancionado"
    // Sin revelar: qui n soy, qu  pa s , cu ndo me verifiqu

    signal output isCompliant;
}
```

1.3. ZK Identity Proofs

1.3.1. Tipos de Pruebas de Identidad

Definición 1.7 (ZK Identity Proof)**.** Prueba de conocimiento cero que demuestra una propiedad relacionada con la identidad sin revelar la identidad en sí.

[Taxonomía de ZK Identity Proofs] **1. Membership Proofs:**

- "Pertenezco al grupo X"
- Ejemplo: Soy empleado de esta empresa
- Técnica: Merkle proof de que mi commitment está en el árbol

2. Non-membership Proofs:

- "No estoy en la lista Y"
- Ejemplo: No estoy en lista de sancionados
- Técnica: Más compleja, requiere estructura especial (sparse Merkle tree)

3. Attribute Proofs:

- "Tengo la propiedad Z"

- Ejemplo: Soy ciudadano de la UE
- Técnica: ZK proof sobre atributo de credencial

4. Range Proofs:

- "Mi valor está en el rango [a, b]"
- Ejemplo: Mi edad está entre 21 y 65
- Técnica: Bulletproofs o circuito de comparación

5. Equality/Inequality Proofs:

- "Mis atributos X e Y son iguales/diferentes"
- Ejemplo: El país de mi pasaporte = país de mi licencia
- Técnica: Commitment equality proof

1.3.2. Semaphore Protocol

Definición 1.8 (Semaphore)**.** Protocolo de código abierto para pruebas anónimas de membresía en grupos, desarrollado originalmente para Ethereum:

- **Creador:** Barry Whitehat et al. (2020)
- **Uso:** Voting, signaling, rate limiting, whistleblowing
- **Técnica:** Identity commitments en Merkle tree + nullifiers
- **Implementación:** Circom + snarkjs

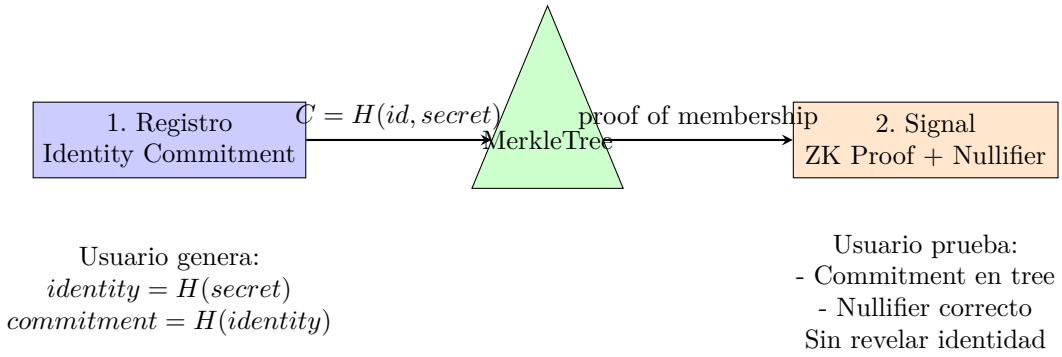


Figura 1.3: Flujo del protocolo Semaphore

[Semaphore Identity] **Generación de identidad:**

1. Usuario genera dos secretos: trapdoor, nullifier_secret
2. Calcula: identity = Poseidon(nullifier_secret, trapdoor)
3. Commitment público: C = Poseidon(identity)

Registro en grupo:

1. Admin del grupo añade C al Merkle tree
2. Root del tree identifica el grupo

Señalización anónima:

1. Usuario genera nullifier: $nf = \text{Poseidon}(\text{external_nullifier}, \text{identity})$
2. Genera ZK proof de:
   - Conoce $(\text{trapdoor}, \text{nullifier_secret})$ tal que C está en el tree
   - El nullifier nf es correcto
3. Publica: (nf, signal, π)

Propiedades:

- **Anonimato:** Nadie sabe quién señaló
- **Doble señalización:** Detectada por nullifier repetido
- **External nullifier:** Permite múltiples señales en diferentes contextos

Listing 1.4: Circuito Semaphore simplificado

```
pragma circom 2.0.0;

include "circomlib/poseidon.circom";
include "circomlib/mux1.circom";

// Merkle tree proof
template MerkleTreeInclusionProof(nLevels) {
    signal input leaf;
    signal input pathIndices[nLevels];
    signal input siblings[nLevels];
    signal output root;

    component hashers[nLevels];
    component mux[nLevels];

    signal hashes[nLevels + 1];
    hashes[0] <== leaf;

    for (var i = 0; i < nLevels; i++) {
        // pathIndices[i] es 0 o 1
        pathIndices[i] * (1 - pathIndices[i]) === 0;

        mux[i] = MultiMux1(2);
        mux[i].c[0][0] <== hashes[i];
        mux[i].c[0][1] <== siblings[i];
        mux[i].c[1][0] <== siblings[i];
        mux[i].c[1][1] <== hashes[i];
        mux[i].s <== pathIndices[i];

        hashers[i] = Poseidon(2);
        hashers[i].inputs[0] <== mux[i].out[0];
        hashers[i].inputs[1] <== mux[i].out[1];

        hashes[i + 1] <== hashers[i].out;
```

```
    }

    root <== hashes[nLevels];
}

// Semaphore main circuit
template Semaphore(nLevels) {
    // ========== INPUTS P BLICOS ==========
    signal input merkleRoot;            // Root del grupo
    signal input nullifierHash;         // Hash del nullifier
    signal input signalHash;            // Hash del mensaje/se al
    signal input externalNullifier;     // Identificador del contexto

    // ========== INPUTS PRIVADOS ==========
    signal input identityNullifier;     // Secreto 1
    signal input identityTrapdoor;      // Secreto 2
    signal input treeSiblings[nLevels];
    signal input treePathIndices[nLevels];

    // ========== C LCULOS ==========

    // 1. Calcular identity commitment
    component identityHasher = Poseidon(2);
    identityHasher.inputs[0] <== identityNullifier;
    identityHasher.inputs[1] <== identityTrapdoor;

    signal identity;
    identity <== identityHasher.out;

    // 2. Calcular commitment (leaf del  rbol )
    component commitmentHasher = Poseidon(1);
    commitmentHasher.inputs[0] <== identity;

    signal commitment;
    commitment <== commitmentHasher.out;

    // 3. Verificar membership en el  rbol
    component inclusionProof = MerkleTreeInclusionProof(nLevels);
    inclusionProof.leaf <== commitment;
    for (var i = 0; i < nLevels; i++) {
        inclusionProof.siblings[i] <== treeSiblings[i];
        inclusionProof.pathIndices[i] <== treePathIndices[i];
    }

    // El root calculado debe coincidir
    inclusionProof.root === merkleRoot;

    // 4. Calcular y verificar nullifier
    component nullifierHasher = Poseidon(2);
    nullifierHasher.inputs[0] <== externalNullifier;
    nullifierHasher.inputs[1] <== identity;
```

```
    nullifierHasher.out === nullifierHash;

    // 5. Vincular al signal (previene front-running)
    // El signalHash es parte de los inputs p blicos
    // No hay constraint adicional, pero es parte del statement
    signal signalSquare;
    signalSquare <== signalHash * signalHash;
}

component main {public [merkleRoot, nullifierHash, signalHash, externalNullifi
```

Listing 1.5: Uso de Semaphore en JavaScript

```
const { Identity } = require("@semaphore-protocol/identity")
const { Group } = require("@semaphore-protocol/group")
const { generateProof, verifyProof } = require("@semaphore-protocol/proof")

async function semaphoreDemo() {
    console.log("=== Semaphore Demo ===\n")

    // 1. Crear identidades
    const identity1 = new Identity()
    const identity2 = new Identity()
    const identity3 = new Identity()

    console.log("Created 3 identities")
    console.log(`Identity 1 commitment: ${identity1.commitment.toString().slic

    // 2. Crear grupo y a adir miembros
    const groupId = 1
    const group = new Group(groupId, 20)  // depth = 20

    group.addMember(identity1.commitment)
    group.addMember(identity2.commitment)
    group.addMember(identity3.commitment)

    console.log(`\nGroup created with ${group.members.length} members`)
    console.log(`Group root: ${group.root.toString().slice(0, 20)}...`)

    // 3. Identity1 quiere se alar an nimamente
    const externalNullifier = 12345  // Contexto (ej: ID de votaci n)
    const signal = "I vote YES"

    console.log(`\n--- Identity 1 signals: "${signal}" ---`)

    // Generar proof
    const proof = await generateProof(
        identity1,
        group,
        externalNullifier,
        signal
```

```
    )

    console.log(`Nullifier hash: ${proof.nullifierHash.toString().slice(0, 20)}...`)
    console.log(`Proof generated (${proof.proof.length} elements)`)

    // 4. Verificar proof
    const isValid = await verifyProof(proof, 20)
    console.log(`\nProof verification: ${isValid}`)

    // 5. Intentar se alar de nuevo (doble se alizaci n)
    console.log("\n--- Identity 1 tries to signal again ---")

    const proof2 = await generateProof(
        identity1,
        group,
        externalNullifier,  // Mismo contexto
        "I vote NO"
    )

    // El nullifier ser  el mismo!
    console.log(`New nullifier: ${proof2.nullifierHash.toString().slice(0, 20)}...`)
    console.log(`Same nullifier: ${proof.nullifierHash === proof2.nullifierHash}`)
    console.log("=> Double signaling detected!")

    // 6. Se alar en diferente contexto (OK)
    console.log("\n--- Identity 1 signals in different context ---")

    const differentContext = 99999
    const proof3 = await generateProof(
        identity1,
        group,
        differentContext,
        "Hello world"
    )

    console.log(`Different nullifier: ${proof3.nullifierHash.toString().slice(0,
    console.log(`Nullifiers different: ${proof.nullifierHash !== proof3.nullifier
    console.log("=> Different context allows new signal")
}

semaphoreDemo().catch(console.error)
```

1.3.3. Aplicaciones de Semaphore

Idea Clave

Casos de uso de Semaphore:

1. **Voting anónimo:**
   - Grupo = votantes elegibles
   - External nullifier = ID de la propuesta

- Signal = voto (sí/no)
- Cada miembro vota una vez por propuesta

2. **Whistleblowing:**
   - Grupo = empleados de empresa
   - Signal = denuncia
   - Nullifier previene spam
   - Identidad permanece oculta
3. **Rate limiting:**
   - Grupo = usuarios verificados
   - External nullifier = timestamp (hora/día)
   - Limita acciones por período sin identificar
4. **Anonymous authentication:**
   - Grupo = usuarios autorizados
   - Probar membresía sin revelar quién
   - Acceso a recursos restringidos

1.3.4. Non-membership Proofs

Definición 1.9 (Non-membership Proof)**.** Prueba de que un elemento NO está en un conjunto:

$$\text{Proof}(x, S) \implies x \notin S$$

Más difícil que membership porque requiere "probar un negativo".

[Técnicas para Non-membership] **1. Sparse Merkle Tree:**

- Árbol con posición predeterminada para cada posible elemento
- Non-membership = mostrar que la posición tiene valor vacío
- Tamaño: 2^{256} posiciones virtuales

2. RSA Accumulators:

- $A = g^{\prod_{x \in S} H(x)} \mod n$
- Non-membership witness más complejo
- Basado en "proofs of exponentiation"

3. Indexed Merkle Tree (Aztec):

- Elementos ordenados con punteros
- Non-membership: mostrar elementos adyacentes
- Más eficiente que sparse tree

Listing 1.6: Sparse Merkle Tree para non-membership proofs

```
from hashlib import sha256
from typing import Dict, List, Tuple, Optional

class SparseMerkleTree:
    """
    Sparse Merkle Tree para membership y non-membership proofs.
    Soporta 2^depth elementos posibles.
    """

    def __init__(self, depth: int = 256):
        self.depth = depth
        self.empty_leaf = b'\x00' * 32
        self.empty_tree_roots = self._precompute_empty_roots()
        self.leaves: Dict[int, bytes] = {}  # position -> value

    def _hash(self, left: bytes, right: bytes) -> bytes:
        return sha256(left + right).digest()

    def _precompute_empty_roots(self) -> List[bytes]:
        """Precomputar roots de sub rboles vac os."""
        roots = [self.empty_leaf]
        for _ in range(self.depth):
            roots.append(self._hash(roots[-1], roots[-1]))
        return roots

    def _get_position(self, key: bytes) -> int:
        """Convertir key a posici n en el  rbol ."""
        return int.from_bytes(sha256(key).digest(), 'big') % (2 ** self.depth)

    def insert(self, key: bytes, value: bytes):
        """Insertar elemento."""
        position = self._get_position(key)
        self.leaves[position] = value

    def remove(self, key: bytes):
        """Remover elemento."""
        position = self._get_position(key)
        if position in self.leaves:
            del self.leaves[position]

    def get_root(self) -> bytes:
        """Calcular root del  rbol ."""
        return self._compute_root(0, self.depth)

    def _compute_root(self, start: int, level: int) -> bytes:
        """Recursivamente calcular root de sub rbol."""
        if level == 0:
            return self.leaves.get(start, self.empty_leaf)

        mid = 2 ** (level - 1)
```

```
        left = self._compute_root(start, level - 1)
        right = self._compute_root(start + mid, level - 1)

        return self._hash(left, right)

    def get_membership_proof(self, key: bytes) -> Dict:
        """
␣␣␣␣␣␣␣␣Generar␣proof␣de␣membership.
␣␣␣␣␣␣␣␣"""
        position = self._get_position(key)

        if position not in self.leaves:
            return {'exists': False}

        proof_path = []
        current_pos = position

        for level in range(self.depth):
            sibling_pos = current_pos ^ 1  # Flip last bit

            # Obtener valor del hermano
            if level == 0:
                sibling = self.leaves.get(sibling_pos, self.empty_leaf)
            else:
                # En niveles superiores, calcular sub rbol
                sibling = self._compute_root(
                    (sibling_pos >> level) << level,
                    level
                )

            proof_path.append({
                'sibling': sibling,
                'position': 'left' if current_pos % 2 == 1 else 'right'
            })

            current_pos //= 2

        return {
            'exists': True,
            'key': key,
            'value': self.leaves[position],
            'proof': proof_path
        }

    def get_non_membership_proof(self, key: bytes) -> Dict:
        """
␣␣␣␣␣␣␣␣Generar␣proof␣de␣NON-membership.
␣␣␣␣␣␣␣␣Demuestra␣que␣la␣posici n␣est ␣vac a.
␣␣␣␣␣␣␣␣"""
        position = self._get_position(key)
```

```
        if position in self.leaves:
            return {'exists': True, 'error': 'Key exists'}

        # Proof de que la posici n tiene el valor vac o
        proof_path = []
        current_pos = position

        for level in range(self.depth):
            sibling_pos = current_pos ^ 1

            if level == 0:
                sibling = self.leaves.get(sibling_pos, self.empty_leaf)
            else:
                sibling = self._compute_root(
                    (sibling_pos >> level) << level,
                    level
                )

            proof_path.append({
                'sibling': sibling,
                'position': 'left' if current_pos % 2 == 1 else 'right'
            })

            current_pos //= 2

        return {
            'exists': False,
            'key': key,
            'empty_position': position,
            'proof': proof_path
        }

    @staticmethod
    def verify_membership(root: bytes, proof: Dict, empty_leaf: bytes) -> bool:
        """Verificar proof de membership."""
        if not proof['exists']:
            return False

        current = proof['value']

        for step in proof['proof']:
            if step['position'] == 'right':
                current = sha256(step['sibling'] + current).digest()
            else:
                current = sha256(current + step['sibling']).digest()

        return current == root

    @staticmethod
    def verify_non_membership(root: bytes, proof: Dict, empty_leaf: bytes) -> bo
        """Verificar proof de non-membership."""
```

```
        if proof.get('exists', True):
            return False

        # Verificar que la posici n tiene valor vac o
        current = empty_leaf

        for step in proof['proof']:
            if step['position'] == 'right':
                current = sha256(step['sibling'] + current).digest()
            else:
                current = sha256(current + step['sibling']).digest()

        return current == root


def sparse_merkle_demo():
    print("=== Sparse Merkle Tree Demo ===\n")

    # Usar depth peque o para demo
    smt = SparseMerkleTree(depth=8)

    # Insertar algunos elementos
    smt.insert(b"alice", b"data_alice")
    smt.insert(b"bob", b"data_bob")
    smt.insert(b"carol", b"data_carol")

    root = smt.get_root()
    print(f"Tree root: {root.hex()[:16]}...")

    # Membership proof
    print("\n-- Membership Proof --")
    membership_proof = smt.get_membership_proof(b"alice")
    print(f"Alice exists: {membership_proof['exists']}")

    is_valid = SparseMerkleTree.verify_membership(
        root, membership_proof, smt.empty_leaf
    )
    print(f"Proof valid: {is_valid}")

    # Non-membership proof
    print("\n-- Non-membership Proof --")
    non_membership_proof = smt.get_non_membership_proof(b"dave")
    print(f"Dave exists: {non_membership_proof['exists']}")

    is_valid_non = SparseMerkleTree.verify_non_membership(
        root, non_membership_proof, smt.empty_leaf
    )
    print(f"Non-membership proof valid: {is_valid_non}")

    # Aplicaci n: Lista de sancionados
    print("\n-- Aplicaci n: Sanctions List --")
```

```
    sanctions = SparseMerkleTree(depth=16)
    sanctions.insert(b"sanctioned_entity_1", b"1")
    sanctions.insert(b"sanctioned_entity_2", b"1")

    sanctions_root = sanctions.get_root()

    # Usuario prueba que NO est  sancionado
    user_proof = sanctions.get_non_membership_proof(b"honest_user")
    print(f"Honest user sanctioned: {user_proof['exists']}")
    print("User can prove they are NOT on sanctions list!")

sparse_merkle_demo()
```

1.4. Soulbound Tokens (SBTs) con Privacidad

1.4.1. El Problema de los SBTs Públicos

Definición 1.10 (Soulbound Token (SBT))**.** Token no transferible vinculado permanentemente a una dirección ("alma"), que representa credenciales, logros, afiliaciones o propiedades de identidad.

Concepto: Introducido por Vitalik Buterin et al. en "Decentralized Society" (2022).
Ejemplos:

- Diploma universitario
- Certificación profesional
- Historial crediticio
- Membresía de DAO
- Prueba de asistencia (POAP vinculado)

Problemas de privacidad con SBTs públicos:

1. **Doxxing por acumulación:**
   - Cada SBT revela información
   - Combinación de SBTs permite perfilar completamente
   - Ejemplo: Universidad X + Empleador Y + Ciudad Z = persona identificable
2. **Discriminación:**
   - SBT de historial médico visible
   - SBT de antecedentes penales
   - Servicios pueden discriminar basándose en SBTs
3. **Vigilancia social:**
   - Gobiernos pueden rastrear credenciales
   - Score social basado en SBTs
   - Sin posibilidad de "borrar" historial
4. **Linkability:**

- Todos los SBTs vinculados a una dirección
- No hay separación de contextos
- Tu empleador ve tus SBTs médicos

1.4.2. SBTs Privados con ZK

Idea Clave

Solución: Private SBTs
En lugar de SBTs públicos visibles para todos:

- **Emisión privada:** SBT emitido como commitment oculto
- **Almacenamiento privado:** Solo el holder conoce sus SBTs
- **Revelación selectiva:** ZK proof de posesión sin mostrar el SBT
- **Unlinkability:** Diferentes pruebas no son vinculables

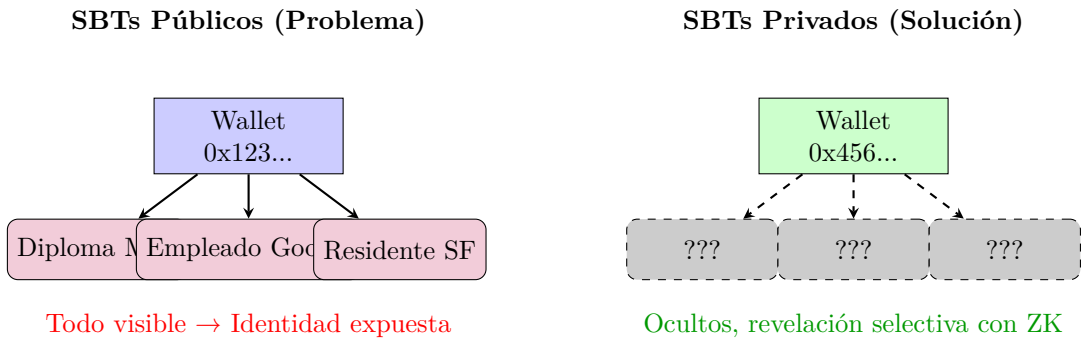


Figura 1.4: SBTs públicos vs privados

Listing 1.7: Private SBT Registry con ZK

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

interface IVerifier {
    function verifyProof(
        bytes calldata proof,
        uint256[] calldata publicInputs
    ) external view returns (bool);
}

contract PrivateSBTRegistry {
    // Verificadores para diferentes tipos de pruebas
    IVerifier public ownershipVerifier;
    IVerifier public attributeVerifier;

    // Merkle tree de SBT commitments
    bytes32 public sbtTreeRoot;
    uint256 public nextSbtIndex;
    mapping(uint256 => bytes32) public sbtCommitments;
```

```
// Nullifiers para prevenir doble uso (opcional)
mapping(bytes32 => bool) public usedNullifiers;

// Issuers autorizados
mapping(address => bool) public authorizedIssuers;
mapping(address => bytes32) public issuerPublicKeys;

// Tipos de SBT
struct SBTType {
    string name;
    address issuer;
    bytes32 schemaHash;
    bool revocable;
}
mapping(uint256 => SBTType) public sbtTypes;

event SBTIssued(
    uint256 indexed sbtTypeId,
    bytes32 indexed commitment,
    uint256 leafIndex,
    bytes encryptedData  // Encriptado para el receptor
);

event SBTRevoked(bytes32 indexed nullifier);

// ========== EMISI N PRIVADA ==========

function issueSBT(
    uint256 sbtTypeId,
    bytes32 commitment,        // H(sbtType, attributes, owner, randomness)
    bytes calldata encryptedData,  // Datos encriptados para el receptor
    bytes calldata issuerSignature
) external {
    require(authorizedIssuers[msg.sender], "Not authorized issuer");
    require(sbtTypes[sbtTypeId].issuer == msg.sender, "Wrong issuer");

    // Verificar firma del issuer sobre el commitment
    // (prueba que el issuer aprob  este SBT espec fico)
    require(
        _verifyIssuerSignature(
            msg.sender,
            commitment,
            issuerSignature
        ),
        "Invalid issuer signature"
    );

    // A adir al  rbol
    uint256 index = nextSbtIndex++;
    sbtCommitments[index] = commitment;
```

```
        _updateMerkleRoot();

        emit SBTIssued(sbtTypeId, commitment, index, encryptedData);
    }

    // ========== VERIFICACI N PRIVADA ==========

    function verifyOwnership(
        bytes calldata proof,
        bytes32 merkleRoot,
        uint256 sbtTypeId
    ) external view returns (bool) {
        require(merkleRoot == sbtTreeRoot, "Invalid root");

        // Public inputs: root, sbtTypeId
        uint256[] memory publicInputs = new uint256[](2);
        publicInputs[0] = uint256(merkleRoot);
        publicInputs[1] = sbtTypeId;

        // La prueba demuestra:
        // 1. Prover conoce un SBT commitment en el  rbol
        // 2. El SBT es del tipo especificado
        // 3. Prover es el owner del SBT
        // Sin revelar: cu l SBT espec fico, otros atributos

        return ownershipVerifier.verifyProof(proof, publicInputs);
    }

    function verifyAttribute(
        bytes calldata proof,
        bytes32 merkleRoot,
        uint256 sbtTypeId,
        bytes32 attributeHash,  // H(attribute_name)
        bytes32 expectedValue   // Valor esperado o commitment
    ) external view returns (bool) {
        require(merkleRoot == sbtTreeRoot, "Invalid root");

        uint256[] memory publicInputs = new uint256[](4);
        publicInputs[0] = uint256(merkleRoot);
        publicInputs[1] = sbtTypeId;
        publicInputs[2] = uint256(attributeHash);
        publicInputs[3] = uint256(expectedValue);

        // La prueba demuestra:
        // 1. Prover tiene SBT v lido del tipo correcto
        // 2. El atributo especificado tiene el valor esperado
        // Sin revelar: identidad, otros atributos

        return attributeVerifier.verifyProof(proof, publicInputs);
    }
```

```
    // Prueba de rango sobre atributo
    function verifyAttributeRange(
        bytes calldata proof,
        bytes32 merkleRoot,
        uint256 sbtTypeId,
        bytes32 attributeHash,
        uint256 minValue,
        uint256 maxValue
    ) external view returns (bool) {
        uint256[] memory publicInputs = new uint256[](5);
        publicInputs[0] = uint256(merkleRoot);
        publicInputs[1] = sbtTypeId;
        publicInputs[2] = uint256(attributeHash);
        publicInputs[3] = minValue;
        publicInputs[4] = maxValue;

        // Verifica: minValue <= attribute <= maxValue
        return attributeVerifier.verifyProof(proof, publicInputs);
    }

    // ========== REVOCACIN ==========

    function revokeSBT(
        bytes32 nullifier,
        bytes calldata revokerProof
    ) external {
        // Solo el issuer puede revocar
        // Proof demuestra que el nullifier corresponde a un SBT que emiti

        require(!usedNullifiers[nullifier], "Already revoked");
        usedNullifiers[nullifier] = true;

        emit SBTRevoked(nullifier);
    }

    // ========== HELPERS ==========

    function _updateMerkleRoot() internal {
        // Actualizar root del Merkle tree
        // (implementaci n simplificada)
    }

    function _verifyIssuerSignature(
        address issuer,
        bytes32 commitment,
        bytes calldata signature
    ) internal view returns (bool) {
        // Verificar firma ECDSA o BLS
        return true; // Placeholder
    }
}
```

Listing 1.8: Circuito para prueba de posesión de SBT privado

```
pragma circom 2.0.0;

include "circomlib/poseidon.circom";

// Prueba de que poseo un SBT de cierto tipo sin revelar cu l
template PrivateSBTOwnership(treeLevels, numAttributes) {
    // ========== PUBLIC INPUTS ==========
    signal input merkleRoot;
    signal input sbtTypeId;

    // ========== PRIVATE INPUTS ==========
    signal input ownerSecret;           // Secreto del owner
    signal input sbtRandomness;         // Randomness del commitment
    signal input attributes[numAttributes];
    signal input merkleProof[treeLevels];
    signal input merkleIndices[treeLevels];

    // ========== CONSTRAINTS ==========

    // 1. Calcular owner commitment
    component ownerHash = Poseidon(1);
    ownerHash.inputs[0] <== ownerSecret;
    signal ownerCommitment;
    ownerCommitment <== ownerHash.out;

    // 2. Calcular SBT commitment
    // commitment = H(sbtTypeId, attributes..., ownerCommitment, randomness)
    component sbtHash = Poseidon(numAttributes + 3);
    sbtHash.inputs[0] <== sbtTypeId;
    for (var i = 0; i < numAttributes; i++) {
        sbtHash.inputs[i + 1] <== attributes[i];
    }
    sbtHash.inputs[numAttributes + 1] <== ownerCommitment;
    sbtHash.inputs[numAttributes + 2] <== sbtRandomness;

    signal sbtCommitment;
    sbtCommitment <== sbtHash.out;

    // 3. Verificar Merkle proof
    component merkleVerifier = MerkleProof(treeLevels);
    merkleVerifier.leaf <== sbtCommitment;
    for (var i = 0; i < treeLevels; i++) {
        merkleVerifier.siblings[i] <== merkleProof[i];
        merkleVerifier.pathIndices[i] <== merkleIndices[i];
    }

    // Root debe coincidir
    merkleVerifier.root === merkleRoot;
}
```

```
// Prueba de atributo espec fico
template PrivateSBTAttributeProof(treeLevels, numAttributes, attrIndex) {
    // PUBLIC
    signal input merkleRoot;
    signal input sbtTypeId;
    signal input expectedAttributeValue;

    // PRIVATE
    signal input ownerSecret;
    signal input sbtRandomness;
    signal input attributes[numAttributes];
    signal input merkleProof[treeLevels];
    signal input merkleIndices[treeLevels];

    // Verificar ownership (mismo que arriba)
    // ...

    // Verificar que el atributo tiene el valor esperado
    attributes[attrIndex] === expectedAttributeValue;
}

component main {public [merkleRoot, sbtTypeId]} = PrivateSBTOwnership(20, 5);
```

1.5. DID + ZK

1.5.1. Decentralized Identifiers (DIDs)

Definición 1.11 (DID - Decentralized Identifier)**.** Identificador único, persistente y verificable criptográficamente, que no requiere una autoridad centralizada:

Formato: `did:method:specific-identifier`

Ejemplos:

- `did:ethr:0x123...abc` (Ethereum)
- `did:key:z6Mk...` (Key-based)
- `did:web:example.com` (Web-based)
- `did:ion:EiD...` (Bitcoin/ION)

Estándar: W3C DID Core Specification

[DID Document] Cada DID resuelve a un **DID Document** que contiene:

```
{
  "@context": "https://www.w3.org/ns/did/v1",
  "id": "did:example:123456789",
  "authentication": [{
    "id": "did:example:123456789#keys-1",
    "type": "Ed25519VerificationKey2020",
    "controller": "did:example:123456789",
    "publicKeyMultibase": "z6Mkf..."
  }],
  "service": [{
```

```
    "id": "did:example:123456789#messaging",
    "type": "MessagingService",
    "serviceEndpoint": "https://example.com/messages"
  }]
}
```

Componentes clave:

- **Verification Methods:** Claves públicas para autenticación
- **Services:** Endpoints para interactuar con el DID
- **Controller:** Quién puede modificar el documento

1.5.2. Verifiable Credentials (VCs)

Definición 1.12 (Verifiable Credential)**.** Credencial digital que puede ser verificada criptográficamente:

- **Issuer:** Entidad que emite la credencial
- **Subject:** Entidad sobre la que se hace el claim
- **Holder:** Quien posee y presenta la credencial
- **Verifier:** Quien verifica la credencial

Estándar: W3C Verifiable Credentials Data Model

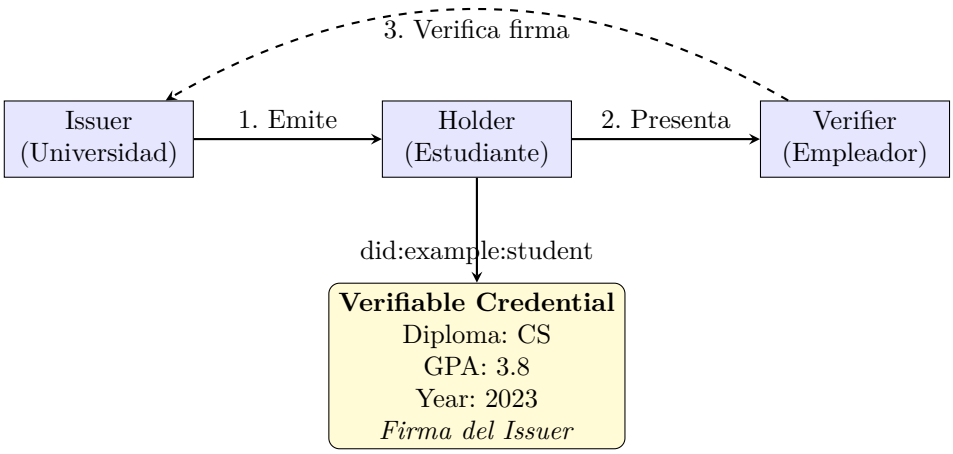


Figura 1.5: Flujo de Verifiable Credentials

1.5.3. Combinando DIDs + VCs + ZK

Idea Clave

El problema con VCs tradicionales:

- Presentar una VC revela todos sus claims
- El verifier puede rastrear al holder
- Múltiples presentaciones son vinculables

Solución: ZK-VCs

- Selective disclosure nativo
- Presentaciones unlinkable
- Predicados sobre atributos
- Privacidad del holder preservada

Listing 1.9: Sistema DID + VC + ZK conceptual

```
from dataclasses import dataclass
from typing import Dict, List, Optional
from hashlib import sha256
import json
from datetime import datetime

@dataclass
class DIDDocument:
    """Documento␣DID␣simplificado."""
    did: str
    verification_methods: List[Dict]
    services: List[Dict]
    created: datetime
    updated: datetime

    def to_dict(self) -> Dict:
        return {
            "@context": "https://www.w3.org/ns/did/v1",
            "id": self.did,
            "verificationMethod": self.verification_methods,
            "service": self.services,
            "created": self.created.isoformat(),
            "updated": self.updated.isoformat()
        }

@dataclass
class VerifiableCredential:
    """Verifiable␣Credential␣W3C-style."""
    context: List[str]
    type: List[str]
    issuer: str  # DID del issuer
    issuance_date: datetime
    expiration_date: Optional[datetime]
    credential_subject: Dict
    proof: Dict

    def to_dict(self) -> Dict:
        return {
            "@context": self.context,
            "type": self.type,
```

```
            "issuer": self.issuer,
            "issuanceDate": self.issuance_date.isoformat(),
            "expirationDate": self.expiration_date.isoformat() if self.expirati
            "credentialSubject": self.credential_subject,
            "proof": self.proof
        }

class ZKVerifiableCredential:
    """
    VC con capacidades ZK para selective disclosure.
    """

    def __init__(self, vc: VerifiableCredential):
        self.vc = vc
        self.commitment = self._create_commitment()

    def _create_commitment(self) -> bytes:
        """Crear commitment de la credencial completa."""
        data = json.dumps(self.vc.to_dict(), sort_keys=True)
        return sha256(data.encode()).digest()

    def create_presentation(
        self,
        disclosed_claims: List[str],
        predicates: Dict[str, tuple],  # claim -> (operator, value)
        holder_secret: bytes,
        verifier_challenge: bytes
    ) -> Dict:
        """
        Crear presentaci n con selective disclosure.

        Args:
            disclosed_claims: Claims a revelar completamente
            predicates: Claims a probar con predicados (ej: age > 18)
            holder_secret: Secreto del holder para binding
            verifier_challenge: Challenge del verifier (previene replay)

        Returns:
            Presentaci n verificable
        """
        presentation = {
            "@context": ["https://www.w3.org/ns/credentials/v2"],
            "type": ["VerifiablePresentation"],
            "verifiableCredential": [],
            "holder": self.vc.credential_subject.get("id"),
        }

        # Datos revelados
        revealed_data = {}
        for claim in disclosed_claims:
            if claim in self.vc.credential_subject:
```

```
                revealed_data[claim] = self.vc.credential_subject[claim]

        # Predicados evaluados
        predicate_results = {}
        for claim, (op, value) in predicates.items():
            if claim in self.vc.credential_subject:
                actual = self.vc.credential_subject[claim]
                if op == ">":
                    predicate_results[f"{claim} > {value}"] = actual > value
                elif op == "<":
                    predicate_results[f"{claim} < {value}"] = actual < value
                elif op == ">=":
                    predicate_results[f"{claim} >= {value}"] = actual >= value
                elif op == "==":
                    predicate_results[f"{claim} == {value}"] = actual == value

        # Crear ZK proof (placeholder)
        zk_proof = self._generate_zk_proof(
            revealed_data,
            predicate_results,
            holder_secret,
            verifier_challenge
        )

        presentation["proof"] = {
            "type": "ZKSelectiveDisclosureProof2024",
            "created": datetime.now().isoformat(),
            "challenge": verifier_challenge.hex(),
            "revealedClaims": revealed_data,
            "predicateResults": predicate_results,
            "zkProof": zk_proof.hex()
        }

        return presentation

    def _generate_zk_proof(
        self,
        revealed: Dict,
        predicates: Dict,
        secret: bytes,
        challenge: bytes
    ) -> bytes:
        """
        Generar ZK proof real.
        En producci n: usar BBS+, Groth16, etc.
        """
        # Placeholder: hash como "proof"
        data = json.dumps({
            'commitment': self.commitment.hex(),
            'revealed': revealed,
            'predicates': predicates,
```

```
            'challenge': challenge.hex()
        }, sort_keys=True)

        return sha256(data.encode() + secret).digest()

    @staticmethod
    def verify_presentation(
        presentation: Dict,
        issuer_public_key: bytes,
        expected_challenge: bytes
    ) -> Dict:
        """
␣␣␣␣␣␣␣␣Verificar una presentaci n ZK.

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣Resultado de verificaci n con claims verificados
␣␣␣␣␣␣␣␣"""
        proof = presentation.get("proof", {})

        # Verificar challenge
        if bytes.fromhex(proof.get("challenge", "")) != expected_challenge:
            return {"valid": False, "error": "Challenge mismatch"}

        # Verificar ZK proof (placeholder)
        # En producci n: verificaci n criptogr fica real

        return {
            "valid": True,
            "revealedClaims": proof.get("revealedClaims", {}),
            "predicateResults": proof.get("predicateResults", {}),
            "issuer": "verified"  # Despu s de verificar firma
        }


def did_vc_zk_demo():
    print("=== DID + VC + ZK Demo ===\n")

    # 1. Universidad crea DID
    university_did = "did:example:university123"
    print(f"University DID: {university_did}")

    # 2. Estudiante crea DID
    student_did = "did:example:student456"
    print(f"Student DID: {student_did}")

    # 3. Universidad emite VC al estudiante
    diploma_vc = VerifiableCredential(
        context=["https://www.w3.org/ns/credentials/v2"],
        type=["VerifiableCredential", "DiplomaCredential"],
        issuer=university_did,
        issuance_date=datetime(2023, 6, 15),
```

```
    expiration_date=None,  # No expira
    credential_subject={
        "id": student_did,
        "name": "Alice Smith",
        "degree": "Computer Science",
        "gpa": 3.85,
        "graduationYear": 2023,
        "honors": True
    },
    proof={
        "type": "Ed25519Signature2020",
        "verificationMethod": f"{university_did}#key-1",
        "proofValue": "z..."  # Firma real
    }
)

print(f"\nVC issued for degree: {diploma_vc.credential_subject['degree']}")

# 4. Crear ZK-VC
zk_vc = ZKVerifiableCredential(diploma_vc)

# 5. Empleador quiere verificar:
# - Que tiene grado CS (revelar)
# - Que GPA > 3.5 (predicado, no revelar exacto)
# - NO revelar nombre

challenge = b"employer_challenge_12345"

presentation = zk_vc.create_presentation(
    disclosed_claims=["degree", "graduationYear"],  # Revelar
    predicates={"gpa": (">=", 3.5)},  # Solo predicado
    holder_secret=b"student_secret",
    verifier_challenge=challenge
)

print("\n--- Presentation Created ---")
print(f"Revealed claims: {presentation['proof']['revealedClaims']}")
print(f"Predicate results: {presentation['proof']['predicateResults']}")
print("Hidden: name, exact GPA, honors status")

# 6. Empleador verifica
result = ZKVerifiableCredential.verify_presentation(
    presentation,
    issuer_public_key=b"university_pubkey",
    expected_challenge=challenge
)

print(f"\n--- Verification Result ---")
print(f"Valid: {result['valid']}")
print(f"Verified claims: {result['revealedClaims']}")
print(f"Verified predicates: {result['predicateResults']}")
```

did_vc_zk_demo()

1.6. Proof of Humanity sin Doxxing

1.6.1. El Problema Sybil

Definición 1.13 (Sybil Attack)**.** Ataque donde un adversario crea múltiples identidades falsas para ganar influencia indebida en un sistema:

- En voting: Un atacante vota múltiples veces
- En airdrops: Farmer reclama múltiples veces
- En DAOs: Una persona controla múltiples votos
- En redes sociales: Bots amplifican mensajes

Definición 1.14 (Proof of Humanity / Proof of Personhood)**.** Mecanismo para demostrar que una entidad digital corresponde a un humano único, sin revelar la identidad de ese humano.

Propiedades deseadas:

1. **Uniqueness:** Una persona = un proof
2. **Privacy:** No revela identidad
3. **Accessibility:** Cualquier humano puede obtenerlo
4. **Resistance:** Difícil de falsificar o transferir

1.6.2. Proyectos de Proof of Humanity

Worldcoin

[Arquitectura Worldcoin] **Proceso:**

1. Usuario escanea iris con "Orb" (dispositivo especial)
2. Orb calcula *iris code* (hash biométrico)
3. Verifica que iris code no existe en base de datos
4. Si único: genera World ID credential
5. Usuario puede probar humanidad con ZK sin revelar iris

Componentes ZK:

- **Semaphore:** Para proofs anónimos de membresía
- **Nullifiers:** Previenen doble-claim por contexto
- **Privacy:** Iris code nunca sale del Orb (en teoría)

Controversias:

- Centralización del Orb y datos biométricos
- Preocupaciones sobre biometric surveillance

- Desigualdad de acceso a Orbs

Listing 1.10: Worldcoin-style Proof of Humanity (conceptual)

```
from hashlib import sha256
import secrets
from typing import Dict, Optional

class WorldIDSimulation:
    """
    Simulaci n simplificada del sistema Worldcoin.
    En producci n: usa Semaphore y verificaci n real de Orb.
    """

    def __init__(self):
        # Base de datos de iris codes (en producci n: rbol Merkle)
        self.iris_codes: set = set()
        self.identity_commitments: Dict[bytes, bytes] = {}  # iris -> commitment
        self.merkle_root: bytes = b'\x00' * 32

    def enroll(self, iris_scan: bytes) -> Optional[Dict]:
        """
        Enrollar nuevo usuario (simulaci n de Orb).

        Args:
            iris_scan: Datos biom tricos del iris

        Returns:
            Credencial World ID o None si ya existe
        """
        # Calcular iris code (en producci n: algoritmo especializado)
        iris_code = sha256(iris_scan).digest()

        # Verificar unicidad
        if iris_code in self.iris_codes:
            return None  # Ya registrado

        # Registrar
        self.iris_codes.add(iris_code)

        # Generar identity secret (usuario guarda esto)
        identity_secret = secrets.token_bytes(32)

        # Generar commitment (p blico)
        commitment = sha256(identity_secret + iris_code).digest()
        self.identity_commitments[iris_code] = commitment

        # Actualizar Merkle tree
        self._update_merkle_root()

        return {
            'identity_secret': identity_secret,
```

```
            'commitment': commitment,
            'iris_code_hash': sha256(iris_code).digest(),  # No revelar iris c
            'enrolled_at': 'timestamp'
        }

    def generate_proof(
        self,
        identity_secret: bytes,
        app_id: str,
        action: str
    ) -> Dict:
        """
␣␣␣␣␣␣␣␣Generar␣ZK␣proof␣de␣humanidad␣para␣una␣acci n.

␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣identity_secret:␣Secreto␣del␣usuario
␣␣␣␣␣␣␣␣␣␣␣␣app_id:␣Identificador␣de␣la␣aplicaci n
␣␣␣␣␣␣␣␣␣␣␣␣action:␣Acci n␣espec fica␣(para␣nullifier)

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣Proof␣verificable
␣␣␣␣␣␣␣␣"""
        # Calcular nullifier ( nico  por app+action)
        nullifier = sha256(
            identity_secret +
            app_id.encode() +
            action.encode()
        ).digest()

        # En producci n: generar Semaphore proof
        # Aqu : simulaci n
        proof = {
            'merkle_root': self.merkle_root.hex(),
            'nullifier': nullifier.hex(),
            'signal': sha256(action.encode()).hexdigest(),
            'proof': 'zk_proof_placeholder'
        }

        return proof

    def verify_proof(
        self,
        proof: Dict,
        app_id: str,
        expected_action: str
    ) -> Dict:
        """
␣␣␣␣␣␣␣␣Verificar␣proof␣de␣humanidad.
␣␣␣␣␣␣␣␣"""
        # Verificar signal
        expected_signal = sha256(expected_action.encode()).hexdigest()
```

```
        if proof['signal'] != expected_signal:
            return {'valid': False, 'error': 'Signal mismatch'}

        # Verificar merkle root es reciente/v lido
        if proof['merkle_root'] != self.merkle_root.hex():
            return {'valid': False, 'error': 'Invalid root'}

        # En producci n: verificar Semaphore proof

        return {
            'valid': True,
            'nullifier': proof['nullifier'],
            'action_verified': expected_action
        }

    def _update_merkle_root(self):
        """Actualizar Merkle root con nuevos commitments."""
        data = b''.join(sorted(self.identity_commitments.values()))
        self.merkle_root = sha256(data).digest()


def worldid_demo():
    print("=== World ID Demo ===\n")

    system = WorldIDSimulation()

    # Usuario A se registra
    print("--- User A enrolls ---")
    iris_a = b"unique_iris_pattern_user_a"
    cred_a = system.enroll(iris_a)
    print(f"Enrolled: {cred_a is not None}")
    print(f"Commitment: {cred_a['commitment'].hex()[:16]}...")

    # Usuario B se registra
    print("\n--- User B enrolls ---")
    iris_b = b"unique_iris_pattern_user_b"
    cred_b = system.enroll(iris_b)
    print(f"Enrolled: {cred_b is not None}")

    # Usuario A intenta registrarse de nuevo (debe fallar)
    print("\n--- User A tries to re-enroll ---")
    cred_a2 = system.enroll(iris_a)
    print(f"Re-enrollment: {cred_a2}")  # None = rejected

    # Usuario A genera proof para votar
    print("\n--- User A votes on proposal ---")
    proof_vote = system.generate_proof(
        identity_secret=cred_a['identity_secret'],
        app_id="dao_xyz",
        action="vote_proposal_123_yes"
    )
```

```
    print(f"Nullifier: {proof_vote['nullifier'][:16]}...")

    # Verificar
    result = system.verify_proof(proof_vote, "dao_xyz", "vote_proposal_123_yes
    print(f"Verification: {result['valid']}")

    # Usuario A intenta votar de nuevo (mismo nullifier detectado)
    print("\n--- User A tries to vote again ---")
    proof_vote2 = system.generate_proof(
        identity_secret=cred_a['identity_secret'],
        app_id="dao_xyz",
        action="vote_proposal_123_yes"
    )
    print(f"Same nullifier: {proof_vote['nullifier'] == proof_vote2['nullifier
    print("=> Double voting detected by nullifier!")

worldid_demo()
```

Otros Enfoques

Proyecto	Método	Ventajas	Desventajas
Worldcoin	Biometría (iris)	Alta unicidad, difícil transferir	Centralizado, hardware especial
BrightID	Grafo social	Descentralizado, sin biometría	Ataques Sybil posibles, lento
Gitcoin Passport	Agregación de señales	Flexible, múltiples fuentes	No garantiza unicidad
Proof of Humanity (PoH)	Video + vouching	Simple, solo necesita webcam	Lento, ataques de colusión
Idena	CAPTCHA periódico	Descentralizado, accesible	Requiere participación regular

Tabla 1.2: Comparación de sistemas de Proof of Humanity

Aplicación en Banco ZK

Proof of Humanity en Banco ZK:
Para prevenir Sybil attacks mientras preserva privacidad:

1. **Integración multi-proveedor:**
   - Acepta proofs de Worldcoin, BrightID, etc.
   - Usuario elige proveedor preferido
   - ZK proof verifica "humanidad" sin revelar proveedor
2. **Rate limiting privado:**
   - Un humano = un depósito de alto valor por período
   - Previene farming de anonymity set
   - Sin vincular depósitos entre sí

3. **Governance:**
   - Un humano = un voto en decisiones de Banco ZK
   - Voting anónimo con Semaphore
   - Previene plutocracy

1.7. Age Verification ZK

1.7.1. Motivación

La verificación de edad es uno de los casos de uso más claros para ZK:
Escenario actual:

- Compras alcohol online: envías copia de ID completo
- Accedes a sitio adulto: proporcionas fecha de nacimiento
- Resultado: Datos sensibles en manos de terceros

Con ZK:

- Demuestras: "Mi edad ≥ 21"
- Sin revelar: Fecha exacta, nombre, documento, foto
- El verificador obtiene SOLO la información necesaria

1.7.2. Arquitectura del Sistema

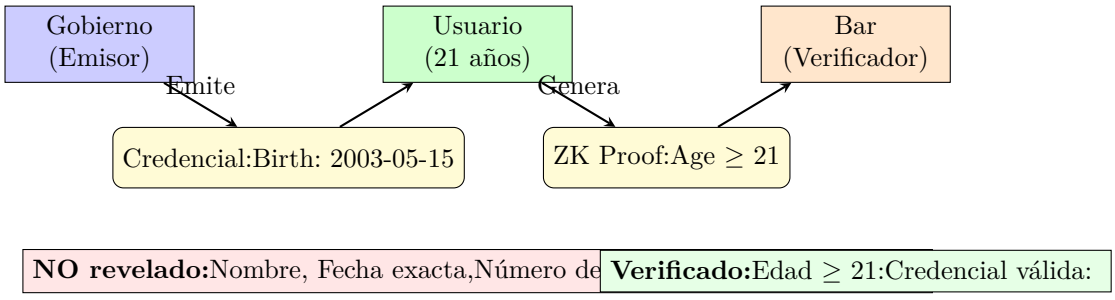


Figura 1.6: Flujo de verificación de edad con ZK

1.7.3. Implementación

Listing 1.11: Circuito completo de verificación de edad

```
pragma circom 2.0.0;

include "circomlib/poseidon.circom";
include "circomlib/comparators.circom";
include "circomlib/bitify.circom";

// Verificar que una fecha es anterior a otra
// Fechas en formato YYYYMMDD como enteros
```

```
template DateBefore() {
    signal input date1;  // Fecha a verificar
    signal input date2;  // Fecha l mite
    signal output isBefore;

    component lt = LessThan(32);
    lt.in[0] <== date1;
    lt.in[1] <== date2;

    isBefore <== lt.out;
}

// Calcular fecha hace N a os
template DateMinusYears(years) {
    signal input currentDate;  // YYYYMMDD
    signal output resultDate;

    // Extraer componentes
    signal year;
    signal monthDay;

    year <-- currentDate \ 10000;
    monthDay <-- currentDate % 10000;

    // Verificar descomposici n
    currentDate === year * 10000 + monthDay;

    // Restar a os
    signal newYear;
    newYear <== year - years;

    // Reconstruir fecha
    resultDate <== newYear * 10000 + monthDay;
}

// Verificaci n completa de edad
template AgeVerificationFull(treeLevels) {
    // ========== PUBLIC INPUTS ==========
    signal input currentDate;           // Fecha actual (YYYYMMDD)
    signal input minimumAge;            // Edad m nima requerida
    signal input issuerPubKeyHash;      // Hash de la clave p blica del emiso
    signal input credentialMerkleRoot;  // Root del  rbol  de credenciales

    // ========== PRIVATE INPUTS ==========
    signal input birthDate;             // Fecha de nacimiento (YYYYMMDD)
    signal input credentialSecret;      // Secreto de la credencial
    signal input issuerSignature[2];    // Firma del emisor
    signal input merkleProof[treeLevels];
    signal input merkleIndices[treeLevels];

    // Otros atributos de la credencial (ocultos)
```

```
signal input name;
signal input documentNumber;
signal input country;

// ========== OUTPUT ==========
signal output isOldEnough;

// ========== VERIFICACIONES ==========

// 1. Calcular commitment de la credencial
component credentialHash = Poseidon(5);
credentialHash.inputs[0] <== birthDate;
credentialHash.inputs[1] <== name;
credentialHash.inputs[2] <== documentNumber;
credentialHash.inputs[3] <== country;
credentialHash.inputs[4] <== credentialSecret;

signal credentialCommitment;
credentialCommitment <== credentialHash.out;

// 2. Verificar que la credencial est  en el  rbol
component merkleVerifier = MerkleProof(treeLevels);
merkleVerifier.leaf <== credentialCommitment;
for (var i = 0; i < treeLevels; i++) {
    merkleVerifier.siblings[i] <== merkleProof[i];
    merkleVerifier.pathIndices[i] <== merkleIndices[i];
}
merkleVerifier.root === credentialMerkleRoot;

// 3. Verificar firma del emisor sobre el commitment
// (simplificado - en producci n usar EdDSA o similar)
component sigVerifier = Poseidon(3);
sigVerifier.inputs[0] <== credentialCommitment;
sigVerifier.inputs[1] <== issuerSignature[0];
sigVerifier.inputs[2] <== issuerSignature[1];
// sigVerifier.out deber a relacionarse con issuerPubKeyHash

// 4. Calcular la fecha de corte (currentDate - minimumAge a os)
component cutoffCalc = DateMinusYears(1);  // Parametrizado
cutoffCalc.currentDate <== currentDate;

// Para minimumAge din mico, necesitar amos l gica m s compleja
// Simplificaci n: calcular cutoff fuera del circuito
signal cutoffDate;
cutoffDate <-- currentDate - minimumAge * 10000;  // Aproximaci n

// Verificar que la aproximaci n es razonable
component yearCheck = LessThan(32);
yearCheck.in[0] <== cutoffDate;
yearCheck.in[1] <== currentDate;
yearCheck.out === 1;
```

```
    // 5. Verificar que birthDate < cutoffDate
    component ageCheck = DateBefore();
    ageCheck.date1 <== birthDate;
    ageCheck.date2 <== cutoffDate;

    isOldEnough <== ageCheck.isBefore;

    // 6. Forzar que sea suficientemente mayor
    isOldEnough === 1;

    // 7. Validaciones adicionales
    // birthDate es fecha v lida (entre 1900 y currentDate)
    component birthValid = LessThan(32);
    birthValid.in[0] <== birthDate;
    birthValid.in[1] <== currentDate;
    birthValid.out === 1;

    component birthNotTooOld = GreaterThan(32);
    birthNotTooOld.in[0] <== birthDate;
    birthNotTooOld.in[1] <== 19000101;
    birthNotTooOld.out === 1;
}

component main {public [currentDate, minimumAge, issuerPubKeyHash, credentialM
    AgeVerificationFull(20);
```

Listing 1.12: Sistema de Age Verification completo

```
from dataclasses import dataclass
from datetime import date, datetime
from typing import Optional, Dict
from hashlib import sha256
import json

@dataclass
class AgeCredential:
    """Credencial de edad emitida por autoridad."""
    subject_id: str  # DID del sujeto
    birth_date: date
    issuer: str  # DID del emisor
    issued_at: datetime
    expiration: Optional[datetime]
    signature: bytes

    # Atributos adicionales (privados)
    full_name: str
    document_number: str
    country: str

class AgeVerificationSystem:
    """Sistema completo de verificaci n de edad con ZK."""
```

```
    def __init__(self):
        self.authorized_issuers: Dict[str, bytes] = {}  # DID -> public key
        self.credential_tree: list = []  # Merkle tree de credenciales

    def register_issuer(self, issuer_did: str, public_key: bytes):
        """Registrar emisor autorizado."""
        self.authorized_issuers[issuer_did] = public_key

    def issue_credential(
        self,
        issuer_did: str,
        issuer_private_key: bytes,
        subject_did: str,
        birth_date: date,
        full_name: str,
        document_number: str,
        country: str
    ) -> AgeCredential:
        """Emitir credencial de edad."""
        if issuer_did not in self.authorized_issuers:
            raise ValueError("Unauthorized issuer")

        # Crear credencial
        credential = AgeCredential(
            subject_id=subject_did,
            birth_date=birth_date,
            issuer=issuer_did,
            issued_at=datetime.now(),
            expiration=None,
            signature=b'',
            full_name=full_name,
            document_number=document_number,
            country=country
        )

        # Firmar
        credential.signature = self._sign_credential(
            credential, issuer_private_key
        )

        # A adir al  rbol  de credenciales
        commitment = self._compute_commitment(credential)
        self.credential_tree.append(commitment)

        return credential

    def _compute_commitment(self, cred: AgeCredential) -> bytes:
        """Calcular commitment de credencial."""
        data = json.dumps({
            'birth_date': cred.birth_date.isoformat(),
```

```
            'name_hash': sha256(cred.full_name.encode()).hexdigest(),
            'doc_hash': sha256(cred.document_number.encode()).hexdigest(),
            'country': cred.country,
            'issuer': cred.issuer
        }, sort_keys=True)
        return sha256(data.encode()).digest()

    def _sign_credential(self, cred: AgeCredential, private_key: bytes) -> byt
        """Firmar credencial (simplificado)."""
        commitment = self._compute_commitment(cred)
        return sha256(commitment + private_key).digest()

    def create_age_proof(
        self,
        credential: AgeCredential,
        minimum_age: int,
        current_date: date,
        verifier_challenge: bytes
    ) -> Dict:
        """
        Crear ZK proof de edad.

        Returns:
            Proof que demuestra edad >= minimum_age sin revelar fecha exacta
        """
        # Calcular edad
        age = self._calculate_age(credential.birth_date, current_date)

        if age < minimum_age:
            return {'success': False, 'error': 'Age requirement not met'}

        # Calcular fecha de corte
        cutoff_year = current_date.year - minimum_age
        cutoff_date = date(cutoff_year, current_date.month, current_date.day)

        # Generar proof (en producci n: SNARK real)
        proof_data = {
            'statement': f'birth_date < {cutoff_date.isoformat()}',
            'current_date': current_date.isoformat(),
            'minimum_age': minimum_age,
            'issuer': credential.issuer,
            'challenge': verifier_challenge.hex()
        }

        # El proof NO contiene:
        # - Fecha exacta de nacimiento
        # - Nombre
        # - N mero de documento
        # Solo prueba: credencial v lida Y edad suficiente

        proof_bytes = sha256(json.dumps(proof_data).encode()).digest()
```

```
        return {
            'success': True,
            'proof': {
                'zk_proof': proof_bytes.hex(),
                'public_inputs': {
                    'minimum_age': minimum_age,
                    'current_date': current_date.isoformat(),
                    'issuer_hash': sha256(credential.issuer.encode()).hexdigest(
                    'challenge': verifier_challenge.hex()
                }
            }
        }

    def verify_age_proof(
        self,
        proof: Dict,
        minimum_age: int,
        current_date: date,
        challenge: bytes
    ) -> Dict:
        """Verificar ZK proof de edad."""
        public_inputs = proof.get('public_inputs', {})

        # Verificar inputs p blicos
        if public_inputs.get('minimum_age') != minimum_age:
            return {'valid': False, 'error': 'Age mismatch'}

        if public_inputs.get('current_date') != current_date.isoformat():
            return {'valid': False, 'error': 'Date mismatch'}

        if public_inputs.get('challenge') != challenge.hex():
            return {'valid': False, 'error': 'Challenge mismatch'}

        # Verificar que el issuer es autorizado
        # (en producci n: verificar contra lista de issuers)

        # Verificar ZK proof (en producci n: verificaci n SNARK)

        return {
            'valid': True,
            'verified_claim': f'Age >= {minimum_age}',
            'issuer_verified': True
        }

    def _calculate_age(self, birth_date: date, current_date: date) -> int:
        """Calcular edad en a os completos."""
        age = current_date.year - birth_date.year
        if (current_date.month, current_date.day) < (birth_date.month, birth_dat
            age -= 1
        return age
```

```
def age_verification_demo():
    print("=== Age Verification Demo ===\n")

    system = AgeVerificationSystem()

    # Registrar emisor (gobierno)
    gov_did = "did:example:government"
    gov_privkey = b"government_secret_key"
    gov_pubkey = sha256(gov_privkey).digest()
    system.register_issuer(gov_did, gov_pubkey)
    print(f"Registered issuer: {gov_did}")

    # Emitir credencial a usuario
    user_did = "did:example:alice"
    credential = system.issue_credential(
        issuer_did=gov_did,
        issuer_private_key=gov_privkey,
        subject_did=user_did,
        birth_date=date(2000, 5, 15),  # 24 a os en 2024
        full_name="Alice Johnson",
        document_number="ABC123456",
        country="US"
    )
    print(f"\nIssued credential to: {user_did}")
    print(f"Birth date: {credential.birth_date}")

    # Escenario 1: Bar verifica edad >= 21
    print("\n--- Scenario 1: Bar (21+) ---")

    current = date(2024, 12, 15)
    challenge1 = b"bar_challenge_12345"

    proof1 = system.create_age_proof(
        credential=credential,
        minimum_age=21,
        current_date=current,
        verifier_challenge=challenge1
    )

    print(f"Proof created: {proof1['success']}")
    print(f"Public inputs: {list(proof1['proof']['public_inputs'].keys())}")
    print("NOT revealed: name, exact birth date, document number")

    result1 = system.verify_age_proof(proof1['proof'], 21, current, challenge1
    print(f"Verification: {result1['valid']}")
    print(f"Claim verified: {result1['verified_claim']}")

    # Escenario 2: Casino verifica edad >= 25
    print("\n--- Scenario 2: Casino (25+) ---")
```

```
    challenge2 = b"casino_challenge_67890"
    proof2 = system.create_age_proof(
        credential=credential,
        minimum_age=25,
        current_date=current,
        verifier_challenge=challenge2
    )

    # Alice tiene 24, no cumple >= 25
    print(f"Proof created: {proof2['success']}")
    if not proof2['success']:
        print(f"Error: {proof2.get('error')}")

    # Escenario 3: Verificaci n fallida por challenge incorrecto
    print("\n--- Scenario 3: Replay attack ---")

    wrong_challenge = b"attacker_challenge"
    result3 = system.verify_age_proof(proof1['proof'], 21, current, wrong_challe
    print(f"Replay attack result: {result3['valid']}")
    print(f"Error: {result3.get('error')}")

age_verification_demo()
```

1.8. KYC Preservando Privacidad

1.8.1. Problemas del KYC Tradicional

Problemas del KYC tradicional:

1. **Replicación de datos:**
   - Cada exchange, banco, servicio tiene copia completa
   - Documentos sensibles en múltiples bases de datos
   - Multiplicación del riesgo de breach
2. **Data breaches:**
   - Exchanges hackeados: datos KYC robados
   - Documentos de identidad en mercado negro
   - Robo de identidad
3. **Surveillance:**
   - Gobiernos pueden solicitar todos los datos
   - Perfiles completos de actividad financiera
   - Sin privacidad real para ciudadanos
4. **Fricciones:**
   - Repetir proceso en cada servicio
   - Documentos físicos, video calls
   - Días/semanas de espera

1.8.2. Modelo de KYC con ZK

Idea Clave

Privacy-Preserving KYC:

1. **KYC una vez:**
   - Usuario completa KYC con proveedor certificado
   - Proveedor verifica documentos reales
   - Usuario recibe credencial ZK firmada
2. **Uso con ZK:**
   - Para usar servicio: genera ZK proof
   - Proof demuestra: "KYC válido" sin datos
   - Servicio NO recibe documentos
3. **Selective disclosure:**
   - Exchange: probar "no PEP, no sancionado"
   - DeFi: probar "residente de país permitido"
   - Casino: probar "mayor de 21"

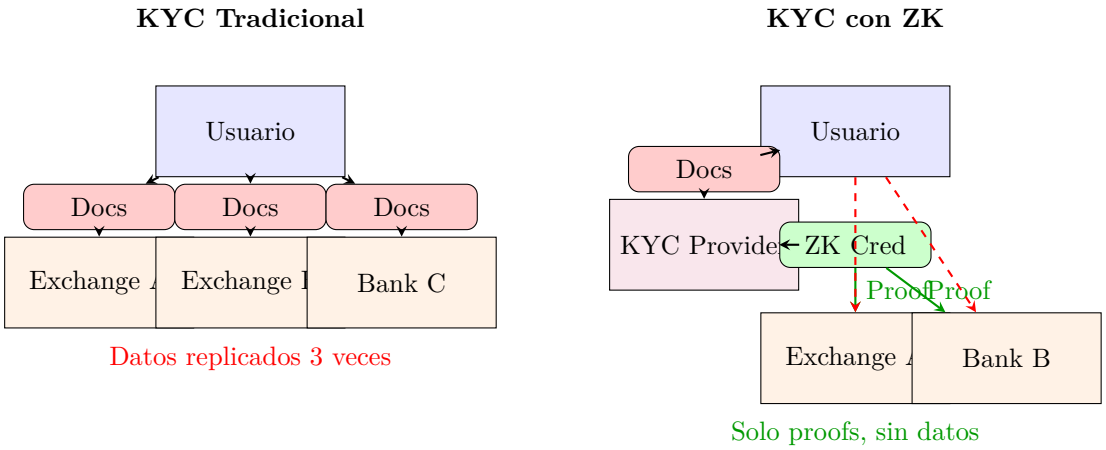


Figura 1.7: KYC tradicional vs KYC con ZK

Listing 1.13: Sistema de Privacy-Preserving KYC

```
from dataclasses import dataclass
from datetime import date, datetime
from typing import Dict, List, Optional, Set
from hashlib import sha256
from enum import Enum
import json

class RiskLevel(Enum):
    LOW = "low"
    MEDIUM = "medium"
```

```
    HIGH = "high"

class KYCStatus(Enum):
    PENDING = "pending"
    VERIFIED = "verified"
    REJECTED = "rejected"
    EXPIRED = "expired"

@dataclass
class KYCCredential:
    """Credencial KYC emitida por proveedor certificado."""
    credential_id: str
    subject_did: str
    provider_did: str

    # Informaci n verificada (privada)
    full_name: str
    birth_date: date
    nationality: str
    document_type: str
    document_number: str
    address_country: str

    # Metadatos de verificaci n
    verification_date: datetime
    expiration_date: datetime
    verification_level: int  # 1, 2, 3

    # Flags de compliance
    is_pep: bool  # Politically Exposed Person
    is_sanctioned: bool
    risk_level: RiskLevel

    # Firma del proveedor
    signature: bytes

class PrivacyKYCSystem:
    """Sistema de KYC preservando privacidad."""

    def __init__(self):
        self.certified_providers: Dict[str, bytes] = {}
        self.sanctioned_countries: Set[str] = {"NK", "IR", "SY"}  # Ejemplo
        self.credential_tree: list = []

    def register_provider(self, provider_did: str, public_key: bytes):
        """Registrar proveedor KYC certificado."""
        self.certified_providers[provider_did] = public_key

    def issue_kyc_credential(
        self,
        provider_did: str,
```

```
        provider_key: bytes,
        user_did: str,
        user_data: Dict
    ) -> KYCCredential:
        """
␣␣␣␣␣␣␣␣Proveedor␣emite␣credencial␣KYC␣despu s␣de␣verificaci n.
␣␣␣␣␣␣␣␣"""
        if provider_did not in self.certified_providers:
            raise ValueError("Uncertified␣provider")

        # Verificaciones del proveedor (en producci n: proceso real)
        is_pep = self._check_pep(user_data['full_name'])
        is_sanctioned = self._check_sanctions(
            user_data['full_name'],
            user_data['nationality']
        )
        risk_level = self._assess_risk(user_data)

        credential = KYCCredential(
            credential_id=sha256(f"{user_did}:{datetime.now()}".encode()).hexd
            subject_did=user_did,
            provider_did=provider_did,
            full_name=user_data['full_name'],
            birth_date=user_data['birth_date'],
            nationality=user_data['nationality'],
            document_type=user_data['document_type'],
            document_number=user_data['document_number'],
            address_country=user_data['address_country'],
            verification_date=datetime.now(),
            expiration_date=datetime(2025, 12, 31),
            verification_level=user_data.get('level', 2),
            is_pep=is_pep,
            is_sanctioned=is_sanctioned,
            risk_level=risk_level,
            signature=b''
        )

        # Firmar credencial
        credential.signature = self._sign_credential(credential, provider_key)

        # A adir al  rbol
        self.credential_tree.append(self._commitment(credential))

        return credential

    def create_compliance_proof(
        self,
        credential: KYCCredential,
        requirements: Dict,
        verifier_challenge: bytes
    ) -> Dict:
```

```
        """
␣␣␣␣␣␣␣␣Crear␣ZK␣proof␣de␣compliance.

␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣credential:␣KYC␣credential␣del␣usuario
␣␣␣␣␣␣␣␣␣␣␣␣requirements:␣Lo␣que␣el␣verifier␣necesita
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣-␣'not_sanctioned':␣bool
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣-␣'not_pep':␣bool
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣-␣'min_verification_level':␣int
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣-␣'allowed_countries':␣List[str]
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣-␣'min_age':␣int
␣␣␣␣␣␣␣␣␣␣␣␣verifier_challenge:␣Challenge␣del␣verifier

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣ZK␣proof␣que␣demuestra␣compliance␣sin␣revelar␣datos
␣␣␣␣␣␣␣␣"""
        # Verificar que credencial no est  expirada
        if credential.expiration_date < datetime.now():
            return {'success': False, 'error': 'Credential␣expired'}

        # Verificar cada requirement
        compliance_results = {}

        if requirements.get('not_sanctioned', False):
            compliance_results['not_sanctioned'] = not credential.is_sanctioned

        if requirements.get('not_pep', False):
            compliance_results['not_pep'] = not credential.is_pep

        if 'min_verification_level' in requirements:
            compliance_results['verification_level_met'] = (
                credential.verification_level >= requirements['min_verification_
            )

        if 'allowed_countries' in requirements:
            compliance_results['country_allowed'] = (
                credential.nationality in requirements['allowed_countries']
            )

        if 'min_age' in requirements:
            age = self._calculate_age(credential.birth_date)
            compliance_results['age_requirement_met'] = age >= requirements['min

        # Verificar que todos los requirements se cumplen
        all_met = all(compliance_results.values())

        if not all_met:
            failed = [k for k, v in compliance_results.items() if not v]
            return {'success': False, 'error': f'Requirements␣not␣met:␣{failed}'

        # Generar ZK proof
```

```
        proof = {
            'zk_proof': self._generate_zk_proof(
                credential, requirements, verifier_challenge
            ),
            'public_inputs': {
                'provider_hash': sha256(credential.provider_did.encode()).hexd
                'expiration': credential.expiration_date.isoformat(),
                'requirements_hash': sha256(json.dumps(requirements, sort_keys
                'challenge': verifier_challenge.hex(),
                'compliance_flags': list(compliance_results.keys())
            }
        }

        return {'success': True, 'proof': proof}

    def verify_compliance_proof(
        self,
        proof: Dict,
        requirements: Dict,
        challenge: bytes
    ) -> Dict:
        """Verificar ZK proof de compliance."""
        public_inputs = proof.get('public_inputs', {})

        # Verificar challenge
        if bytes.fromhex(public_inputs.get('challenge', '')) != challenge:
            return {'valid': False, 'error': 'Invalid challenge'}

        # Verificar requirements match
        expected_hash = sha256(json.dumps(requirements, sort_keys=True).encode
        if public_inputs.get('requirements_hash') != expected_hash:
            return {'valid': False, 'error': 'Requirements mismatch'}

        # Verificar no expirado
        expiration = datetime.fromisoformat(public_inputs.get('expiration', '2
        if expiration < datetime.now():
            return {'valid': False, 'error': 'Credential expired'}

        # Verificar ZK proof (placeholder)
        # En producci n: verificaci n SNARK real

        return {
            'valid': True,
            'verified_compliance': public_inputs.get('compliance_flags', []),
            'provider_verified': True,
            'expiration': public_inputs.get('expiration')
        }

    def _check_pep(self, name: str) -> bool:
        """Verificar si es PEP (simplificado)."""
        # En producci n: consultar bases de datos de PEPs
```

```
        return False

    def _check_sanctions(self, name: str, country: str) -> bool:
        """Verificar listas de sanciones."""
        # En producci n: OFAC, UN, EU sanctions lists
        return country in self.sanctioned_countries

    def _assess_risk(self, data: Dict) -> RiskLevel:
        """Evaluar nivel de riesgo."""
        if data['nationality'] in self.sanctioned_countries:
            return RiskLevel.HIGH
        return RiskLevel.LOW

    def _commitment(self, cred: KYCCredential) -> bytes:
        """Calcular commitment de credencial."""
        data = f"{cred.subject_did}:{cred.verification_date}"
        return sha256(data.encode()).digest()

    def _sign_credential(self, cred: KYCCredential, key: bytes) -> bytes:
        """Firmar credencial."""
        return sha256(self._commitment(cred) + key).digest()

    def _generate_zk_proof(self, cred, reqs, challenge) -> str:
        """Generar ZK proof (placeholder)."""
        data = f"{cred.credential_id}:{json.dumps(reqs)}:{challenge.hex()}"
        return sha256(data.encode()).hexdigest()

    def _calculate_age(self, birth_date: date) -> int:
        today = date.today()
        return today.year - birth_date.year - (
            (today.month, today.day) < (birth_date.month, birth_date.day)
        )


def privacy_kyc_demo():
    print("=== Privacy-Preserving KYC Demo ===\n")

    system = PrivacyKYCSystem()

    # Registrar proveedor KYC
    provider_did = "did:example:kycprovider"
    provider_key = b"provider_secret"
    system.register_provider(provider_did, sha256(provider_key).digest())
    print(f"Registered KYC provider: {provider_did}")

    # Usuario completa KYC una vez
    user_did = "did:example:alice"
    user_data = {
        'full_name': 'Alice Johnson',
        'birth_date': date(1990, 3, 15),
        'nationality': 'US',
```

```
    'document_type': 'passport',
    'document_number': 'AB1234567',
    'address_country': 'US',
    'level': 2
}

credential = system.issue_kyc_credential(
    provider_did, provider_key, user_did, user_data
)
print(f"\nKYC credential issued to: {user_did}")
print(f"Verification level: {credential.verification_level}")
print(f"Expires: {credential.expiration_date}")

# Escenario 1: Exchange quiere verificar no-sanctions + age
print("\n── Scenario 1: Exchange Verification ──")

exchange_requirements = {
    'not_sanctioned': True,
    'not_pep': True,
    'min_age': 18,
    'allowed_countries': ['US', 'UK', 'DE', 'FR', 'ES', 'JP']
}

challenge1 = b"exchange_challenge_abc"

proof1 = system.create_compliance_proof(
    credential, exchange_requirements, challenge1
)

print(f"Proof created: {proof1['success']}")
print(f"Compliance flags: {proof1['proof']['public_inputs']['compliance_fl
print("\nNOT revealed to exchange:")
print("  - Full name")
print("  - Birth date")
print("  - Document number")
print("  - Address")

result1 = system.verify_compliance_proof(
    proof1['proof'], exchange_requirements, challenge1
)
print(f"\nExchange verification: {result1['valid']}")

# Escenario 2: DeFi solo necesita not-sanctioned
print("\n── Scenario 2: DeFi (minimal requirements) ──")

defi_requirements = {
    'not_sanctioned': True
}

challenge2 = b"defi_challenge_xyz"
```

```
    proof2 = system.create_compliance_proof(
        credential, defi_requirements, challenge2
    )

    print(f"Minimal proof created: {proof2['success']}")
    print(f"Only verifies: {proof2['proof']['public_inputs']['compliance_flags']
    print("Even less data exposure than exchange scenario!")

privacy_kyc_demo()
```

Resumen del Bloque

Idea Clave

Puntos clave de Identidad y Credenciales Anónimas (Bloque 14):
1. Anonymous Credentials:

- Credenciales verificables sin revelar identidad
- Sistemas: Idemix (CL), U-Prove, BBS+
- Propiedades: Unforgeability, anonymity, unlinkability

2. Selective Disclosure:

- Revelar solo atributos necesarios
- Predicados: range proofs, comparaciones
- Implementaciones: Merkle trees, BBS+ proofs

3. ZK Identity Proofs:

- Membership, non-membership, attribute proofs
- Semaphore: voting, whistleblowing, rate limiting
- Nullifiers para prevenir doble uso

4. Soulbound Tokens + ZK:

- SBTs públicos = problemas de privacidad
- Solución: Private SBTs con ownership proofs
- Selective disclosure de atributos de SBT

5. DIDs + VCs + ZK:

- DIDs como ancla de identidad descentralizada
- VCs como credenciales verificables
- ZK para presentaciones privadas

6. Proof of Humanity:

- Worldcoin: biometría + ZK

- BrightID: grafo social
- Trade-off: unicidad vs privacidad vs accesibilidad

7. Age Verification:

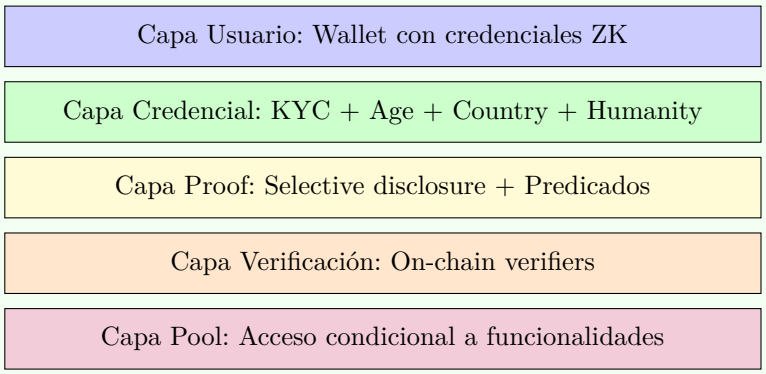
- Probar edad $\geq$ umbral sin revelar fecha exacta
- Circuitos de comparación de fechas
- Aplicaciones: alcohol, gambling, contenido adulto

8. Privacy-Preserving KYC:

- KYC una vez, usar con ZK múltiples veces
- No replicación de documentos sensibles
- Compliance sin surveillance

Aplicación en Banco ZK

Sistema de Identidad en Banco ZK:



Flujo de uso:

1. Usuario obtiene credenciales de proveedores certificados
2. Almacena en wallet personal (nunca en Banco ZK)
3. Para operar: genera ZK proof según requerimientos
4. Verificador on-chain valida proof
5. Usuario accede a funcionalidad sin revelar datos

Casos de uso:

- **Depósitos grandes:** Proof de KYC válido
- **Pools premium:** Proof de país permitido
- **Governance:** Proof of humanity (1 persona = 1 voto)
- **Compliance:** Proof de no-sanctions para withdrawals

Técnica	**Privacidad**	**Verificabilidad**	**Complejidad**	**Madurez**
BBS+ Signatures	Alta	Alta	Media	Alta
Semaphore	Alta	Alta	Media	Alta
Private SBTs	Alta	Alta	Alta	Media
ZK-VCs	Alta	Alta	Alta	Media
Worldcoin	Media	Alta	Alta	Media
Privacy KYC	Alta	Alta	Alta	Baja

Tabla 1.3: Resumen de tecnologías de identidad ZK

Capítulo 2

Compliance vs Privacidad

Bloque 15 de 20

Introducción

El debate entre compliance regulatorio y privacidad financiera es uno de los más importantes en el ecosistema crypto. Este bloque explora cómo las pruebas de conocimiento cero pueden resolver esta aparente dicotomía, permitiendo cumplimiento normativo sin sacrificar la privacidad de usuarios legítimos.

Idea Clave

La falsa dicotomía:
Argumento regulatorio tradicional:

- "Privacidad = facilitar crimen"
- "Transparencia total necesaria para compliance"
- "Si no tienes nada que ocultar..."

Realidad con ZK:

- Privacidad Y compliance son compatibles
- ZK permite probar cumplimiento sin revelar datos
- La vigilancia masiva no es necesaria para AML/CFT
- Privacidad es derecho fundamental, no indicador de criminalidad

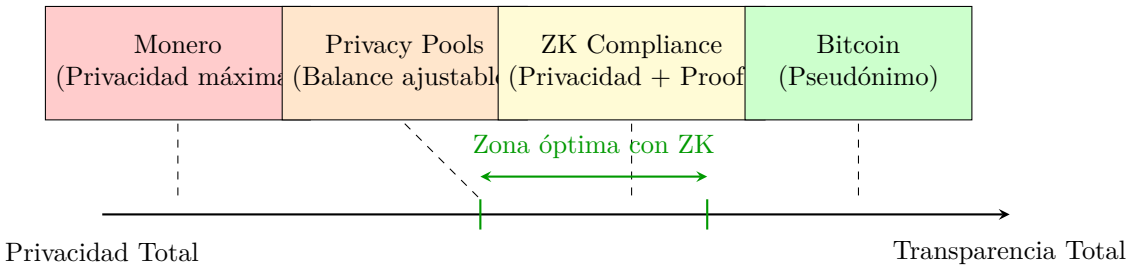


Figura 2.1: Espectro de privacidad-compliance y la zona óptima habilitada por ZK

[Marco Regulatorio Relevante] **Principales regulaciones:**

- **FATF Travel Rule:** Compartir info de origen/destino para txs > $1,000
- **AML (Anti-Money Laundering):** Prevención de lavado de dinero
- **KYC (Know Your Customer):** Identificación de clientes
- **CFT (Counter-Financing of Terrorism):** Anti-financiamiento terrorista
- **OFAC Sanctions:** Lista de entidades sancionadas (EEUU)
- **EU MiCA:** Regulación de criptoactivos en Europa
- **GDPR:** Protección de datos personales (tensión con transparencia)

El desafío: Cumplir con AML/KYC/FATF mientras se respeta GDPR y el derecho a la privacidad.

2.1. Proof-of-Innocence

2.1.1. Concepto y Motivación

Definición 2.1 (Proof-of-Innocence)**.** Prueba criptográfica de que los fondos de un usuario no provienen de fuentes ilícitas, sin revelar el origen específico de los fondos ni la identidad del usuario.

Formalmente:

$$\text{PoI} = \text{ZKProof}\left(\exists \text{ deposit } d : d \in \text{LegitSet} \wedge \text{user owns } d\right)$$

Sin revelar cuál d específico.

Imagina que usaste un mixer hace 2 años. Ahora un regulador te pregunta: "¿De dónde vienen esos fondos?"

Sin ZK:

- Debes revelar tu depósito específico
- Pierdes toda la privacidad del mixer
- O te niegas y pareces sospechoso

Con Proof-of-Innocence:

- Demuestras: "Mis fondos estaban en el mixer ANTES del hack de Exchange X"
- O: "Mis fondos NO vienen de direcciones sancionadas"
- Sin revelar cuál depósito específico era tuyo

2.1.2. Tipos de Proof-of-Innocence

[Categorías de PoI] **1. Proof-of-Pre-Event (Temporal):**

- "Mis fondos entraron al pool antes del timestamp T"
- Útil cuando un evento específico "contamina" el pool
- Ejemplo: Hack de exchange, sanción a entidad

2. Proof-of-Non-Inclusion (Exclusión):

- "Mis fondos NO vienen de direcciones en lista L"
- Requiere sparse Merkle tree o accumulator
- Ejemplo: Lista OFAC, direcciones de hacks conocidos

3. Proof-of-Inclusion (Whitelist):

- "Mis fondos vienen de un set S de fuentes legítimas"
- Set definido por comunidad o regulador
- Ejemplo: Solo exchanges regulados, solo direcciones KYC'd

4. Proof-of-Source-Category:

- "Mis fondos vienen de categoría C"
- Sin revelar cuál fuente específica
- Ejemplo: "De un exchange regulado" (sin decir cuál)

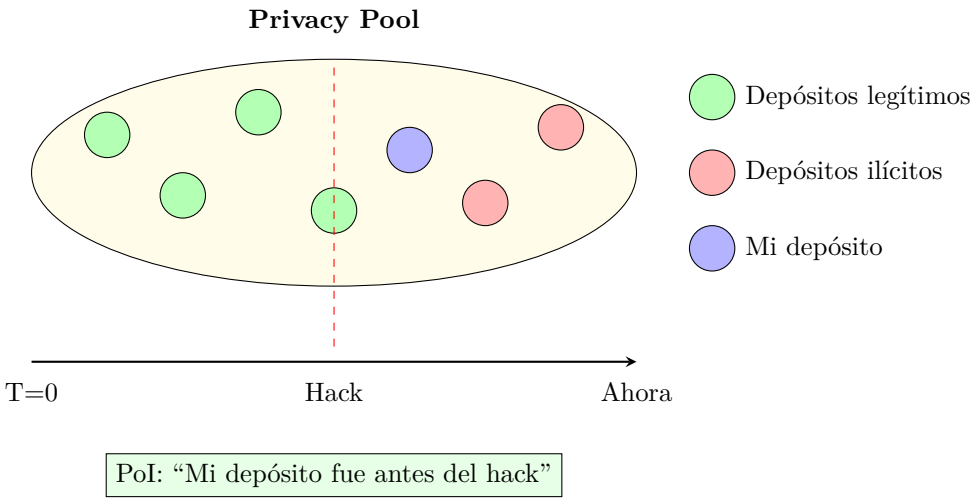


Figura 2.2: Proof-of-Innocence temporal: probar que el depósito ocurrió antes de un evento contaminante

2.1.3. Implementación de Proof-of-Innocence

Listing 2.1: Circuito de Proof-of-Innocence temporal

```
pragma circom 2.0.0;

include "circomlib/poseidon.circom";
include "circomlib/comparators.circom";

// Proof-of-Innocence: Probar que un dep sito ocurri  antes de timestamp T
template ProofOfInnocenceTemporal(treeLevels) {
    // ========== PUBLIC INPUTS ==========
    signal input merkleRoot;            // Root actual del pool
    signal input nullifierHash;         // Nullifier del withdrawal
    signal input cutoffTimestamp;       // Timestamp del evento (hack, sanci n
```

```
    signal input currentTimestamp;      // Timestamp actual

    // ========== PRIVATE INPUTS ==========
    signal input depositTimestamp;      // Cuando se hizo el dep sito
    signal input depositAmount;         // Monto depositado
    signal input secret;                // Secreto del usuario
    signal input nullifier;             // Nullifier secret
    signal input merkleProof[treeLevels];
    signal input merkleIndices[treeLevels];

    // ========== VERIFICACIONES ==========

    // 1. Calcular commitment del dep sito
    // commitment = H(amount, timestamp, secret, nullifier)
    component commitmentHash = Poseidon(4);
    commitmentHash.inputs[0] <== depositAmount;
    commitmentHash.inputs[1] <== depositTimestamp;
    commitmentHash.inputs[2] <== secret;
    commitmentHash.inputs[3] <== nullifier;

    signal commitment;
    commitment <== commitmentHash.out;

    // 2. Verificar que commitment est  en el Merkle tree
    component merkleVerifier = MerkleProof(treeLevels);
    merkleVerifier.leaf <== commitment;
    for (var i = 0; i < treeLevels; i++) {
        merkleVerifier.siblings[i] <== merkleProof[i];
        merkleVerifier.pathIndices[i] <== merkleIndices[i];
    }
    merkleVerifier.root === merkleRoot;

    // 3. Verificar nullifier hash
    component nullifierHasher = Poseidon(2);
    nullifierHasher.inputs[0] <== nullifier;
    nullifierHasher.inputs[1] <== secret;
    nullifierHasher.out === nullifierHash;

    // 4. PROOF-OF-INNOCENCE: dep sito ANTES del cutoff
    component beforeCutoff = LessThan(64);
    beforeCutoff.in[0] <== depositTimestamp;
    beforeCutoff.in[1] <== cutoffTimestamp;
    beforeCutoff.out === 1;  // Debe ser antes del cutoff

    // 5. Verificar que el dep sito es v lido (timestamp razonable)
    component validDeposit = LessThan(64);
    validDeposit.in[0] <== depositTimestamp;
    validDeposit.in[1] <== currentTimestamp;
    validDeposit.out === 1;  // Dep sito antes de ahora
}
```

```
component main {public [merkleRoot, nullifierHash, cutoffTimestamp, currentTim
    ProofOfInnocenceTemporal(20);
\end{lstlisting>

\begin{lstlisting}[language=Python, caption={Sistema completo de Proof-of-Inno
from dataclasses import dataclass
from datetime import datetime
from typing import Dict, List, Set, Optional
from hashlib import sha256
from enum import Enum
import json

class InnocenceType(Enum):
    TEMPORAL = "temporal"           # Antes de evento
    NON_INCLUSION = "non_inclusion" # No en blacklist
    INCLUSION = "inclusion"         # En whitelist
    CATEGORY = "category"           # De categor a espec fica

@dataclass
class Deposit:
    commitment: bytes
    timestamp: datetime
    amount: int
    source_address: str
    source_category: str  # 'exchange', 'defi', 'personal', etc.
    leaf_index: int

@dataclass
class InnocenceProof:
    proof_type: InnocenceType
    nullifier_hash: bytes
    public_inputs: Dict
    zk_proof: bytes
    metadata: Dict

class ProofOfInnocenceSystem:
    """
    Sistema completo de Proof-of-Innocence para privacy pools.
    """

    def __init__(self):
        # Merkle tree de dep sitos
        self.deposits: List[Deposit] = []
        self.merkle_root: bytes = b'\x00' * 32

        # Blacklists y whitelists
        self.blacklisted_addresses: Set[str] = set()
        self.sanctioned_addresses: Set[str] = set()
        self.whitelisted_sources: Set[str] = set()

        # Eventos conocidos (hacks, sanciones)
```

```
        self.contamination_events: List[Dict] = []

        # Categor as de fuentes
        self.source_categories: Dict[str, Set[str]] = {
            'regulated_exchange': set(),
            'defi_protocol': set(),
            'kyc_verified': set(),
            'unknown': set()
        }

    def register_deposit(
        self,
        amount: int,
        source_address: str,
        secret: bytes,
        nullifier: bytes
    ) -> Deposit:
        """Registrar un nuevo dep sito en el pool."""
        timestamp = datetime.now()

        # Determinar categor a de la fuente
        category = self._categorize_source(source_address)

        # Calcular commitment
        commitment = self._compute_commitment(
            amount, timestamp, secret, nullifier
        )

        deposit = Deposit(
            commitment=commitment,
            timestamp=timestamp,
            amount=amount,
            source_address=source_address,
            source_category=category,
            leaf_index=len(self.deposits)
        )

        self.deposits.append(deposit)
        self._update_merkle_root()

        return deposit

    def register_contamination_event(
        self,
        event_type: str,
        timestamp: datetime,
        affected_addresses: List[str],
        description: str
    ):
        """Registrar un evento que contamina ciertos fondos."""
        event = {
```

```
            'type': event_type,
            'timestamp': timestamp,
            'affected_addresses': affected_addresses,
            'description': description
        }
        self.contamination_events.append(event)

        # A adir a blacklist
        for addr in affected_addresses:
            self.blacklisted_addresses.add(addr)

    def add_to_sanctions_list(self, address: str):
        """A adir direcci n a lista de sanciones."""
        self.sanctioned_addresses.add(address)
        self.blacklisted_addresses.add(address)

    def add_whitelisted_source(self, address: str, category: str):
        """A adir fuente a whitelist."""
        self.whitelisted_sources.add(address)
        if category in self.source_categories:
            self.source_categories[category].add(address)

    # ========== GENERACIN DE PROOFS ==========

    def generate_temporal_poi(
        self,
        deposit: Deposit,
        secret: bytes,
        nullifier: bytes,
        cutoff_event: Dict
    ) -> Optional[InnocenceProof]:
        """
        Generar Proof-of-Innocence temporal.
        Demuestra que el dep sito fue ANTES del evento de contaminaci n.
        """
        cutoff_timestamp = cutoff_event['timestamp']

        # Verificar que el dep sito fue antes
        if deposit.timestamp >= cutoff_timestamp:
            return None  # No puede generar proof

        # Generar nullifier hash
        nullifier_hash = self._compute_nullifier_hash(nullifier, secret)

        # Generar ZK proof (placeholder)
        public_inputs = {
            'merkle_root': self.merkle_root.hex(),
            'nullifier_hash': nullifier_hash.hex(),
            'cutoff_timestamp': cutoff_timestamp.isoformat(),
            'current_timestamp': datetime.now().isoformat()
        }
```

```
        zk_proof = self._generate_zk_proof(
            'temporal',
            deposit,
            secret,
            nullifier,
            public_inputs
        )

        return InnocenceProof(
            proof_type=InnocenceType.TEMPORAL,
            nullifier_hash=nullifier_hash,
            public_inputs=public_inputs,
            zk_proof=zk_proof,
            metadata={
                'event_type': cutoff_event['type'],
                'event_description': cutoff_event['description']
            }
        )

    def generate_non_inclusion_poi(
        self,
        deposit: Deposit,
        secret: bytes,
        nullifier: bytes,
        blacklist_root: bytes
    ) -> Optional[InnocenceProof]:
        """
        Generar Proof-of-Innocence de no-inclusi n.
        Demuestra que la fuente NO est  en la blacklist.
        """
        # Verificar que la fuente no est  blacklisted
        if deposit.source_address in self.blacklisted_addresses:
            return None  # No puede generar proof

        nullifier_hash = self._compute_nullifier_hash(nullifier, secret)

        public_inputs = {
            'merkle_root': self.merkle_root.hex(),
            'nullifier_hash': nullifier_hash.hex(),
            'blacklist_root': blacklist_root.hex()
        }

        zk_proof = self._generate_zk_proof(
            'non_inclusion',
            deposit,
            secret,
            nullifier,
            public_inputs
        )
```

```
        return InnocenceProof(
            proof_type=InnocenceType.NON_INCLUSION,
            nullifier_hash=nullifier_hash,
            public_inputs=public_inputs,
            zk_proof=zk_proof,
            metadata={'blacklist_size': len(self.blacklisted_addresses)}
        )

    def generate_inclusion_poi(
        self,
        deposit: Deposit,
        secret: bytes,
        nullifier: bytes,
        whitelist_name: str
    ) -> Optional[InnocenceProof]:
        """
        Generar Proof-of-Innocence de inclusi n.
        Demuestra que la fuente EST  en la whitelist.
        """
        if deposit.source_address not in self.whitelisted_sources:
            return None

        nullifier_hash = self._compute_nullifier_hash(nullifier, secret)

        # Calcular whitelist root
        whitelist_root = self._compute_set_root(self.whitelisted_sources)

        public_inputs = {
            'merkle_root': self.merkle_root.hex(),
            'nullifier_hash': nullifier_hash.hex(),
            'whitelist_root': whitelist_root.hex(),
            'whitelist_name': whitelist_name
        }

        zk_proof = self._generate_zk_proof(
            'inclusion',
            deposit,
            secret,
            nullifier,
            public_inputs
        )

        return InnocenceProof(
            proof_type=InnocenceType.INCLUSION,
            nullifier_hash=nullifier_hash,
            public_inputs=public_inputs,
            zk_proof=zk_proof,
            metadata={'whitelist_name': whitelist_name}
        )

    def generate_category_poi(
```

```
        self,
        deposit: Deposit,
        secret: bytes,
        nullifier: bytes,
        required_category: str
    ) -> Optional[InnocenceProof]:
        """
        Generar Proof-of-Innocence de categor a.
        Demuestra que la fuente es de cierta categor a.
        """
        if deposit.source_category != required_category:
            return None

        nullifier_hash = self._compute_nullifier_hash(nullifier, secret)

        # Root de la categor a
        category_addresses = self.source_categories.get(required_category, set()
        category_root = self._compute_set_root(category_addresses)

        public_inputs = {
            'merkle_root': self.merkle_root.hex(),
            'nullifier_hash': nullifier_hash.hex(),
            'category': required_category,
            'category_root': category_root.hex()
        }

        zk_proof = self._generate_zk_proof(
            'category',
            deposit,
            secret,
            nullifier,
            public_inputs
        )

        return InnocenceProof(
            proof_type=InnocenceType.CATEGORY,
            nullifier_hash=nullifier_hash,
            public_inputs=public_inputs,
            zk_proof=zk_proof,
            metadata={'category': required_category}
        )

    # ========== VERIFICACIN ==========

    def verify_poi(self, proof: InnocenceProof) -> Dict:
        """Verificar un Proof-of-Innocence."""
        # Verificar merkle root es v lido
        if proof.public_inputs.get('merkle_root') != self.merkle_root.hex():
            return {'valid': False, 'error': 'Invalid merkle root'}

        # Verificar ZK proof (placeholder)
```

```
        zk_valid = self._verify_zk_proof(proof)

        if not zk_valid:
            return {'valid': False, 'error': 'Invalid ZK proof'}

        return {
            'valid': True,
            'proof_type': proof.proof_type.value,
            'verified_claim': self._get_claim_description(proof),
            'metadata': proof.metadata
        }

    def _get_claim_description(self, proof: InnocenceProof) -> str:
        """Obtener descripci n del claim verificado."""
        if proof.proof_type == InnocenceType.TEMPORAL:
            return f"Deposit before {proof.public_inputs.get('cutoff_timestamp
        elif proof.proof_type == InnocenceType.NON_INCLUSION:
            return "Source not in blacklist"
        elif proof.proof_type == InnocenceType.INCLUSION:
            return f"Source in whitelist: {proof.metadata.get('whitelist_name'
        elif proof.proof_type == InnocenceType.CATEGORY:
            return f"Source category: {proof.metadata.get('category')}"
        return "Unknown claim"

    # ========== HELPERS ==========

    def _compute_commitment(
        self,
        amount: int,
        timestamp: datetime,
        secret: bytes,
        nullifier: bytes
    ) -> bytes:
        data = f"{amount}:{timestamp.isoformat()}:{secret.hex()}:{nullifier.he
        return sha256(data.encode()).digest()

    def _compute_nullifier_hash(self, nullifier: bytes, secret: bytes) -> byte
        return sha256(nullifier + secret).digest()

    def _compute_set_root(self, address_set: Set[str]) -> bytes:
        sorted_addresses = sorted(address_set)
        data = ''.join(sorted_addresses)
        return sha256(data.encode()).digest()

    def _update_merkle_root(self):
        if not self.deposits:
            self.merkle_root = b'\x00' * 32
            return

        commitments = [d.commitment for d in self.deposits]
        data = b''.join(commitments)
```

```
        self.merkle_root = sha256(data).digest()

    def _categorize_source(self, address: str) -> str:
        for category, addresses in self.source_categories.items():
            if address in addresses:
                return category
        return 'unknown'

    def _generate_zk_proof(
        self,
        proof_type: str,
        deposit: Deposit,
        secret: bytes,
        nullifier: bytes,
        public_inputs: Dict
    ) -> bytes:
        """Generar ZK proof (placeholder)."""
        data = f"{proof_type}:{deposit.commitment.hex()}:{json.dumps(public_inpu
        return sha256(data.encode() + secret).digest()

    def _verify_zk_proof(self, proof: InnocenceProof) -> bool:
        """Verificar ZK proof (placeholder)."""
        return True


def poi_demo():
    print("=== Proof-of-Innocence Demo ===\n")

    system = ProofOfInnocenceSystem()

    # Configurar fuentes conocidas
    system.add_whitelisted_source("0xCoinbase", "regulated_exchange")
    system.add_whitelisted_source("0xKraken", "regulated_exchange")
    system.add_whitelisted_source("0xUniswap", "defi_protocol")

    # Simular dep sitos
    print("--- Registering deposits ---")

    secret1 = b"user1_secret_key"
    nullifier1 = b"user1_nullifier"

    deposit1 = system.register_deposit(
        amount=1000000,
        source_address="0xCoinbase",
        secret=secret1,
        nullifier=nullifier1
    )
    print(f"Deposit 1: {deposit1.amount} from {deposit1.source_address}")
    print(f"  Timestamp: {deposit1.timestamp}")
    print(f"  Category: {deposit1.source_category}")
```

```
# Simular evento de contaminaci n (hack)
import time
time.sleep(0.1)  # Peque a pausa

hack_event = {
    'type': 'hack',
    'timestamp': datetime.now(),
    'affected_addresses': ['0xHackerWallet', '0xStolenFunds'],
    'description': 'Exchange X hack - $100M stolen'
}
system.register_contamination_event(**hack_event)
print(f"\n--- Contamination event registered ---")
print(f"  Type: {hack_event['type']}")
print(f"  Timestamp: {hack_event['timestamp']}")

# Dep sito despu s del hack (no puede probar innocence temporal)
secret2 = b"user2_secret_key"
nullifier2 = b"user2_nullifier"

deposit2 = system.register_deposit(
    amount=500000,
    source_address="0xUnknown",
    secret=secret2,
    nullifier=nullifier2
)
print(f"\nDeposit 2 (after hack): from {deposit2.source_address}")

# Generar proofs
print("\n--- Generating Proofs ---")

# Proof temporal para deposit1 (antes del hack)
temporal_proof = system.generate_temporal_poi(
    deposit1, secret1, nullifier1, hack_event
)
if temporal_proof:
    print(f"\nTemporal PoI for Deposit 1:")
    print(f"  Type: {temporal_proof.proof_type.value}")
    print(f"  Claim: Deposit before hack")

    verification = system.verify_poi(temporal_proof)
    print(f"  Verification: {verification['valid']}")

# Proof de inclusi n en whitelist
inclusion_proof = system.generate_inclusion_poi(
    deposit1, secret1, nullifier1, "regulated_exchanges"
)
if inclusion_proof:
    print(f"\nInclusion PoI for Deposit 1:")
    print(f"  Type: {inclusion_proof.proof_type.value}")
    print(f"  Claim: Source in whitelist")
```

```
    # Proof de categor a
    category_proof = system.generate_category_poi(
        deposit1, secret1, nullifier1, "regulated_exchange"
    )
    if category_proof:
        print(f"\nCategory PoI for Deposit 1:")
        print(f"  Type: {category_proof.proof_type.value}")
        print(f"  Category: {category_proof.metadata['category']}")

    # Intentar proof temporal para deposit2 (despu s del hack - debe fallar)
    temporal_proof2 = system.generate_temporal_poi(
        deposit2, secret2, nullifier2, hack_event
    )
    print(f"\nTemporal PoI for Deposit 2 (after hack): {temporal_proof2}")
    print("  Cannot prove innocence - deposit was after the hack!")

poi_demo()
```

2.2. Selective De-anonymization

2.2.1. Concepto y Necesidad

Definición 2.2 (Selective De-anonymization)**.** Mecanismo criptográfico que permite revelar la identidad de un usuario bajo condiciones específicas y predefinidas, mientras mantiene la privacidad en circunstancias normales.

Propiedades:

1. **Privacidad por defecto:** Usuario anónimo en operaciones normales
2. **Condiciones claras:** De-anonymization solo bajo circunstancias específicas
3. **Proceso verificable:** No puede abusarse sin dejar rastro
4. **Proporcionalidad:** Solo se revela lo necesario

Riesgos de de-anonymization mal implementada:

1. **Backdoors:** Si una entidad puede de-anonimizar unilateralmente, no hay privacidad real
2. **Coerción:** Gobiernos pueden forzar uso de mecanismo
3. **Filtraciones:** Claves de escrow pueden ser robadas
4. **Abuso:** "Condiciones especiales" pueden expandirse

Mitigaciones:

- Threshold schemes: Requiere múltiples partes
- Timelock encryption: Transparencia temporal
- Auditoría pública de usos
- Límites técnicos en scope de revelación

2.2.2. Mecanismos de Selective De-anonymization

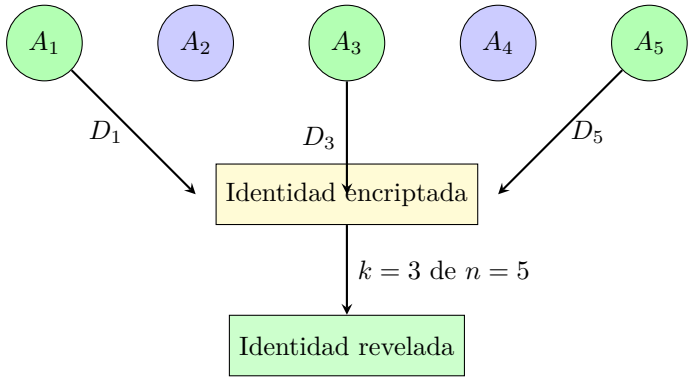
Threshold Decryption

Definición 2.3 (Threshold Decryption)**.** Sistema donde k de n autoridades deben cooperar para descifrar información:

$$\text{Decrypt}(C) = \text{Combine}\left(\text{PartialDecrypt}_1(C), ..., \text{PartialDecrypt}_k(C)\right)$$

Configuraciones comunes:

- 3 de 5: Mayoría simple
- 5 de 7: Supermayoría
- 2 de 3: Mínimo viable



Solo con 3+ autoridades cooperando

Figura 2.3: Threshold decryption: 3 de 5 autoridades revelan la identidad

Listing 2.2: Threshold decryption con Shamir's Secret Sharing

```
from typing import List, Tuple
import secrets
from functools import reduce

class ThresholdDecryption:
    """
␣␣␣␣Sistema␣de␣threshold␣decryption␣usando␣Shamir's␣Secret␣Sharing.
␣␣␣␣"""

    def __init__(self, threshold: int, total: int, prime: int = None):
        """
␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣threshold:␣N mero␣m nimo␣de␣shares␣para␣reconstruir␣(k)
␣␣␣␣␣␣␣␣␣␣␣␣total:␣N mero␣total␣de␣shares␣(n)
␣␣␣␣␣␣␣␣␣␣␣␣prime:␣Primo␣para␣el␣campo␣finito
␣␣␣␣␣␣␣␣"""
        self.k = threshold
        self.n = total
        self.p = prime or self._generate_prime()
```

```
    def _generate_prime(self) -> int:
        """Generar primo seguro."""
        # En producci n: usar primo criptogr fico grande
        return 2**127 - 1  # Mersenne prime

    def _mod_inverse(self, a: int, p: int) -> int:
        """Inverso multiplicativo modular."""
        return pow(a, p - 2, p)

    def split_secret(self, secret: int) -> List[Tuple[int, int]]:
        """
        Dividir secreto en n shares, donde k son suficientes para reconstruir.

        Returns:
            Lista de (x, y) shares
        """
        if secret >= self.p:
            raise ValueError("Secret must be smaller than prime")

        # Generar coeficientes aleatorios para polinomio de grado k-1
        # f(x) = secret + a1*x + a2*x^2 + ... + a(k-1)*x^(k-1)
        coefficients = [secret] + [
            secrets.randbelow(self.p)
            for _ in range(self.k - 1)
        ]

        # Evaluar polinomio en n puntos
        shares = []
        for x in range(1, self.n + 1):
            y = sum(
                coef * pow(x, i, self.p)
                for i, coef in enumerate(coefficients)
            ) % self.p
            shares.append((x, y))

        return shares

    def reconstruct_secret(self, shares: List[Tuple[int, int]]) -> int:
        """
        Reconstruir secreto usando interpolaci n de Lagrange.

        Args:
            shares: Al menos k shares (x, y)

        Returns:
            Secreto original
        """
        if len(shares) < self.k:
            raise ValueError(f"Need at least {self.k} shares")
```

```
        # Usar solo k shares
        shares = shares[:self.k]

        secret = 0
        for i, (xi, yi) in enumerate(shares):
            # Calcular base de Lagrange
            numerator = 1
            denominator = 1

            for j, (xj, _) in enumerate(shares):
                if i != j:
                    numerator = (numerator * (-xj)) % self.p
                    denominator = (denominator * (xi - xj)) % self.p

            # Li(0) = numerator / denominator
            lagrange_basis = (numerator * self._mod_inverse(denominator, self.

            # Sumar contribuci n
            secret = (secret + yi * lagrange_basis) % self.p

        return secret


class SelectiveDeAnonymization:
    """
    Sistema de de-anonimizaci n selectiva para privacy pools.
    """

    def __init__(self, authorities: List[str], threshold: int):
        self.authorities = authorities
        self.threshold = threshold
        self.total = len(authorities)
        self.shamir = ThresholdDecryption(threshold, self.total)

        # Storage de datos encriptados
        self.encrypted_identities: dict = {}  # nullifier -> encrypted_data
        self.authority_shares: dict = {}  # nullifier -> {authority: share}

        # Audit log
        self.decryption_log: List[dict] = []

    def register_with_escrow(
        self,
        nullifier: bytes,
        identity_data: dict,
        encryption_key: int
    ) -> dict:
        """
        Registrar identidad en escrow al depositar.

        Args:
```

```
            nullifier: Nullifier del dep sito
            identity_data: Datos de identidad a proteger
            encryption_key: Clave de encriptaci n (secreta)

        Returns:
            Confirmaci n con shares distribuidos
        """
        # Encriptar identidad (simplificado - usar AES en producci n)
        identity_bytes = str(identity_data).encode()
        encrypted = self._encrypt(identity_bytes, encryption_key)

        # Dividir clave entre autoridades
        shares = self.shamir.split_secret(encryption_key)

        # Almacenar
        self.encrypted_identities[nullifier] = encrypted
        self.authority_shares[nullifier] = {
            self.authorities[i]: share
            for i, share in enumerate(shares)
        }

        return {
            'nullifier': nullifier.hex(),
            'shares_distributed': len(shares),
            'threshold_required': self.threshold
        }

    def request_decryption(
        self,
        nullifier: bytes,
        reason: str,
        requesting_authority: str,
        court_order: Optional[bytes] = None
    ) -> dict:
        """
        Iniciar proceso de de-anonimizaci n.
        """
        if nullifier not in self.encrypted_identities:
            return {'success': False, 'error': 'Unknown nullifier'}

        request_id = secrets.token_hex(16)

        # Registrar solicitud
        request = {
            'request_id': request_id,
            'nullifier': nullifier.hex(),
            'reason': reason,
            'requester': requesting_authority,
            'court_order': court_order.hex() if court_order else None,
            'timestamp': datetime.now().isoformat(),
            'status': 'pending',
```

```
            'approvals': []
        }

        self.decryption_log.append(request)

        return {
            'success': True,
            'request_id': request_id,
            'threshold_needed': self.threshold,
            'authorities_to_contact': self.authorities
        }

    def approve_decryption(
        self,
        request_id: str,
        authority: str,
        authority_signature: bytes
    ) -> dict:
        """
␣␣␣␣␣␣␣␣Una␣autoridad␣aprueba␣la␣solicitud␣de␣decryption.
␣␣␣␣␣␣␣␣"""
        # Encontrar request
        request = next(
            (r for r in self.decryption_log if r['request_id'] == request_id),
            None
        )

        if not request:
            return {'success': False, 'error': 'Unknown␣request'}

        if authority not in self.authorities:
            return {'success': False, 'error': 'Not␣an␣authority'}

        if authority in [a['authority'] for a in request['approvals']]:
            return {'success': False, 'error': 'Already␣approved'}

        # A adir aprobaci n
        nullifier = bytes.fromhex(request['nullifier'])
        share = self.authority_shares[nullifier][authority]

        request['approvals'].append({
            'authority': authority,
            'share': share,
            'signature': authority_signature.hex(),
            'timestamp': datetime.now().isoformat()
        })

        # Verificar si hay suficientes aprobaciones
        if len(request['approvals']) >= self.threshold:
            request['status'] = 'ready'
            return {
```

```
                'success': True,
                'message': 'Threshold␣reached,␣decryption␣ready'
            }

        return {
            'success': True,
            'approvals': len(request['approvals']),
            'needed': self.threshold
        }

    def execute_decryption(self, request_id: str) -> dict:
        """
␣␣␣␣␣␣␣␣Ejecutar␣decryption␣si␣hay␣suficientes␣aprobaciones.
␣␣␣␣␣␣␣␣"""
        request = next(
            (r for r in self.decryption_log if r['request_id'] == request_id),
            None
        )

        if not request or request['status'] != 'ready':
            return {'success': False, 'error': 'Not␣ready␣for␣decryption'}

        # Recolectar shares
        shares = [a['share'] for a in request['approvals']]

        # Reconstruir clave
        encryption_key = self.shamir.reconstruct_secret(shares)

        # Descifrar
        nullifier = bytes.fromhex(request['nullifier'])
        encrypted = self.encrypted_identities[nullifier]
        decrypted = self._decrypt(encrypted, encryption_key)

        # Actualizar log
        request['status'] = 'completed'
        request['completed_at'] = datetime.now().isoformat()

        return {
            'success': True,
            'identity_data': decrypted.decode(),
            'request_id': request_id,
            'authorities_involved': [a['authority'] for a in request['approvals'
        }

    def _encrypt(self, data: bytes, key: int) -> bytes:
        """Encriptar␣datos␣(simplificado)."""
        # En producci n: usar AES-GCM
        key_bytes = key.to_bytes(32, 'big')
        return bytes(a ^ b for a, b in zip(data, key_bytes * (len(data) // 32 +

    def _decrypt(self, data: bytes, key: int) -> bytes:
```

```
        """Descifrar datos."""
        return self._encrypt(data, key)  # XOR es sim trico


def threshold_demo():
    print("=== Selective De-anonymization Demo ===\n")

    # Setup: 3 de 5 autoridades
    authorities = [
        "Court_Authority",
        "Financial_Regulator",
        "Privacy_Ombudsman",
        "Industry_Representative",
        "Civil_Liberties_Org"
    ]

    system = SelectiveDeAnonymization(authorities, threshold=3)
    print(f"System setup: {system.threshold} of {system.total} authorities req
    print(f"Authorities: {authorities}\n")

    # Usuario deposita con escrow de identidad
    user_nullifier = secrets.token_bytes(32)
    user_identity = {
        'name': 'John Doe',
        'country': 'US',
        'kyc_provider': 'VerifyInc',
        'verified_at': '2024-01-15'
    }
    encryption_key = secrets.randbelow(2**128)

    escrow_result = system.register_with_escrow(
        user_nullifier,
        user_identity,
        encryption_key
    )
    print(f"Identity escrowed:")
    print(f"  Shares distributed: {escrow_result['shares_distributed']}")
    print(f"  Threshold: {escrow_result['threshold_required']}")

    # Simular solicitud de de-anonimizaci n
    print("\n-- De-anonymization Request --")

    request = system.request_decryption(
        user_nullifier,
        reason="Suspected money laundering - Case #12345",
        requesting_authority="Financial_Regulator",
        court_order=b"court_order_document_hash"
    )

    print(f"Request ID: {request['request_id']}")
    print(f"Threshold needed: {request['threshold_needed']}")
```

```
    # Autoridades aprueban
    print("\n── Authority Approvals ──")

    approving_authorities = ["Court_Authority", "Financial_Regulator", "Privacy_C

    for auth in approving_authorities:
        result = system.approve_decryption(
            request['request_id'],
            auth,
            secrets.token_bytes(64)  # Firma
        )
        print(f"{auth}: Approved ({result.get('approvals', 'ready')}/{system.thr

    # Ejecutar decryption
    print("\n── Executing Decryption ──")

    decryption_result = system.execute_decryption(request['request_id'])

    if decryption_result['success']:
        print(f"Identity revealed: {decryption_result['identity_data']}")
        print(f"Authorities involved: {decryption_result['authorities_involved']

    # Audit log
    print("\n── Audit Log ──")
    for log in system.decryption_log:
        print(f"Request: {log['request_id'][:16]}...")
        print(f"  Status: {log['status']}")
        print(f"  Reason: {log['reason']}")
        print(f"  Approvals: {len(log['approvals'])}")

threshold_demo()
```

Timelock Encryption

Definición 2.4 (Timelock Encryption)**.** Encriptación que se vuelve descifrable automáticamente después de cierto tiempo:

- **Puzzle-based:** Requiere computación que toma tiempo T
- **Witness encryption:** Descifrable cuando evento ocurre
- **Blockchain-based:** Clave liberada en bloque futuro

[Timelock Puzzles (Rivest, Shamir, Wagner)] **Construcción básica:**

1. Elegir RSA modulus $n = pq$
2. Calcular $\phi(n) = (p-1)(q-1)$
3. Elegir tiempo T (número de squarings)
4. Calcular $e = 2^T \mod \phi(n)$ (rápido si conoces $\phi(n)$)
5. Puzzle: $C = m \cdot a^e \mod n$

Resolver (sin conocer $\phi(n)$):

- Calcular $a^{2^T} \mod n$ requiere T squarings secuenciales
- No paralelizable (inherentemente secuencial)
- Tiempo predecible: T operaciones

Court-Order Escrow

Definición 2.5 (Court-Order Escrow)**.** Sistema donde la clave de descifrado está en custodia de entidad neutral, liberada solo por orden judicial válida:

- Clave en HSM (Hardware Security Module)
- Proceso legal requerido para acceso
- Registro público de todas las órdenes

Aplicación en Banco ZK

Selective De-anonymization en Banco ZK:
Banco ZK implementa de-anonimización selectiva opcional:

1. **Modo estándar (default):**
   - Privacidad total
   - Sin escrow de identidad
   - Para usuarios que no lo necesitan
2. **Modo compliance (opt-in):**
   - Usuario escoge depositar con escrow
   - Identidad encriptada con threshold scheme
   - Autoridades: 3 de 5 (2 reguladores, 2 industria, 1 civil)
   - Beneficio: Acceso a pools de mayor liquidez
3. **Proceso de revelación:**
   - Requiere orden judicial + 3 autoridades
   - Audit log público (hash de cada solicitud)
   - Solo se revela identidad, no historial de txs

Trade-off: Usuarios compliance-mode tienen más opciones pero menos privacidad potencial.

2.3. Viewing Keys y Audit Keys

2.3.1. Jerarquía de Claves de Privacidad

Definición 2.6 (Viewing Key)**.** Clave derivada que permite ver (descifrar) transacciones asociadas a una cuenta sin poder gastar los fondos:

$$\text{ViewingKey} = \text{Derive}(\text{SpendingKey}, \text{“view”})$$

Capacidades:

- Ver balances entrantes
- Descifrar memos de transacciones
- Calcular balance total
- NO puede firmar transacciones

Definición 2.7 (Audit Key)**.** Extensión del viewing key con capacidades adicionales para auditoría:

- Todo lo de viewing key
- Generar reportes verificables (proofs)
- Certificar rangos de tiempo
- Puede tener scope limitado (solo ciertas txs)

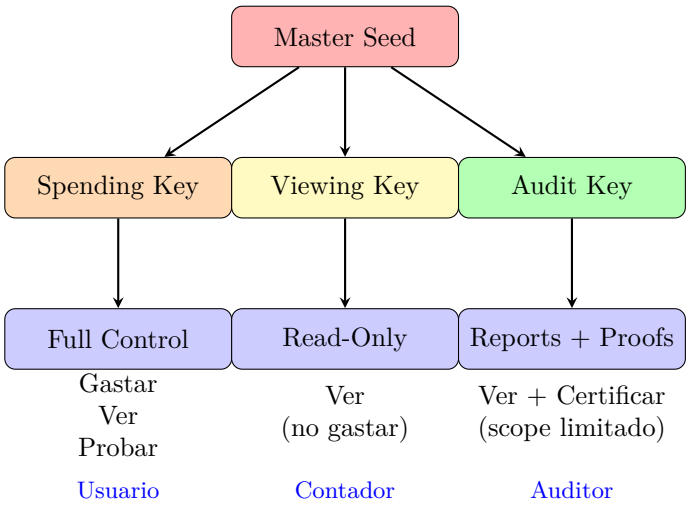


Figura 2.4: Jerarquía de claves: spending, viewing, audit

2.3.2. Implementación de Viewing Keys

[Derivación de Viewing Keys (estilo Zcash)] En Zcash Sapling:
Spending Key (sk):

- 256 bits aleatorios
- Controla todo

Proof Authorizing Key (ak) y Nullifier Key (nk):

$$ak = \text{SpendAuthSig.DerivePublic}(sk)$$

$$nk = \text{PRF}^{nk}_{sk}(\text{“nk”})$$

Full Viewing Key (fvk):

$$fvk = (ak, nk, ovk)$$

donde ovk = outgoing viewing key
Incoming Viewing Key (ivk):

$$ivk = \text{CRH}^{ivk}(ak, nk)$$

Con ivk puedes:

- Detectar pagos entrantes a tu dirección
- Descifrar memos de esos pagos
- Calcular tu balance

NO puedes:

- Ver pagos salientes (necesitas ovk)
- Gastar fondos (necesitas sk)
- Generar nullifiers (necesitas nk completo)

Listing 2.3: Sistema de Viewing Keys y Audit Keys

```
from dataclasses import dataclass
from datetime import datetime, date
from typing import List, Dict, Optional, Tuple
from hashlib import sha256, sha512
import hmac
import secrets
from enum import Enum

class KeyType(Enum):
    SPENDING = "spending"
    VIEWING = "viewing"
    INCOMING_VIEWING = "incoming_viewing"
    OUTGOING_VIEWING = "outgoing_viewing"
    AUDIT = "audit"

@dataclass
class Transaction:
    tx_id: bytes
    timestamp: datetime
    amount: int
    direction: str  # 'incoming' or 'outgoing'
    memo: bytes
    encrypted_memo: bytes
    sender_address: Optional[str]
    recipient_address: str

@dataclass
class AuditReport:
    period_start: date
    period_end: date
    total_incoming: int
    total_outgoing: int
    transaction_count: int
    proof: bytes  # ZK proof de los totales

class ViewingKeySystem:
    """
    Sistema de viewing keys con diferentes niveles de acceso.
```

```
    """

    def __init__(self, master_seed: bytes):
        self.master_seed = master_seed
        self.keys = self._derive_all_keys()

        # Transaction storage (encriptado)
        self.transactions: List[Transaction] = []

    def _derive_all_keys(self) -> Dict[KeyType, bytes]:
        """Derivar todas las claves desde master seed."""
        keys = {}

        # Spending key (m ximo privilegio)
        keys[KeyType.SPENDING] = self._derive_key("spending")

        # Viewing key (full viewing)
        keys[KeyType.VIEWING] = self._derive_key("viewing")

        # Incoming viewing key (solo recibidos)
        keys[KeyType.INCOMING_VIEWING] = self._derive_key("incoming_view")

        # Outgoing viewing key (solo enviados)
        keys[KeyType.OUTGOING_VIEWING] = self._derive_key("outgoing_view")

        # Audit key (con capacidad de generar proofs)
        keys[KeyType.AUDIT] = self._derive_key("audit")

        return keys

    def _derive_key(self, purpose: str) -> bytes:
        """Derivar clave para prop sito espec fico."""
        return hmac.new(
            self.master_seed,
            purpose.encode(),
            sha512
        ).digest()[:32]

    def get_key(self, key_type: KeyType) -> bytes:
        """Obtener clave de tipo espec fico."""
        return self.keys[key_type]

    def export_viewing_key(self, key_type: KeyType) -> Dict:
        """
        Exportar viewing key para compartir.
        """
        if key_type == KeyType.SPENDING:
            raise ValueError("Cannot export spending key as viewing key")

        key = self.keys[key_type]
```

```
        return {
            'type': key_type.value,
            'key': key.hex(),
            'capabilities': self._get_capabilities(key_type),
            'created_at': datetime.now().isoformat()
        }

    def _get_capabilities(self, key_type: KeyType) -> List[str]:
        """Obtener capacidades de cada tipo de clave."""
        capabilities = {
            KeyType.SPENDING: ['spend', 'view_all', 'generate_proofs', 'sign']
            KeyType.VIEWING: ['view_incoming', 'view_outgoing', 'view_memos'],
            KeyType.INCOMING_VIEWING: ['view_incoming', 'view_incoming_memos']
            KeyType.OUTGOING_VIEWING: ['view_outgoing', 'view_outgoing_memos']
            KeyType.AUDIT: ['view_all', 'generate_proofs', 'generate_reports']
        }
        return capabilities.get(key_type, [])

    # ========== ENCRIPTACIN DE TRANSACCIONES ==========

    def encrypt_transaction(self, tx: Transaction) -> Transaction:
        """Encriptar memo de transacci n."""
        # En producci n: usar encryption asim trica apropiada
        if tx.direction == 'incoming':
            key = self.keys[KeyType.INCOMING_VIEWING]
        else:
            key = self.keys[KeyType.OUTGOING_VIEWING]

        encrypted = self._encrypt(tx.memo, key)
        tx.encrypted_memo = encrypted
        return tx

    def record_transaction(
        self,
        amount: int,
        direction: str,
        memo: str,
        counterparty: str
    ):
        """Registrar nueva transacci n."""
        tx = Transaction(
            tx_id=secrets.token_bytes(32),
            timestamp=datetime.now(),
            amount=amount,
            direction=direction,
            memo=memo.encode(),
            encrypted_memo=b'',
            sender_address=counterparty if direction == 'incoming' else 'self'
            recipient_address='self' if direction == 'incoming' else counterpa
        )
```

```
        tx = self.encrypt_transaction(tx)
        self.transactions.append(tx)
        return tx

    # ========== VIEWING CON DIFERENTES CLAVES ==========

    def view_transactions(
        self,
        viewing_key: bytes,
        key_type: KeyType
    ) -> List[Dict]:
        """
␣␣␣␣␣␣␣␣Ver␣transacciones␣usando␣una␣viewing␣key.
␣␣␣␣␣␣␣␣"""
        visible_txs = []

        for tx in self.transactions:
            can_view = False
            decrypted_memo = None

            if key_type in [KeyType.VIEWING, KeyType.AUDIT, KeyType.SPENDING]:
                can_view = True
                # Puede descifrar ambas direcciones
                try:
                    if tx.direction == 'incoming':
                        decrypted_memo = self._decrypt(
                            tx.encrypted_memo,
                            self.keys[KeyType.INCOMING_VIEWING]
                        )
                    else:
                        decrypted_memo = self._decrypt(
                            tx.encrypted_memo,
                            self.keys[KeyType.OUTGOING_VIEWING]
                        )
                except:
                    decrypted_memo = b'[encrypted]'

            elif key_type == KeyType.INCOMING_VIEWING:
                if tx.direction == 'incoming':
                    can_view = True
                    decrypted_memo = self._decrypt(tx.encrypted_memo, viewing_ke

            elif key_type == KeyType.OUTGOING_VIEWING:
                if tx.direction == 'outgoing':
                    can_view = True
                    decrypted_memo = self._decrypt(tx.encrypted_memo, viewing_ke

            if can_view:
                visible_txs.append({
                    'tx_id': tx.tx_id.hex()[:16],
                    'timestamp': tx.timestamp.isoformat(),
```

```
                    'amount': tx.amount,
                    'direction': tx.direction,
                    'memo': decrypted_memo.decode() if decrypted_memo else '[h
                    'counterparty': tx.sender_address if tx.direction == 'inco
                })

        return visible_txs

    def calculate_balance(self, viewing_key: bytes, key_type: KeyType) -> Dict
        """Calcular balance usando viewing key."""
        txs = self.view_transactions(viewing_key, key_type)

        incoming = sum(tx['amount'] for tx in txs if tx['direction'] == 'incom
        outgoing = sum(tx['amount'] for tx in txs if tx['direction'] == 'outgo

        return {
            'total_incoming': incoming,
            'total_outgoing': outgoing,
            'balance': incoming - outgoing,
            'transaction_count': len(txs)
        }

    # ========== AUDIT KEYS Y REPORTES ==========

    def generate_audit_report(
        self,
        audit_key: bytes,
        period_start: date,
        period_end: date
    ) -> AuditReport:
        """
        Generar reporte de auditor a verificable.
        """
        # Verificar que es audit key v lida
        if audit_key != self.keys[KeyType.AUDIT]:
            raise ValueError("Invalid audit key")

        # Filtrar transacciones del per odo
        period_txs = [
            tx for tx in self.transactions
            if period_start <= tx.timestamp.date() <= period_end
        ]

        total_incoming = sum(
            tx.amount for tx in period_txs
            if tx.direction == 'incoming'
        )
        total_outgoing = sum(
            tx.amount for tx in period_txs
            if tx.direction == 'outgoing'
        )
```

```
        # Generar proof ZK de los totales
        proof = self._generate_audit_proof(
            period_txs,
            total_incoming,
            total_outgoing,
            period_start,
            period_end
        )

        return AuditReport(
            period_start=period_start,
            period_end=period_end,
            total_incoming=total_incoming,
            total_outgoing=total_outgoing,
            transaction_count=len(period_txs),
            proof=proof
        )

    def verify_audit_report(self, report: AuditReport) -> bool:
        """Verificar reporte de auditor a."""
        # Verificar ZK proof
        # En producci n: verificaci n SNARK real
        return len(report.proof) > 0

    # ========== HELPERS ==========

    def _encrypt(self, data: bytes, key: bytes) -> bytes:
        """Encriptar datos (simplificado)."""
        return bytes(a ^ b for a, b in zip(data, key * (len(data) // len(key) +

    def _decrypt(self, data: bytes, key: bytes) -> bytes:
        """Descifrar datos."""
        return self._encrypt(data, key)

    def _generate_audit_proof(
        self,
        txs: List[Transaction],
        total_in: int,
        total_out: int,
        start: date,
        end: date
    ) -> bytes:
        """Generar ZK proof para auditor a."""
        # En producci n: SNARK que prueba:
        # 1. Los totales corresponden a transacciones reales
        # 2. Todas las txs est n en el per odo especificado
        # 3. No se omiti  ninguna tx del per odo

        data = f"{total_in}:{total_out}:{start}:{end}:{len(txs)}"
        return sha256(data.encode()).digest()
```

```
def viewing_keys_demo():
    print("=== Viewing Keys Demo ===\n")

    # Crear sistema con master seed
    master_seed = secrets.token_bytes(32)
    system = ViewingKeySystem(master_seed)

    print("Key hierarchy created:")
    for key_type in KeyType:
        caps = system._get_capabilities(key_type)
        print(f"  {key_type.value}: {caps}")

    # Registrar transacciones
    print("\n--- Recording transactions ---")

    system.record_transaction(1000000, 'incoming', 'Salary payment', 'employer_
    system.record_transaction(50000, 'outgoing', 'Coffee purchase', 'coffee_sh
    system.record_transaction(200000, 'incoming', 'Freelance work', 'client_ad
    system.record_transaction(100000, 'outgoing', 'Rent payment', 'landlord_ad

    print(f"Recorded {len(system.transactions)} transactions")

    # Compartir viewing key con contador
    print("\n--- Accountant's View (Incoming Only) ---")

    ivk = system.get_key(KeyType.INCOMING_VIEWING)
    accountant_view = system.view_transactions(ivk, KeyType.INCOMING_VIEWING)

    for tx in accountant_view:
        print(f"  {tx['direction']}: {tx['amount']} - {tx['memo']}")

    print(f"\nAccountant can see: {len(accountant_view)} transactions")
    print("Cannot see: outgoing transactions")

    # Full viewing key
    print("\n--- Full Viewing Key View ---")

    fvk = system.get_key(KeyType.VIEWING)
    full_view = system.view_transactions(fvk, KeyType.VIEWING)

    print(f"Full view: {len(full_view)} transactions")
    balance = system.calculate_balance(fvk, KeyType.VIEWING)
    print(f"Balance: {balance['balance']}")
    print(f"  Incoming: {balance['total_incoming']}")
    print(f"  Outgoing: {balance['total_outgoing']}")

    # Audit report
    print("\n--- Audit Report ---")
```

```
    audit_key = system.get_key(KeyType.AUDIT)
    report = system.generate_audit_report(
        audit_key,
        date(2024, 1, 1),
        date(2024, 12, 31)
    )

    print(f"Period: {report.period_start} to {report.period_end}")
    print(f"Total incoming: {report.total_incoming}")
    print(f"Total outgoing: {report.total_outgoing}")
    print(f"Transaction count: {report.transaction_count}")
    print(f"Proof valid: {system.verify_audit_report(report)}")

viewing_keys_demo()
```

2.3.3. Casos de Uso

Usuario	Clave	Puede Ver	Caso de Uso
Propietario	Spending Key	Todo + gastar	Control total
Contador	Incoming Viewing	Solo recibidos	Calcular ingresos para impuestos
Auditor	Full Viewing + Proof	Todo + reportes	Auditoría financiera
Regulador	Audit Key (temporal)	Período específico	Investigación con orden
Exchange	Viewing Key (scope)	Depósitos del usuario	Verificar origen de fondos

Tabla 2.1: Casos de uso para diferentes tipos de viewing keys

2.4. Regulatory-Compliant Mixers

2.4.1. El Desafío

Los mixers tradicionales (Tornado Cash) fueron sancionados porque:

- No había forma de distinguir usuarios legítimos de criminales
- No había mecanismo de exclusión de fondos ilícitos
- No había forma de cooperar con investigaciones legítimas

Pregunta: ¿Es posible un mixer que preserve privacidad Y sea compliance-friendly?
Respuesta: Sí, con diseño cuidadoso:

- Proof-of-innocence obligatorio
- Exclusión de blacklisted addresses
- Viewing keys opcionales para auditoría
- Association sets “limpios”

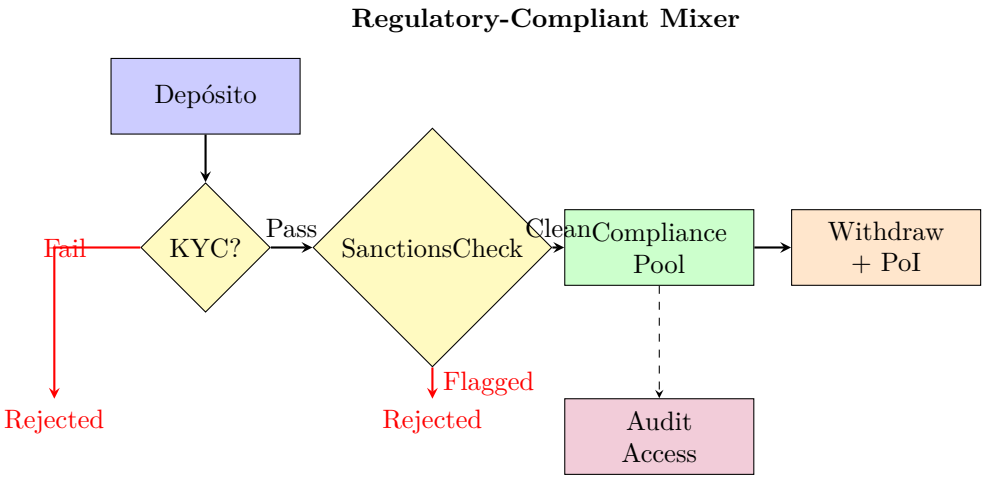


Figura 2.5: Arquitectura de mixer con compliance integrado

2.4.2. Arquitectura de Mixer Compliant

Listing 2.4: Regulatory-Compliant Mixer Contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

interface IVerifier {
    function verifyProof(
        bytes calldata proof,
        uint256[] calldata publicInputs
    ) external view returns (bool);
}

interface ISanctionsOracle {
    function isSanctioned(address addr) external view returns (bool);
}

interface IKYCRegistry {
    function isVerified(bytes32 identityCommitment) external view returns (boo
}

contract CompliantMixer is AccessControl, ReentrancyGuard {
    bytes32 public constant COMPLIANCE_ROLE = keccak256("COMPLIANCE_ROLE");
    bytes32 public constant AUDITOR_ROLE = keccak256("AUDITOR_ROLE");

    // Configuraci n
    IERC20 public immutable token;
    uint256 public immutable denomination;
    IVerifier public withdrawVerifier;
    IVerifier public innocenceVerifier;
    ISanctionsOracle public sanctionsOracle;
```

```
    IKYCRegistry public kycRegistry;

    // Estado del pool
    bytes32 public merkleRoot;
    uint256 public depositCount;
    mapping(bytes32 => bool) public nullifiers;
    mapping(bytes32 => bool) public commitments;

    // Compliance data
    mapping(bytes32 => DepositMetadata) public depositMetadata;
    mapping(bytes32 => bool) public blacklistedCommitments;

    // Association sets (Privacy Pools)
    mapping(bytes32 => bool) public associationSets;
    bytes32[] public approvedAssociationSets;

    struct DepositMetadata {
        uint256 timestamp;
        bytes32 kycCommitment;  // Commitment de KYC (no revela identidad)
        bool hasEscrow;         // Si tiene escrow de identidad
    }

    event Deposit(
        bytes32 indexed commitment,
        uint256 leafIndex,
        uint256 timestamp
    );

    event Withdrawal(
        bytes32 indexed nullifier,
        address indexed recipient,
        bytes32 associationSetRoot
    );

    event ComplianceAlert(
        bytes32 indexed commitment,
        string reason
    );

    constructor(
        address _token,
        uint256 _denomination,
        address _sanctionsOracle,
        address _kycRegistry
    ) {
        token = IERC20(_token);
        denomination = _denomination;
        sanctionsOracle = ISanctionsOracle(_sanctionsOracle);
        kycRegistry = IKYCRegistry(_kycRegistry);

        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
```

```
        _grantRole(COMPLIANCE_ROLE, msg.sender);
    }

    // ========== DEPSITO CON COMPLIANCE ==========

    function deposit(
        bytes32 commitment,
        bytes32 kycCommitment,
        bool useEscrow,
        bytes calldata escrowData
    ) external nonReentrant {
        require(!commitments[commitment], "Commitment exists");
        require(commitment != bytes32(0), "Invalid commitment");

        // 1. Verificar sanctions
        require(
            !sanctionsOracle.isSanctioned(msg.sender),
            "Sanctioned address"
        );

        // 2. Verificar KYC (opcional pero recomendado)
        bool hasKYC = kycRegistry.isVerified(kycCommitment);

        // 3. Transferir fondos
        require(
            token.transferFrom(msg.sender, address(this), denomination),
            "Transfer failed"
        );

        // 4. Guardar metadata de compliance
        depositMetadata[commitment] = DepositMetadata({
            timestamp: block.timestamp,
            kycCommitment: hasKYC ? kycCommitment : bytes32(0),
            hasEscrow: useEscrow
        });

        // 5. Si usa escrow, almacenar datos encriptados
        if (useEscrow && escrowData.length > 0) {
            _storeEscrowData(commitment, escrowData);
        }

        // 6. A adir al  rbol
        commitments[commitment] = true;
        _insertCommitment(commitment);

        emit Deposit(commitment, depositCount++, block.timestamp);
    }

    // ========== RETIRO CON PROOF-OF-INNOCENCE ==========

    function withdraw(
```

```
        bytes calldata withdrawProof,
        bytes calldata innocenceProof,
        bytes32 nullifier,
        bytes32 root,
        address recipient,
        bytes32 associationSetRoot
    ) external nonReentrant {
        require(!nullifiers[nullifier], "Already spent");
        require(_isKnownRoot(root), "Unknown root");
        require(
            _isApprovedAssociationSet(associationSetRoot),
            "Association set not approved"
        );

        // 1. Verificar recipient no sancionado
        require(
            !sanctionsOracle.isSanctioned(recipient),
            "Sanctioned recipient"
        );

        // 2. Verificar withdraw proof
        uint256[] memory withdrawInputs = new uint256[](3);
        withdrawInputs[0] = uint256(root);
        withdrawInputs[1] = uint256(nullifier);
        withdrawInputs[2] = uint256(uint160(recipient));

        require(
            withdrawVerifier.verifyProof(withdrawProof, withdrawInputs),
            "Invalid withdraw proof"
        );

        // 3. Verificar proof-of-innocence
        uint256[] memory innocenceInputs = new uint256[](2);
        innocenceInputs[0] = uint256(root);
        innocenceInputs[1] = uint256(associationSetRoot);

        require(
            innocenceVerifier.verifyProof(innocenceProof, innocenceInputs),
            "Invalid innocence proof"
        );

        // 4. Marcar nullifier
        nullifiers[nullifier] = true;

        // 5. Transferir fondos
        require(
            token.transfer(recipient, denomination),
            "Transfer failed"
        );

        emit Withdrawal(nullifier, recipient, associationSetRoot);
```

```
    }

    // ========== GESTI N DE ASSOCIATION SETS ==========

    function approveAssociationSet(
        bytes32 setRoot,
        string calldata description
    ) external onlyRole(COMPLIANCE_ROLE) {
        associationSets[setRoot] = true;
        approvedAssociationSets.push(setRoot);
    }

    function revokeAssociationSet(bytes32 setRoot)
        external onlyRole(COMPLIANCE_ROLE)
    {
        associationSets[setRoot] = false;
    }

    function _isApprovedAssociationSet(bytes32 setRoot) internal view returns
        return associationSets[setRoot];
    }

    // ========== BLACKLISTING ==========

    function blacklistCommitment(
        bytes32 commitment,
        string calldata reason
    ) external onlyRole(COMPLIANCE_ROLE) {
        blacklistedCommitments[commitment] = true;
        emit ComplianceAlert(commitment, reason);
    }

    // ========== AUDIT ACCESS ==========

    function getDepositMetadata(bytes32 commitment)
        external view
        onlyRole(AUDITOR_ROLE)
        returns (DepositMetadata memory)
    {
        return depositMetadata[commitment];
    }

    function requestEscrowDecryption(
        bytes32 commitment,
        bytes calldata courtOrder
    ) external onlyRole(COMPLIANCE_ROLE) {
        // Iniciar proceso de threshold decryption
        // (implementaci n externa)
    }

    // ========== HELPERS ==========
```

```
    function _insertCommitment(bytes32 commitment) internal {
        // Actualizar Merkle tree
    }

    function _isKnownRoot(bytes32 root) internal view returns (bool) {
        return root == merkleRoot;  // Simplificado
    }

    function _storeEscrowData(bytes32 commitment, bytes calldata data) internal
        // Almacenar datos de escrow encriptados
    }
}
```

2.5. Travel Rule y Soluciones ZK

2.5.1. Requisitos de la Travel Rule

Definición 2.8 (Travel Rule (FATF Recommendation 16))**.** Requisito regulatorio que obliga a VASPs (Virtual Asset Service Providers) a compartir información del originador y beneficiario para transferencias sobre cierto umbral:

Información requerida del originador:

- Nombre
- Número de cuenta (dirección)
- Dirección física, ID nacional, o fecha/lugar de nacimiento

Información del beneficiario:

- Nombre
- Número de cuenta

Umbral: $1,000 USD (varía por jurisdicción)

Problemas de la Travel Rule tradicional:

1. **Privacidad destruida:**
   - Datos personales compartidos entre VASPs
   - Múltiples copias de información sensible
   - Riesgo de data breaches
2. **Incompatible con DeFi:**
   - Smart contracts no tienen "nombre"
   - Self-custody no tiene VASP intermediario
   - ¿Quién comparte info en swap descentralizado?
3. **Honeypot de datos:**
   - Bases de datos centralizadas de todas las transacciones
   - Target para hackers y vigilancia

2.5.2. Soluciones ZK para Travel Rule

Idea Clave

ZK Travel Rule Compliance:
En lugar de compartir datos personales directamente:

1. **Proof of KYC validity:**
   - Probar que originador/beneficiario tienen KYC válido
   - Sin revelar los datos de KYC
   - VASP receptor verifica proof
2. **Encrypted data + conditional access:**
   - Datos encriptados en tránsito
   - Solo descifrable por regulador con orden
   - VASPs solo ven proofs
3. **Notary with ZK verification:**
   - Tercero trusted verifica compliance
   - Emite attestation sin ver datos
   - Usando hardware seguro (SGX, etc.)

Listing 2.5: ZK Travel Rule Implementation

```
from dataclasses import dataclass
from datetime import datetime
from typing import Optional, Dict
from hashlib import sha256
import secrets

@dataclass
class TravelRuleData:
    """Datos requeridos por Travel Rule."""
    originator_name: str
    originator_account: str
    originator_address: str
    originator_id: str
    beneficiary_name: str
    beneficiary_account: str
    amount: int
    currency: str

@dataclass
class ZKTravelRuleProof:
    """Proof de compliance sin revelar datos."""
    originator_kyc_valid: bool
    beneficiary_kyc_valid: bool
    amount_above_threshold: bool
    sanctions_check_passed: bool
    proof: bytes
    encrypted_data: bytes  # Datos encriptados para regulador
```

```
class ZKTravelRuleSystem:
    """
␣␣␣␣Sistema␣de␣Travel␣Rule␣con␣privacidad␣usando␣ZK.
␣␣␣␣"""

    def __init__(self, threshold_amount: int = 1000):
        self.threshold = threshold_amount

        # KYC registry (commitment -> verified)
        self.kyc_commitments: Dict[bytes, bool] = {}

        # Sanctions list
        self.sanctioned_accounts: set = set()

        # Regulator public key for encryption
        self.regulator_pubkey: bytes = b''

    def register_kyc(self, identity_data: Dict) -> bytes:
        """Registrar␣KYC␣y␣obtener␣commitment."""
        # Calcular commitment de identidad
        data_str = str(sorted(identity_data.items()))
        commitment = sha256(data_str.encode()).digest()

        self.kyc_commitments[commitment] = True
        return commitment

    def create_zk_travel_rule_proof(
        self,
        travel_data: TravelRuleData,
        originator_kyc_commitment: bytes,
        beneficiary_kyc_commitment: bytes,
        originator_kyc_proof: bytes,  # ZK proof de validez
        beneficiary_kyc_proof: bytes
    ) -> ZKTravelRuleProof:
        """
␣␣␣␣␣␣␣␣Crear␣proof␣de␣Travel␣Rule␣compliance␣sin␣revelar␣datos.
␣␣␣␣␣␣␣␣"""
        # 1. Verificar KYC v lido para ambas partes
        originator_valid = self._verify_kyc_proof(
            originator_kyc_commitment,
            originator_kyc_proof
        )
        beneficiary_valid = self._verify_kyc_proof(
            beneficiary_kyc_commitment,
            beneficiary_kyc_proof
        )

        # 2. Verificar sanctions
        sanctions_ok = (
            travel_data.originator_account not in self.sanctioned_accounts and
```

```
            travel_data.beneficiary_account not in self.sanctioned_accounts
        )

        # 3. Verificar monto sobre threshold
        above_threshold = travel_data.amount >= self.threshold

        # 4. Encriptar datos para regulador
        encrypted = self._encrypt_for_regulator(travel_data)

        # 5. Generar ZK proof
        proof = self._generate_zk_proof(
            originator_valid,
            beneficiary_valid,
            sanctions_ok,
            above_threshold
        )

        return ZKTravelRuleProof(
            originator_kyc_valid=originator_valid,
            beneficiary_kyc_valid=beneficiary_valid,
            amount_above_threshold=above_threshold,
            sanctions_check_passed=sanctions_ok,
            proof=proof,
            encrypted_data=encrypted
        )

    def verify_travel_rule_proof(
        self,
        proof: ZKTravelRuleProof
    ) -> Dict:
        """
␣␣␣␣␣␣␣␣Verificar␣proof␣de␣Travel␣Rule␣(por␣VASP␣receptor).
␣␣␣␣␣␣␣␣"""
        # Verificar ZK proof
        zk_valid = self._verify_zk_proof(proof.proof)

        if not zk_valid:
            return {
                'compliant': False,
                'error': 'Invalid␣ZK␣proof'
            }

        # Verificar flags de compliance
        all_checks = (
            proof.originator_kyc_valid and
            proof.beneficiary_kyc_valid and
            proof.sanctions_check_passed
        )

        return {
            'compliant': all_checks,
```

```
            'originator_kyc': proof.originator_kyc_valid,
            'beneficiary_kyc': proof.beneficiary_kyc_valid,
            'sanctions_ok': proof.sanctions_check_passed,
            'threshold_transfer': proof.amount_above_threshold,
            'data_available': len(proof.encrypted_data) > 0
        }

    def regulator_decrypt(
        self,
        encrypted_data: bytes,
        regulator_privkey: bytes,
        court_order: bytes
    ) -> Optional[TravelRuleData]:
        """
␣␣␣␣␣␣␣␣Regulador␣descifra␣datos␣con␣orden␣judicial.
␣␣␣␣␣␣␣␣"""
        # Verificar orden judicial (en producci n: verificaci n real)
        if not self._verify_court_order(court_order):
            return None

        # Descifrar
        decrypted = self._decrypt_with_key(encrypted_data, regulator_privkey)

        # Parsear datos
        # ... (deserializaci n)

        return decrypted

    # ========== HELPERS ==========

    def _verify_kyc_proof(self, commitment: bytes, proof: bytes) -> bool:
        """Verificar␣ZK␣proof␣de␣KYC␣v lido."""
        return commitment in self.kyc_commitments and self.kyc_commitments[commi

    def _encrypt_for_regulator(self, data: TravelRuleData) -> bytes:
        """Encriptar␣datos␣para␣acceso␣regulatorio."""
        # En producci n: encryption asim trica con regulator pubkey
        plaintext = str(data).encode()
        return sha256(plaintext + self.regulator_pubkey).digest() + plaintext

    def _decrypt_with_key(self, encrypted: bytes, key: bytes) -> TravelRuleData:
        """Descifrar␣con␣clave␣privada."""
        # Placeholder
        return None

    def _generate_zk_proof(self, *args) -> bytes:
        """Generar␣ZK␣proof."""
        data = ':'.join(str(a) for a in args)
        return sha256(data.encode()).digest()

    def _verify_zk_proof(self, proof: bytes) -> bool:
```

```
        """Verificar ZK proof."""
        return len(proof) == 32

    def _verify_court_order(self, order: bytes) -> bool:
        """Verificar orden judicial."""
        return len(order) > 0


def travel_rule_demo():
    print("=== ZK Travel Rule Demo ===\n")

    system = ZKTravelRuleSystem(threshold_amount=1000)

    # Registrar KYC de Alice y Bob
    alice_kyc = system.register_kyc({
        'name': 'Alice Smith',
        'country': 'US',
        'id': 'ABC123'
    })

    bob_kyc = system.register_kyc({
        'name': 'Bob Jones',
        'country': 'UK',
        'id': 'XYZ789'
    })

    print("KYC registered for Alice and Bob")

    # Alice env a $5000 a Bob
    travel_data = TravelRuleData(
        originator_name="Alice Smith",
        originator_account="0xAlice...",
        originator_address="123 Main St",
        originator_id="ABC123",
        beneficiary_name="Bob Jones",
        beneficiary_account="0xBob...",
        amount=5000,
        currency="USDC"
    )

    print(f"\nTransfer: ${travel_data.amount} from Alice to Bob")

    # Crear ZK proof
    zk_proof = system.create_zk_travel_rule_proof(
        travel_data,
        alice_kyc,
        bob_kyc,
        b"alice_kyc_proof",
        b"bob_kyc_proof"
    )
```

```
    print("\n── ZK Travel Rule Proof ──")
    print(f"Originator KYC valid: {zk_proof.originator_kyc_valid}")
    print(f"Beneficiary KYC valid: {zk_proof.beneficiary_kyc_valid}")
    print(f"Sanctions check: {zk_proof.sanctions_check_passed}")
    print(f"Above threshold: {zk_proof.amount_above_threshold}")
    print(f"Encrypted data available: {len(zk_proof.encrypted_data)} bytes")

    # VASP receptor verifica
    print("\n── Receiving VASP Verification ──")
    verification = system.verify_travel_rule_proof(zk_proof)

    print(f"Compliant: {verification['compliant']}")
    print("\nWhat receiving VASP knows:")
    print("     Originator has valid KYC")
    print("     Beneficiary has valid KYC")
    print("     Neither party is sanctioned")
    print("     Originator's actual name")
    print("     Originator's address")
    print("     Originator's ID number")
    print("\n     Privacy preserved while compliant!")

travel_rule_demo()
```

2.6. Chainalysis y Contramedidas

2.6.1. Técnicas de Blockchain Analytics

Definición 2.9 (Blockchain Analytics / Chainalysis)**.** Conjunto de técnicas para analizar blockchains públicas con el objetivo de:

- Identificar propietarios de direcciones
- Rastrear flujo de fondos
- Detectar actividad sospechosa
- Vincular transacciones con identidades reales

Principales empresas: Chainalysis, Elliptic, CipherTrace, TRM Labs

[Técnicas de Análisis] **1. Heurísticas de Clustering:**

- **Common Input Ownership:** Inputs de misma tx = mismo dueño
- **Change Detection:** Identificar output de cambio
- **Address Reuse:** Vincular múltiples txs de misma dirección
- **Script Fingerprinting:** Identificar tipo de wallet por script

2. Taint Analysis:

- Rastrear "contaminación" desde fuentes conocidas
- Propagar taint a través de transacciones
- Métodos: FIFO, LIFO, proporcional, haircut

3. Exchange Deposits:

- Exchanges conocen identidad de usuarios (KYC)
- Depósito a exchange = posible identificación
- Colaboración exchange-analytics

4. Machine Learning:

- Clasificación de direcciones (exchange, mixer, darknet)
- Detección de patrones anómalos
- Predicción de comportamiento

5. Off-chain Intelligence:

- Scraped addresses de foros, redes sociales
- Direcciones de donación públicas
- Investigación tradicional

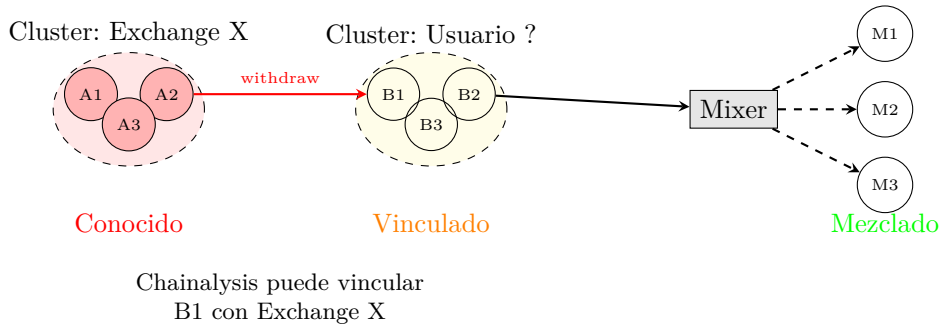


Figura 2.6: Técnicas de clustering y taint analysis

Listing 2.6: Simulación de técnicas de Chainalysis

```
from dataclasses import dataclass
from typing import Dict, List, Set, Tuple
from collections import defaultdict
import networkx as nx

@dataclass
class Transaction:
    tx_id: str
    inputs: List[Tuple[str, int]]    # (address, amount)
    outputs: List[Tuple[str, int]]   # (address, amount)
    timestamp: int

class BlockchainAnalytics:
    """
␣␣␣␣Simulaci n␣de␣t cnicas␣de␣blockchain␣analytics.
␣␣␣␣"""

    def __init__(self):
```

```
        # Grafo de transacciones
        self.graph = nx.DiGraph()

        # Clusters de direcciones
        self.clusters: Dict[str, Set[str]] = {}
        self.address_to_cluster: Dict[str, str] = {}

        # Entidades conocidas
        self.known_entities: Dict[str, str] = {}  # address -> entity_name

        # Taint tracking
        self.taint: Dict[str, Dict[str, float]] = defaultdict(dict)

        # Transacciones
        self.transactions: List[Transaction] = []

    def add_transaction(self, tx: Transaction):
        """A adir transacci n al an lisis."""
        self.transactions.append(tx)

        # A adir al grafo
        for in_addr, _ in tx.inputs:
            for out_addr, _ in tx.outputs:
                self.graph.add_edge(in_addr, out_addr, tx_id=tx.tx_id)

    def label_address(self, address: str, entity: str):
        """Etiquetar direcci n como perteneciente a entidad."""
        self.known_entities[address] = entity

    # ========== CLUSTERING ==========

    def apply_common_input_heuristic(self):
        """
        Heur stica 1: Inputs de misma tx = mismo due o.
        """
        for tx in self.transactions:
            if len(tx.inputs) > 1:
                # Todas las direcciones de input van al mismo cluster
                input_addrs = [addr for addr, _ in tx.inputs]
                self._merge_addresses(input_addrs)

    def apply_change_heuristic(self):
        """
        Heur stica 2: Detectar y vincular change addresses.
        """
        for tx in self.transactions:
            if len(tx.outputs) == 2:
                # Analizar cu l es change
                out1, out2 = tx.outputs

                # Heur stica: output con monto no redondo es change
```

```
                is_round_1 = self._is_round_amount(out1[1])
                is_round_2 = self._is_round_amount(out2[1])

                change_addr = None
                if is_round_1 and not is_round_2:
                    change_addr = out2[0]
                elif is_round_2 and not is_round_1:
                    change_addr = out1[0]

                if change_addr:
                    # Vincular change con inputs
                    input_addrs = [addr for addr, _ in tx.inputs]
                    self._merge_addresses(input_addrs + [change_addr])

    def _merge_addresses(self, addresses: List[str]):
        """Fusionar direcciones en un cluster."""
        # Encontrar clusters existentes
        existing_clusters = set()
        for addr in addresses:
            if addr in self.address_to_cluster:
                existing_clusters.add(self.address_to_cluster[addr])

        # Crear nuevo cluster o usar existente
        if existing_clusters:
            main_cluster = list(existing_clusters)[0]
            # Fusionar otros clusters
            for cluster_id in existing_clusters:
                if cluster_id != main_cluster and cluster_id in self.clusters:
                    self.clusters[main_cluster].update(self.clusters[cluster_i
                    del self.clusters[cluster_id]
        else:
            main_cluster = f"cluster_{len(self.clusters)}"
            self.clusters[main_cluster] = set()

        # A adir todas las direcciones
        for addr in addresses:
            self.clusters[main_cluster].add(addr)
            self.address_to_cluster[addr] = main_cluster

    def _is_round_amount(self, amount: int) -> bool:
        """Verificar si monto es 'redondo'."""
        # Redondo si es m ltiplo de 1000 (simplificado)
        return amount % 1000 == 0

    # ========== TAINT ANALYSIS ==========

    def mark_tainted(self, address: str, source: str):
        """Marcar direcci n como origen de taint."""
        self.taint[address] = {source: 1.0}

    def propagate_taint(self):
```

```
        """Propagar␣taint␣a␣trav s␣de␣transacciones."""
        for tx in sorted(self.transactions, key=lambda x: x.timestamp):
            # Calcular taint total de inputs
            total_input = sum(amount for _, amount in tx.inputs)
            combined_taint: Dict[str, float] = defaultdict(float)

            for addr, amount in tx.inputs:
                if addr in self.taint:
                    weight = amount / total_input
                    for source, prop in self.taint[addr].items():
                        combined_taint[source] += prop * weight

            if not combined_taint:
                continue

            # Distribuir a outputs
            total_output = sum(amount for _, amount in tx.outputs)
            for addr, amount in tx.outputs:
                out_weight = amount / total_output
                self.taint[addr] = {
                    source: prop * out_weight
                    for source, prop in combined_taint.items()
                }

    def get_taint(self, address: str) -> Dict[str, float]:
        """Obtener␣taint␣de␣una␣direcci n."""
        return dict(self.taint.get(address, {}))

    # ========== AN LISIS ==========

    def identify_entity(self, address: str) -> str:
        """Intentar␣identificar␣entidad␣propietaria␣de␣direcci n."""
        # M todo 1: Direcci n directamente conocida
        if address in self.known_entities:
            return self.known_entities[address]

        # M todo 2: Otra direcci n del cluster es conocida
        if address in self.address_to_cluster:
            cluster_id = self.address_to_cluster[address]
            for addr in self.clusters.get(cluster_id, set()):
                if addr in self.known_entities:
                    return f"{self.known_entities[addr]}␣(cluster)"

        return "Unknown"

    def trace_funds(self, address: str, depth: int = 5) -> List[str]:
        """Rastrear␣flujo␣de␣fondos␣desde␣una␣direcci n."""
        path = []
        current = address

        for _ in range(depth):
```

```
            path.append(current)

            # Encontrar siguiente direcci n
            successors = list(self.graph.successors(current))
            if not successors:
                break

            # Elegir la de mayor monto (simplificado)
            current = successors[0]

        return path

    def get_cluster_stats(self) -> Dict:
        """Obtener estad sticas de clustering."""
        total_addresses = sum(len(c) for c in self.clusters.values())
        identified = sum(
            1 for addr in self.known_entities
            if addr in self.address_to_cluster
        )

        return {
            'num_clusters': len(self.clusters),
            'total_addresses_clustered': total_addresses,
            'clusters_with_known_entity': identified,
            'largest_cluster_size': max(
                (len(c) for c in self.clusters.values()),
                default=0
            )
        }


def chainalysis_demo():
    print("=== Chainalysis Simulation Demo ===\n")

    analytics = BlockchainAnalytics()

    # Simular transacciones
    # Exchange -> Usuario
    tx1 = Transaction(
        tx_id="tx1",
        inputs=[("exchange_hot_wallet", 100000)],
        outputs=[
            ("user_addr_1", 50000),  # Pago redondo
            ("exchange_change", 50000)
        ],
        timestamp=1
    )

    # Usuario hace m ltiples transacciones
    tx2 = Transaction(
        tx_id="tx2",
```

```
        inputs=[
            ("user_addr_1", 30000),
            ("user_addr_2", 20000)  # Common input!
        ],
        outputs=[
            ("merchant", 45000),
            ("user_addr_3", 4999)  # Change (no redondo)
        ],
        timestamp=2
    )

    # Hacia mixer
    tx3 = Transaction(
        tx_id="tx3",
        inputs=[("user_addr_3", 4999)],
        outputs=[("mixer_deposit", 4999)],
        timestamp=3
    )

    for tx in [tx1, tx2, tx3]:
        analytics.add_transaction(tx)

    # Etiquetar direcciones conocidas
    analytics.label_address("exchange_hot_wallet", "Binance")
    analytics.label_address("exchange_change", "Binance")
    analytics.label_address("mixer_deposit", "Tornado Cash")

    # Aplicar heur sticas
    print("— Applying Heuristics —")
    analytics.apply_common_input_heuristic()
    analytics.apply_change_heuristic()

    stats = analytics.get_cluster_stats()
    print(f"Clusters formed: {stats['num_clusters']}")
    print(f"Addresses clustered: {stats['total_addresses_clustered']}")

    # Identificar usuario
    print("\n— Entity Identification —")
    for addr in ["user_addr_1", "user_addr_2", "user_addr_3"]:
        entity = analytics.identify_entity(addr)
        print(f"{addr}: {entity}")

    # Taint analysis
    print("\n— Taint Analysis —")
    analytics.mark_tainted("exchange_hot_wallet", "binance_funds")
    analytics.propagate_taint()

    for addr in ["user_addr_1", "user_addr_3", "mixer_deposit"]:
        taint = analytics.get_taint(addr)
        if taint:
            print(f"{addr}: {taint}")
```

```
    # Tracing
    print("\n--- Fund Tracing ---")
    path = analytics.trace_funds("exchange_hot_wallet")
    print(f"Path from exchange: {' -> '.join(path)}")

chainalysis_demo()
```

2.6.2. Contramedidas

Contramedida	Cómo Funciona	Limitaciones	Efectividad
Direcciones nuevas	Usar dirección nueva cada tx	Change aún vinculable	Media
CoinJoin	Mezclar inputs de múltiples usuarios	Timing/amount correlation	Media-Alta
Privacy coins	Privacidad nativa (Monero, Zcash)	Menor liquidez, regulación	Alta
Mixers/Pools	Pool de fondos anónimo	Puntos entrada/salida visibles	Alta
Timing delays	Randomizar timing de txs	Correlación por montos	Media
Amount splitting	Dividir en múltiples txs	Más fees, complejidad	Media
P2P/DEX	Evitar exchanges centralizados	Menor liquidez	Media
Tor/VPN	Ocultar IP al crear tx	Solo protege metadata de red	Baja

Tabla 2.2: Contramedidas contra análisis de blockchain

2.7. Privacy Pools (Propuesta de Vitalik)

2.7.1. Concepto y Motivación

Definición 2.10 (Privacy Pools)**.** Sistema propuesto por Vitalik Buterin, Jacob Illum, et al. que permite privacidad financiera mientras mantiene la capacidad de demostrar desasociación de fondos ilícitos.

Paper: "Blockchain Privacy and Regulatory Compliance: Towards a Practical Equilibrium" (2023)

Idea central: Los usuarios pueden probar que sus fondos pertenecen a un "association set" (conjunto de depósitos) que excluye fuentes ilícitas conocidas.

Idea Clave

El dilema resuelto:
Problema con mixers tradicionales:

- Todos los depósitos en el mismo set
- Incluye fondos de hacks, ransomware, etc.
- Usuario legítimo no puede probar "inocencia"

Solución Privacy Pools:

- Múltiples "association sets" definidos por comunidad

- Usuario elige de qué set quiere probar membresía
- Set puede excluir direcciones conocidas como ilícitas
- Prueba ZK: "Mis fondos están en set X" sin revelar cuál depósito

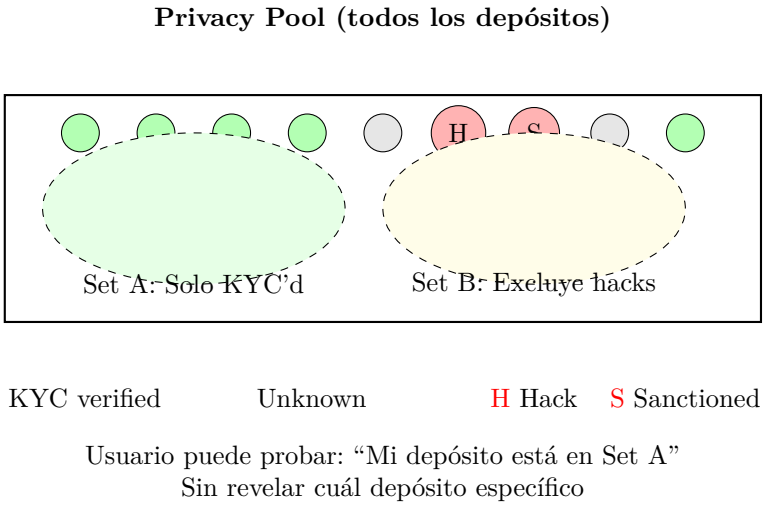


Figura 2.7: Privacy Pools: múltiples association sets con diferentes niveles de restricción

2.7.2. Arquitectura Técnica

[Componentes de Privacy Pools] **1. Deposit Tree:**

- Merkle tree de todos los depósitos
- Root: R_{all}
- Cada depósito tiene commitment C_i

2. Association Set Providers (ASPs):

- Entidades que definen y publican association sets
- Cada set es subset de depósitos
- Publicado como Merkle root: R_{set}

3. Withdrawal Proof:

- ZK proof estándar de posesión de depósito
- Prueba: conoce (v, r) tal que $C = \text{Commit}(v, r)$ y $C \in R_{all}$

4. Association Proof:

- ZK proof adicional de membresía en association set
- Prueba: El mismo C también está en R_{set}
- Sin revelar cuál C

Full Proof:

$$\pi = \text{ZKProof}\left(\begin{array}{l} \text{Conozco } (v, r, \text{path}_{all}, \text{path}_{set}) \\ \text{tal que } C = \text{Commit}(v, r) \\ \wedge \text{MerkleVerify}(C, \text{path}_{all}, R_{all}) \\ \wedge \text{MerkleVerify}(C, \text{path}_{set}, R_{set}) \end{array}\right)$$

Listing 2.7: Circuito de Privacy Pools

```
pragma circom 2.0.0;

include "circomlib/poseidon.circom";

// Verificar que un commitment pertenece tanto al pool completo
// como a un association set espec fico
template PrivacyPoolWithdraw(poolLevels, setLevels) {
    // ========== PUBLIC INPUTS ==========
    signal input poolRoot;            // Root del pool completo
    signal input associationSetRoot; // Root del association set elegido
    signal input nullifierHash;       // Hash del nullifier
    signal input recipient;           // Direcci n de destino

    // ========== PRIVATE INPUTS ==========
    signal input amount;
    signal input secret;
    signal input nullifier;

    // Merkle proofs
    signal input poolPathElements[poolLevels];
    signal input poolPathIndices[poolLevels];
    signal input setPathElements[setLevels];
    signal input setPathIndices[setLevels];

    // ========== COMPUTATIONS ==========

    // 1. Calcular commitment
    component commitmentHasher = Poseidon(3);
    commitmentHasher.inputs[0] <== amount;
    commitmentHasher.inputs[1] <== secret;
    commitmentHasher.inputs[2] <== nullifier;

    signal commitment;
    commitment <== commitmentHasher.out;

    // 2. Verificar membership en pool completo
    component poolMerkle = MerkleProof(poolLevels);
    poolMerkle.leaf <== commitment;
    for (var i = 0; i < poolLevels; i++) {
        poolMerkle.pathElements[i] <== poolPathElements[i];
        poolMerkle.pathIndices[i] <== poolPathIndices[i];
    }
    poolMerkle.root === poolRoot;

    // 3. Verificar membership en association set
```

```
    component setMerkle = MerkleProof(setLevels);
    setMerkle.leaf <== commitment;
    for (var i = 0; i < setLevels; i++) {
        setMerkle.pathElements[i] <== setPathElements[i];
        setMerkle.pathIndices[i] <== setPathIndices[i];
    }
    setMerkle.root === associationSetRoot;

    // 4. Calcular y verificar nullifier hash
    component nullifierHasher = Poseidon(2);
    nullifierHasher.inputs[0] <== nullifier;
    nullifierHasher.inputs[1] <== secret;
    nullifierHasher.out === nullifierHash;

    // 5. Vincular recipient (previene front-running)
    signal recipientSquare;
    recipientSquare <== recipient * recipient;
}

component main {public [poolRoot, associationSetRoot, nullifierHash, recipient]}
    PrivacyPoolWithdraw(20, 16);
```

Listing 2.8: Sistema completo de Privacy Pools

```
from dataclasses import dataclass
from typing import Dict, List, Set, Optional
from hashlib import sha256
from datetime import datetime
import secrets

@dataclass
class Deposit:
    commitment: bytes
    timestamp: datetime
    source_address: str
    amount: int
    leaf_index: int

@dataclass
class AssociationSet:
    name: str
    description: str
    root: bytes
    members: Set[bytes]  # Commitments incluidos
    exclusion_criteria: str
    provider: str
    created_at: datetime

class PrivacyPoolSystem:
    """
␣␣␣␣Implementaci n␣de␣Privacy␣Pools␣seg n␣propuesta␣de␣Vitalik␣et␣al.
␣␣␣␣"""
```

```
    def __init__(self):
        # Pool principal
        self.deposits: List[Deposit] = []
        self.pool_root: bytes = b'\x00' * 32
        self.nullifiers: Set[bytes] = set()

        # Association sets
        self.association_sets: Dict[str, AssociationSet] = {}

        # Providers de association sets
        self.approved_providers: Set[str] = set()

        # Blacklists para construir sets
        self.known_hacks: Set[str] = set()  # Direcciones de hacks
        self.sanctioned: Set[str] = set()   # Direcciones sancionadas
        self.kyc_verified: Set[str] = set() # Direcciones con KYC

    def register_provider(self, provider_name: str):
        """Registrar proveedor de association sets."""
        self.approved_providers.add(provider_name)

    def deposit(
        self,
        amount: int,
        source_address: str,
        secret: bytes,
        nullifier: bytes
    ) -> Deposit:
        """Depositar en el pool."""
        # Calcular commitment
        commitment = self._compute_commitment(amount, secret, nullifier)

        deposit = Deposit(
            commitment=commitment,
            timestamp=datetime.now(),
            source_address=source_address,
            amount=amount,
            leaf_index=len(self.deposits)
        )

        self.deposits.append(deposit)
        self._update_pool_root()

        return deposit

    # ========== ASSOCIATION SET MANAGEMENT ==========

    def create_association_set(
        self,
        provider: str,
```

```
        name: str,
        description: str,
        inclusion_criteria: str
    ) -> AssociationSet:
        """
␣␣␣␣␣␣␣␣Crear␣nuevo␣association␣set␣basado␣en␣criterios.
␣␣␣␣␣␣␣␣"""
        if provider not in self.approved_providers:
            raise ValueError("Unapproved␣provider")

        # Determinar qu  commitments incluir
        members = set()

        for deposit in self.deposits:
            include = True

            if inclusion_criteria == "exclude_hacks":
                if deposit.source_address in self.known_hacks:
                    include = False

            elif inclusion_criteria == "exclude_sanctioned":
                if deposit.source_address in self.sanctioned:
                    include = False

            elif inclusion_criteria == "only_kyc":
                if deposit.source_address not in self.kyc_verified:
                    include = False

            elif inclusion_criteria == "exclude_all_bad":
                if (deposit.source_address in self.known_hacks or
                    deposit.source_address in self.sanctioned):
                    include = False

            if include:
                members.add(deposit.commitment)

        # Calcular root del set
        root = self._compute_set_root(members)

        association_set = AssociationSet(
            name=name,
            description=description,
            root=root,
            members=members,
            exclusion_criteria=inclusion_criteria,
            provider=provider,
            created_at=datetime.now()
        )

        self.association_sets[name] = association_set
        return association_set
```

```
    def update_association_set(self, set_name: str):
        """Actualizar association set con nuevos dep sitos."""
        if set_name not in self.association_sets:
            return

        aset = self.association_sets[set_name]

        # Re-evaluar todos los dep sitos
        new_members = set()
        for deposit in self.deposits:
            if self._should_include(deposit, aset.exclusion_criteria):
                new_members.add(deposit.commitment)

        aset.members = new_members
        aset.root = self._compute_set_root(new_members)

    def _should_include(self, deposit: Deposit, criteria: str) -> bool:
        """Evaluar si dep sito cumple criterios de inclusi n."""
        if criteria == "exclude_hacks":
            return deposit.source_address not in self.known_hacks
        elif criteria == "exclude_sanctioned":
            return deposit.source_address not in self.sanctioned
        elif criteria == "only_kyc":
            return deposit.source_address in self.kyc_verified
        elif criteria == "exclude_all_bad":
            return (deposit.source_address not in self.known_hacks and
                    deposit.source_address not in self.sanctioned)
        return True

    # ========== WITHDRAWAL WITH ASSOCIATION PROOF ==========

    def withdraw(
        self,
        deposit: Deposit,
        secret: bytes,
        nullifier: bytes,
        association_set_name: str
    ) -> Dict:
        """
        Retirar con proof de association set.
        """
        # Verificar nullifier no usado
        nullifier_hash = sha256(nullifier + secret).digest()
        if nullifier_hash in self.nullifiers:
            return {'success': False, 'error': 'Already spent'}

        # Verificar que deposit est  en el set elegido
        if association_set_name not in self.association_sets:
            return {'success': False, 'error': 'Unknown association set'}
```

```
        aset = self.association_sets[association_set_name]
        if deposit.commitment not in aset.members:
            return {'success': False, 'error': 'Deposit␣not␣in␣association␣set'}

        # Generar proofs
        pool_proof = self._generate_pool_membership_proof(deposit)
        set_proof = self._generate_set_membership_proof(deposit, aset)

        # Marcar como gastado
        self.nullifiers.add(nullifier_hash)

        return {
            'success': True,
            'nullifier_hash': nullifier_hash.hex(),
            'pool_root': self.pool_root.hex(),
            'association_set_root': aset.root.hex(),
            'association_set_name': aset.name,
            'pool_proof': pool_proof,
            'set_proof': set_proof,
            'verified_claim': f"Deposit␣in␣'{aset.name}'␣set␣({aset.exclusion_cr
        }

    def verify_withdrawal(self, withdrawal_data: Dict) -> Dict:
        """
␣␣␣␣␣␣␣␣Verificar␣withdrawal␣proof.
␣␣␣␣␣␣␣␣"""
        # Verificar pool root
        if withdrawal_data['pool_root'] != self.pool_root.hex():
            return {'valid': False, 'error': 'Invalid␣pool␣root'}

        # Verificar association set root
        set_name = withdrawal_data.get('association_set_name')
        if set_name in self.association_sets:
            expected_root = self.association_sets[set_name].root.hex()
            if withdrawal_data['association_set_root'] != expected_root:
                return {'valid': False, 'error': 'Invalid␣association␣set␣root'}

        # Verificar ZK proofs (placeholder)

        return {
            'valid': True,
            'association_set': withdrawal_data.get('association_set_name'),
            'claim_verified': withdrawal_data.get('verified_claim')
        }

    # ========= BLACKLIST MANAGEMENT =========

    def report_hack(self, address: str, description: str):
        """Reportar␣direcci n␣como␣parte␣de␣hack."""
        self.known_hacks.add(address)
        # Actualizar todos los sets relevantes
```

```
        for set_name in self.association_sets:
            self.update_association_set(set_name)

    def add_sanction(self, address: str):
        """A adir direcci n a lista de sanciones."""
        self.sanctioned.add(address)
        for set_name in self.association_sets:
            self.update_association_set(set_name)

    def verify_kyc(self, address: str):
        """Marcar direcci n como KYC verificada."""
        self.kyc_verified.add(address)

    # ========== HELPERS ==========

    def _compute_commitment(self, amount: int, secret: bytes, nullifier: bytes
        data = f"{amount}:{secret.hex()}:{nullifier.hex()}"
        return sha256(data.encode()).digest()

    def _compute_set_root(self, members: Set[bytes]) -> bytes:
        if not members:
            return b'\x00' * 32
        sorted_members = sorted(members)
        data = b''.join(sorted_members)
        return sha256(data).digest()

    def _update_pool_root(self):
        commitments = [d.commitment for d in self.deposits]
        self.pool_root = self._compute_set_root(set(commitments))

    def _generate_pool_membership_proof(self, deposit: Deposit) -> bytes:
        # Placeholder para Merkle proof real
        return sha256(deposit.commitment + self.pool_root).digest()

    def _generate_set_membership_proof(self, deposit: Deposit, aset: Associati
        return sha256(deposit.commitment + aset.root).digest()

    def get_stats(self) -> Dict:
        """Obtener estad sticas del sistema."""
        return {
            'total_deposits': len(self.deposits),
            'total_nullifiers_spent': len(self.nullifiers),
            'association_sets': len(self.association_sets),
            'known_hack_addresses': len(self.known_hacks),
            'sanctioned_addresses': len(self.sanctioned),
            'kyc_verified_addresses': len(self.kyc_verified)
        }


def privacy_pools_demo():
    print("=== Privacy Pools Demo ===\n")
```

```
system = PrivacyPoolSystem()

# Registrar providers
system.register_provider("Chainalysis")
system.register_provider("CommunityDAO")

# Simular algunas direcciones KYC'd
system.verify_kyc("0xAlice")
system.verify_kyc("0xBob")
system.verify_kyc("0xCarol")

# Simular hack conocido
system.report_hack("0xHacker", "Exchange X hack")

# Simular sanci n
system.add_sanction("0xSanctioned")

# Dep sitos
print("— Making Deposits —")

# Usuario leg timo con KYC
secret1, null1 = secrets.token_bytes(32), secrets.token_bytes(32)
deposit1 = system.deposit(1000000, "0xAlice", secret1, null1)
print(f"Deposit 1 (Alice, KYC): {deposit1.commitment.hex()[:16]}...")

# Usuario sin KYC pero leg timo
secret2, null2 = secrets.token_bytes(32), secrets.token_bytes(32)
deposit2 = system.deposit(500000, "0xDave", secret2, null2)
print(f"Deposit 2 (Dave, no KYC): {deposit2.commitment.hex()[:16]}...")

# Hacker
secret3, null3 = secrets.token_bytes(32), secrets.token_bytes(32)
deposit3 = system.deposit(2000000, "0xHacker", secret3, null3)
print(f"Deposit 3 (Hacker): {deposit3.commitment.hex()[:16]}...")

# Crear association sets
print("\n— Creating Association Sets —")

set1 = system.create_association_set(
    "Chainalysis",
    "clean_funds",
    "Excludes known hacks and sanctions",
    "exclude_all_bad"
)
print(f"Set 'clean_funds': {len(set1.members)} deposits")

set2 = system.create_association_set(
    "CommunityDAO",
    "kyc_only",
    "Only KYC verified addresses",
```

```
        "only_kyc"
    )
    print(f"Set 'kyc_only': {len(set2.members)} deposits")

    # Withdrawals
    print("\n— Withdrawals —")

    # Alice puede retirar de cualquier set
    print("\nAlice withdraws from 'clean_funds':")
    result1 = system.withdraw(deposit1, secret1, null1, "clean_funds")
    print(f"  Success: {result1['success']}")
    print(f"  Verified claim: {result1.get('verified_claim')}")

    # Alice tambi n podr a usar 'kyc_only'
    print("\nAlice could also use 'kyc_only' set")

    # Dave solo puede usar 'clean_funds' (no tiene KYC)
    print("\nDave withdraws from 'clean_funds':")
    result2 = system.withdraw(deposit2, secret2, null2, "clean_funds")
    print(f"  Success: {result2['success']}")

    print("\nDave tries 'kyc_only':")
    result3 = system.withdraw(deposit2, secrets.token_bytes(32), secrets.token
    # Necesita nuevo nullifier porque ya us  el anterior
    deposit2_new = system.deposit(100000, "0xDave", secrets.token_bytes(32), s
    result3 = system.withdraw(deposit2_new, secrets.token_bytes(32), secrets.t
    print(f"  Success: {result3['success']}")
    print(f"  Error: {result3.get('error')}")

    # Hacker no puede usar ning n set
    print("\nHacker tries 'clean_funds':")
    result4 = system.withdraw(deposit3, secret3, null3, "clean_funds")
    print(f"  Success: {result4['success']}")
    print(f"  Error: {result4.get('error')}")

    # Stats
    print("\n— System Stats —")
    stats = system.get_stats()
    for k, v in stats.items():
        print(f"  {k}: {v}")

privacy_pools_demo()
```

2.7.3. Trade-offs de Privacy Pools

Aplicación en Banco ZK

Privacy Pools en Banco ZK:
Banco ZK implementa Privacy Pools como sistema central:

1. **Association Set Providers aprobados:**

Association Set	Ventajas	Desventajas
"All deposits"	Máxima privacidad (mayor anonymity set)	Sin exclusión de malos actores
"Exclude hacks"	Balance razonable	Requiere identificación de hacks
"Only KYC"	Máximo compliance	Menor anonymity set, barrera entrada
"Only regulated"	Aceptable por reguladores	Muy restrictivo

Tabla 2.3: Trade-offs de diferentes association sets

- Chainalysis (exclusión de hacks)
- TRM Labs (sanctions screening)
- Community DAO (governance por usuarios)

2. **Sets disponibles:**
   - `banco_zk_standard`: Excluye hacks y sanciones
   - `banco_zk_kyc`: Solo usuarios con KYC
   - `banco_zk_defi`: Para integración DeFi
3. **UX simplificado:**
   - Wallet sugiere mejor set según contexto
   - Regulador puede requerir set específico
   - Usuario siempre ve opciones disponibles

2.8. Opt-in Transparency

2.8.1. Concepto Fundamental

Definición 2.11 (Opt-in Transparency)**.** Modelo donde la privacidad es el estado predeterminado, pero el usuario puede elegir voluntariamente revelar información cuando le beneficia:

- **Default:** Todas las transacciones son privadas
- **Opt-in:** Usuario decide qué, cuándo y a quién revelar
- **Beneficio:** Transparencia solo cuando aporta valor
- **Control:** Usuario mantiene soberanía sobre sus datos

Analogía con la vida real:
En el mundo físico:

- Por defecto, nadie sabe cuánto dinero tienes
- Pero puedes mostrar tu extracto bancario al solicitar hipoteca
- O revelar tu historial crediticio para mejor tasa de interés
- La revelación es siempre tu elección

En crypto con opt-in transparency:

- Por defecto, transacciones privadas
- Puedes generar proof de solvencia para contraparte
- O revelar historial limpio para obtener servicios
- Siempre mantienes control

2.8.2. Casos de Uso

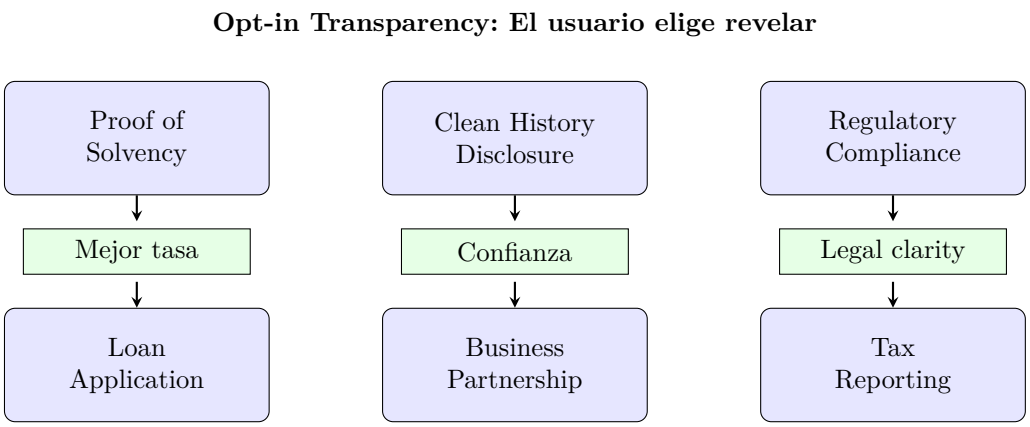


Figura 2.8: Casos de uso de opt-in transparency

[Tipos de Revelación Opt-in] **1. Proof of Reserves/Solvency:**

- Probar: "Tengo al menos X fondos"
- Sin revelar: Monto exacto, fuentes, otras cuentas
- Uso: Colateral para préstamos, confianza de contraparte

2. Transaction History Disclosure:

- Probar: "Todas mis txs en período P fueron legítimas"
- Sin revelar: Contrapartes específicas, otros períodos
- Uso: Due diligence para socios, auditorías

3. Source of Funds:

- Probar: "Mis fondos vienen de fuentes legítimas"
- Sin revelar: Cuál fuente específica
- Uso: Exchanges, servicios regulados

4. Compliance Attestation:

- Probar: "Cumplo con requisito X"
- Sin revelar: Datos subyacentes
- Uso: Requisitos regulatorios específicos

5. Selective Identity Reveal:

- Probar: "Soy la persona que hizo tx Y"
- Con propósito específico
- Uso: Reclamar derechos, disputas

2.8.3. Implementación

Listing 2.9: Sistema de Opt-in Transparency

```
from dataclasses import dataclass
from datetime import datetime, date
from typing import Dict, List, Optional, Set
from hashlib import sha256
from enum import Enum
import secrets

class DisclosureType(Enum):
    SOLVENCY = "solvency"
    HISTORY = "history"
    SOURCE = "source"
    COMPLIANCE = "compliance"
    IDENTITY = "identity"

@dataclass
class DisclosureRequest:
    request_id: str
    requester: str
    disclosure_type: DisclosureType
    parameters: Dict
    expires_at: datetime
    purpose: str

@dataclass
class DisclosureProof:
    request_id: str
    disclosure_type: DisclosureType
    claims: Dict
    zk_proof: bytes
    created_at: datetime
    valid_until: datetime

class OptInTransparencySystem:
    """
␣␣␣␣Sistema␣de␣transparencia␣opt-in␣para␣privacidad␣selectiva.
␣␣␣␣"""

    def __init__(self):
        # Usuario data (privado por defecto)
        self.balances: Dict[str, int] = {}
        self.transactions: List[Dict] = []
        self.kyc_data: Dict[str, Dict] = {}

        # Disclosure tracking
        self.disclosure_requests: Dict[str, DisclosureRequest] = {}
        self.issued_proofs: List[DisclosureProof] = []

        # Revocaciones
```

```
        self.revoked_proofs: Set[str] = set()

    def set_user_data(
        self,
        user_id: str,
        balance: int,
        transactions: List[Dict],
        kyc: Dict
    ):
        """Configurar datos de usuario (privados)."""
        self.balances[user_id] = balance
        self.transactions = transactions
        self.kyc_data[user_id] = kyc

    # ========== DISCLOSURE REQUESTS ==========

    def create_disclosure_request(
        self,
        requester: str,
        disclosure_type: DisclosureType,
        parameters: Dict,
        purpose: str,
        valid_hours: int = 24
    ) -> DisclosureRequest:
        """
        Crear solicitud de disclosure.
        El usuario debe aprobarla explícitamente.
        """
        request = DisclosureRequest(
            request_id=secrets.token_hex(16),
            requester=requester,
            disclosure_type=disclosure_type,
            parameters=parameters,
            expires_at=datetime.now() + timedelta(hours=valid_hours),
            purpose=purpose
        )

        self.disclosure_requests[request.request_id] = request
        return request

    # ========== USER APPROVES DISCLOSURE ==========

    def approve_solvency_disclosure(
        self,
        user_id: str,
        request_id: str,
        minimum_balance: int
    ) -> Optional[DisclosureProof]:
        """
        Usuario aprueba disclosure de solvencia.
        Solo revela: "tengo al menos X"
```

```
        """
        request = self.disclosure_requests.get(request_id)
        if not request or datetime.now() > request.expires_at:
            return None

        actual_balance = self.balances.get(user_id, 0)

        # Verificar que cumple
        if actual_balance < minimum_balance:
            return None  # No puede probar lo que no tiene

        # Generar proof
        claims = {
            'minimum_balance': minimum_balance,
            'balance_sufficient': True,
            # NO incluye: balance_actual
        }

        proof = self._generate_zk_proof(
            'solvency',
            {'balance': actual_balance, 'threshold': minimum_balance}
        )

        disclosure = DisclosureProof(
            request_id=request_id,
            disclosure_type=DisclosureType.SOLVENCY,
            claims=claims,
            zk_proof=proof,
            created_at=datetime.now(),
            valid_until=request.expires_at
        )

        self.issued_proofs.append(disclosure)
        return disclosure

    def approve_history_disclosure(
        self,
        user_id: str,
        request_id: str,
        period_start: date,
        period_end: date,
        required_properties: List[str]
    ) -> Optional[DisclosureProof]:
        """
        Usuario aprueba disclosure de historial.
        Revela propiedades agregadas del periodo.
        """
        request = self.disclosure_requests.get(request_id)
        if not request:
            return None
```

```
        # Filtrar transacciones del periodo
        period_txs = [
            tx for tx in self.transactions
            if period_start <= tx['date'] <= period_end
        ]

        claims = {}

        if 'total_volume' in required_properties:
            claims['total_volume'] = sum(tx['amount'] for tx in period_txs)

        if 'transaction_count' in required_properties:
            claims['transaction_count'] = len(period_txs)

        if 'no_suspicious' in required_properties:
            # Verificar que ninguna tx est  marcada como sospechosa
            suspicious = [tx for tx in period_txs if tx.get('suspicious', Fals
            claims['no_suspicious'] = len(suspicious) == 0

        if 'no_sanctioned' in required_properties:
            # Verificar ninguna contraparte sancionada
            claims['no_sanctioned'] = True  # Placeholder

        proof = self._generate_zk_proof('history', claims)

        disclosure = DisclosureProof(
            request_id=request_id,
            disclosure_type=DisclosureType.HISTORY,
            claims=claims,
            zk_proof=proof,
            created_at=datetime.now(),
            valid_until=request.expires_at
        )

        self.issued_proofs.append(disclosure)
        return disclosure

    def approve_source_disclosure(
        self,
        user_id: str,
        request_id: str,
        amount_to_verify: int
    ) -> Optional[DisclosureProof]:
        """
        Usuario aprueba disclosure de fuente de fondos.
        Revela: "estos fondos vienen de fuentes leg timas"
        """
        request = self.disclosure_requests.get(request_id)
        if not request:
            return None
```

```
        # Verificar que puede demostrar origen leg timo
        # (En producci n: an lisis real de procedencia)

        claims = {
            'amount_verified': amount_to_verify,
            'source_legitimate': True,
            'source_categories': ['regulated_exchange', 'employment'],
            # NO incluye: cu l exchange espec fico, empleador, etc.
        }

        proof = self._generate_zk_proof('source', claims)

        disclosure = DisclosureProof(
            request_id=request_id,
            disclosure_type=DisclosureType.SOURCE,
            claims=claims,
            zk_proof=proof,
            created_at=datetime.now(),
            valid_until=request.expires_at
        )

        self.issued_proofs.append(disclosure)
        return disclosure

    def approve_compliance_disclosure(
        self,
        user_id: str,
        request_id: str,
        compliance_requirements: List[str]
    ) -> Optional[DisclosureProof]:
        """
␣␣␣␣␣␣␣␣Usuario␣aprueba␣disclosure␣de␣compliance.
␣␣␣␣␣␣␣␣"""
        request = self.disclosure_requests.get(request_id)
        if not request:
            return None

        kyc = self.kyc_data.get(user_id, {})
        claims = {}

        for req in compliance_requirements:
            if req == 'kyc_verified':
                claims['kyc_verified'] = kyc.get('verified', False)
            elif req == 'not_pep':
                claims['not_pep'] = not kyc.get('is_pep', False)
            elif req == 'not_sanctioned':
                claims['not_sanctioned'] = not kyc.get('is_sanctioned', False)
            elif req == 'country_allowed':
                allowed_countries = request.parameters.get('allowed_countries',
                claims['country_allowed'] = kyc.get('country') in allowed_countr
            elif req == 'age_verified':
```

```
                claims['age_verified'] = kyc.get('age_verified', False)

        proof = self._generate_zk_proof('compliance', claims)

        disclosure = DisclosureProof(
            request_id=request_id,
            disclosure_type=DisclosureType.COMPLIANCE,
            claims=claims,
            zk_proof=proof,
            created_at=datetime.now(),
            valid_until=request.expires_at
        )

        self.issued_proofs.append(disclosure)
        return disclosure

    # ========== VERIFICATION ==========

    def verify_disclosure(self, proof: DisclosureProof) -> Dict:
        """
␣␣␣␣␣␣␣␣Verificar␣disclosure␣proof.
␣␣␣␣␣␣␣␣"""
        # Verificar no revocado
        if proof.request_id in self.revoked_proofs:
            return {'valid': False, 'error': 'Proof␣revoked'}

        # Verificar no expirado
        if datetime.now() > proof.valid_until:
            return {'valid': False, 'error': 'Proof␣expired'}

        # Verificar ZK proof
        zk_valid = self._verify_zk_proof(proof.zk_proof)

        if not zk_valid:
            return {'valid': False, 'error': 'Invalid␣ZK␣proof'}

        return {
            'valid': True,
            'disclosure_type': proof.disclosure_type.value,
            'claims': proof.claims,
            'issued_at': proof.created_at.isoformat(),
            'valid_until': proof.valid_until.isoformat()
        }

    # ========== REVOCATION ==========

    def revoke_disclosure(self, user_id: str, request_id: str) -> bool:
        """
␣␣␣␣␣␣␣␣Usuario␣revoca␣un␣disclosure␣previamente␣emitido.
␣␣␣␣␣␣␣␣"""
        # Verificar que el proof existe y pertenece al usuario
```

```
        proof = next(
            (p for p in self.issued_proofs if p.request_id == request_id),
            None
        )

        if not proof:
            return False

        self.revoked_proofs.add(request_id)
        return True

    # ========== HELPERS ==========

    def _generate_zk_proof(self, proof_type: str, data: Dict) -> bytes:
        """Generar ZK proof (placeholder)."""
        data_str = f"{proof_type}:{str(sorted(data.items()))}"
        return sha256(data_str.encode()).digest()

    def _verify_zk_proof(self, proof: bytes) -> bool:
        """Verificar ZK proof."""
        return len(proof) == 32


from datetime import timedelta

def opt_in_demo():
    print("=== Opt-in Transparency Demo ===\n")

    system = OptInTransparencySystem()

    # Configurar datos de usuario (PRIVADOS)
    system.set_user_data(
        user_id="alice",
        balance=1000000,  # 1M
        transactions=[
            {'date': date(2024, 1, 15), 'amount': 50000, 'type': 'salary'},
            {'date': date(2024, 2, 15), 'amount': 50000, 'type': 'salary'},
            {'date': date(2024, 3, 1), 'amount': -10000, 'type': 'payment'},
        ],
        kyc={
            'verified': True,
            'country': 'US',
            'is_pep': False,
            'is_sanctioned': False,
            'age_verified': True
        }
    )

    print("Alice's private data configured")
    print("(Balance, transactions, KYC - all private by default)\n")
```

```
# ========== ESCENARIO 1: Pr stamo ==========
print("── Scenario 1: Loan Application ──")

loan_request = system.create_disclosure_request(
    requester="DeFi_Lending_Protocol",
    disclosure_type=DisclosureType.SOLVENCY,
    parameters={'minimum_required': 100000},
    purpose="Collateral verification for loan"
)
print(f"Loan protocol requests proof of minimum 100,000")

# Alice aprueba
solvency_proof = system.approve_solvency_disclosure(
    user_id="alice",
    request_id=loan_request.request_id,
    minimum_balance=100000
)

print(f"\nAlice approves disclosure:")
print(f"  Revealed: balance >= 100,000: {solvency_proof.claims['balance_s
print(f"  NOT revealed: actual balance (1,000,000)")

# Verificar
verification = system.verify_disclosure(solvency_proof)
print(f"\nVerification result: {verification['valid']}")

# ========== ESCENARIO 2: Partnership ==========
print("\n── Scenario 2: Business Partnership ──")

partner_request = system.create_disclosure_request(
    requester="Potential_Partner_Inc",
    disclosure_type=DisclosureType.HISTORY,
    parameters={},
    purpose="Due diligence for partnership"
)

history_proof = system.approve_history_disclosure(
    user_id="alice",
    request_id=partner_request.request_id,
    period_start=date(2024, 1, 1),
    period_end=date(2024, 3, 31),
    required_properties=['transaction_count', 'no_suspicious', 'no_sanctio
)

print(f"Partner requests transaction history disclosure")
print(f"\nAlice approves:")
print(f"  Transaction count: {history_proof.claims.get('transaction_count'
print(f"  No suspicious activity: {history_proof.claims.get('no_suspicious
print(f"  No sanctioned parties: {history_proof.claims.get('no_sanctioned'
print(f"  NOT revealed: specific transactions, counterparties, amounts")
```

```
    # ========== ESCENARIO 3: Exchange ==========
    print("\n── Scenario 3: Exchange Compliance ──")

    exchange_request = system.create_disclosure_request(
        requester="Regulated_Exchange",
        disclosure_type=DisclosureType.COMPLIANCE,
        parameters={'allowed_countries': ['US', 'UK', 'EU']},
        purpose="Regulatory compliance check"
    )

    compliance_proof = system.approve_compliance_disclosure(
        user_id="alice",
        request_id=exchange_request.request_id,
        compliance_requirements=['kyc_verified', 'not_pep', 'not_sanctioned', 'c
    )

    print(f"Exchange requests compliance verification")
    print(f"\nAlice approves:")
    for claim, value in compliance_proof.claims.items():
        print(f"  {claim}: {value}")
    print(f"  NOT revealed: name, address, ID number, date of birth")

    # ========== REVOCACIN ==========
    print("\n── Scenario 4: Revocation ──")

    revoked = system.revoke_disclosure("alice", loan_request.request_id)
    print(f"Alice revokes solvency proof: {revoked}")

    # Intentar verificar despu s de revocaci n
    reverify = system.verify_disclosure(solvency_proof)
    print(f"Verification after revocation: {reverify}")

opt_in_demo()
```

2.8.4. Principios de Diseño

Idea Clave

Principios de Opt-in Transparency:

1. **Privacy by Default:**
   - Sin acción del usuario, todo es privado
   - La transparencia requiere consentimiento explícito
   - Nunca revelar más de lo solicitado
2. **Granular Control:**
   - Usuario decide qué, cuándo, a quién, por cuánto tiempo
   - Diferentes niveles de revelación disponibles
   - Revocación posible en cualquier momento

3. **Purpose Binding:**
   - Disclosure vinculado a propósito específico
   - No reutilizable para otros fines
   - Auditable quién solicitó qué
4. **Minimal Disclosure:**
   - Revelar solo lo mínimo necesario
   - Preferir proofs sobre datos raw
   - Agregar cuando sea posible

Resumen del Bloque

Idea Clave

Puntos clave de Compliance vs Privacidad (Bloque 15):
1. Proof-of-Innocence:

- Probar que fondos no son ilícitos sin revelar origen
- Tipos: temporal, non-inclusion, inclusion, category
- Circuitos ZK para cada tipo de proof

2. Selective De-anonymization:

- Threshold decryption: k de n autoridades
- Timelock encryption: revelación temporal
- Court-order escrow: acceso judicial

3. Viewing Keys:

- Jerarquía: spending >viewing >audit
- Diferentes capacidades por tipo de clave
- Reportes verificables sin revelar todo

4. Regulatory-Compliant Mixers:

- Sanctions screening en entrada/salida
- KYC opcional con beneficios
- Association sets para proof-of-innocence

5. Travel Rule + ZK:

- Proof de KYC válido sin compartir datos
- Datos encriptados para regulador
- Compliance sin surveillance masiva

6. Chainalysis y Contramedidas:

- Técnicas: clustering, taint, ML
- Contramedidas: mixers, direcciones nuevas, timing
- Trade-off privacidad vs usabilidad

7. Privacy Pools:

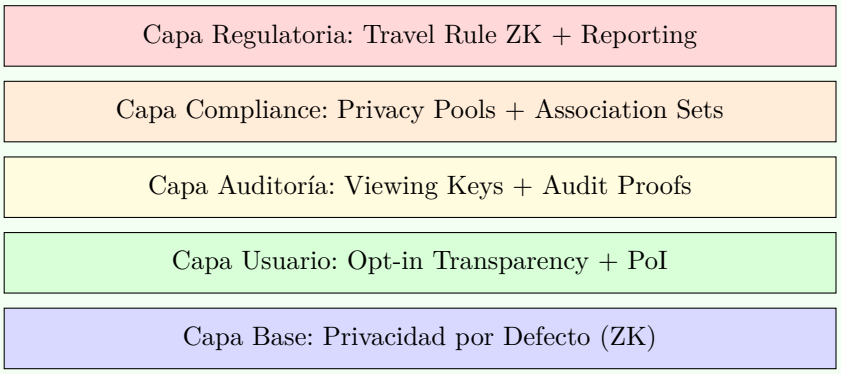
- Association sets definidos por comunidad
- Usuario elige nivel de restricción
- Balance ajustable privacidad/compliance

8. Opt-in Transparency:

- Privacidad por defecto
- Revelación cuando beneficia al usuario
- Control granular y revocable

Aplicación en Banco ZK

Compliance Framework de Banco ZK:



Flujo de compliance:

1. Usuario deposita (sanctions check automático)
2. Opera con privacidad total dentro del pool
3. Al retirar, elige association set apropiado
4. Si necesita, genera proofs de compliance
5. Viewing keys disponibles para auditoría voluntaria

Configuraciones de usuario:

- **Modo Privado:** Sin compliance extra, usa set más amplio
- **Modo Compliant:** KYC + escrow, acceso a pools premium

- **Modo Institucional:** Full viewing keys, reportes automáticos

Métricas objetivo:

Métrica	Target
Sanctions check latency	< 1 segundo
Association set update	Cada 100 bloques
PoI verification time	< 500 ms
Viewing key derivation	Instantáneo
Audit report generation	< 5 segundos

Técnica	Privacidad	Compliance	Complejidad
Proof-of-Innocence	Alta (no revela origen)	Alta (excluye ilícitos)	Media
Threshold De-anon	Media (revela con k/n)	Alta (acceso judicial)	Alta
Viewing Keys	Ajustable	Alta (auditoría)	Baja
Privacy Pools	Ajustable por set	Ajustable por set	Media
Opt-in Transparency	Alta (control usuario)	Media (voluntario)	Baja
ZK Travel Rule	Alta	Alta	Alta

Tabla 2.4: Resumen de técnicas de balance compliance-privacidad

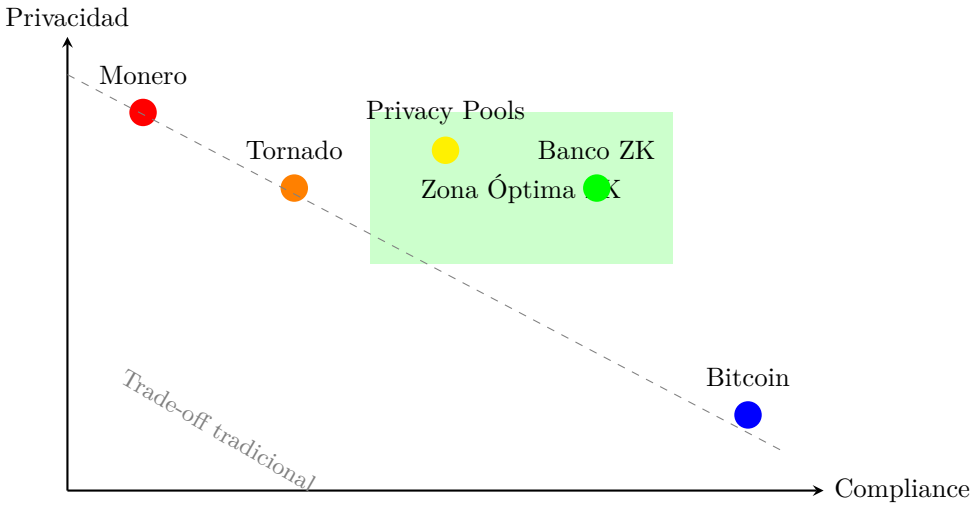


Figura 2.9: ZK permite superar el trade-off tradicional entre privacidad y compliance

Capítulo 3

Metadata y Network Privacy

Bloque 16 de 20

Introducción

Las pruebas de conocimiento cero protegen la privacidad a nivel de transacción, pero la metadata de red puede revelar información crítica sobre los usuarios. Este bloque explora las amenazas a nivel de red y las técnicas para proteger la privacidad más allá del contenido de las transacciones.

Idea Clave

El problema de la metadata:
Protección a nivel transacción (ZK):

- Monto oculto
- Remitente/receptor ocultos
- Origen de fondos no vinculable

Metadata que aún se filtra:

- IP del usuario al enviar tx
- Timing de cuándo se envía
- Patrones de actividad
- Con quién se comunica
- Tamaño de los mensajes

Consecuencia: Un adversario de red puede vincular tu identidad con tus transacciones anónimas observando CÓMO te comunicas, no QUÉ comunicas.

[Modelo de Amenazas de Red] **Adversarios potenciales:**

1. **ISP (Internet Service Provider):**
   - Ve todo tu tráfico de red
   - Conoce tu IP real y ubicación
   - Puede registrar timing de todas las conexiones

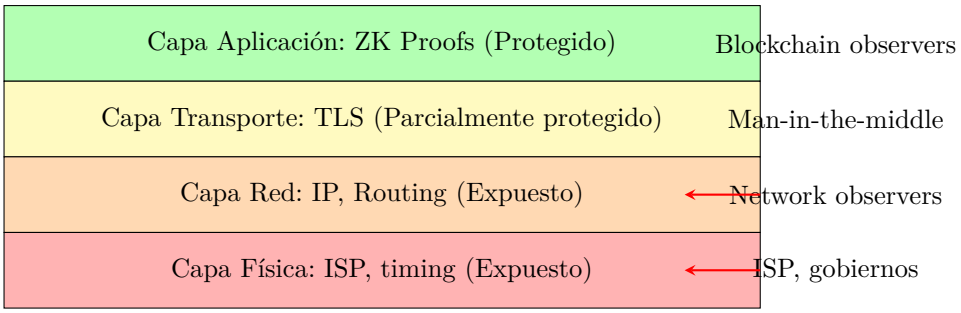


Figura 3.1: Capas de privacidad: ZK protege la aplicación pero la red sigue expuesta

2. **Nodos de la red P2P:**
   - Ven IP de quien envía transacciones
   - Pueden inferir el origen de una tx
   - Posibles nodos maliciosos (sybil)
3. **Proveedores de RPC:**
   - Infura, Alchemy ven todas tus queries
   - Pueden vincular direcciones con IP
   - Conocen tu actividad on-chain
4. **Global Passive Adversary (GPA):**
   - Puede observar todo el tráfico de Internet
   - Correlación de tráfico entre endpoints
   - Agencias de inteligencia nacionales

Capacidades del adversario:

- Observar: Ver tráfico que pasa
- Registrar: Almacenar para análisis futuro
- Correlacionar: Vincular diferentes flujos de tráfico
- Modificar: Inyectar, dropear, o retrasar paquetes

3.1. IP Leakage y Protección

3.1.1. Anatomía del IP Leakage

Definición 3.1 (IP Leakage)**.** Exposición no intencional de la dirección IP real de un usuario durante la interacción con sistemas blockchain, permitiendo vincular la identidad real con actividad on-chain.

Consecuencias:

- Geolocalización del usuario
- Vinculación de múltiples sesiones
- Identificación personal (con cooperación del ISP)
- Correlación de direcciones on-chain con identidad

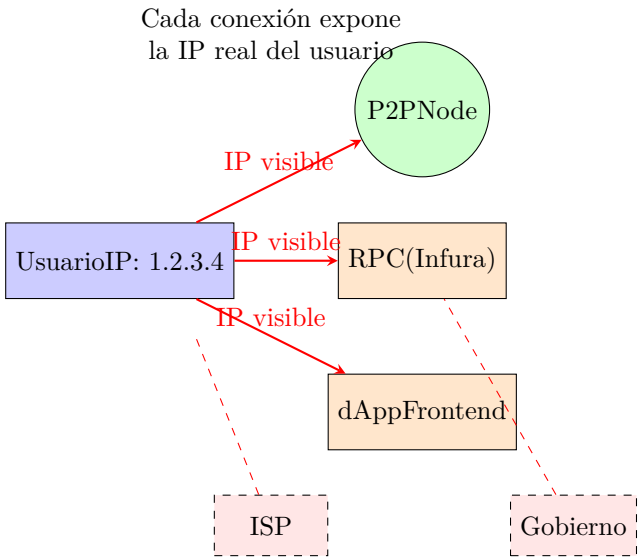


Figura 3.2: Vectores de IP leakage en interacción con blockchain

3.1.2. Vectores de IP Leakage

[Vectores Detallados] **1. Conexión directa a nodos P2P:**

- Al sincronizar blockchain, conectas a múltiples peers
- Cada peer ve tu IP
- Nodos maliciosos pueden registrar IPs de quien envía txs
- "First-seen" heuristic: el primer nodo que ve una tx probablemente es el origen

2. Uso de RPC públicos:

- Infura, Alchemy, QuickNode procesan tus requests
- Ven: IP + queries + direcciones consultadas
- Pueden construir perfil completo de actividad
- Términos de servicio permiten logging

3. Broadcast de transacciones:

- Al enviar tx, el primer nodo conoce tu IP
- Incluso con multiple peers, timing puede revelar origen
- Transacciones "frescas" más fáciles de rastrear

4. Conexión a dApps:

- Frontend web conoce tu IP (logs del servidor)
- Analytics de terceros (Google Analytics, etc.)
- WebRTC puede filtrar IP real incluso con VPN
- Cookies vinculan sesiones

5. DNS Leaks:

- Queries DNS revelan qué dominios visitas
- ISP ve todas las consultas DNS por defecto
- Incluso con VPN, DNS puede filtrar por configuración incorrecta

Listing 3.1: Detector de IP Leakage

```
from dataclasses import dataclass
from datetime import datetime
from typing import List, Dict, Set, Optional
from enum import Enum
import re

class LeakageVector(Enum):
    P2P_CONNECTION = "p2p_connection"
    RPC_REQUEST = "rpc_request"
    DAPP_FRONTEND = "dapp_frontend"
    TX_BROADCAST = "tx_broadcast"
    DNS_QUERY = "dns_query"
    WEBRTC = "webrtc"

@dataclass
class LeakageEvent:
    vector: LeakageVector
    timestamp: datetime
    ip_exposed: str
    destination: str
    metadata: Dict
    risk_level: str  # 'low', 'medium', 'high', 'critical'

class IPLeakageDetector:
    """
    Detector de eventos de IP leakage en interacci n con blockchain.
    """

    def __init__(self, user_real_ip: str):
        self.real_ip = user_real_ip
        self.events: List[LeakageEvent] = []

        # Servicios conocidos y su riesgo
        self.known_rpc_providers = {
            'infura.io': 'high',
            'alchemy.com': 'high',
            'quicknode.com': 'medium',
            'cloudflare-eth.com': 'medium',
        }

        self.known_analytics = {
            'google-analytics.com': 'high',
            'segment.io': 'high',
```

```
            'mixpanel.com': 'medium',
        }

        # Nodos P2P conocidos como honeypots
        self.suspicious_nodes: Set[str] = set()

    def analyze_connection(
        self,
        destination_ip: str,
        destination_domain: Optional[str],
        connection_type: str,
        headers: Optional[Dict] = None
    ) -> Optional[LeakageEvent]:
        """
␣␣␣␣␣␣␣␣Analizar una conexión saliente para detectar leakage.
␣␣␣␣␣␣␣␣"""
        risk_level = 'low'
        vector = None
        metadata = {}

        # Determinar vector
        if connection_type == 'p2p':
            vector = LeakageVector.P2P_CONNECTION

            # Verificar si es nodo sospechoso
            if destination_ip in self.suspicious_nodes:
                risk_level = 'critical'
                metadata['warning'] = 'Known honeypot node'
            else:
                risk_level = 'medium'

        elif connection_type == 'http' and destination_domain:
            # Verificar si es RPC
            for provider, provider_risk in self.known_rpc_providers.items():
                if provider in destination_domain:
                    vector = LeakageVector.RPC_REQUEST
                    risk_level = provider_risk
                    metadata['provider'] = provider
                    break

            # Verificar analytics
            for analytics, analytics_risk in self.known_analytics.items():
                if analytics in destination_domain:
                    vector = LeakageVector.DAPP_FRONTEND
                    risk_level = 'high'
                    metadata['analytics'] = analytics
                    break

            if not vector:
                vector = LeakageVector.DAPP_FRONTEND
                risk_level = 'low'
```

```
        elif connection_type == 'dns':
            vector = LeakageVector.DNS_QUERY
            risk_level = 'medium'
            metadata['query'] = destination_domain

        elif connection_type == 'webrtc':
            vector = LeakageVector.WEBRTC
            risk_level = 'critical'
            metadata['warning'] = 'WebRTC can leak real IP even with VPN'

        if vector:
            event = LeakageEvent(
                vector=vector,
                timestamp=datetime.now(),
                ip_exposed=self.real_ip,
                destination=destination_domain or destination_ip,
                metadata=metadata,
                risk_level=risk_level
            )
            self.events.append(event)
            return event

        return None

    def analyze_tx_broadcast(
        self,
        tx_hash: str,
        connected_peers: List[str],
        broadcast_method: str
    ) -> LeakageEvent:
        """
        Analizar broadcast de transacci n.
        """
        risk_level = 'high'
        metadata = {
            'tx_hash': tx_hash,
            'num_peers': len(connected_peers),
            'method': broadcast_method
        }

        # M todos m s seguros
        if broadcast_method == 'dandelion':
            risk_level = 'medium'
            metadata['protection'] = 'Dandelion routing'
        elif broadcast_method == 'tor':
            risk_level = 'low'
            metadata['protection'] = 'Tor network'
        elif broadcast_method == 'direct':
            risk_level = 'critical'
            metadata['warning'] = 'Direct broadcast exposes origin'
```

```
        # Verificar honeypots en peers
        honeypot_count = sum(1 for p in connected_peers if p in self.suspicious_
        if honeypot_count > 0:
            risk_level = 'critical'
            metadata['honeypots_detected'] = honeypot_count

        event = LeakageEvent(
            vector=LeakageVector.TX_BROADCAST,
            timestamp=datetime.now(),
            ip_exposed=self.real_ip,
            destination=f"{len(connected_peers)} peers",
            metadata=metadata,
            risk_level=risk_level
        )

        self.events.append(event)
        return event

    def get_risk_summary(self) -> Dict:
        """
        Obtener resumen de riesgos detectados.
        """
        risk_counts = {'low': 0, 'medium': 0, 'high': 0, 'critical': 0}
        vector_counts = {}

        for event in self.events:
            risk_counts[event.risk_level] += 1
            vector_name = event.vector.value
            vector_counts[vector_name] = vector_counts.get(vector_name, 0) + 1

        overall_risk = 'low'
        if risk_counts['critical'] > 0:
            overall_risk = 'critical'
        elif risk_counts['high'] > 0:
            overall_risk = 'high'
        elif risk_counts['medium'] > 0:
            overall_risk = 'medium'

        return {
            'overall_risk': overall_risk,
            'total_events': len(self.events),
            'risk_breakdown': risk_counts,
            'vectors': vector_counts,
            'recommendations': self._get_recommendations(risk_counts)
        }

    def _get_recommendations(self, risk_counts: Dict) -> List[str]:
        """Generar recomendaciones basadas en riesgos."""
        recommendations = []
```

```
        if risk_counts['critical'] > 0:
            recommendations.append("URGENT: Use Tor for all blockchain interac
            recommendations.append("Disable WebRTC in browser")

        if risk_counts['high'] > 0:
            recommendations.append("Consider running your own RPC node")
            recommendations.append("Use privacy-focused RPC providers")

        if any(e.vector == LeakageVector.TX_BROADCAST and e.risk_level != 'low
               for e in self.events):
            recommendations.append("Enable Dandelion++ for transaction broadca

        if any(e.vector == LeakageVector.DNS_QUERY for e in self.events):
            recommendations.append("Use encrypted DNS (DoH/DoT)")

        return recommendations


def ip_leakage_demo():
    print("=== IP Leakage Detection Demo ===\n")

    detector = IPLeakageDetector(user_real_ip="192.168.1.100")

    # Simular actividad t pica de usuario
    print("--- Simulating user activity ---\n")

    # Conexi n a RPC
    event1 = detector.analyze_connection(
        destination_ip="13.56.163.26",
        destination_domain="mainnet.infura.io",
        connection_type="http"
    )
    print(f"RPC Connection: {event1.risk_level} risk")

    # Conexi n P2P
    event2 = detector.analyze_connection(
        destination_ip="45.76.33.21",
        destination_domain=None,
        connection_type="p2p"
    )
    print(f"P2P Connection: {event2.risk_level} risk")

    # dApp con analytics
    event3 = detector.analyze_connection(
        destination_ip="142.250.190.46",
        destination_domain="www.google-analytics.com",
        connection_type="http"
    )
    print(f"Analytics: {event3.risk_level} risk")

    # Broadcast de tx
```

```
    event4 = detector.analyze_tx_broadcast(
        tx_hash="0xabc123...",
        connected_peers=["45.76.33.21", "185.199.108.153", "140.82.114.3"],
        broadcast_method="direct"
    )
    print(f"TX Broadcast: {event4.risk_level} risk")

    # Resumen
    print("\n--- Risk Summary ---")
    summary = detector.get_risk_summary()
    print(f"Overall Risk: {summary['overall_risk'].upper()}")
    print(f"Total Events: {summary['total_events']}")
    print(f"Risk Breakdown: {summary['risk_breakdown']}")

    print("\n--- Recommendations ---")
    for rec in summary['recommendations']:
        print(f"  {rec}")

ip_leakage_demo()
```

3.1.3. Soluciones de Protección

VPN (Virtual Private Network)

Definición 3.2 (VPN)**.** Red privada virtual que encripta el tráfico y lo enruta a través de un servidor intermedio, ocultando la IP real del usuario.

Modelo de seguridad:

- Tráfico encriptado entre usuario y servidor VPN
- Destino ve IP del servidor VPN, no del usuario
- Requiere confiar en el proveedor de VPN

Limitaciones de VPN:

1. **Confianza centralizada:** El proveedor VPN ve TODO tu tráfico
2. **Logging:** Muchos proveedores mantienen logs (aunque digan que no)
3. **Jurisdicción:** Pueden ser forzados a entregar datos
4. **Single point of failure:** Si VPN se compromete, todo se compromete
5. **DNS leaks:** Configuración incorrecta puede filtrar DNS
6. **WebRTC leaks:** Navegadores pueden filtrar IP real

Para privacidad blockchain: VPN es insuficiente como única protección.

Tor (The Onion Router)

Definición 3.3 (Tor)**.** Red de anonimato que enruta tráfico a través de múltiples nodos (relays), aplicando capas de encriptación:

Arquitectura:

1. **Guard node:** Primer nodo, conoce IP real pero no destino

2. **Middle node:** Nodo intermedio, no conoce origen ni destino
3. **Exit node:** Último nodo, conoce destino pero no origen

Propiedades:

- Ningún nodo conoce tanto origen como destino
- Requiere compromiso de 3 nodos para desanonimizar
- Código abierto y auditado

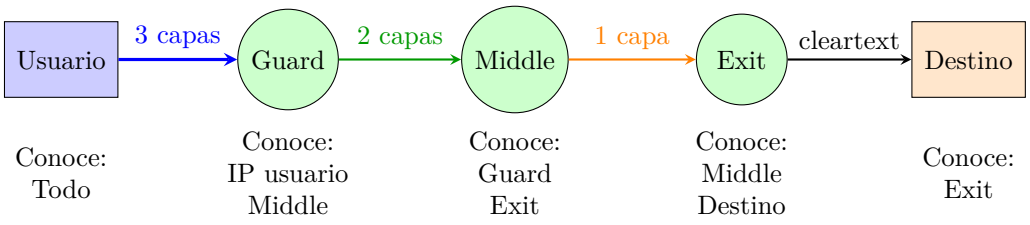


Figura 3.3: Onion routing en Tor: cada relay solo conoce el anterior y siguiente

I2P (Invisible Internet Project)

Definición 3.4 (I2P)**.** Red overlay diseñada para comunicación anónima interna (servicios ocultos):

Diferencias con Tor:

- Optimizado para servicios internos, no exit a clearnet
- Usa "garlic routing" (múltiples mensajes empaquetados)
- Todos los usuarios son routers (más descentralizado)
- Conexiones unidireccionales (túneles separados in/out)

Uso en blockchain:

- Monero integra I2P (Kovri)
- Mejor para comunicación P2P entre nodos
- Menos adecuado para acceder a servicios web externos

Nodo Propio

Idea Clave

Ejecutar nodo propio:
La forma más directa de evitar IP leakage a terceros es ejecutar tu propio nodo:

- **Full node:** Verifica todas las transacciones localmente
- **Sin RPC externos:** No dependes de Infura/Alchemy
- **Broadcast privado:** Puedes combinar con Tor
- **Sincronización:** Requiere ancho de banda y almacenamiento

Configuración recomendada:

1. Full node detrás de Tor
2. Conexiones P2P solo a través de .onion
3. Broadcast de txs con delay aleatorio
4. Sin conexiones entrantes (solo outbound)

3.2. Timing Correlation Attacks

3.2.1. Anatomía del Ataque

Definición 3.5 (Timing Correlation Attack)**.** Ataque que vincula eventos aparentemente no relacionados basándose en la correlación temporal entre ellos.

Principio: Si el evento A y el evento B ocurren consistentemente juntos en el tiempo, probablemente están relacionados.

Formalización:

$$P(\text{vinculados}|\Delta t < \epsilon) \check{} P(\text{vinculados}|\Delta t > \epsilon)$$

donde Δt es la diferencia temporal entre eventos.

[Tipos de Timing Correlation] **1. Deposit-Withdraw Correlation:**

- Usuario deposita en mixer, retira poco después
- Correlación temporal revela vínculo
- Ejemplo: Deposita 1 ETH a las 14:00, retira 1 ETH a las 14:05

2. Login-Transaction Correlation:

- Usuario entra a exchange, segundos después aparece tx
- El exchange puede vincular la tx con el usuario
- Incluso si la tx viene de dirección "anónima"

3. Behavioral Patterns:

- Usuario siempre activo a las 9am (zona horaria)
- Patrones de fin de semana diferentes
- Correlación con eventos del mundo real

4. Cross-Service Correlation:

- Actividad en servicio A correlaciona con servicio B
- Mismo timing = mismo usuario
- Aunque use diferentes direcciones/identidades

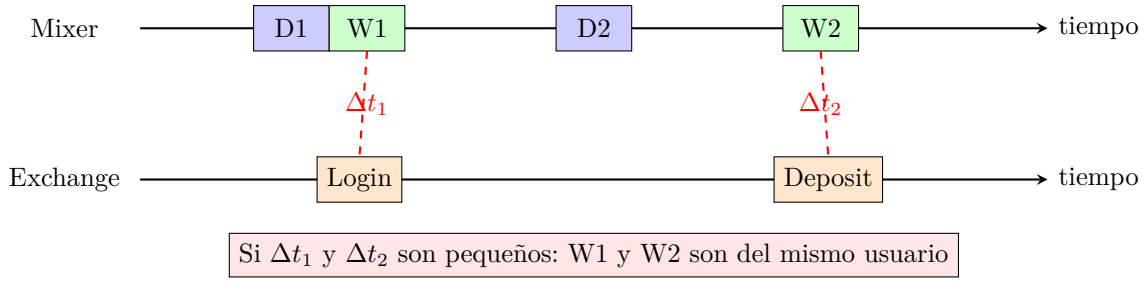


Figura 3.4: Timing correlation: pequeños Δt revelan vinculación

Listing 3.2: Sistema de análisis y defensa contra timing correlation

```
from dataclasses import dataclass
from datetime import datetime, timedelta
from typing import List, Dict, Tuple, Optional
import random
import math
from collections import defaultdict

@dataclass
class Event:
    event_id: str
    event_type: str
    timestamp: datetime
    metadata: Dict

class TimingCorrelationAnalyzer:
    """
␣␣␣␣Analizador␣de␣correlaciones␣temporales␣para␣detectar␣vinculaciones.
␣␣␣␣"""

    def __init__(self, correlation_threshold_seconds: float = 60):
        self.threshold = correlation_threshold_seconds
        self.events: List[Event] = []

    def add_event(self, event: Event):
        """A adir␣evento␣para␣an lisis."""
        self.events.append(event)

    def find_correlated_pairs(
        self,
        event_type_a: str,
        event_type_b: str,
        max_delta_seconds: float = None
    ) -> List[Tuple[Event, Event, float]]:
        """
␣␣␣␣␣␣␣␣Encontrar␣pares␣de␣eventos␣correlacionados␣temporalmente.
␣␣␣␣␣␣␣␣"""
        max_delta = max_delta_seconds or self.threshold

        events_a = [e for e in self.events if e.event_type == event_type_a]
```

```
        events_b = [e for e in self.events if e.event_type == event_type_b]

        correlated = []

        for ea in events_a:
            for eb in events_b:
                delta = abs((eb.timestamp - ea.timestamp).total_seconds())
                if delta <= max_delta:
                    correlated.append((ea, eb, delta))

        # Ordenar por delta (m s correlacionados primero)
        correlated.sort(key=lambda x: x[2])

        return correlated

    def calculate_correlation_score(
        self,
        events_a: List[Event],
        events_b: List[Event],
        window_seconds: float = 300
    ) -> float:
        """
        Calcular score de correlaci n entre dos series de eventos.
        Score alto = probable vinculaci n.
        """
        if not events_a or not events_b:
            return 0.0

        # Contar co-ocurrencias en ventanas de tiempo
        co_occurrences = 0

        for ea in events_a:
            for eb in events_b:
                delta = abs((eb.timestamp - ea.timestamp).total_seconds())
                if delta <= window_seconds:
                    # Peso inversamente proporcional al delta
                    weight = 1.0 - (delta / window_seconds)
                    co_occurrences += weight

        # Normalizar por n mero de eventos
        max_possible = min(len(events_a), len(events_b))
        if max_possible == 0:
            return 0.0

        score = co_occurrences / max_possible
        return min(1.0, score)

    def detect_behavioral_patterns(
        self,
        user_events: List[Event],
        min_occurrences: int = 3
```

```
    ) -> Dict:
        """
        Detectar patrones de comportamiento temporal.
        """
        patterns = {
            'hourly': defaultdict(int),
            'daily': defaultdict(int),
            'intervals': []
        }

        # Distribuci n por hora del d a
        for event in user_events:
            hour = event.timestamp.hour
            patterns['hourly'][hour] += 1

        # Distribuci n por d a de semana
        for event in user_events:
            day = event.timestamp.strftime('%A')
            patterns['daily'][day] += 1

        # Intervalos entre eventos consecutivos
        sorted_events = sorted(user_events, key=lambda x: x.timestamp)
        for i in range(1, len(sorted_events)):
            interval = (sorted_events[i].timestamp - sorted_events[i-1].timesta
            patterns['intervals'].append(interval)

        # Detectar patrones significativos
        findings = []

        # Hora m s com n
        if patterns['hourly']:
            peak_hour = max(patterns['hourly'], key=patterns['hourly'].get)
            if patterns['hourly'][peak_hour] >= min_occurrences:
                findings.append(f"Peak activity at hour {peak_hour}")

        # D a m s com n
        if patterns['daily']:
            peak_day = max(patterns['daily'], key=patterns['daily'].get)
            if patterns['daily'][peak_day] >= min_occurrences:
                findings.append(f"Most active on {peak_day}")

        # Intervalo regular
        if patterns['intervals']:
            avg_interval = sum(patterns['intervals']) / len(patterns['interval
            variance = sum((x - avg_interval)**2 for x in patterns['intervals'
            if variance < avg_interval * 0.1:  # Baja varianza = patr n regul
                findings.append(f"Regular interval pattern: ~{avg_interval:.0f

        return {
            'hourly_distribution': dict(patterns['hourly']),
            'daily_distribution': dict(patterns['daily']),
```

```
            'avg_interval': sum(patterns['intervals']) / len(patterns['intervals
            'findings': findings
        }


class TimingDefense:
    """
    T cnicas de defensa contra timing correlation attacks.
    """

    def __init__(self, min_delay: float = 60, max_delay: float = 3600):
        self.min_delay = min_delay
        self.max_delay = max_delay
        self.pending_actions: List[Dict] = []

    def schedule_with_random_delay(
        self,
        action: callable,
        delay_distribution: str = 'exponential'
    ) -> datetime:
        """
        Programar acci n con delay aleatorio.

        Args:
            action: Funci n a ejecutar
            delay_distribution: 'uniform', 'exponential', 'poisson'

        Returns:
            Timestamp programado
        """
        if delay_distribution == 'uniform':
            delay = random.uniform(self.min_delay, self.max_delay)

        elif delay_distribution == 'exponential':
            # Lambda tal que media = (min + max) / 2
            mean = (self.min_delay + self.max_delay) / 2
            delay = random.expovariate(1 / mean)
            delay = max(self.min_delay, min(self.max_delay, delay))

        elif delay_distribution == 'poisson':
            # Similar a exponencial pero discretizado
            mean = (self.min_delay + self.max_delay) / 2
            delay = random.expovariate(1 / mean)
            delay = max(self.min_delay, min(self.max_delay, delay))

        else:
            delay = random.uniform(self.min_delay, self.max_delay)

        scheduled_time = datetime.now() + timedelta(seconds=delay)

        self.pending_actions.append({
```

```
            'action': action,
            'scheduled': scheduled_time,
            'delay': delay
        })

        return scheduled_time

    def batch_actions(
        self,
        actions: List[callable],
        batch_window: float = 3600
    ) -> List[datetime]:
        """
        Agrupar múltiples acciones en una ventana de tiempo.
        Todas se ejecutan en momentos aleatorios dentro de la ventana.
        """
        window_start = datetime.now() + timedelta(seconds=self.min_delay)
        scheduled_times = []

        for action in actions:
            # Tiempo aleatorio dentro de la ventana
            offset = random.uniform(0, batch_window)
            scheduled = window_start + timedelta(seconds=offset)

            self.pending_actions.append({
                'action': action,
                'scheduled': scheduled,
                'batch_id': id(actions)
            })
            scheduled_times.append(scheduled)

        return scheduled_times

    def generate_cover_activity(
        self,
        real_action: callable,
        num_decoys: int = 5,
        time_window: float = 3600
    ) -> List[Dict]:
        """
        Generar actividad de cobertura junto con acción real.
        El adversario no puede distinguir cuál es la real.
        """
        activities = []

        # Programar acción real
        real_time = self.schedule_with_random_delay(real_action, 'exponential'
        activities.append({
            'type': 'real',
            'scheduled': real_time
        })
```

```
        # Generar decoys
        window_start = datetime.now()
        for i in range(num_decoys):
            offset = random.uniform(0, time_window)
            decoy_time = window_start + timedelta(seconds=offset)

            # Decoy action (no-op o actividad falsa)
            def decoy_action():
                pass  # O enviar transacci n falsa a s  mismo

            activities.append({
                'type': 'decoy',
                'scheduled': decoy_time
            })

        # Mezclar para que no se pueda distinguir por orden
        random.shuffle(activities)

        return activities

    def calculate_anonymity_improvement(
        self,
        base_anonymity_set: int,
        delay_seconds: float,
        events_per_hour: float
    ) -> Dict:
        """
        Calcular mejora en anonymity set por delay.
        """
        # Eventos adicionales en el tiempo de delay
        hours = delay_seconds / 3600
        additional_events = events_per_hour * hours

        new_anonymity_set = base_anonymity_set + additional_events
        improvement = (new_anonymity_set / base_anonymity_set - 1) * 100

        return {
            'base_anonymity_set': base_anonymity_set,
            'delay_hours': hours,
            'additional_events': additional_events,
            'new_anonymity_set': new_anonymity_set,
            'improvement_percent': improvement
        }


def timing_correlation_demo():
    print("=== Timing Correlation Demo ===\n")

    analyzer = TimingCorrelationAnalyzer(correlation_threshold_seconds=60)
```

```
# Simular eventos de usuario (mixer + exchange)
base_time = datetime.now()

# Usuario deposita en mixer, luego va al exchange (mal patr n)
events = [
    Event("d1", "mixer_deposit", base_time, {}),
    Event("w1", "mixer_withdraw", base_time + timedelta(seconds=30), {}),
    Event("l1", "exchange_login", base_time + timedelta(seconds=35), {}),

    Event("d2", "mixer_deposit", base_time + timedelta(hours=2), {}),
    Event("w2", "mixer_withdraw", base_time + timedelta(hours=2, seconds=4
    Event("l2", "exchange_login", base_time + timedelta(hours=2, seconds=5
]

for e in events:
    analyzer.add_event(e)

# Detectar correlaciones
print("--- Correlation Analysis ---")
correlated = analyzer.find_correlated_pairs(
    "mixer_withdraw", "exchange_login", max_delta_seconds=30
)

print(f"Found {len(correlated)} correlated pairs:")
for ea, eb, delta in correlated:
    print(f"  {ea.event_id} <-> {eb.event_id}: t ={delta:.1f}s")

# Score de correlaci n
withdraws = [e for e in events if e.event_type == "mixer_withdraw"]
logins = [e for e in events if e.event_type == "exchange_login"]

score = analyzer.calculate_correlation_score(withdraws, logins)
print(f"\nCorrelation score: {score:.2f}")
if score > 0.5:
    print("  WARNING: High correlation suggests same user!")

# Defensa
print("\n--- Timing Defense ---")
defense = TimingDefense(min_delay=300, max_delay=3600)

# Programar retiro con delay
scheduled = defense.schedule_with_random_delay(
    lambda: print("Executing withdraw"),
    delay_distribution='exponential'
)
print(f"Withdraw scheduled at: {scheduled}")
print(f"Delay: {(scheduled - datetime.now()).seconds}s")

# Calcular mejora en anonimato
improvement = defense.calculate_anonymity_improvement(
    base_anonymity_set=100,
```

```
        delay_seconds=1800,  # 30 min
        events_per_hour=20
    )
    print(f"\nAnonymity improvement with 30min delay:")
    print(f"  Base set: {improvement['base_anonymity_set']}")
    print(f"  New set: {improvement['new_anonymity_set']:.0f}")
    print(f"  Improvement: {improvement['improvement_percent']:.1f}%")

timing_correlation_demo()
```

3.2.2. Defensas contra Timing Correlation

Defensa	Descripción	Efectividad	Usabilidad
Random delay	Esperar tiempo aleatorio antes de actuar	Alta	Media
Batching	Agrupar acciones con otras	Alta	Baja
Cover traffic	Generar actividad falsa de cobertura	Muy alta	Baja
Time-locked actions	Acciones que solo se ejecutan en futuro	Alta	Baja
Poisson timing	Intervalos con distribución Poisson	Alta	Alta

Tabla 3.1: Defensas contra timing correlation

3.3. Traffic Analysis

3.3.1. Fundamentos del Análisis de Tráfico

Definición 3.6 (Traffic Analysis)**.** Técnica de inteligencia que infiere información a partir de los patrones del tráfico de red, sin necesidad de descifrar el contenido:

Observables:

- Tamaño de paquetes
- Timing entre paquetes
- Frecuencia de comunicación
- Direcciones de origen/destino
- Volumen de datos transferidos

Principio: Incluso el tráfico encriptado "habla" a través de sus metadatos.

[Modelo de Traffic Analysis] **Flujo de tráfico como señal:**
Un flujo de tráfico puede modelarse como una serie temporal:

$$F = \{(t_1, s_1), (t_2, s_2), ..., (t_n, s_n)\}$$

donde t_i es el timestamp y s_i es el tamaño del paquete i.

Features para análisis:

- **Inter-arrival time:** $\Delta t_i = t_{i+1} - t_i$

- **Distribución de tamaños:** hist$(s_1, ..., s_n)$
- **Burstiness:** Varianza en Δt
- **Total volume:** $V = \sum_i s_i$
- **Direction ratio:** Bytes enviados vs recibidos

Fingerprinting: Cada tipo de actividad genera un "fingerprint" distintivo:

- Video streaming: Alto volumen, paquetes regulares grandes
- Chat: Bajo volumen, paquetes pequeños, bidireccional
- Transacción blockchain: Patrón específico de tamaños

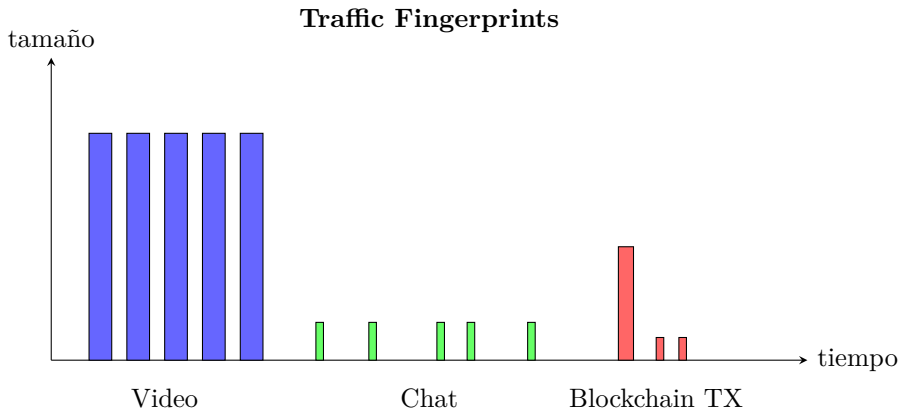


Figura 3.5: Diferentes actividades generan patrones de tráfico distintivos

3.3.2. Ataques de Traffic Analysis

Listing 3.3: Sistema de Traffic Analysis

```
from dataclasses import dataclass
from datetime import datetime, timedelta
from typing import List, Dict, Tuple, Optional
import numpy as np
from collections import defaultdict
import statistics

@dataclass
class Packet:
    timestamp: datetime
    size: int
    direction: str  # 'sent' or 'received'
    encrypted: bool

@dataclass
class TrafficFlow:
    flow_id: str
    source_ip: str
    dest_ip: str
```

```
    packets: List[Packet]
    start_time: datetime
    end_time: datetime

class TrafficAnalyzer:
    """
    Analizador de patrones de tr fico para inferir actividad.
    """

    def __init__(self):
        # Fingerprints conocidos de diferentes actividades
        self.known_fingerprints = {
            'blockchain_tx': {
                'size_range': (200, 2000),
                'packet_count': (3, 10),
                'bidirectional': True,
                'duration_seconds': (1, 30)
            },
            'mixer_deposit': {
                'size_range': (500, 3000),
                'packet_count': (5, 15),
                'bidirectional': True,
                'duration_seconds': (5, 60)
            },
            'rpc_query': {
                'size_range': (100, 500),
                'packet_count': (2, 4),
                'bidirectional': True,
                'duration_seconds': (0.1, 2)
            },
            'p2p_sync': {
                'size_range': (1000, 65000),
                'packet_count': (100, 10000),
                'bidirectional': True,
                'duration_seconds': (60, 3600)
            }
        }

    def extract_features(self, flow: TrafficFlow) -> Dict:
        """
        Extraer caracter sticas de un flujo de tr fico.
        """
        if not flow.packets:
            return {}

        packets = sorted(flow.packets, key=lambda p: p.timestamp)
        sizes = [p.size for p in packets]

        # Inter-arrival times
        iats = []
        for i in range(1, len(packets)):
```

```
            iat = (packets[i].timestamp - packets[i-1].timestamp).total_second
            iats.append(iat)

        # Direcci n de paquetes
        sent = [p for p in packets if p.direction == 'sent']
        received = [p for p in packets if p.direction == 'received']

        features = {
            # Estad sticas de tama o
            'size_mean': statistics.mean(sizes) if sizes else 0,
            'size_std': statistics.stdev(sizes) if len(sizes) > 1 else 0,
            'size_min': min(sizes) if sizes else 0,
            'size_max': max(sizes) if sizes else 0,

            # Estad sticas de timing
            'iat_mean': statistics.mean(iats) if iats else 0,
            'iat_std': statistics.stdev(iats) if len(iats) > 1 else 0,

            # Conteos
            'packet_count': len(packets),
            'sent_count': len(sent),
            'received_count': len(received),

            # Volumen
            'total_bytes': sum(sizes),
            'sent_bytes': sum(p.size for p in sent),
            'received_bytes': sum(p.size for p in received),

            # Duraci n
            'duration': (flow.end_time - flow.start_time).total_seconds(),

            # Ratios
            'direction_ratio': len(sent) / len(received) if received else floa
            'bytes_ratio': sum(p.size for p in sent) / sum(p.size for p in rec
        }

        return features

    def classify_activity(self, flow: TrafficFlow) -> Tuple[str, float]:
        """
        Clasificar tipo de actividad bas ndose en fingerprint.
        """
        features = self.extract_features(flow)

        if not features:
            return ('unknown', 0.0)

        best_match = 'unknown'
        best_score = 0.0

        for activity, fingerprint in self.known_fingerprints.items():
```

```
            score = self._match_fingerprint(features, fingerprint)
            if score > best_score:
                best_score = score
                best_match = activity

        return (best_match, best_score)

    def _match_fingerprint(self, features: Dict, fingerprint: Dict) -> float:
        """
␣␣␣␣␣␣␣␣Calcular␣score␣de␣coincidencia␣con␣fingerprint.
␣␣␣␣␣␣␣␣"""
        score = 0.0
        checks = 0

        # Verificar rango de tama o
        if 'size_range' in fingerprint:
            min_size, max_size = fingerprint['size_range']
            if min_size <= features.get('size_mean', 0) <= max_size:
                score += 1
            checks += 1

        # Verificar conteo de paquetes
        if 'packet_count' in fingerprint:
            min_count, max_count = fingerprint['packet_count']
            if min_count <= features.get('packet_count', 0) <= max_count:
                score += 1
            checks += 1

        # Verificar duraci n
        if 'duration_seconds' in fingerprint:
            min_dur, max_dur = fingerprint['duration_seconds']
            if min_dur <= features.get('duration', 0) <= max_dur:
                score += 1
            checks += 1

        # Verificar bidireccionalidad
        if fingerprint.get('bidirectional', False):
            if features.get('sent_count', 0) > 0 and features.get('received_coun
                score += 1
            checks += 1

        return score / checks if checks > 0 else 0.0

    def detect_correlation(
        self,
        flow_a: TrafficFlow,
        flow_b: TrafficFlow,
        time_threshold: float = 60
    ) -> Dict:
        """
␣␣␣␣␣␣␣␣Detectar␣correlaci n␣entre␣dos␣flujos␣de␣tr fico.
```

```
        """
        # Correlación temporal
        time_diff = abs((flow_a.start_time - flow_b.start_time).total_seconds(
        temporal_correlation = max(0, 1 - time_diff / time_threshold)

        # Correlación de volumen
        vol_a = sum(p.size for p in flow_a.packets)
        vol_b = sum(p.size for p in flow_b.packets)
        if vol_a > 0 and vol_b > 0:
            volume_ratio = min(vol_a, vol_b) / max(vol_a, vol_b)
        else:
            volume_ratio = 0

        # Correlación de duración
        dur_a = (flow_a.end_time - flow_a.start_time).total_seconds()
        dur_b = (flow_b.end_time - flow_b.start_time).total_seconds()
        if dur_a > 0 and dur_b > 0:
            duration_ratio = min(dur_a, dur_b) / max(dur_a, dur_b)
        else:
            duration_ratio = 0

        # Score combinado
        overall_correlation = (
            temporal_correlation * 0.5 +
            volume_ratio * 0.25 +
            duration_ratio * 0.25
        )

        return {
            'temporal_correlation': temporal_correlation,
            'volume_ratio': volume_ratio,
            'duration_ratio': duration_ratio,
            'overall_score': overall_correlation,
            'likely_related': overall_correlation > 0.7
        }

    def website_fingerprinting_attack(
        self,
        target_flow: TrafficFlow,
        known_websites: Dict[str, List[TrafficFlow]]
    ) -> List[Tuple[str, float]]:
        """
        Intento de identificar qu sitio web visit el usuario
        bas ndose en patrones de tr fico (Website Fingerprinting).
        """
        target_features = self.extract_features(target_flow)

        matches = []

        for website, sample_flows in known_websites.items():
            # Calcular features promedio del sitio
```

```
            all_features = [self.extract_features(f) for f in sample_flows]

            # Calcular distancia al target
            similarity = self._calculate_similarity(target_features, all_features
            matches.append((website, similarity))

        # Ordenar por similitud
        matches.sort(key=lambda x: x[1], reverse=True)
        return matches

    def _calculate_similarity(
        self,
        target: Dict,
        samples: List[Dict]
    ) -> float:
        """
        Calcular similitud entre target y muestras.
        """
        if not samples:
            return 0.0

        # Calcular media de cada feature en samples
        mean_features = {}
        for key in target.keys():
            values = [s.get(key, 0) for s in samples if isinstance(s.get(key, 0)
            if values:
                mean_features[key] = statistics.mean(values)

        # Calcular distancia euclidiana normalizada
        distance = 0
        count = 0

        for key, target_val in target.items():
            if key in mean_features and isinstance(target_val, (int, float)):
                mean_val = mean_features[key]
                if mean_val > 0:
                    normalized_diff = abs(target_val - mean_val) / mean_val
                    distance += normalized_diff
                    count += 1

        if count == 0:
            return 0.0

        avg_distance = distance / count
        similarity = max(0, 1 - avg_distance)

        return similarity


def traffic_analysis_demo():
    print("=== Traffic Analysis Demo ===\n")
```

```
analyzer = TrafficAnalyzer()

# Simular flujo de transacci n blockchain
base_time = datetime.now()

blockchain_flow = TrafficFlow(
    flow_id="flow_1",
    source_ip="192.168.1.100",
    dest_ip="13.56.163.26",  # Infura
    packets=[
        Packet(base_time, 450, 'sent', True),
        Packet(base_time + timedelta(milliseconds=50), 200, 'received', Tr
        Packet(base_time + timedelta(milliseconds=100), 800, 'sent', True)
        Packet(base_time + timedelta(milliseconds=200), 150, 'received', T
        Packet(base_time + timedelta(milliseconds=300), 100, 'received', T
    ],
    start_time=base_time,
    end_time=base_time + timedelta(milliseconds=300)
)

# Extraer features
print("--- Feature Extraction ---")
features = analyzer.extract_features(blockchain_flow)
print(f"Packet count: {features['packet_count']}")
print(f"Mean size: {features['size_mean']:.0f} bytes")
print(f"Duration: {features['duration']:.3f}s")
print(f"Direction ratio: {features['direction_ratio']:.2f}")

# Clasificar actividad
print("\n--- Activity Classification ---")
activity, confidence = analyzer.classify_activity(blockchain_flow)
print(f"Classified as: {activity}")
print(f"Confidence: {confidence:.2f}")

# Simular flujo de mixer
mixer_flow = TrafficFlow(
    flow_id="flow_2",
    source_ip="192.168.1.100",
    dest_ip="tornado.cash.onion",
    packets=[
        Packet(base_time + timedelta(seconds=5), 1500, 'sent', True),
        Packet(base_time + timedelta(seconds=5.1), 500, 'received', True),
        Packet(base_time + timedelta(seconds=5.5), 2000, 'sent', True),
        Packet(base_time + timedelta(seconds=6), 800, 'received', True),
        Packet(base_time + timedelta(seconds=6.5), 1200, 'sent', True),
        Packet(base_time + timedelta(seconds=7), 300, 'received', True),
        Packet(base_time + timedelta(seconds=8), 200, 'received', True),
    ],
    start_time=base_time + timedelta(seconds=5),
    end_time=base_time + timedelta(seconds=8)
```

```
    )

    activity2, confidence2 = analyzer.classify_activity(mixer_flow)
    print(f"\nSecond flow classified as: {activity2} ({confidence2:.2f})")

    # Detectar correlaci n
    print("\n—— Correlation Detection ——")
    correlation = analyzer.detect_correlation(blockchain_flow, mixer_flow)
    print(f"Temporal correlation: {correlation['temporal_correlation']:.2f}")
    print(f"Volume ratio: {correlation['volume_ratio']:.2f}")
    print(f"Likely related: {correlation['likely_related']}")

traffic_analysis_demo()
```

3.4. Mixnets vs Onion Routing

3.4.1. Onion Routing (Tor)

Definición 3.7 (Onion Routing)**.** Técnica de comunicación anónima donde el mensaje se encripta en múltiples capas, cada una descifrada por un nodo diferente en la ruta:

Características:

- Circuito establecido antes de enviar datos
- Baja latencia (segundos)
- Paquetes correlacionables por timing
- Vulnerable a análisis de tráfico global

Modelo de seguridad:

- Seguro contra adversario local (observa parte de la red)
- Vulnerable a Global Passive Adversary (GPA)
- No usa cover traffic

[Vulnerabilidades de Onion Routing] **1. End-to-end timing correlation:**
Si un adversario observa tanto la entrada como la salida del circuito:

$$\text{Corr}(T_{\text{entry}}, T_{\text{exit}}) \approx 1$$

Los paquetes tienen el mismo patrón temporal, solo retrasados.
2. Traffic confirmation attack:
El adversario controla nodos de entrada y salida:

1. Modifica timing en entrada (watermark)
2. Detecta watermark en salida
3. Confirma que el mismo usuario

3. Website fingerprinting:
Incluso sin ver contenido, el patrón de tráfico (tamaño, timing) identifica el sitio visitado con alta precisión ($> 90\,\%$ en condiciones controladas).

3.4.2. Mixnets

Definición 3.8 (Mixnet)**.** Red de comunicación anónima donde los mensajes se mezclan con otros mensajes en cada nodo, rompiendo la correlación temporal:

Características:

- Cada nodo acumula mensajes y los reordena
- Alta latencia (minutos a horas)
- Resiste análisis de timing
- Usa cover traffic

Modelo de seguridad:

- Seguro incluso contra GPA (con suficiente tráfico)
- Requiere confianza en que no todos los nodos coludan
- Cover traffic oculta patrones de actividad

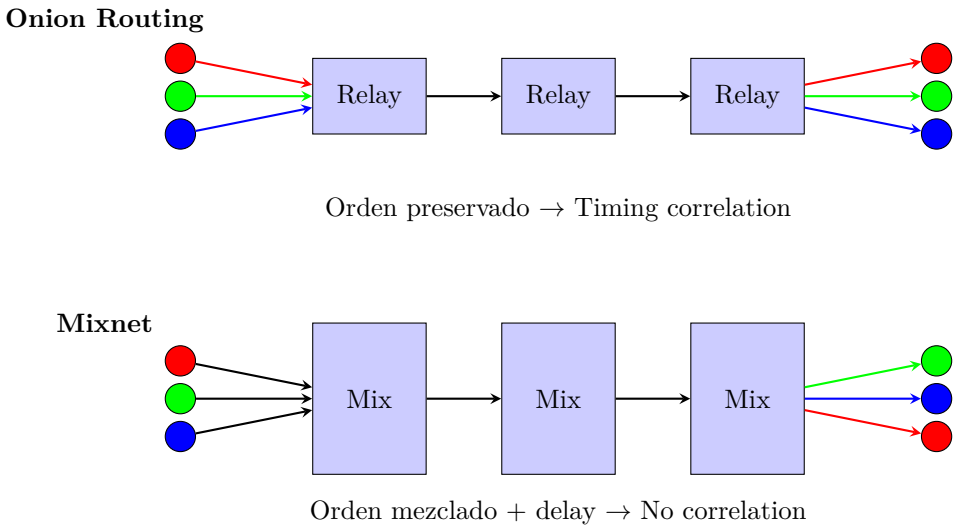


Figura 3.6: Onion routing vs Mixnet: el mixing rompe la correlación temporal

3.4.3. Comparación Detallada

3.4.4. Nym Mixnet

Definición 3.9 (Nym)**.** Mixnet moderna con incentivos económicos y cover traffic integrado:

Arquitectura:

1. **Mix nodes:** 3 capas de mixing
2. **Gateways:** Puntos de entrada/salida
3. **Validators:** Red de consenso para incentivos

Características únicas:

- Sphinx packet format (tamaño uniforme)

Aspecto	Onion Routing (Tor)	Mixnet (Nym)
Latencia	Baja (100ms - 1s)	Alta (segundos a minutos)
Modelo de mensaje	Stream-based (circuitos)	Packet-based (mensajes individuales)
Timing resistance	Parcial (vulnerable a GPA)	Alta (mixing + delays)
Cover traffic	No	Sí (parte del protocolo)
Casos de uso	Browsing web, streaming	Messaging, transacciones
Escalabilidad	Alta (millones de usuarios)	Media (requiere más overhead)
Incentivos	Voluntarios	Económicos (tokens)
Madurez	Alta (20+ años)	Emergente

Tabla 3.2: Comparación entre Onion Routing y Mixnets

- Poisson mixing (delays aleatorios)
- Loop cover traffic (mensajes a sí mismo)
- NYM token para pago e incentivos

Listing 3.4: Simulación conceptual de Mixnet

```
from dataclasses import dataclass
from datetime import datetime, timedelta
from typing import List, Dict, Optional, Tuple
import random
import secrets
from collections import deque
import time

@dataclass
class SphinxPacket:
    """
    Paquete Sphinx con tama o uniforme.
    """
    packet_id: bytes
    encrypted_payload: bytes
    routing_info: bytes  # Encriptado para cada hop
    size: int = 2048  # Tama o fijo

    def is_cover_traffic(self) -> bool:
        """Verificar si es cover traffic (payload vac o)."""
        return self.encrypted_payload == b'\x00' * len(self.encrypted_payload)

@dataclass
class MixNode:
    """
    Nodo de mixing en la mixnet.
    """
    node_id: str
    layer: int
    public_key: bytes
```

```
    # Pool de mensajes para mixing
    message_pool: deque
    pool_size: int = 10

    # Estad sticas
    messages_processed: int = 0

    def __post_init__(self):
        self.message_pool = deque(maxlen=self.pool_size)

    def receive_packet(self, packet: SphinxPacket) -> Optional[SphinxPacket]:
        """
␣␣␣␣␣␣␣␣Recibir␣paquete,␣a adir␣al␣pool,␣potencialmente␣emitir.
␣␣␣␣␣␣␣␣"""
        # Descifrar capa correspondiente a este nodo
        decrypted = self._decrypt_layer(packet)

        # A adir al pool
        self.message_pool.append(decrypted)
        self.messages_processed += 1

        # Emitir si pool est  lleno (mixing)
        if len(self.message_pool) >= self.pool_size:
            return self._emit_random()

        # O emitir con probabilidad (Poisson)
        if random.random() < 0.3:  # Lambda del proceso Poisson
            return self._emit_random()

        return None

    def _decrypt_layer(self, packet: SphinxPacket) -> SphinxPacket:
        """Descifrar␣capa␣de␣routing␣para␣este␣nodo."""
        # En producci n: descifrado real con clave privada
        return SphinxPacket(
            packet_id=packet.packet_id,
            encrypted_payload=packet.encrypted_payload,
            routing_info=packet.routing_info[32:],  # Remover info de este hop
            size=packet.size
        )

    def _emit_random(self) -> Optional[SphinxPacket]:
        """Emitir␣paquete␣aleatorio␣del␣pool."""
        if not self.message_pool:
            return None

        # Selecci n aleatoria (no FIFO!)
        idx = random.randint(0, len(self.message_pool) - 1)
        packets = list(self.message_pool)
        selected = packets[idx]
```

```
        # Remover del pool
        self.message_pool = deque(
            [p for i, p in enumerate(packets) if i != idx],
            maxlen=self.pool_size
        )

        return selected


class MixnetSimulator:
    """
␣␣␣␣Simulador␣de␣mixnet␣con␣m ltiples␣capas.
␣␣␣␣"""

    def __init__(self, num_layers: int = 3, nodes_per_layer: int = 5):
        self.num_layers = num_layers
        self.nodes: Dict[int, List[MixNode]] = {}

        # Crear nodos por capa
        for layer in range(num_layers):
            self.nodes[layer] = []
            for i in range(nodes_per_layer):
                node = MixNode(
                    node_id=f"node_{layer}_{i}",
                    layer=layer,
                    public_key=secrets.token_bytes(32),
                    pool_size=5
                )
                self.nodes[layer].append(node)

    def create_packet(
        self,
        payload: bytes,
        destination: str,
        is_cover: bool = False
    ) -> Tuple[SphinxPacket, List[MixNode]]:
        """
␣␣␣␣␣␣␣␣Crear␣paquete␣Sphinx␣con␣ruta␣aleatoria.
␣␣␣␣␣␣␣␣"""
        # Seleccionar ruta (un nodo por capa)
        route = []
        for layer in range(self.num_layers):
            node = random.choice(self.nodes[layer])
            route.append(node)

        # Construir routing info (encriptado para cada hop)
        # En producci n: ECIES o similar
        routing_info = b''.join(node.public_key for node in route)

        # Payload (uniforme, padded)
```

```
        if is_cover:
            encrypted_payload = b'\x00' * 1024
        else:
            padded = payload.ljust(1024, b'\x00')
            encrypted_payload = padded  # En producci n: encriptar

        packet = SphinxPacket(
            packet_id=secrets.token_bytes(16),
            encrypted_payload=encrypted_payload,
            routing_info=routing_info
        )

        return packet, route

    def send_packet(
        self,
        packet: SphinxPacket,
        route: List[MixNode],
        add_delay: bool = True
    ) -> Dict:
        """
        Enviar paquete a trav s de la mixnet.
        """
        current_packet = packet
        delays = []

        for i, node in enumerate(route):
            # Delay de Poisson antes de procesar
            if add_delay:
                delay = random.expovariate(1.0)  # media = 1 segundo
                delays.append(delay)

            # Procesar en nodo
            output = node.receive_packet(current_packet)

            if output:
                current_packet = output

        return {
            'packet_id': packet.packet_id.hex()[:16],
            'route_length': len(route),
            'total_delay': sum(delays),
            'nodes_visited': [n.node_id for n in route]
        }

    def generate_cover_traffic(self, num_packets: int = 10) -> List[Dict]:
        """
        Generar tr fico de cobertura.
        """
        results = []
```

```
        for _ in range(num_packets):
            packet, route = self.create_packet(
                payload=b'',
                destination='self',
                is_cover=True
            )
            result = self.send_packet(packet, route)
            result['is_cover'] = True
            results.append(result)

        return results

    def analyze_anonymity(
        self,
        observed_input: SphinxPacket,
        observed_outputs: List[SphinxPacket]
    ) -> Dict:
        """
        Analizar qu tan dif cil es correlacionar input con output.
        """
        # Sin timing correlation
        # Todos los outputs son igualmente probables
        anonymity_set = len(observed_outputs)

        # Entrop a
        entropy = 0
        if anonymity_set > 0:
            import math
            prob = 1 / anonymity_set
            entropy = -anonymity_set * prob * math.log2(prob)

        return {
            'anonymity_set_size': anonymity_set,
            'entropy_bits': entropy,
            'probability_correct_guess': 1 / anonymity_set if anonymity_set > 0
        }


def mixnet_demo():
    print("=== Mixnet Simulation Demo ===\n")

    # Crear mixnet
    mixnet = MixnetSimulator(num_layers=3, nodes_per_layer=5)

    print(f"Created mixnet with {mixnet.num_layers} layers")
    print(f"Nodes per layer: {len(mixnet.nodes[0])}")

    # Enviar mensaje real
    print("\n--- Sending Real Message ---")

    packet, route = mixnet.create_packet(
```

```
        payload=b"Secret transaction data",
        destination="0xRecipient",
        is_cover=False
    )

    result = mixnet.send_packet(packet, route)
    print(f"Packet sent through: {result['nodes_visited']}")
    print(f"Total delay: {result['total_delay']:.2f}s")

    # Generar cover traffic
    print("\n— Generating Cover Traffic —")

    cover_results = mixnet.generate_cover_traffic(num_packets=10)
    print(f"Generated {len(cover_results)} cover packets")

    avg_delay = sum(r['total_delay'] for r in cover_results) / len(cover_resul
    print(f"Average cover delay: {avg_delay:.2f}s")

    # An lisis de anonimato
    print("\n— Anonymity Analysis —")

    # Simular observador que ve entrada y 10 salidas posibles
    anonymity = mixnet.analyze_anonymity(
        observed_input=packet,
        observed_outputs=[packet] + [p for p, _ in [mixnet.create_packet(b'',
    )

    print(f"Anonymity set size: {anonymity['anonymity_set_size']}")
    print(f"Entropy: {anonymity['entropy_bits']:.2f} bits")
    print(f"Probability of correct guess: {anonymity['probability_correct_gues

mixnet_demo()
```

3.5. VPN vs Tor vs I2P vs Nym

3.5.1. Comparación Comprehensiva

Característica	VPN	Tor	I2P	Nym
Modelo de confianza	Centralizado	Distribuido	Distribuido	Distribuido
Conoce tu IP	Proveedor VPN	Guard node	Ninguno	Gateway
Latencia típica	10-50ms	200-500ms	500ms-2s	1-10s
Cover traffic	No	No	Limitado	Sí
Timing resistance	No	Parcial	Parcial	Alta
Exit a clearnet	Sí	Sí	Limitado	Sí
Servicios ocultos	No	.onion	.i2p	–
Incentivos	Pago al provider	Voluntarios	Voluntarios	Token NYM
Uso en blockchain	Básico	Recomendado	Monero/Kovri	Emergente
Resistencia a GPA	No	No	Parcial	Alta

Tabla 3.3: Comparación de soluciones de privacidad de red

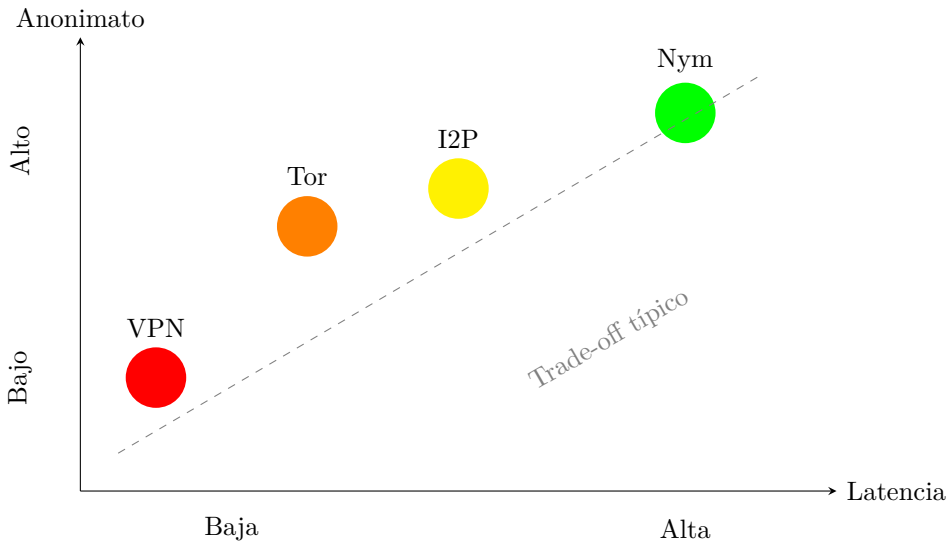


Figura 3.7: Trade-off entre latencia y nivel de anonimato

Idea Clave

Recomendaciones según caso de uso:

1. **Browsing web general:**
   - VPN para privacidad básica del ISP
   - Tor para anonimato real
2. **Transacciones blockchain:**
   - Mínimo: Tor para broadcast de txs
   - Mejor: Nodo propio + Tor
   - Óptimo: Mixnet para transacciones sensibles
3. **Servicios ocultos:**
   - Tor .onion para accesibilidad
   - I2P .i2p para comunidad específica
4. **Mensajería privada:**
   - Nym para máxima privacidad
   - Tolerable latencia alta

3.6. Packet Padding

3.6.1. Concepto y Motivación

Definición 3.10 (Packet Padding)**.** Técnica de añadir datos dummy a los paquetes de red para uniformizar su tamaño, eliminando la información que un adversario podría inferir del tamaño variable de los mensajes.

Objetivo: Si todos los paquetes tienen el mismo tamaño, el observador no puede distinguir entre:

- Transacción grande vs pequeña
- Mensaje de texto vs imagen
- Consulta simple vs compleja

[Análisis de Tamaño de Paquetes] **Sin padding:**

- Transacción simple: ∼250 bytes
- Transacción con contrato: ∼2,000 bytes
- Despliegue de contrato: ∼10,000+ bytes

Un observador puede inferir el tipo de operación solo por el tamaño.
Con padding uniforme:

$$\forall \text{ paquete } p : |p| = S_{\text{fijo}}$$

Todos los paquetes tienen tamaño S_{fijo}, eliminando el side-channel de tamaño.
Trade-offs:

- **Overhead de bandwidth:** Overhead $= \frac{S_{\text{fijo}} - S_{\text{real}}}{S_{\text{real}}} \times 100\,\%$
- **Fragmentación:** Mensajes grandes requieren múltiples paquetes
- **Elección de S_{fijo}:** Muy pequeño = mucha fragmentación, muy grande = mucho overhead

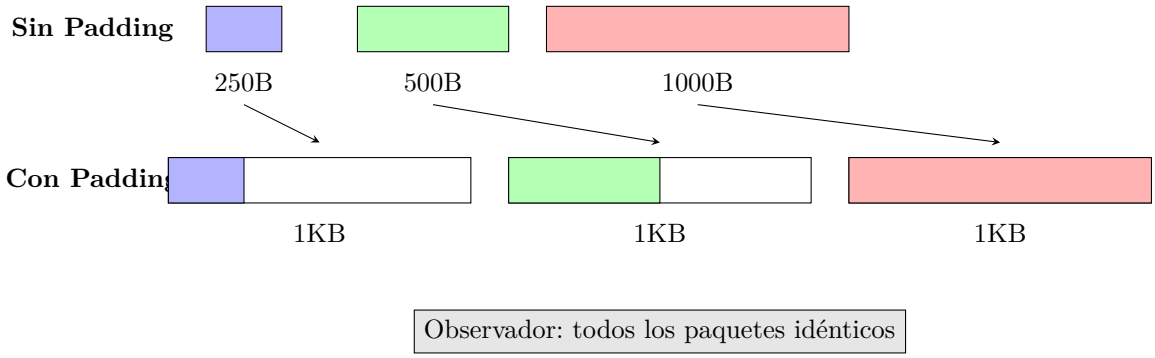


Figura 3.8: Packet padding uniformiza el tamaño visible de los paquetes

Listing 3.5: Sistema de Packet Padding

```
from dataclasses import dataclass
from typing import List, Tuple, Optional
import secrets
import struct

@dataclass
class PaddedPacket:
    """Paquete con padding uniforme."""
    payload: bytes
    padding: bytes
    total_size: int
    sequence_number: int
    is_final: bool  # Para mensajes fragmentados
```

```
class PacketPaddingSystem:
    """
␣␣␣␣Sistema␣de␣padding␣de␣paquetes␣para␣uniformizar␣tama os.
␣␣␣␣"""

    def __init__(self, packet_size: int = 2048):
        """
␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣packet_size:␣Tama o␣fijo␣de␣todos␣los␣paquetes␣(bytes)
␣␣␣␣␣␣␣␣"""
        self.packet_size = packet_size
        self.header_size = 16  # sequence(4) + flags(4) + length(4) + checksum(4
        self.max_payload = packet_size - self.header_size

    def pad_message(self, message: bytes) -> List[PaddedPacket]:
        """
␣␣␣␣␣␣␣␣Convertir␣mensaje␣en␣lista␣de␣paquetes␣padded.
␣␣␣␣␣␣␣␣"""
        packets = []

        # Dividir mensaje si es necesario
        num_packets = (len(message) + self.max_payload - 1) // self.max_payload
        num_packets = max(1, num_packets)  # Al menos 1 paquete

        for i in range(num_packets):
            start = i * self.max_payload
            end = min(start + self.max_payload, len(message))
            chunk = message[start:end]

            # Calcular padding necesario
            padding_size = self.max_payload - len(chunk)
            padding = secrets.token_bytes(padding_size)

            packet = PaddedPacket(
                payload=chunk,
                padding=padding,
                total_size=self.packet_size,
                sequence_number=i,
                is_final=(i == num_packets - 1)
            )
            packets.append(packet)

        return packets

    def serialize_packet(self, packet: PaddedPacket) -> bytes:
        """
␣␣␣␣␣␣␣␣Serializar␣paquete␣a␣bytes␣con␣header.
␣␣␣␣␣␣␣␣"""
        # Header
        flags = 1 if packet.is_final else 0
```

```
        header = struct.pack(
            '>IIII',
            packet.sequence_number,
            flags,
            len(packet.payload),
            self._checksum(packet.payload)
        )

        # Payload + padding
        data = packet.payload + packet.padding

        return header + data

    def deserialize_packet(self, data: bytes) -> PaddedPacket:
        """
␣␣␣␣␣␣␣␣Deserializar␣bytes␣a␣paquete.
␣␣␣␣␣␣␣␣"""
        # Parse header
        seq, flags, payload_len, checksum = struct.unpack('>IIII', data[:16])

        # Extract payload (sin padding)
        payload = data[16:16 + payload_len]
        padding = data[16 + payload_len:]

        # Verificar checksum
        if self._checksum(payload) != checksum:
            raise ValueError("Checksum␣mismatch")

        return PaddedPacket(
            payload=payload,
            padding=padding,
            total_size=len(data),
            sequence_number=seq,
            is_final=(flags & 1) == 1
        )

    def reassemble_message(self, packets: List[PaddedPacket]) -> bytes:
        """
␣␣␣␣␣␣␣␣Reensamblar␣mensaje␣desde␣paquetes.
␣␣␣␣␣␣␣␣"""
        # Ordenar por n mero de secuencia
        sorted_packets = sorted(packets, key=lambda p: p.sequence_number)

        # Concatenar payloads
        message = b''.join(p.payload for p in sorted_packets)

        return message

    def _checksum(self, data: bytes) -> int:
        """Calcular␣checksum␣simple."""
        return sum(data) & 0xFFFFFFFF
```

```
    def calculate_overhead(self, message_size: int) -> dict:
        """
        Calcular overhead de padding para un mensaje.
        """
        packets = self.pad_message(b'\x00' * message_size)

        total_transmitted = len(packets) * self.packet_size
        overhead_bytes = total_transmitted - message_size
        overhead_percent = (overhead_bytes / message_size) * 100 if message_size

        return {
            'message_size': message_size,
            'num_packets': len(packets),
            'total_transmitted': total_transmitted,
            'overhead_bytes': overhead_bytes,
            'overhead_percent': overhead_percent
        }


class VariablePaddingSystem:
    """
    Sistema de padding con tama os variables predefinidos.
    Menor overhead que padding fijo, pero revela categor a de tama o.
    """

    def __init__(self, size_buckets: List[int] = None):
        """
        Args:
            size_buckets: Lista de tama os permitidos (ordenados)
        """
        self.buckets = size_buckets or [64, 256, 1024, 4096, 16384]

    def pad_to_bucket(self, data: bytes) -> Tuple[bytes, int]:
        """
        Pad data al siguiente bucket de tama o.
        """
        data_size = len(data)

        # Encontrar bucket apropiado
        target_size = self.buckets[-1]  # Default al m s grande
        for bucket in self.buckets:
            if bucket >= data_size:
                target_size = bucket
                break

        # Aplicar padding
        padding_size = target_size - data_size
        padding = secrets.token_bytes(padding_size)

        # Prepend length for extraction
```

```
        length_prefix = struct.pack('>I', data_size)
        padded = length_prefix + data + padding

        return padded, target_size

    def extract_from_bucket(self, padded_data: bytes) -> bytes:
        """
␣␣␣␣␣␣␣␣Extraer␣datos␣originales␣de␣bucket␣padded.
␣␣␣␣␣␣␣␣"""
        # Extract length
        original_length = struct.unpack('>I', padded_data[:4])[0]

        # Extract data
        return padded_data[4:4 + original_length]

    def analyze_leakage(self) -> dict:
        """
␣␣␣␣␣␣␣␣Analizar␣cu nta␣informaci n␣filtra␣este␣esquema.
␣␣␣␣␣␣␣␣"""
        import math

        # Entrop a de la distribuci n de buckets
        # Si hay n buckets, un adversario puede distinguir n casos
        num_buckets = len(self.buckets)
        max_entropy = math.log2(num_buckets)

        return {
            'num_buckets': num_buckets,
            'size_categories': self.buckets,
            'bits_leaked': max_entropy,
            'anonymity_set': num_buckets
        }


def packet_padding_demo():
    print("═══␣Packet␣Padding␣Demo␣═══\n")

    # Sistema de padding fijo
    system = PacketPaddingSystem(packet_size=2048)

    # Diferentes mensajes
    messages = [
        (b"short␣message", "Short"),
        (b"A" * 500, "Medium␣(500B)"),
        (b"B" * 2000, "Large␣(2KB)"),
        (b"C" * 5000, "Very␣Large␣(5KB)")
    ]

    print("──␣Fixed␣Size␣Padding␣(2KB␣packets)␣──\n")

    for msg, desc in messages:
```

```
        packets = system.pad_message(msg)
        overhead = system.calculate_overhead(len(msg))

        print(f"{desc}:")
        print(f"  Original: {len(msg)} bytes")
        print(f"  Packets: {len(packets)}")
        print(f"  Transmitted: {overhead['total_transmitted']} bytes")
        print(f"  Overhead: {overhead['overhead_percent']:.1f}%")

        # Verificar todos los paquetes tienen mismo tama o
        serialized = [system.serialize_packet(p) for p in packets]
        sizes = [len(s) for s in serialized]
        print(f"  All packets same size: {len(set(sizes)) == 1}")
        print()

    # Sistema de buckets
    print("--- Variable Bucket Padding ---\n")

    bucket_system = VariablePaddingSystem()
    leakage = bucket_system.analyze_leakage()

    print(f"Buckets: {leakage['size_categories']}")
    print(f"Information leaked: {leakage['bits_leaked']:.2f} bits")

    for msg, desc in messages:
        padded, bucket = bucket_system.pad_to_bucket(msg)
        print(f"\n{desc}:")
        print(f"  Original: {len(msg)} bytes -> Bucket: {bucket} bytes")

packet_padding_demo()
```

3.7. Dummy Traffic (Cover Traffic)

3.7.1. Concepto y Tipos

Definición 3.11 (Dummy Traffic / Cover Traffic)**.** Tráfico artificial generado para ocultar patrones de comunicación real. El adversario no puede distinguir entre tráfico real y dummy.

Propósitos:

- Ocultar cuándo ocurre comunicación real
- Uniformizar patrones de actividad
- Prevenir timing correlation
- Aumentar anonymity set

[Tipos de Cover Traffic] **1. Constant-Rate Cover Traffic:**

- Enviar paquetes a tasa constante (ej: 1/segundo)
- Paquetes reales reemplazan dummies
- Máxima resistencia a timing analysis

- Alto overhead de bandwidth

2. Poisson Cover Traffic:

- Intervalos entre paquetes siguen distribución Poisson
- Más eficiente que constant-rate
- Estadísticamente indistinguible del tráfico real

3. Loop Traffic:

- Mensajes enviados a uno mismo a través de la red
- Aumenta tráfico sin revelar destinos
- Usado en Nym mixnet

4. Chaff Traffic:

- Paquetes indistinguibles de reales pero descartados
- Solo el receptor puede identificar chaff
- Previene traffic analysis end-to-end

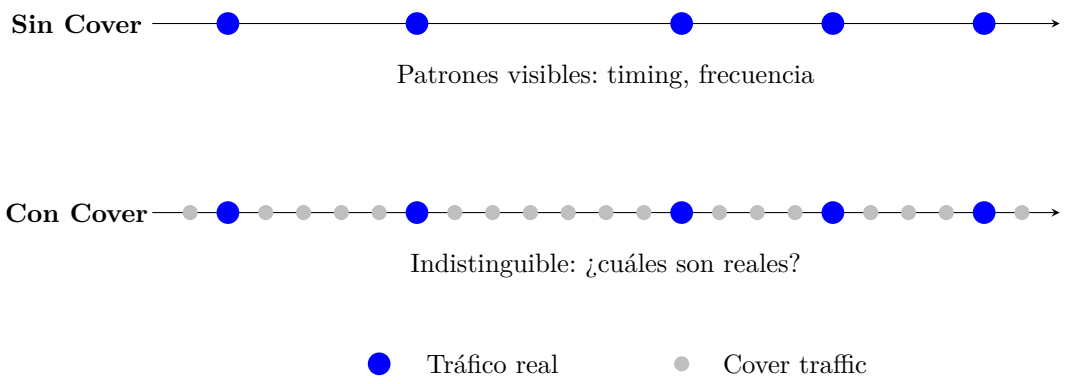


Figura 3.9: Cover traffic oculta los patrones del tráfico real

Listing 3.6: Sistema de Cover Traffic

```
from dataclasses import dataclass
from datetime import datetime, timedelta
from typing import List, Callable, Optional
import random
import threading
import time
from enum import Enum

class TrafficType(Enum):
    REAL = "real"
    COVER = "cover"
    LOOP = "loop"
    CHAFF = "chaff"

@dataclass
```

```
class TrafficPacket:
    packet_id: str
    traffic_type: TrafficType
    payload: bytes
    timestamp: datetime
    destination: str

class CoverTrafficGenerator:
    """
␣␣␣␣Generador␣de␣cover␣traffic␣para␣ocultar␣patrones␣de␣comunicaci n.
␣␣␣␣"""

    def __init__(
        self,
        packet_size: int = 2048,
        base_rate: float = 1.0  # paquetes por segundo
    ):
        self.packet_size = packet_size
        self.base_rate = base_rate
        self.running = False
        self.packet_queue: List[TrafficPacket] = []
        self.sent_packets: List[TrafficPacket] = []

        # Estad sticas
        self.stats = {
            'real_sent': 0,
            'cover_sent': 0,
            'loop_sent': 0,
            'total_bytes': 0
        }

    def start_constant_rate(self, destinations: List[str]):
        """
␣␣␣␣␣␣␣␣Iniciar␣generaci n␣de␣cover␣traffic␣a␣tasa␣constante.
␣␣␣␣␣␣␣␣"""
        self.running = True

        while self.running:
            # Verificar si hay paquete real pendiente
            real_packet = self._get_pending_real()

            if real_packet:
                # Enviar paquete real
                self._send_packet(real_packet)
            else:
                # Enviar cover packet
                cover = self._generate_cover_packet(
                    random.choice(destinations)
                )
                self._send_packet(cover)
```

```
            # Esperar intervalo fijo
            time.sleep(1.0 / self.base_rate)

    def start_poisson_rate(self, destinations: List[str], mean_interval: float
        """
␣␣␣␣␣␣␣␣Iniciar␣generaci n␣con␣intervalos␣Poisson.
␣␣␣␣␣␣␣␣M s␣eficiente␣y␣estad sticamente␣m s␣dif cil␣de␣analizar.
␣␣␣␣␣␣␣␣"""
        self.running = True

        while self.running:
            real_packet = self._get_pending_real()

            if real_packet:
                self._send_packet(real_packet)
            else:
                cover = self._generate_cover_packet(
                    random.choice(destinations)
                )
                self._send_packet(cover)

            # Intervalo Poisson (exponencial)
            interval = random.expovariate(1.0 / mean_interval)
            time.sleep(interval)

    def queue_real_packet(self, payload: bytes, destination: str):
        """
␣␣␣␣␣␣␣␣Encolar␣paquete␣real␣para␣env o␣en␣siguiente␣slot.
␣␣␣␣␣␣␣␣"""
        packet = TrafficPacket(
            packet_id=f"real_{len(self.sent_packets)}",
            traffic_type=TrafficType.REAL,
            payload=self._pad_payload(payload),
            timestamp=datetime.now(),
            destination=destination
        )
        self.packet_queue.append(packet)

    def generate_loop_traffic(self, own_address: str, num_loops: int = 5):
        """
␣␣␣␣␣␣␣␣Generar␣loop␣traffic␣(mensajes␣a␣uno␣mismo).
␣␣␣␣␣␣␣␣"""
        for i in range(num_loops):
            loop_packet = TrafficPacket(
                packet_id=f"loop_{i}",
                traffic_type=TrafficType.LOOP,
                payload=self._random_payload(),
                timestamp=datetime.now(),
                destination=own_address
            )
            self._send_packet(loop_packet)
```

```
            # Intervalo aleatorio
            time.sleep(random.uniform(0.1, 1.0))

    def _generate_cover_packet(self, destination: str) -> TrafficPacket:
        """Generar paquete de cover traffic."""
        return TrafficPacket(
            packet_id=f"cover_{self.stats['cover_sent']}",
            traffic_type=TrafficType.COVER,
            payload=self._random_payload(),
            timestamp=datetime.now(),
            destination=destination
        )

    def _get_pending_real(self) -> Optional[TrafficPacket]:
        """Obtener siguiente paquete real pendiente."""
        if self.packet_queue:
            return self.packet_queue.pop(0)
        return None

    def _send_packet(self, packet: TrafficPacket):
        """Enviar paquete (simulaci n)."""
        self.sent_packets.append(packet)

        # Actualizar estad sticas
        if packet.traffic_type == TrafficType.REAL:
            self.stats['real_sent'] += 1
        elif packet.traffic_type == TrafficType.COVER:
            self.stats['cover_sent'] += 1
        elif packet.traffic_type == TrafficType.LOOP:
            self.stats['loop_sent'] += 1

        self.stats['total_bytes'] += len(packet.payload)

    def _pad_payload(self, payload: bytes) -> bytes:
        """Pad payload a tama o fijo."""
        if len(payload) >= self.packet_size:
            return payload[:self.packet_size]
        padding = bytes(self.packet_size - len(payload))
        return payload + padding

    def _random_payload(self) -> bytes:
        """Generar payload aleatorio para cover traffic."""
        return bytes(random.getrandbits(8) for _ in range(self.packet_size))

    def stop(self):
        """Detener generaci n de cover traffic."""
        self.running = False

    def get_statistics(self) -> dict:
        """Obtener estad sticas de tr fico."""
```

```
        total_packets = self.stats['real_sent'] + self.stats['cover_sent'] + s

        return {
            **self.stats,
            'total_packets': total_packets,
            'real_ratio': self.stats['real_sent'] / total_packets if total_pac
            'overhead_ratio': 1 - (self.stats['real_sent'] / total_packets) if
            'bandwidth_overhead_bytes': (self.stats['cover_sent'] + self.stats
        }

    def analyze_timing_resistance(self) -> dict:
        """
␣␣␣␣␣␣␣␣Analizar␣qu ␣tan␣bien␣el␣cover␣traffic␣oculta␣el␣timing␣del␣tr fico␣
␣␣␣␣␣␣␣"""
        real_timestamps = [
            p.timestamp for p in self.sent_packets
            if p.traffic_type == TrafficType.REAL
        ]

        all_timestamps = [p.timestamp for p in self.sent_packets]

        if len(all_timestamps) < 2:
            return {'insufficient_data': True}

        # Calcular intervalos
        def get_intervals(timestamps):
            sorted_ts = sorted(timestamps)
            return [
                (sorted_ts[i+1] - sorted_ts[i]).total_seconds()
                for i in range(len(sorted_ts) - 1)
            ]

        all_intervals = get_intervals(all_timestamps)

        import statistics

        return {
            'total_packets': len(all_timestamps),
            'real_packets': len(real_timestamps),
            'mean_interval': statistics.mean(all_intervals),
            'std_interval': statistics.stdev(all_intervals) if len(all_interva
            'timing_uniformity': 1 - (statistics.stdev(all_intervals) / statis
                                 if len(all_intervals) > 1 and statistics.mean(
        }


def cover_traffic_demo():
    print("===␣Cover␣Traffic␣Demo␣===\n")

    generator = CoverTrafficGenerator(packet_size=2048, base_rate=10)
```

```
destinations = ["node_a", "node_b", "node_c", "node_d"]

# Simular env o de paquetes reales
print("── Queueing Real Packets ──")

for i in range(5):
    generator.queue_real_packet(
        payload=f"Real message {i}".encode(),
        destination=random.choice(destinations)
    )

print(f"Queued {len(generator.packet_queue)} real packets")

# Simular constant rate durante algunos "ticks"
print("\n── Constant Rate Simulation ──")

for tick in range(30):
    # Simular un tick del generador
    real_packet = generator._get_pending_real()

    if real_packet:
        generator._send_packet(real_packet)
    else:
        cover = generator._generate_cover_packet(
            random.choice(destinations)
        )
        generator._send_packet(cover)

# Estad sticas
print("\n── Statistics ──")
stats = generator.get_statistics()

print(f"Total packets sent: {stats['total_packets']}")
print(f"Real packets: {stats['real_sent']}")
print(f"Cover packets: {stats['cover_sent']}")
print(f"Real ratio: {stats['real_ratio']:.2%}")
print(f"Overhead ratio: {stats['overhead_ratio']:.2%}")

# An lisis de timing
print("\n── Timing Analysis ──")
timing = generator.analyze_timing_resistance()

print(f"Mean interval: {timing.get('mean_interval', 0):.4f}s")
print(f"Std interval: {timing.get('std_interval', 0):.4f}s")
print(f"Timing uniformity: {timing.get('timing_uniformity', 0):.2%}")

print("\n── Adversary's View ──")
print("From observer's perspective:")
print(f"  Sees {stats['total_packets']} identical packets")
print(f"  Cannot distinguish which {stats['real_sent']} are real")
print(f"  Effective anonymity set: {stats['total_packets']}")
```

```
cover_traffic_demo()
```

3.8. P2P Network Privacy

3.8.1. libp2p

Definición 3.12 (libp2p)**.** Framework modular de networking peer-to-peer desarrollado por Protocol Labs:

Usado por:

- IPFS (InterPlanetary File System)
- Ethereum 2.0 (consensus layer)
- Filecoin
- Polkadot

Características:

- Transport agnostic (TCP, QUIC, WebSocket)
- Multiplexing de conexiones
- NAT traversal
- Peer discovery (DHT, mDNS)
- Encryption (Noise protocol)

Limitaciones de privacidad en libp2p:

1. **IP visible a peers:**
   - Cada peer conoce la IP de sus conexiones directas
   - No hay onion routing nativo
2. **Gossip protocol:**
   - Mensajes se propagan a múltiples peers
   - Primer receptor puede inferir origen
3. **DHT queries:**
   - Queries revelan qué datos buscas
   - Nodos DHT pueden perfilar usuarios
4. **Peer IDs:**
   - Identificadores persistentes
   - Vinculan sesiones a través del tiempo

Listing 3.7: Análisis de privacidad en redes P2P

```
from dataclasses import dataclass
from typing import Dict, List, Set
from collections import defaultdict

@dataclass
class Peer:
    peer_id: str
    ip_address: str
    connected_peers: Set[str]
    messages_originated: List[str]
    messages_relayed: List[str]

class P2PPrivacyAnalyzer:
    """
    Analizador de privacidad para redes P2P tipo libp2p.
    """

    def __init__(self):
        self.peers: Dict[str, Peer] = {}
        self.message_first_seen: Dict[str, str] = {}  # msg_id -> first peer
        self.ip_to_peer: Dict[str, str] = {}

    def add_peer(self, peer: Peer):
        """Registrar peer en la red."""
        self.peers[peer.peer_id] = peer
        self.ip_to_peer[peer.ip_address] = peer.peer_id

    def record_message_propagation(
        self,
        message_id: str,
        peer_id: str,
        is_originator: bool
    ):
        """
        Registrar propagaci n de mensaje.
        """
        if message_id not in self.message_first_seen:
            self.message_first_seen[message_id] = peer_id

        if peer_id in self.peers:
            if is_originator:
                self.peers[peer_id].messages_originated.append(message_id)
            else:
                self.peers[peer_id].messages_relayed.append(message_id)

    def analyze_ip_exposure(self) -> Dict:
        """
        Analizar exposici n de IPs a otros peers.
        """
        exposure = {}
```

```
        for peer_id, peer in self.peers.items():
            # IPs que este peer conoce (sus conexiones directas)
            known_ips = set()
            for connected_id in peer.connected_peers:
                if connected_id in self.peers:
                    known_ips.add(self.peers[connected_id].ip_address)

            # IPs que conocen este peer
            exposing_peers = [
                p.peer_id for p in self.peers.values()
                if peer_id in p.connected_peers
            ]

            exposure[peer_id] = {
                'ip': peer.ip_address,
                'knows_ips': len(known_ips),
                'exposed_to': len(exposing_peers),
                'exposure_peers': exposing_peers[:5]  # Primeros 5
            }

        return exposure

    def analyze_first_seen_heuristic(self) -> Dict:
        """
        Analizar efectividad del heur stico "first-seen" para
        identificar originadores de mensajes.
        """
        correct_attributions = 0
        total_messages = 0

        for peer in self.peers.values():
            for msg_id in peer.messages_originated:
                total_messages += 1
                first_seen = self.message_first_seen.get(msg_id)
                if first_seen == peer.peer_id:
                    correct_attributions += 1

        return {
            'total_messages': total_messages,
            'correctly_attributed': correct_attributions,
            'attribution_accuracy': correct_attributions / total_messages if t
            'privacy_level': 1 - (correct_attributions / total_messages) if to
        }

    def recommend_mitigations(self) -> List[str]:
        """
        Recomendar mitigaciones de privacidad.
        """
        recommendations = []
```

```
        exposure = self.analyze_ip_exposure()
        first_seen = self.analyze_first_seen_heuristic()

        # Analizar exposici n promedio
        avg_exposure = sum(e['exposed_to'] for e in exposure.values()) / len(exp

        if avg_exposure > 10:
            recommendations.append(
                "High peer exposure - consider reducing max connections"
            )

        if first_seen['attribution_accuracy'] > 0.5:
            recommendations.append(
                "High first-seen attribution - implement Dandelion protocol"
            )

        recommendations.extend([
            "Use Tor transport for libp2p connections",
            "Rotate peer IDs periodically",
            "Add random delays before message propagation",
            "Use encrypted DHT queries"
        ])

        return recommendations


def p2p_privacy_demo():
    print("=== P2P Privacy Analysis Demo ===\n")

    analyzer = P2PPrivacyAnalyzer()

    # Crear red de ejemplo
    peers = [
        Peer(f"peer_{i}", f"192.168.1.{i}", set(), [], [])
        for i in range(10)
    ]

    # Conectar peers (topolog a de malla parcial)
    import random
    for i, peer in enumerate(peers):
        # Cada peer conecta a 3-5 otros
        num_connections = random.randint(3, 5)
        possible_peers = [p.peer_id for p in peers if p.peer_id != peer.peer_id]
        peer.connected_peers = set(random.sample(possible_peers, num_connections

    for peer in peers:
        analyzer.add_peer(peer)

    # Simular propagaci n de mensajes
    print("-- Simulating Message Propagation --")
```

```
    for i in range(20):
        originator = random.choice(peers)
        msg_id = f"msg_{i}"

        # Registrar mensaje
        analyzer.record_message_propagation(msg_id, originator.peer_id, True)

        # Propagar a peers conectados
        for connected_id in originator.connected_peers:
            analyzer.record_message_propagation(msg_id, connected_id, False)

    # An lisis de exposici n IP
    print("\n── IP Exposure Analysis ──")
    exposure = analyzer.analyze_ip_exposure()

    for peer_id, data in list(exposure.items())[:3]:
        print(f"\n{peer_id} ({data['ip']}):")
        print(f"  Knows IPs of: {data['knows_ips']} peers")
        print(f"  Exposed to: {data['exposed_to']} peers")

    # An lisis first-seen
    print("\n── First-Seen Heuristic Analysis ──")
    first_seen = analyzer.analyze_first_seen_heuristic()

    print(f"Messages analyzed: {first_seen['total_messages']}")
    print(f"Correctly attributed: {first_seen['correctly_attributed']}")
    print(f"Attribution accuracy: {first_seen['attribution_accuracy']:.2%}")
    print(f"Privacy level: {first_seen['privacy_level']:.2%}")

    # Recomendaciones
    print("\n── Privacy Recommendations ──")
    for rec in analyzer.recommend_mitigations():
        print(f"  {rec}")

p2p_privacy_demo()
```

3.8.2. Waku

Definición 3.13 (Waku)**.** Protocolo de mensajería descentralizado desarrollado por Status, diseñado como evolución de Whisper con mejor privacidad y escalabilidad:

Versiones:

- Waku v1: Compatible con Whisper
- Waku v2: Rediseño con mejor privacidad

Características de privacidad:

- **Content topics:** Agrupación de mensajes sin revelar contenido
- **Encryption:** Mensajes encriptados end-to-end
- **Relay nodes:** No ven contenido, solo metadata limitada
- **Anonymity set:** Todos los usuarios del mismo topic

Aspecto	libp2p básico	Waku v2
IP visible a peers	Sí	Relay oculta
Contenido visible a relay	Sí	No (E2E encrypted)
Metadata visible	Alta	Limitada a topic
Sender anonymity	Baja	Media-Alta
Receiver anonymity	Baja	Alta

Tabla 3.4: Comparación de privacidad entre libp2p y Waku

Resumen del Bloque

Idea Clave

Puntos clave de Metadata y Network Privacy (Bloque 16):
1. IP Leakage:

- Múltiples vectores: P2P, RPC, dApps, DNS
- Protecciones: VPN (básica), Tor (mejor), nodo propio
- IP revela geolocalización e identidad potencial

2. Timing Correlation:

- Vincula eventos por proximidad temporal
- Defensas: delays aleatorios, batching, cover traffic
- Crítico en mixers y privacidad financiera

3. Traffic Analysis:

- Fingerprinting por tamaño, timing, patrones
- Website fingerprinting efectivo incluso con Tor
- Defensas: padding, cover traffic, mixing

4. Onion Routing vs Mixnets:

- Tor: baja latencia, vulnerable a GPA
- Mixnets: alta latencia, resiste timing analysis
- Trade-off latencia vs anonimato

5. VPN/Tor/I2P/Nym:

- VPN: confianza centralizada, básico
- Tor: distribuido, buen balance
- I2P: servicios ocultos, P2P
- Nym: mixnet con incentivos, máximo anonimato

6. Packet Padding:

- Uniformiza tamaños para ocultar contenido

- Trade-off overhead vs privacidad
- Buckets variables como compromiso

7. Cover Traffic:

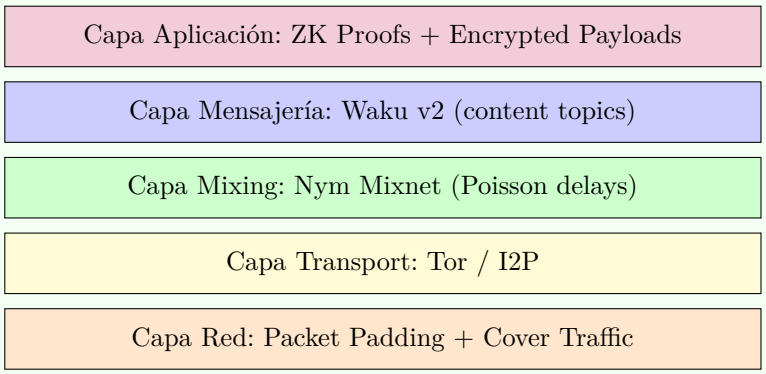
- Tráfico falso indistinguible del real
- Tipos: constant-rate, Poisson, loop, chaff
- Esencial para máxima privacidad temporal

8. P2P Privacy:

- libp2p: funcional pero sin privacidad nativa
- Waku: mensajería con privacidad mejorada
- Mitigaciones: Tor transport, Dandelion, delays

Aplicación en Banco ZK

Stack de Network Privacy en Banco ZK:



Configuraciones de privacidad:

1. **Modo Estándar:**
   - Tor para conexiones
   - Nodo propio recomendado
   - Delays aleatorios en broadcast
2. **Modo Alto Anonimato:**
   - Nym mixnet para transacciones
   - Cover traffic activo
   - Packet padding uniforme
   - Mayor latencia (minutos)
3. **Modo Máximo:**
   - Todo lo anterior +

- Loop traffic continuo
- Conexiones solo via .onion
- Rotación de identidad de red

Métricas de privacidad de red:

Métrica	Target
IP exposure a terceros	0 (via Tor)
Timing correlation resistance	$> 95\,\%$
Cover traffic ratio	$\geq 50\,\%$
Packet size uniformity	$100\,\%$
First-seen attribution	$< 10\,\%$

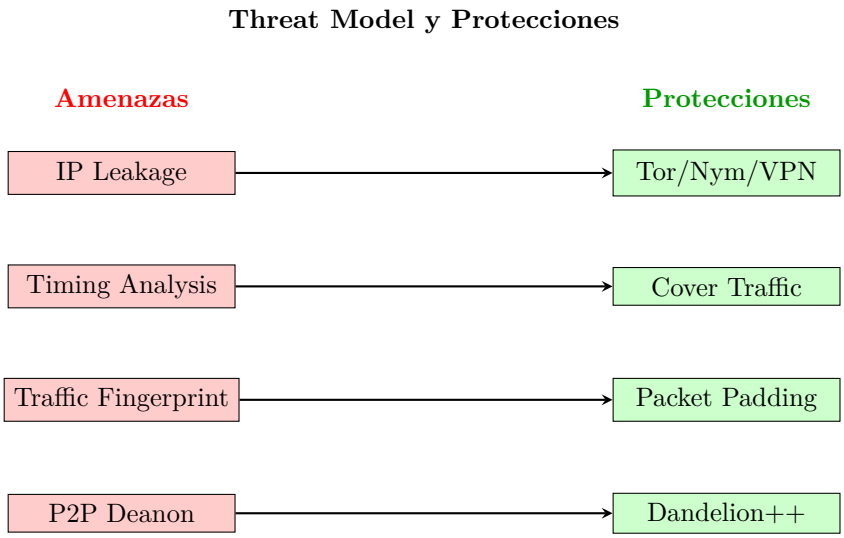


Figura 3.10: Mapeo de amenazas de red a protecciones correspondientes

Capítulo 4

Private Smart Contracts

Bloque 17 de 20

Introducción

Los smart contracts tradicionales (Ethereum, Solana) ejecutan código de forma transparente: todos pueden ver el estado, los inputs y los outputs de cada transacción. Esta transparencia, aunque útil para auditoría, es incompatible con muchos casos de uso que requieren confidencialidad. Este bloque explora las diferentes aproximaciones para lograr smart contracts privados.

Idea Clave

El problema de la transparencia en smart contracts:
En Ethereum tradicional, todos pueden ver:

- Estado del contrato (balances, mappings, variables)
- Inputs de cada llamada (parámetros de función)
- Outputs de cada transacción
- Historial completo de interacciones

Casos de uso afectados:

- Subastas (pujas visibles = frontrunning)
- Votación (votos públicos = coerción)
- Trading (órdenes visibles = MEV extraction)
- DeFi privado (posiciones visibles = liquidaciones predatorias)
- Datos sensibles (salud, finanzas, identidad)

[Comparación de Modelos de Confianza] **1. Zero-Knowledge Proofs (ZK):**

- Trust: Solo en matemáticas y criptografía
- Asunción: Problema computacionalmente difícil (DLP, factorización)
- Verificación: On-chain, cualquiera puede verificar

2. Trusted Execution Environment (TEE):

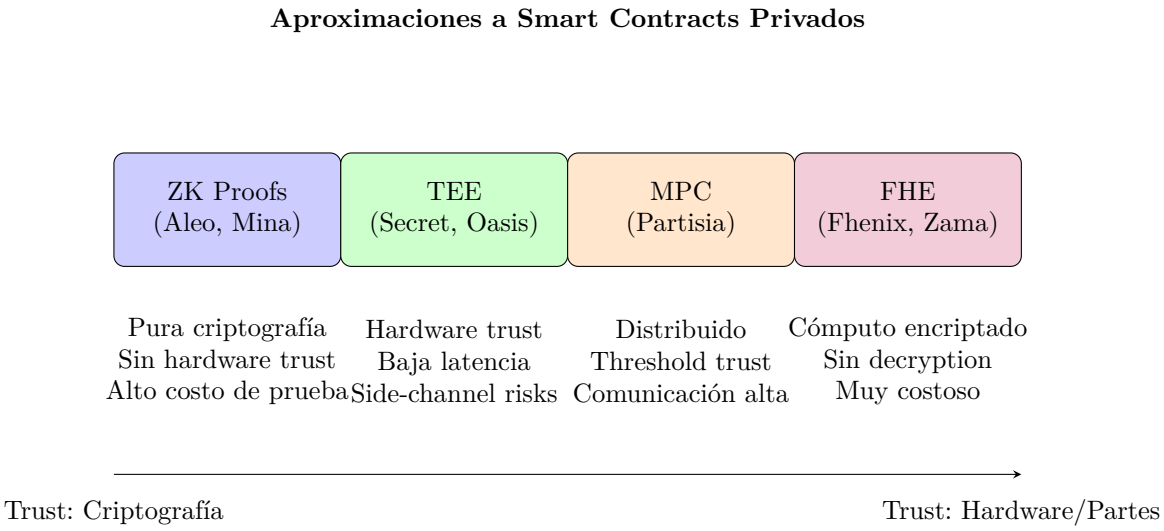


Figura 4.1: Cuatro aproximaciones principales a smart contracts privados

- Trust: Hardware del fabricante (Intel SGX, AMD SEV)
- Asunción: Enclave no comprometido, sin side-channels explotables
- Verificación: Attestation del enclave

3. Multi-Party Computation (MPC):

- Trust: Threshold de partes honestas (t de n)
- Asunción: No más de $t - 1$ partes coludan
- Verificación: Protocolo garantiza correctitud

4. Fully Homomorphic Encryption (FHE):

- Trust: Solo en criptografía
- Asunción: Problema LWE/RLWE es difícil
- Verificación: El resultado descifrado es correcto

4.1. Secret Network

4.1.1. Arquitectura y Diseño

Definición 4.1 (Secret Network)**.** Blockchain Layer-1 basada en Cosmos SDK que permite smart contracts con estado encriptado mediante Trusted Execution Environments (TEE), específicamente Intel SGX.

Características principales:

- Smart contracts "secretos" con estado encriptado
- Solo el enclave SGX puede ver datos en claro
- Viewing keys para acceso selectivo a datos
- Compatibilidad con ecosistema Cosmos (IBC)
- Token nativo: SCRT

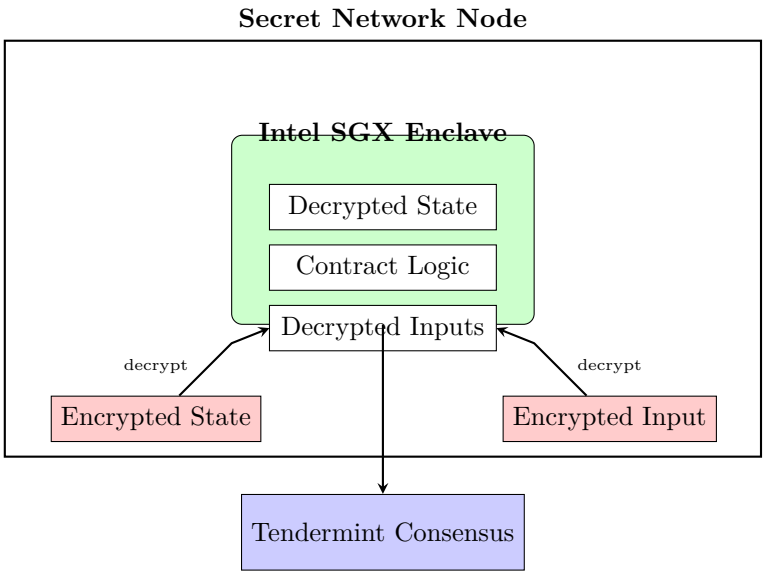


Figura 4.2: Arquitectura de Secret Network: estado encriptado, procesamiento en enclave SGX

4.1.2. Intel SGX y Trusted Execution

Definición 4.2 (Intel SGX (Software Guard Extensions))**.** Conjunto de instrucciones de CPU que permite crear "enclaves" - regiones de memoria aisladas y encriptadas donde el código se ejecuta de forma protegida.

Propiedades:

- **Confidencialidad:** Datos en enclave encriptados en RAM
- **Integridad:** Código no puede ser modificado
- **Attestation:** Prueba criptográfica de qué código ejecuta
- **Sealing:** Datos persistentes encriptados a enclave específico

Vulnerabilidades conocidas de SGX:

1. **Side-channel attacks:**
   - Spectre, Meltdown, Foreshadow
   - Timing attacks, cache attacks
   - Requieren parches continuos
2. **Rollback attacks:**
   - OS controla persistencia
   - Puede restaurar estado anterior
3. **Trust en Intel:**
   - Intel genera claves de attestation
   - Backdoors teóricamente posibles
4. **Disponibilidad:**
   - Intel puede revocar attestation keys

- Dependencia de proveedor único

Mitigación en Secret Network:

- Múltiples nodos con diferentes CPUs/versiones
- Threshold schemes para claves críticas
- Actualizaciones frecuentes de enclaves

4.1.3. Desarrollo de Secret Contracts

Listing 4.1: Secret Contract básico en Rust (CosmWasm modificado)

```
// Estructura del contrato secreto
use cosmwasm_std::{
    entry_point, to_binary, Binary, Deps, DepsMut, Env,
    MessageInfo, Response, StdResult,
};
use secret_toolkit::viewing_key::{ViewingKey, ViewingKeyStore};

// Estado encriptado (solo visible dentro del enclave)
#[derive(Serialize, Deserialize, Clone)]
pub struct State {
    pub owner: Addr,
    pub secret_data: String,
    pub private_balances: HashMap<Addr, Uint128>,
    pub total_supply: Uint128,
}

// Mensajes de inicializaci n
#[derive(Serialize, Deserialize)]
pub struct InstantiateMsg {
    pub initial_supply: Uint128,
}

// Mensajes de ejecuci n (inputs encriptados)
#[derive(Serialize, Deserialize)]
pub enum ExecuteMsg {
    // Transferencia privada
    Transfer {
        recipient: String,
        amount: Uint128,
    },
    // Crear viewing key para el usuario
    CreateViewingKey {
        entropy: String,
    },
    // Depositar (p blico -> privado)
    Deposit {},
    // Retirar (privado -> p blico)
    Withdraw {
        amount: Uint128,
```

```
    },
}

// Mensajes de consulta (requieren viewing key para datos privados)
#[derive(Serialize, Deserialize)]
pub enum QueryMsg {
    // Consulta p blica
    TotalSupply {},
    // Consulta privada (requiere viewing key)
    Balance {
        address: String,
        key: String,
    },
    // Historial de transacciones privado
    TransactionHistory {
        address: String,
        key: String,
        page: u32,
        page_size: u32,
    },
}

// ========== ENTRY POINTS ==========

#[entry_point]
pub fn instantiate(
    deps: DepsMut,
    _env: Env,
    info: MessageInfo,
    msg: InstantiateMsg,
) -> StdResult<Response> {
    let state = State {
        owner: info.sender.clone(),
        secret_data: String::new(),
        private_balances: HashMap::new(),
        total_supply: msg.initial_supply,
    };

    // Guardar estado (ser  encriptado por el enclave)
    STATE.save(deps.storage, &state)?;

    Ok(Response::new()
        .add_attribute("action", "instantiate")
        .add_attribute("owner", info.sender))
}

#[entry_point]
pub fn execute(
    deps: DepsMut,
    env: Env,
    info: MessageInfo,
```

```
    msg: ExecuteMsg,
) -> StdResult<Response> {
    match msg {
        ExecuteMsg::Transfer { recipient, amount } => {
            execute_transfer(deps, env, info, recipient, amount)
        }
        ExecuteMsg::CreateViewingKey { entropy } => {
            execute_create_viewing_key(deps, env, info, entropy)
        }
        ExecuteMsg::Deposit {} => {
            execute_deposit(deps, env, info)
        }
        ExecuteMsg::Withdraw { amount } => {
            execute_withdraw(deps, env, info, amount)
        }
    }
}

// Transferencia privada
fn execute_transfer(
    deps: DepsMut,
    _env: Env,
    info: MessageInfo,
    recipient: String,
    amount: Uint128,
) -> StdResult<Response> {
    let mut state = STATE.load(deps.storage)?;

    let sender = info.sender.clone();
    let recipient_addr = deps.api.addr_validate(&recipient)?;

    // Verificar balance (dentro del enclave, no visible externamente)
    let sender_balance = state.private_balances
        .get(&sender)
        .copied()
        .unwrap_or(Uint128::zero());

    if sender_balance < amount {
        return Err(StdError::generic_err("Insufficient balance"));
    }

    // Actualizar balances (estado encriptado)
    state.private_balances.insert(
        sender.clone(),
        sender_balance - amount
    );

    let recipient_balance = state.private_balances
        .get(&recipient_addr)
        .copied()
        .unwrap_or(Uint128::zero());
```

```
    state.private_balances.insert(
        recipient_addr.clone(),
        recipient_balance + amount
    );

    STATE.save(deps.storage, &state)?;

    // Log m nimo (no revela montos ni direcciones)
    Ok(Response::new()
        .add_attribute("action", "transfer"))
}

// Crear viewing key
fn execute_create_viewing_key(
    deps: DepsMut,
    env: Env,
    info: MessageInfo,
    entropy: String,
) -> StdResult<Response> {
    // Generar viewing key  nica  para este usuario
    let key = ViewingKey::create(
        deps.storage,
        &info,
        &env,
        info.sender.as_str(),
        entropy.as_bytes(),
    );

    Ok(Response::new()
        .add_attribute("action", "create_viewing_key")
        .set_data(to_binary(&CreateViewingKeyResponse { key })?))
}

// ========== QUERIES ==========

#[entry_point]
pub fn query(deps: Deps, _env: Env, msg: QueryMsg) -> StdResult<Binary> {
    match msg {
        QueryMsg::TotalSupply {} => {
            // Consulta p blica
            let state = STATE.load(deps.storage)?;
            to_binary(&TotalSupplyResponse {
                total_supply: state.total_supply,
            })
        }
        QueryMsg::Balance { address, key } => {
            // Consulta privada - requiere viewing key v lida
            query_balance(deps, address, key)
        }
        QueryMsg::TransactionHistory { address, key, page, page_size } => {
```

```
            query_transaction_history(deps, address, key, page, page_size)
        }
    }
}

fn query_balance(deps: Deps, address: String, key: String) -> StdResult<Binary> {
    let addr = deps.api.addr_validate(&address)?;

    // Verificar viewing key
    ViewingKey::check(deps.storage, &address, &key)?;

    let state = STATE.load(deps.storage)?;
    let balance = state.private_balances
        .get(&addr)
        .copied()
        .unwrap_or(Uint128::zero());

    to_binary(&BalanceResponse { balance })
}
```

Listing 4.2: Cliente para interactuar con Secret Network

```
from dataclasses import dataclass
from typing import Dict, Optional, List
import json
import hashlib
import secrets
from base64 import b64encode, b64decode

@dataclass
class SecretContract:
    """Representaci n de un contrato secreto."""
    address: str
    code_hash: str

@dataclass
class ViewingKey:
    """Viewing key para acceder a datos privados."""
    key: str
    address: str
    contract: str

class SecretNetworkClient:
    """
    Cliente para interactuar con Secret Network.
    """

    def __init__(self, endpoint: str, wallet_address: str, wallet_key: bytes):
        self.endpoint = endpoint
        self.address = wallet_address
        self.key = wallet_key
```

```
        # Almac n de viewing keys
        self.viewing_keys: Dict[str, ViewingKey] = {}

        # Cache de contratos
        self.contracts: Dict[str, SecretContract] = {}

    def encrypt_input(
        self,
        contract: SecretContract,
        msg: Dict
    ) -> bytes:
        """
␣␣␣␣␣␣␣␣Encriptar␣input␣para␣el␣contrato.
␣␣␣␣␣␣␣␣Los␣inputs␣se␣encriptan␣con␣la␣clave␣p blica␣del␣enclave.
␣␣␣␣␣␣␣␣"""
        # En producci n: usar clave p blica del enclave
        # y ECDH para derivar clave de encriptaci n

        plaintext = json.dumps(msg).encode()

        # Placeholder: en realidad se usa x25519 + ChaCha20Poly1305
        nonce = secrets.token_bytes(12)

        # Simular encriptaci n
        encrypted = self._aead_encrypt(plaintext, contract.code_hash.encode(),

        return nonce + encrypted

    def decrypt_output(
        self,
        contract: SecretContract,
        encrypted_output: bytes
    ) -> Dict:
        """
␣␣␣␣␣␣␣␣Descifrar␣output␣del␣contrato.
␣␣␣␣␣␣␣␣"""
        nonce = encrypted_output[:12]
        ciphertext = encrypted_output[12:]

        plaintext = self._aead_decrypt(ciphertext, contract.code_hash.encode()

        return json.loads(plaintext)

    # ========== TRANSACCIONES ==========

    def execute_contract(
        self,
        contract_address: str,
        msg: Dict,
        funds: Optional[List[Dict]] = None
    ) -> Dict:
```

```
        """
␣␣␣␣␣␣␣␣Ejecutar␣funci n␣del␣contrato␣con␣input␣encriptado.
␣␣␣␣␣␣␣␣"""
        contract = self._get_contract(contract_address)

        # Encriptar mensaje
        encrypted_msg = self.encrypt_input(contract, msg)

        # Construir transacci n
        tx = {
            'type': 'wasm/MsgExecuteContract',
            'sender': self.address,
            'contract': contract_address,
            'msg': b64encode(encrypted_msg).decode(),
            'sent_funds': funds or []
        }

        # Firmar y enviar (simulado)
        result = self._broadcast_tx(tx)

        return result

    def transfer_private(
        self,
        contract_address: str,
        recipient: str,
        amount: int
    ) -> Dict:
        """
␣␣␣␣␣␣␣␣Transferencia␣privada␣de␣tokens.
␣␣␣␣␣␣␣␣"""
        msg = {
            'transfer': {
                'recipient': recipient,
                'amount': str(amount)
            }
        }

        return self.execute_contract(contract_address, msg)

    # ========== VIEWING KEYS ==========

    def create_viewing_key(self, contract_address: str) -> ViewingKey:
        """
␣␣␣␣␣␣␣␣Crear␣viewing␣key␣para␣un␣contrato.
␣␣␣␣␣␣␣␣"""
        entropy = secrets.token_hex(32)

        msg = {
            'create_viewing_key': {
                'entropy': entropy
```

```
            }
        }

        result = self.execute_contract(contract_address, msg)

        # Extraer key del resultado
        key_data = result.get('data', {})
        viewing_key = ViewingKey(
            key=key_data.get('key', ''),
            address=self.address,
            contract=contract_address
        )

        self.viewing_keys[contract_address] = viewing_key

        return viewing_key

    # ========== QUERIES ==========

    def query_contract(
        self,
        contract_address: str,
        msg: Dict,
        use_viewing_key: bool = False
    ) -> Dict:
        """
        Consultar contrato.
        """
        contract = self._get_contract(contract_address)

        # A adir viewing key si es necesario
        if use_viewing_key and contract_address in self.viewing_keys:
            vk = self.viewing_keys[contract_address]
            msg['key'] = vk.key
            msg['address'] = vk.address

        # Encriptar query
        encrypted_query = self.encrypt_input(contract, msg)

        # Enviar query (simulado)
        encrypted_result = self._query(contract_address, encrypted_query)

        # Descifrar resultado
        return self.decrypt_output(contract, encrypted_result)

    def query_balance(self, contract_address: str) -> int:
        """
        Consultar balance privado (requiere viewing key).
        """
        vk = self.viewing_keys.get(contract_address)
        if not vk:
```

```
            raise ValueError("No viewing key for this contract")

        msg = {
            'balance': {
                'address': self.address,
                'key': vk.key
            }
        }

        result = self.query_contract(contract_address, msg)
        return int(result.get('balance', 0))

    def query_transaction_history(
        self,
        contract_address: str,
        page: int = 0,
        page_size: int = 10
    ) -> List[Dict]:
        """
        Consultar historial de transacciones privado.
        """
        vk = self.viewing_keys.get(contract_address)
        if not vk:
            raise ValueError("No viewing key for this contract")

        msg = {
            'transaction_history': {
                'address': self.address,
                'key': vk.key,
                'page': page,
                'page_size': page_size
            }
        }

        result = self.query_contract(contract_address, msg)
        return result.get('transactions', [])

    # ========== HELPERS ==========

    def _get_contract(self, address: str) -> SecretContract:
        """Obtener info del contrato."""
        if address not in self.contracts:
            # En producci n: query a la blockchain
            self.contracts[address] = SecretContract(
                address=address,
                code_hash=hashlib.sha256(address.encode()).hexdigest()
            )
        return self.contracts[address]

    def _aead_encrypt(self, plaintext: bytes, aad: bytes, nonce: bytes) -> bytes
        """Encriptaci n AEAD (placeholder)."""
```

```
        # En producci n: ChaCha20Poly1305
        key = hashlib.sha256(aad + nonce).digest()
        return bytes(p ^ k for p, k in zip(plaintext, key * (len(plaintext) //

    def _aead_decrypt(self, ciphertext: bytes, aad: bytes, nonce: bytes) -> by
        """Descifrado AEAD."""
        return self._aead_encrypt(ciphertext, aad, nonce)  # XOR es sim trico

    def _broadcast_tx(self, tx: Dict) -> Dict:
        """Broadcast transacci n (simulado)."""
        return {'status': 'success', 'tx_hash': secrets.token_hex(32)}

    def _query(self, contract: str, encrypted_query: bytes) -> bytes:
        """Query al contrato (simulado)."""
        # Simular respuesta encriptada
        response = json.dumps({'balance': '1000000'}).encode()
        nonce = secrets.token_bytes(12)
        contract_info = self._get_contract(contract)
        encrypted = self._aead_encrypt(response, contract_info.code_hash.encod
        return nonce + encrypted


def secret_network_demo():
    print("=== Secret Network Demo ===\n")

    # Crear cliente
    client = SecretNetworkClient(
        endpoint="https://lcd.secret.network",
        wallet_address="secret1abc...",
        wallet_key=secrets.token_bytes(32)
    )

    contract_address = "secret1xyz..."

    print("-- Creating Viewing Key --")
    vk = client.create_viewing_key(contract_address)
    print(f"Viewing key created: {vk.key[:20]}...")

    print("\n-- Private Transfer --")
    result = client.transfer_private(
        contract_address,
        recipient="secret1recipient...",
        amount=1000000
    )
    print(f"Transfer result: {result['status']}")
    print("Note: Amount and recipient NOT visible on blockchain")

    print("\n-- Query Private Balance --")
    # El balance solo es visible con viewing key
    try:
        balance = client.query_balance(contract_address)
```

```
        print(f"Balance: {balance}")
    except Exception as e:
        print(f"Query error: {e}")

    print("\n—— What's visible on blockchain ——")
    print("     Contract was called")
    print("     Who called it (sender address)")
    print("     Gas used")
    print("     Transfer amount")
    print("     Recipient")
    print("     User balances")

secret_network_demo()
```

4.2. Oasis Network

4.2.1. Arquitectura Separada

Definición 4.3 (Oasis Network)**.** Blockchain Layer-1 con arquitectura separada entre consenso y ejecución, permitiendo ParaTimes especializados incluyendo computación confidencial.

Componentes:

- **Consensus Layer:** Proof-of-Stake con Tendermint BFT
- **ParaTimes:** Entornos de ejecución paralelos
- **Cipher ParaTime:** Ejecución confidencial con TEE
- **Sapphire ParaTime:** EVM confidencial

Token nativo: ROSE

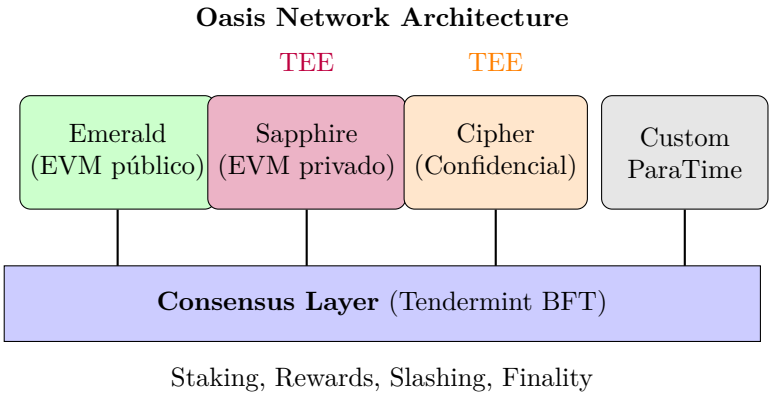


Figura 4.3: Arquitectura de Oasis: consenso separado de ejecución

4.2.2. Sapphire: EVM Confidencial

Definición 4.4 (Oasis Sapphire)**.** ParaTime compatible con EVM que proporciona computación confidencial:

Características:

- Contratos Solidity estándar

- Estado encriptado por defecto
- Llamadas confidenciales opcionales
- Compatible con herramientas Ethereum (Hardhat, Remix)
- RNG seguro on-chain

Listing 4.3: Contrato confidencial en Oasis Sapphire

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

// Imports especficos de Sapphire
import "@oasisprotocol/sapphire-contracts/contracts/Sapphire.sol";

/**
 * @title PrivateAuction
 * @notice Subasta privada donde las pujas son confidenciales hasta el cierre
 */
contract PrivateAuction {
    // ========== ESTADO PRIVADO ==========
    // En Sapphire, el estado es encriptado por defecto

    address public owner;
    string public itemDescription;
    uint256 public auctionEndTime;
    bool public ended;

    // Pujas privadas (no visibles en blockchain)
    mapping(address => uint256) private bids;
    address[] private bidders;

    // Ganador (solo revelado despus del cierre)
    address private winner;
    uint256 private winningBid;
    bool private winnerRevealed;

    // Compromiso de hash para verificacin
    mapping(address => bytes32) private bidCommitments;

    // ========== EVENTOS ==========
    // Los eventos en Sapphire pueden ser confidenciales

    event BidPlaced(address indexed bidder); // No revela monto
    event AuctionEnded(address winner, uint256 amount);

    // ========== CONSTRUCTOR ==========

    constructor(string memory _item, uint256 _duration) {
        owner = msg.sender;
        itemDescription = _item;
        auctionEndTime = block.timestamp + _duration;
```

```
}

// ========== FUNCIONES P BLICAS ==========

/**
 * @notice Realizar puja confidencial
 * @dev El monto se almacena encriptado, no visible en mempool ni estado
 */
function placeBid() external payable {
    require(block.timestamp < auctionEndTime, "Auction ended");
    require(msg.value > 0, "Bid must be positive");

    // Si ya tiene puja, a adir al total
    if (bids[msg.sender] == 0) {
        bidders.push(msg.sender);
    }

    bids[msg.sender] += msg.value;

    // Crear commitment para verificaci n posterior
    bidCommitments[msg.sender] = keccak256(
        abi.encodePacked(msg.sender, bids[msg.sender], getRandomBytes32())
    );

    emit BidPlaced(msg.sender);
}

/**
 * @notice Revelar ganador despu s del cierre
 * @dev Solo el propietario puede revelar
 */
function revealWinner() external {
    require(msg.sender == owner, "Only owner");
    require(block.timestamp >= auctionEndTime, "Auction not ended");
    require(!winnerRevealed, "Already revealed");

    // Encontrar la puja m s alta
    uint256 highestBid = 0;
    address highestBidder = address(0);

    for (uint256 i = 0; i < bidders.length; i++) {
        if (bids[bidders[i]] > highestBid) {
            highestBid = bids[bidders[i]];
            highestBidder = bidders[i];
        }
    }

    winner = highestBidder;
    winningBid = highestBid;
    winnerRevealed = true;
    ended = true;
```

```
        emit AuctionEnded(winner, winningBid);
    }

    /**
     * @notice Reclamar fondos si no ganaste
     */
    function claimRefund() external {
        require(ended, "Auction not ended");
        require(msg.sender != winner, "Winner cannot refund");

        uint256 amount = bids[msg.sender];
        require(amount > 0, "No bid to refund");

        bids[msg.sender] = 0;
        payable(msg.sender).transfer(amount);
    }

    // ========== FUNCIONES DE CONSULTA PRIVADA ==========

    /**
     * @notice Ver tu propia puja (solo t  puedes ver)
     * @dev En Sapphire, msg.sender es verificado en el enclave
     */
    function getMyBid() external view returns (uint256) {
        return bids[msg.sender];
    }

    /**
     * @notice Obtener n mero de participantes (p blico)
     */
    function getBidderCount() external view returns (uint256) {
        return bidders.length;
    }

    /**
     * @notice Obtener ganador (solo despu s de revelar)
     */
    function getWinner() external view returns (address, uint256) {
        require(winnerRevealed, "Winner not revealed yet");
        return (winner, winningBid);
    }

    // ========== FUNCIONES AUXILIARES SAPPHIRE ==========

    /**
     * @notice Generar bytes aleatorios seguros
     * @dev Sapphire proporciona RNG seguro en el enclave
     */
    function getRandomBytes32() internal view returns (bytes32) {
        return Sapphire.randomBytes(32, abi.encodePacked(block.timestamp));
```

```
    }
}

/**
 * @title PrivateVoting
 * @notice Sistema de votaci n con votos confidenciales
 */
contract PrivateVoting {
    struct Proposal {
        string description;
        uint256 yesVotes;
        uint256 noVotes;
        uint256 deadline;
        bool finalized;
    }

    mapping(uint256 => Proposal) public proposals;
    uint256 public proposalCount;

    // Votos privados
    mapping(uint256 => mapping(address => bool)) private hasVoted;
    mapping(uint256 => mapping(address => bool)) private votes; // true = yes

    // Lista de votantes elegibles
    mapping(address => bool) public eligibleVoters;

    event ProposalCreated(uint256 indexed proposalId, string description);
    event VoteCast(uint256 indexed proposalId, address voter); // No revela voto
    event ProposalFinalized(uint256 indexed proposalId, bool passed);

    constructor(address[] memory _voters) {
        for (uint256 i = 0; i < _voters.length; i++) {
            eligibleVoters[_voters[i]] = true;
        }
    }

    function createProposal(string memory description, uint256 duration) externa
        proposalCount++;
        proposals[proposalCount] = Proposal({
            description: description,
            yesVotes: 0,
            noVotes: 0,
            deadline: block.timestamp + duration,
            finalized: false
        });

        emit ProposalCreated(proposalCount, description);
    }

    /**
     * @notice Votar en propuesta (voto privado)
```

```
     * @param proposalId ID de la propuesta
     * @param vote true = s   , false = no
     */
    function vote(uint256 proposalId, bool vote) external {
        require(eligibleVoters[msg.sender], "Not eligible");
        require(!hasVoted[proposalId][msg.sender], "Already voted");
        require(block.timestamp < proposals[proposalId].deadline, "Voting ende

        hasVoted[proposalId][msg.sender] = true;
        votes[proposalId][msg.sender] = vote;

        // Incrementar conteo (pero no se revela qui n vot   qu )
        if (vote) {
            proposals[proposalId].yesVotes++;
        } else {
            proposals[proposalId].noVotes++;
        }

        emit VoteCast(proposalId, msg.sender);
    }

    /**
     * @notice Verificar si votaste (solo t   puedes ver tu voto)
     */
    function getMyVote(uint256 proposalId) external view returns (bool voted,
        return (hasVoted[proposalId][msg.sender], votes[proposalId][msg.sender
    }

    /**
     * @notice Finalizar propuesta y revelar resultado
     */
    function finalize(uint256 proposalId) external {
        Proposal storage prop = proposals[proposalId];
        require(block.timestamp >= prop.deadline, "Voting not ended");
        require(!prop.finalized, "Already finalized");

        prop.finalized = true;
        bool passed = prop.yesVotes > prop.noVotes;

        emit ProposalFinalized(proposalId, passed);
    }
}
```

Listing 4.4: Deployment y uso de contratos Sapphire con Hardhat

```
// hardhat.config.js para Oasis Sapphire
require("@oasisprotocol/sapphire-hardhat");

module.exports = {
  solidity: "0.8.20",
  networks: {
    sapphire: {
```

```
      url: "https://sapphire.oasis.io",
      accounts: [process.env.PRIVATE_KEY],
      chainId: 0x5afe,
    },
    sapphire_testnet: {
      url: "https://testnet.sapphire.oasis.dev",
      accounts: [process.env.PRIVATE_KEY],
      chainId: 0x5aff,
    },
  },
};

// deploy.js
const { ethers } = require("hardhat");

async function main() {
  console.log("Deploying to Oasis Sapphire...");

  // Deploy subasta privada
  const PrivateAuction = await ethers.getContractFactory("PrivateAuction");
  const auction = await PrivateAuction.deploy(
    "Rare NFT Collection",
    86400 // 1 d a
  );

  await auction.waitForDeployment();
  console.log("PrivateAuction deployed to:", await auction.getAddress());

  // Deploy votaci n
  const [owner, voter1, voter2, voter3] = await ethers.getSigners();

  const PrivateVoting = await ethers.getContractFactory("PrivateVoting");
  const voting = await PrivateVoting.deploy([
    voter1.address,
    voter2.address,
    voter3.address
  ]);

  await voting.waitForDeployment();
  console.log("PrivateVoting deployed to:", await voting.getAddress());
}

// interact.js - Interacci n confidencial
async function interactWithAuction() {
  const [bidder1, bidder2] = await ethers.getSigners();

  const auction = await ethers.getContractAt(
    "PrivateAuction",
    "0x..." // contract address
  );
```

```
  // Realizar pujas confidenciales
  // Las pujas NO son visibles en la blockchain ni en el mempool

  console.log("Bidder 1 placing confidential bid...");
  const tx1 = await auction.connect(bidder1).placeBid({
    value: ethers.parseEther("1.5")
  });
  await tx1.wait();

  console.log("Bidder 2 placing confidential bid...");
  const tx2 = await auction.connect(bidder2).placeBid({
    value: ethers.parseEther("2.0")
  });
  await tx2.wait();

  // Cada usuario solo puede ver su propia puja
  const myBid1 = await auction.connect(bidder1).getMyBid();
  console.log(`Bidder 1's bid: ${ethers.formatEther(myBid1)} ETH`);

  // Intentar ver puja de otro usuario falla
  // (Sapphire verifica msg.sender en el enclave)

  // El n mero de participantes es p blico
  const count = await auction.getBidderCount();
  console.log(`Total bidders: ${count}`);

  // Pero los montos individuales son privados hasta el cierre
  console.log("Individual bid amounts: [ENCRYPTED]");
}

main().catch(console.error);
```

4.2.3. Comparación Secret Network vs Oasis

Aspecto	Secret Network	Oasis Sapphire
Lenguaje	Rust (CosmWasm modificado)	Solidity estándar
Compatibilidad	Cosmos ecosystem (IBC)	Ethereum ecosystem (EVM)
TEE	Intel SGX	Intel SGX
Privacidad	Todo privado por defecto	Configurable por función
Viewing Keys	Sí, sistema elaborado	Limitado (msg.sender check)
Tooling	SecretJS, custom tools	Hardhat, Remix, Ethers.js
Madurez	Más maduro (2020)	Más reciente (2022)
DeFi	Secret DeFi (ShadeSwap)	Emergente

Tabla 4.1: Comparación entre Secret Network y Oasis Sapphire

4.3. Aleo

4.3.1. ZK-Native Blockchain

Definición 4.5 (Aleo)**.** Blockchain nativa en zero-knowledge donde toda la ejecución de programas genera pruebas ZK, sin depender de hardware de confianza.

Características principales:

- **Modelo de Records:** Similar a notes de Zcash
- **Lenguaje Leo:** DSL similar a Rust para ZK
- **snarkVM:** Máquina virtual que ejecuta y genera pruebas
- **Sin TEE:** Privacidad puramente criptográfica
- **Token nativo:** ALEO

Idea Clave

Diferencia fundamental: ZK vs TEE
TEE (Secret, Oasis):

- Estado encriptado almacenado en blockchain
- Ejecución en hardware de confianza
- Vulnerable a ataques al hardware
- Baja latencia de ejecución

ZK (Aleo):

- No hay estado en blockchain (solo commitments)
- Ejecución local, verificación on-chain
- Seguridad puramente matemática
- Mayor costo computacional (generación de proof)

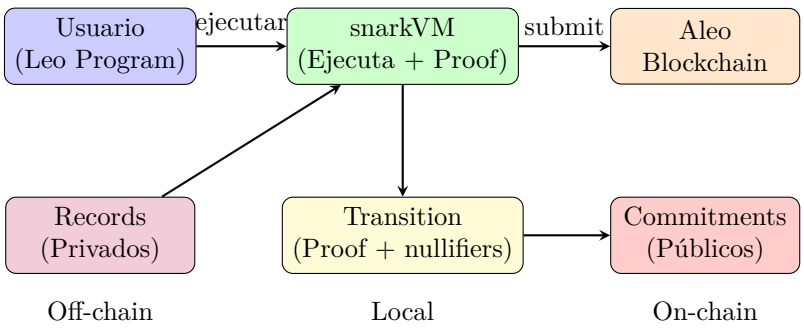


Figura 4.4: Arquitectura de Aleo: ejecución off-chain, verificación on-chain

4.3.2. Modelo de Records

Definición 4.6 (Record en Aleo)**.** Estructura de datos que representa estado privado, similar a las notes de Zcash:

$$\text{Record} = \{\text{owner}, \text{data}, \text{nonce}\}$$

Propiedades:

- **Propiedad:** Solo el owner puede consumir el record
- **Unicidad:** Cada record tiene nonce único
- **Consumo:** Records se "gastan" al usarse (modelo UTXO)
- **Privacidad:** Contenido encriptado al owner

[Transiciones de Estado en Aleo] Una transición consume records de entrada y produce records de salida:

$$\text{Transition} : \{R_{\text{in}_1}, ..., R_{\text{in}_n}\} \rightarrow \{R_{\text{out}_1}, ..., R_{\text{out}_m}\}$$

Lo que se publica on-chain:

- Nullifiers de records consumidos: $\{\text{nf}(R_{\text{in}_i})\}$
- Commitments de records creados: $\{\text{cm}(R_{\text{out}_j})\}$
- ZK Proof de ejecución correcta

Lo que permanece privado:

- Contenido de records (owner, data)
- Mapping entre inputs y outputs
- Lógica específica ejecutada

4.3.3. Lenguaje Leo

Listing 4.5: Programa Leo: Token privado

```
// token.leo - Token privado en Aleo

program token.aleo {
    // Record que representa balance de token
    record Token {
        owner: address,
        amount: u64,
    }

    // Mapping p blico para supply total (opcional)
    mapping total_supply: u8 => u64;

    // ========== FUNCIONES DE TRANSICI N ==========

    // Mint: crear nuevos tokens (solo admin)
    transition mint_private(
        receiver: address,
```

```
    amount: u64
) -> Token {
    // Verificar autorizaci n (simplificado)
    assert(self.caller == aleo1admin...);

    // Crear record para el receptor
    return Token {
        owner: receiver,
        amount: amount,
    };
}

// Transfer: transferencia privada
transition transfer_private(
    token: Token,           // Record de entrada
    receiver: address,
    amount: u64
) -> (Token, Token) {       // Records de salida
    // Verificar que el caller es due o
    assert_eq(token.owner, self.caller);

    // Verificar balance suficiente
    assert(token.amount >= amount);

    // Calcular cambio
    let change: u64 = token.amount - amount;

    // Record para el receptor
    let recipient_token: Token = Token {
        owner: receiver,
        amount: amount,
    };

    // Record de cambio para el sender
    let sender_token: Token = Token {
        owner: self.caller,
        amount: change,
    };

    // Retornar ambos records
    // El record de entrada se consume (nullifier publicado)
    return (recipient_token, sender_token);
}

// Split: dividir un record en dos
transition split(
    token: Token,
    amount1: u64
) -> (Token, Token) {
    assert_eq(token.owner, self.caller);
    assert(token.amount >= amount1);
```

```
        let amount2: u64 = token.amount - amount1;

        let token1: Token = Token {
            owner: self.caller,
            amount: amount1,
        };

        let token2: Token = Token {
            owner: self.caller,
            amount: amount2,
        };

        return (token1, token2);
    }

    // Join: combinar dos records en uno
    transition join(
        token1: Token,
        token2: Token
    ) -> Token {
        assert_eq(token1.owner, self.caller);
        assert_eq(token2.owner, self.caller);

        let total: u64 = token1.amount + token2.amount;

        return Token {
            owner: self.caller,
            amount: total,
        };
    }
}
```

Listing 4.6: Leo: Subasta privada con ZK

```
// private_auction.leo - Subasta donde las pujas son privadas

program private_auction.aleo {
    // Record de puja (privado)
    record Bid {
        owner: address,
        auction_id: field,
        amount: u64,
        timestamp: u64,
    }

    // Record de ganador (revelado solo al ganador)
    record WinnerClaim {
        owner: address,
        auction_id: field,
        winning_amount: u64,
    }
```

```
// Estado p blico de la subasta
struct AuctionState {
    item_hash: field,
    end_time: u64,
    highest_bid_commitment: field,
    bid_count: u64,
    is_finalized: bool,
}

mapping auctions: field => AuctionState;
mapping bid_commitments: field => bool;  // commitment => exists

// ========== CREAR SUBASTA ==========

transition create_auction(
    public item_description: field,
    public duration: u64
) -> field {
    // Generar ID  nico
    let auction_id: field = BHP256::hash_to_field(
        item_description + self.caller.to_field() + duration as field
    );

    return auction_id then finalize(auction_id, item_description, duration);
}

finalize create_auction(
    auction_id: field,
    item_hash: field,
    duration: u64
) {
    let state: AuctionState = AuctionState {
        item_hash: item_hash,
        end_time: block.height + duration,
        highest_bid_commitment: 0field,
        bid_count: 0u64,
        is_finalized: false,
    };

    auctions.set(auction_id, state);
}

// ========== REALIZAR PUJA PRIVADA ==========

transition place_bid(
    auction_id: field,
    amount: u64,
    secret: field  // Salt para el commitment
) -> Bid {
    // Crear commitment de la puja (p blico)
```

```
    let bid_commitment: field = Poseidon2::hash_to_field(
        self.caller.to_field() + amount as field + secret
    );

    // Crear record privado de la puja
    let bid: Bid = Bid {
        owner: self.caller,
        auction_id: auction_id,
        amount: amount,
        timestamp: 0u64,  // Se actualiza en finalize
    };

    return bid then finalize(auction_id, bid_commitment);
}

finalize place_bid(
    auction_id: field,
    bid_commitment: field
) {
    // Verificar que la subasta existe y est  activa
    let state: AuctionState = auctions.get(auction_id);
    assert(!state.is_finalized);
    assert(block.height < state.end_time);

    // Registrar commitment
    bid_commitments.set(bid_commitment, true);

    // Actualizar conteo
    let updated_state: AuctionState = AuctionState {
        item_hash: state.item_hash,
        end_time: state.end_time,
        highest_bid_commitment: state.highest_bid_commitment,
        bid_count: state.bid_count + 1u64,
        is_finalized: false,
    };

    auctions.set(auction_id, updated_state);
}

// ========== REVELAR Y RECLAMAR ==========

// El ganador prueba que tiene la puja m s alta
transition claim_winner(
    bid: Bid,
    secret: field,
    all_other_bids: [u64; 10]  // Otras pujas (debe probar que la suya es
) -> WinnerClaim {
    // Verificar que todas las otras pujas son menores
    for i: u8 in 0u8..10u8 {
        assert(bid.amount > all_other_bids[i] || all_other_bids[i] == 0u64
    }
```

```
        return WinnerClaim {
            owner: bid.owner,
            auction_id: bid.auction_id,
            winning_amount: bid.amount,
        } then finalize(bid.auction_id);
    }

    finalize claim_winner(auction_id: field) {
        let state: AuctionState = auctions.get(auction_id);
        assert(block.height >= state.end_time);
        assert(!state.is_finalized);

        let final_state: AuctionState = AuctionState {
            item_hash: state.item_hash,
            end_time: state.end_time,
            highest_bid_commitment: state.highest_bid_commitment,
            bid_count: state.bid_count,
            is_finalized: true,
        };

        auctions.set(auction_id, final_state);
    }
}
```

4.4. Mina Protocol

4.4.1. Blockchain Sucinta

Definición 4.7 (Mina Protocol)**.** Blockchain con tamaño constante de 22 KB gracias a recursive SNARKs, donde toda la historia de la cadena se comprime en una sola prueba.

Características:

- **Tamaño constante:** Siempre 22 KB independiente del historial
- **Recursive SNARKs:** Pruebas que verifican otras pruebas
- **zkApps:** Smart contracts con privacidad
- **Off-chain execution:** Estado off-chain, proofs on-chain
- **o1js:** SDK TypeScript/JavaScript para desarrollo

[Recursive SNARKs en Mina] **Idea clave:** Un SNARK puede verificar otro SNARK.

Prueba de estado de blockchain:

$$\pi_n = \text{SNARK.Prove}\left(\pi_{n-1} \text{ es válida} \wedge B_n \text{ es bloque válido}\right)$$

Verificación:

- Verificar π_n toma tiempo constante
- π_n implica que TODOS los bloques anteriores son válidos
- No necesitas descargar la historia completa

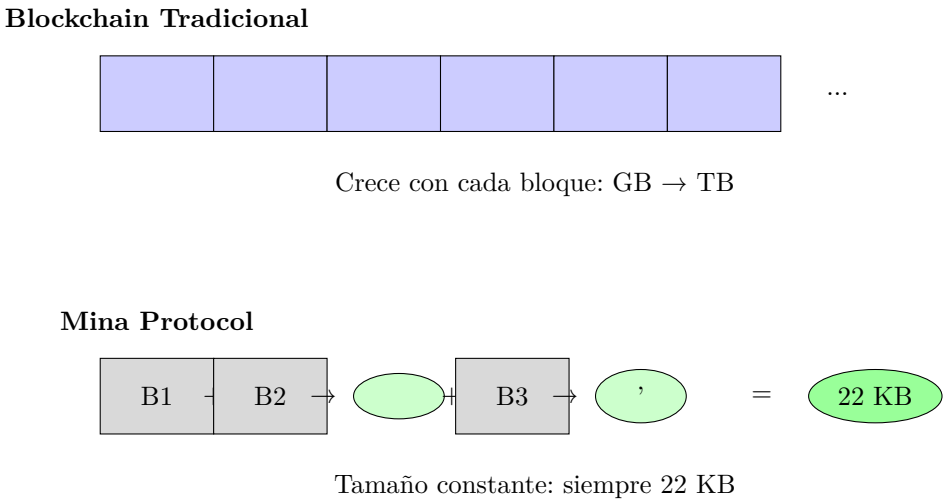


Figura 4.5: Mina comprime toda la blockchain en una prueba de 22 KB

Implicaciones:

- Light clients verdaderos (22 KB = estado completo)
- Sincronización instantánea
- Escalabilidad extrema para verificadores

4.4.2. zkApps: Smart Contracts con ZK

Definición 4.8 (zkApp)**.** Smart contract en Mina donde:

- Estado almacenado off-chain (Merkle tree)
- Solo root del estado on-chain
- Transiciones de estado prueban correctitud con ZK
- Ejecución local, verificación on-chain

Listing 4.7: zkApp básico con o1js

```
import {
  Field,
  SmartContract,
  state,
  State,
  method,
  Poseidon,
  PublicKey,
  Signature,
  UInt64,
  Struct,
  MerkleWitness,
  Bool,
  CircuitString,
  Provable,
} from 'o1js';
```

```
// Altura del Merkle tree para balances
const TREE_HEIGHT = 8;
class BalanceWitness extends MerkleWitness(TREE_HEIGHT) {}

// Estructura para balance de usuario (off-chain)
class UserBalance extends Struct({
  publicKey: PublicKey,
  balance: UInt64,
  nonce: Field,
}) {
  hash(): Field {
    return Poseidon.hash([
      ...this.publicKey.toFields(),
      this.balance.value,
      this.nonce,
    ]);
  }
}

/**
 * PrivateToken: Token con balances privados
 * - Balances almacenados off-chain en Merkle tree
 * - Solo el root est  on-chain
 * - Transferencias prueban actualizaci n correcta del tree
 */
class PrivateToken extends SmartContract {
  // Root del Merkle tree de balances (on-chain)
  @state(Field) balancesRoot = State<Field>();

  // Supply total (p blico)
  @state(UInt64) totalSupply = State<UInt64>();

  // Nullifier set root (para prevenir double-spending)
  @state(Field) nullifiersRoot = State<Field>();

  init() {
    super.init();
    // Inicializar con  rbol  vac o
    this.balancesRoot.set(Field(0));
    this.totalSupply.set(UInt64.zero);
    this.nullifiersRoot.set(Field(0));
  }

  /**
   * Transferencia privada
   * - Sender y receiver prueban posesi n de sus balances
   * - Se actualiza el root del tree
   * - Montos y direcciones no visibles on-chain
   */
  @method async transfer(
```

```
    // Datos del sender (privados)
    senderBalance: UserBalance,
    senderWitness: BalanceWitness,
    senderSignature: Signature,

    // Datos del receiver (privados)
    receiverBalance: UserBalance,
    receiverWitness: BalanceWitness,

    // Monto (privado)
    amount: UInt64,

    // Nuevo balance sender
    newSenderBalance: UserBalance,

    // Nuevo balance receiver
    newReceiverBalance: UserBalance
  ) {
    // 1. Verificar root actual
    const currentRoot = this.balancesRoot.get();
    this.balancesRoot.requireEquals(currentRoot);

    // 2. Verificar que sender tiene el balance que dice
    const senderLeafHash = senderBalance.hash();
    const computedSenderRoot = senderWitness.calculateRoot(senderLeafHash);
    computedSenderRoot.assertEquals(currentRoot);

    // 3. Verificar firma del sender
    senderSignature.verify(
      senderBalance.publicKey,
      [amount.value, senderBalance.nonce]
    ).assertTrue();

    // 4. Verificar balance suficiente
    senderBalance.balance.assertGreaterThanOrEqual(amount);

    // 5. Verificar nuevos balances correctos
    newSenderBalance.balance.assertEquals(
      senderBalance.balance.sub(amount)
    );
    newReceiverBalance.balance.assertEquals(
      receiverBalance.balance.add(amount)
    );

    // 6. Verificar nonce incrementado
    newSenderBalance.nonce.assertEquals(senderBalance.nonce.add(1));

    // 7. Calcular nuevo root con balances actualizados
    // (Simplificado - en realidad se actualizan ambas hojas)
    const newSenderLeafHash = newSenderBalance.hash();
    const intermediateRoot = senderWitness.calculateRoot(newSenderLeafHash);
```

```
    const newReceiverLeafHash = newReceiverBalance.hash();
    const newRoot = receiverWitness.calculateRoot(newReceiverLeafHash);

    // 8. Actualizar root on-chain
    this.balancesRoot.set(newRoot);
  }

  /**
   * Mint privado (solo admin)
   */
  @method async mint(
    receiver: PublicKey,
    amount: UInt64,
    currentReceiverBalance: UserBalance,
    witness: BalanceWitness,
    newReceiverBalance: UserBalance
  ) {
    // Verificar admin (simplificado)
    this.sender.getAndRequireSignature().assertEquals(this.address);

    // Verificar balance actual en tree
    const currentRoot = this.balancesRoot.get();
    this.balancesRoot.requireEquals(currentRoot);

    const leafHash = currentReceiverBalance.hash();
    witness.calculateRoot(leafHash).assertEquals(currentRoot);

    // Verificar nuevo balance correcto
    newReceiverBalance.balance.assertEquals(
      currentReceiverBalance.balance.add(amount)
    );

    // Actualizar root
    const newLeafHash = newReceiverBalance.hash();
    const newRoot = witness.calculateRoot(newLeafHash);
    this.balancesRoot.set(newRoot);

    // Actualizar supply
    const currentSupply = this.totalSupply.get();
    this.totalSupply.requireEquals(currentSupply);
    this.totalSupply.set(currentSupply.add(amount));
  }
}

/**
 * PrivateVoting: Votaci n con votos confidenciales
 */
class PrivateVoting extends SmartContract {
  // Merkle root de votantes elegibles
  @state(Field) votersRoot = State<Field>();
```

```
// Conteos de votos (p blicos, pero no qui n vot  qu )
@state(UInt64) yesVotes = State<UInt64>();
@state(UInt64) noVotes = State<UInt64>();

// Nullifier root (para prevenir doble voto)
@state(Field) nullifiersRoot = State<Field>();

@method async vote(
  // Prueba de elegibilidad (privada)
  voterPublicKey: PublicKey,
  eligibilityWitness: BalanceWitness,

  // Voto (privado)
  voteValue: Bool, // true = yes, false = no

  // Nullifier (para prevenir doble voto)
  nullifier: Field,
  nullifierWitness: BalanceWitness
) {
  // 1. Verificar elegibilidad
  const votersRoot = this.votersRoot.get();
  this.votersRoot.requireEquals(votersRoot);

  const voterHash = Poseidon.hash(voterPublicKey.toFields());
  eligibilityWitness.calculateRoot(voterHash).assertEquals(votersRoot);

  // 2. Verificar que no ha votado (nullifier no existe)
  const nullifiersRoot = this.nullifiersRoot.get();
  this.nullifiersRoot.requireEquals(nullifiersRoot);

  // El nullifier debe llevar a hoja vac a
  nullifierWitness.calculateRoot(Field(0)).assertEquals(nullifiersRoot);

  // 3. Calcular nuevo nullifier root
  const newNullifiersRoot = nullifierWitness.calculateRoot(nullifier);
  this.nullifiersRoot.set(newNullifiersRoot);

  // 4. Actualizar conteos
  const currentYes = this.yesVotes.get();
  const currentNo = this.noVotes.get();
  this.yesVotes.requireEquals(currentYes);
  this.noVotes.requireEquals(currentNo);

  // Increment condicionalmente basado en voto
  const newYes = Provable.if(voteValue, currentYes.add(1), currentYes);
  const newNo = Provable.if(voteValue, currentNo, currentNo.add(1));

  this.yesVotes.set(newYes);
  this.noVotes.set(newNo);
}
```

```
}

// Ejemplo de uso
async function main() {
  console.log('Compiling PrivateToken zkApp...');
  const { verificationKey } = await PrivateToken.compile();

  console.log('Verification key size:', verificationKey.data.length);
  console.log('This proof verifies the entire contract logic!');
}
```

4.5. Fhenix y FHE

4.5.1. Fully Homomorphic Encryption

Definición 4.9 (Fully Homomorphic Encryption (FHE))**.** Sistema de encriptación que permite realizar operaciones sobre datos cifrados sin descifrarlos:

$$\text{Dec}(f(\text{Enc}(x), \text{Enc}(y))) = f(x, y)$$

Operaciones homomórficas:

- **Suma:** $\text{Enc}(x) \oplus \text{Enc}(y) = \text{Enc}(x + y)$
- **Multiplicación:** $\text{Enc}(x) \otimes \text{Enc}(y) = \text{Enc}(x \times y)$
- Con suma y multiplicación, se puede computar cualquier función

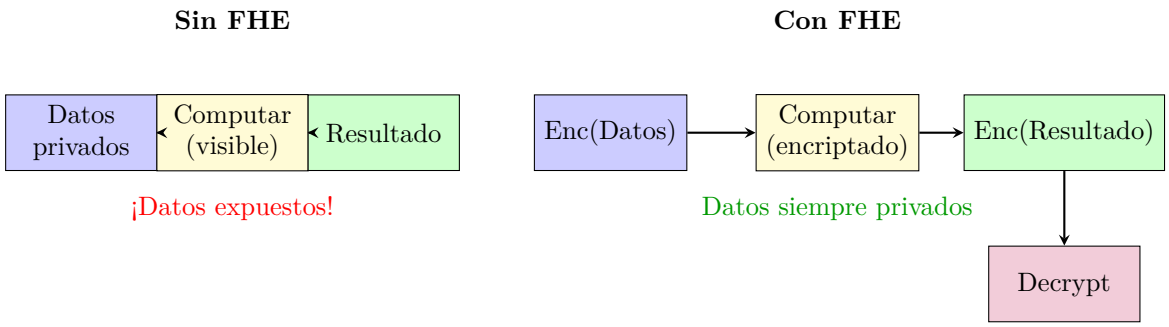


Figura 4.6: FHE permite computar sobre datos encriptados

[Esquemas FHE Modernos] **1. TFHE (Torus FHE):**

- Operaciones sobre bits individuales
- Bootstrap rápido (10ms)
- Usado por Fhenix

2. BGV/BFV:

- Operaciones sobre enteros
- Más eficiente para aritmética
- Usado en muchas implementaciones

3. CKKS:

- Operaciones sobre números reales (aproximados)
- Útil para machine learning
- Menor precisión pero más flexible

Limitaciones actuales:

- Muy costoso computacionalmente (1000-10000x más lento)
- Ciphertexts grandes (KB por dato simple)
- "Noise" acumulativo requiere bootstrapping

Definición 4.10 (Fhenix)**.** Blockchain Layer-2 que integra FHE para smart contracts completamente encriptados:

Características:

- Estado del contrato encriptado
- Computación sobre datos encriptados
- Sin necesidad de descifrar durante ejecución
- Basado en TFHE
- EVM compatible (modificado)

Listing 4.8: Conceptual: Smart contract con FHE (estilo Fhenix)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

// Tipos FHE (proporcionados por la blockchain)
// euint8, euint16, euint32, euint64 = encrypted unsigned integers
// ebool = encrypted boolean

import "@fhenix/contracts/FHE.sol";

/**
 * @title FHEToken
 * @notice Token donde TODOS los balances est n encriptados
 * @dev Incluso el contrato no conoce los balances en claro
 */
contract FHEToken {
    // Balances encriptados
    mapping(address => euint64) private encryptedBalances;

    // Supply total (puede ser p blico o encriptado)
    euint64 private encryptedTotalSupply;

    // ========== OPERACIONES SOBRE DATOS ENCRIPTADOS ==========

    /**
     * @notice Transferencia con montos encriptados
```

```
     * @param to Destinatario
     * @param encryptedAmount Monto encriptado (solo el sender conoce el valor r
     */
    function transfer(address to, inEuint64 calldata encryptedAmount) external {
        euint64 amount = FHE.asEuint64(encryptedAmount);

        // Verificar balance suficiente (comparaci n encriptada)
        // Retorna ebool - no sabemos el resultado, pero el circuito s
        ebool hasEnough = FHE.gte(encryptedBalances[msg.sender], amount);

        // Require encriptado - falla si la condici n es false
        // Pero no revela cu l es el balance ni el monto
        FHE.req(hasEnough);

        // Restar del sender (operaci n sobre encriptados)
        encryptedBalances[msg.sender] = FHE.sub(
            encryptedBalances[msg.sender],
            amount
        );

        // Sumar al receiver
        encryptedBalances[to] = FHE.add(
            encryptedBalances[to],
            amount
        );
    }

    /**
     * @notice Verificar si tienes suficiente balance (sin revelar cu nto)
     * @param encryptedThreshold Umbral encriptado a verificar
     * @return ebool Resultado encriptado (solo t  puedes descifrarlo)
     */
    function hasMinimumBalance(
        inEuint64 calldata encryptedThreshold
    ) external view returns (ebool) {
        euint64 threshold = FHE.asEuint64(encryptedThreshold);
        return FHE.gte(encryptedBalances[msg.sender], threshold);
    }

    /**
     * @notice Obtener tu balance encriptado (para descifrar localmente)
     */
    function getMyEncryptedBalance() external view returns (euint64) {
        // Requiere que el caller sea el due o
        // El resultado solo puede ser descifrado por el caller
        return FHE.seal(encryptedBalances[msg.sender], msg.sender);
    }

    /**
     * @notice Mint tokens (admin only)
     */
```

```
    function mint(address to, inEuint64 calldata encryptedAmount) external {
        // Solo admin...
        euint64 amount = FHE.asEuint64(encryptedAmount);

        encryptedBalances[to] = FHE.add(encryptedBalances[to], amount);
        encryptedTotalSupply = FHE.add(encryptedTotalSupply, amount);
    }
}

/**
 * @title FHEAuction
 * @notice Subasta donde nadie (ni siquiera el contrato) conoce las pujas
 */
contract FHEAuction {
    address public owner;
    uint256 public endTime;
    bool public ended;

    // Pujas encriptadas
    mapping(address => euint64) private encryptedBids;
    address[] private bidders;

    // Puja m s alta actual (encriptada)
    euint64 private highestBid;
    address private highestBidder; // Se conoce, pero no el monto

    constructor(uint256 duration) {
        owner = msg.sender;
        endTime = block.timestamp + duration;
    }

    /**
     * @notice Realizar puja encriptada
     */
    function bid(inEuint64 calldata encryptedBidAmount) external payable {
        require(block.timestamp < endTime, "Auction ended");

        euint64 bidAmount = FHE.asEuint64(encryptedBidAmount);

        // Si ya ten a puja, sumar
        if (encryptedBids[msg.sender].length > 0) {
            encryptedBids[msg.sender] = FHE.add(
                encryptedBids[msg.sender],
                bidAmount
            );
        } else {
            encryptedBids[msg.sender] = bidAmount;
            bidders.push(msg.sender);
        }

        // Actualizar l der si esta puja es mayor
```

```
        ebool isHigher = FHE.gt(encryptedBids[msg.sender], highestBid);

        // Actualizaci n condicional encriptada
        highestBid = FHE.select(
            isHigher,
            encryptedBids[msg.sender],
            highestBid
        );

        // Nota: highestBidder se actualiza, pero esto revela
        // que hubo un nuevo l der (aunque no el monto)
    }

    /**
     * @notice Finalizar y revelar ganador
     * @dev Solo el monto del ganador se revela al final
     */
    function finalize() external {
        require(block.timestamp >= endTime, "Not ended");
        require(!ended, "Already finalized");

        ended = true;

        // Descifrar solo la puja ganadora
        // (requiere interacci n con servicio de descifrado)
    }

    /**
     * @notice Ver tu puja (solo t  puedes descifrar)
     */
    function getMyBid() external view returns (euint64) {
        return FHE.seal(encryptedBids[msg.sender], msg.sender);
    }
}
```

4.5.2. Comparación de Aproximaciones

Aspecto	Secret/Oasis	Aleo	Mina	Fhenix
Tecnología	TEE (SGX)	ZK Proofs	Recursive ZK	FHE
Trust Model	Hardware	Criptografía	Criptografía	Criptografía
Estado	Encriptado on-chain	Off-chain/Records	Off-chain	Encriptado on-chain
Latencia	Baja	Media	Media	Alta
Costo	Bajo	Alto (proving)	Medio	Muy alto
Madurez	Alta	Media	Media	Experimental
EVM Compat	Sapphire: Sí	No (Leo)	No (o1js)	Modificado
Riesgo	Side-channels	Ninguno conocido	Ninguno conocido	Ninguno conocido

Tabla 4.2: Comparación de plataformas de smart contracts privados

4.6. Private Function Calls

4.6.1. Modelos de Ejecución Privada

Definición 4.11 (Private Function Call)**.** Ejecución de una función de smart contract donde los inputs, outputs, o ambos permanecen confidenciales:

$$\text{PrivateCall}(f, x) \to \{y, \pi\}$$

donde y puede ser público o privado, y π es una prueba de ejecución correcta.

Niveles de privacidad:

- **Input privacy:** x oculto, y público
- **Output privacy:** x público, y oculto
- **Full privacy:** x y y ocultos
- **Function privacy:** Incluso qué función se ejecutó es privado

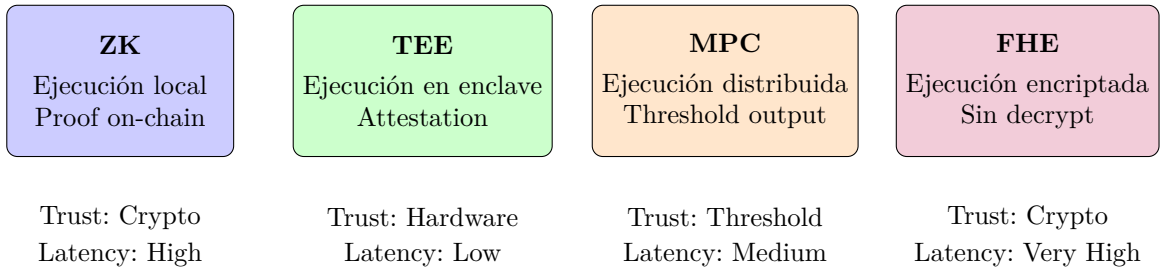


Figura 4.7: Cuatro modelos de ejecución privada de funciones

Listing 4.9: Framework para private function calls

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Any, Dict, List, Optional, Tuple
from enum import Enum
import hashlib
import secrets

class PrivacyLevel(Enum):
    INPUT_PRIVATE = "input_private"
    OUTPUT_PRIVATE = "output_private"
    FULL_PRIVATE = "full_private"
    FUNCTION_PRIVATE = "function_private"

class ExecutionModel(Enum):
    ZK = "zero_knowledge"
    TEE = "trusted_execution"
    MPC = "multi_party_computation"
    FHE = "fully_homomorphic"

@dataclass
class ExecutionResult:
    output: Any
```

```
    proof: Optional[bytes]
    public_inputs: List[Any]
    execution_model: ExecutionModel
    privacy_level: PrivacyLevel

class PrivateFunctionExecutor(ABC):
    """
␣␣␣␣Clase␣base␣abstracta␣para␣ejecutores␣de␣funciones␣privadas.
␣␣␣␣"""

    @abstractmethod
    def execute(
        self,
        function_name: str,
        private_inputs: Dict[str, Any],
        public_inputs: Dict[str, Any]
    ) -> ExecutionResult:
        pass

    @abstractmethod
    def verify(self, result: ExecutionResult) -> bool:
        pass


class ZKExecutor(PrivateFunctionExecutor):
    """
␣␣␣␣Ejecutor␣basado␣en␣Zero-Knowledge␣Proofs.
␣␣␣␣-␣Ejecuci n␣local
␣␣␣␣-␣Genera␣proof␣de␣ejecuci n␣correcta
␣␣␣␣-␣Verifier␣on-chain␣no␣ve␣inputs␣privados
␣␣␣␣"""

    def __init__(self, circuit_path: str):
        self.circuit_path = circuit_path
        self.proving_key = None
        self.verification_key = None

    def setup(self):
        """Generar␣proving␣y␣verification␣keys."""
        # En producci n: usar snarkjs, arkworks, etc.
        self.proving_key = secrets.token_bytes(64)
        self.verification_key = secrets.token_bytes(32)

    def execute(
        self,
        function_name: str,
        private_inputs: Dict[str, Any],
        public_inputs: Dict[str, Any]
    ) -> ExecutionResult:
        # 1. Ejecutar funci n localmente
        output = self._execute_locally(function_name, private_inputs, public_inp
```

```
        # 2. Generar witness (private + public inputs + output)
        witness = self._generate_witness(private_inputs, public_inputs, output

        # 3. Generar ZK proof
        proof = self._generate_proof(witness)

        return ExecutionResult(
            output=output,
            proof=proof,
            public_inputs=list(public_inputs.values()) + [output],
            execution_model=ExecutionModel.ZK,
            privacy_level=PrivacyLevel.INPUT_PRIVATE
        )

    def verify(self, result: ExecutionResult) -> bool:
        """Verificar ZK proof."""
        # En producci n: verificaci n SNARK real
        return result.proof is not None and len(result.proof) == 128

    def _execute_locally(
        self,
        function_name: str,
        private_inputs: Dict,
        public_inputs: Dict
    ) -> Any:
        """Ejecutar la funci n localmente."""
        # Simulaci n de ejecuci n
        all_inputs = {**private_inputs, **public_inputs}

        if function_name == "transfer":
            balance = all_inputs.get('balance', 0)
            amount = all_inputs.get('amount', 0)
            return balance - amount  # Nuevo balance

        return None

    def _generate_witness(
        self,
        private_inputs: Dict,
        public_inputs: Dict,
        output: Any
    ) -> bytes:
        """Generar witness para el circuito."""
        data = str(private_inputs) + str(public_inputs) + str(output)
        return hashlib.sha256(data.encode()).digest()

    def _generate_proof(self, witness: bytes) -> bytes:
        """Generar ZK proof (simulado)."""
        return hashlib.sha256(witness + self.proving_key).digest() * 4
```

```
class TEEExecutor(PrivateFunctionExecutor):
    """
    Ejecutor basado en Trusted Execution Environment.
    - Ejecuci n en enclave SGX/SEV
    - Attestation del enclave
    - Baja latencia, trust en hardware
    """

    def __init__(self, enclave_id: str):
        self.enclave_id = enclave_id
        self.enclave_key = secrets.token_bytes(32)

    def execute(
        self,
        function_name: str,
        private_inputs: Dict[str, Any],
        public_inputs: Dict[str, Any]
    ) -> ExecutionResult:
        # 1. Enviar datos encriptados al enclave
        encrypted_inputs = self._encrypt_for_enclave(private_inputs)

        # 2. Ejecutar en enclave (simulado)
        output, attestation = self._execute_in_enclave(
            function_name,
            encrypted_inputs,
            public_inputs
        )

        return ExecutionResult(
            output=output,
            proof=attestation,
            public_inputs=list(public_inputs.values()),
            execution_model=ExecutionModel.TEE,
            privacy_level=PrivacyLevel.FULL_PRIVATE
        )

    def verify(self, result: ExecutionResult) -> bool:
        """Verificar attestation del enclave."""
        # En producci n: verificar con Intel Attestation Service
        return result.proof is not None and len(result.proof) == 96

    def _encrypt_for_enclave(self, data: Dict) -> bytes:
        """Encriptar datos para el enclave."""
        plaintext = str(data).encode()
        # En producci n: ECDH + AES-GCM
        return bytes(b ^ k for b, k in zip(plaintext, self.enclave_key * 10))

    def _execute_in_enclave(
        self,
        function_name: str,
```

```
        encrypted_inputs: bytes,
        public_inputs: Dict
    ) -> Tuple[Any, bytes]:
        """Simular ejecuci n en enclave."""
        # En producci n: llamada real a enclave SGX

        # Ejecutar funci n
        output = "execution_result"

        # Generar attestation
        attestation = hashlib.sha256(
            self.enclave_id.encode() +
            encrypted_inputs +
            str(output).encode()
        ).digest() * 3

        return output, attestation


class MPCExecutor(PrivateFunctionExecutor):
    """
    Ejecutor basado en Multi-Party Computation.
    - Ejecuci n distribuida entre m ltiples partes
    - Ninguna parte ve los inputs completos
    - Threshold de partes honestas
    """

    def __init__(self, parties: List[str], threshold: int):
        self.parties = parties
        self.threshold = threshold
        self.num_parties = len(parties)

    def execute(
        self,
        function_name: str,
        private_inputs: Dict[str, Any],
        public_inputs: Dict[str, Any]
    ) -> ExecutionResult:
        # 1. Secret-share los inputs entre partes
        shares = self._secret_share_inputs(private_inputs)

        # 2. Cada parte procesa su share
        partial_results = []
        for i, party in enumerate(self.parties):
            partial = self._party_compute(party, shares[i], function_name)
            partial_results.append(partial)

        # 3. Combinar resultados (threshold)
        output = self._combine_results(partial_results)

        # 4. Generar proof de correctitud
```

```
        proof = self._generate_mpc_proof(partial_results)

        return ExecutionResult(
            output=output,
            proof=proof,
            public_inputs=list(public_inputs.values()),
            execution_model=ExecutionModel.MPC,
            privacy_level=PrivacyLevel.FULL_PRIVATE
        )

    def verify(self, result: ExecutionResult) -> bool:
        """Verificar que al menos threshold partes participaron."""
        return result.proof is not None

    def _secret_share_inputs(self, inputs: Dict) -> List[Dict]:
        """Dividir inputs en shares."""
        shares = []
        for i in range(self.num_parties):
            party_share = {}
            for key, value in inputs.items():
                if isinstance(value, int):
                    # Shamir secret sharing (simplificado)
                    party_share[key] = value + i * secrets.randbelow(1000)
            shares.append(party_share)
        return shares

    def _party_compute(
        self,
        party: str,
        share: Dict,
        function_name: str
    ) -> Dict:
        """Computaci n de una parte."""
        return {'party': party, 'partial_result': sum(share.values())}

    def _combine_results(self, partial_results: List[Dict]) -> Any:
        """Combinar resultados de las partes."""
        # Lagrange interpolation (simplificado)
        total = sum(r['partial_result'] for r in partial_results)
        return total // len(partial_results)

    def _generate_mpc_proof(self, partial_results: List[Dict]) -> bytes:
        """Generar proof de MPC correcto."""
        data = str(partial_results)
        return hashlib.sha256(data.encode()).digest()


def private_function_demo():
    print("=== Private Function Calls Demo ===\n")

    # 1. ZK Executor
```

```
    print("── ZK Executor ──")
    zk_exec = ZKExecutor("transfer_circuit.r1cs")
    zk_exec.setup()

    result_zk = zk_exec.execute(
        function_name="transfer",
        private_inputs={'balance': 1000, 'secret_key': 'abc123'},
        public_inputs={'amount': 100, 'recipient': '0xBob'}
    )

    print(f"Output: {result_zk.output}")
    print(f"Proof size: {len(result_zk.proof)} bytes")
    print(f"Verified: {zk_exec.verify(result_zk)}")
    print(f"Privacy: {result_zk.privacy_level.value}")

    # 2. TEE Executor
    print("\n── TEE Executor ──")
    tee_exec = TEEExecutor("sgx_enclave_001")

    result_tee = tee_exec.execute(
        function_name="compute_private",
        private_inputs={'salary': 100000, 'ssn': '123-45-6789'},
        public_inputs={'tax_rate': 0.25}
    )

    print(f"Output: {result_tee.output}")
    print(f"Attestation size: {len(result_tee.proof)} bytes")
    print(f"Verified: {tee_exec.verify(result_tee)}")

    # 3. MPC Executor
    print("\n── MPC Executor ──")
    mpc_exec = MPCExecutor(
        parties=["party_a", "party_b", "party_c"],
        threshold=2
    )

    result_mpc = mpc_exec.execute(
        function_name="aggregate",
        private_inputs={'value_a': 100, 'value_b': 200, 'value_c': 300},
        public_inputs={}
    )

    print(f"Aggregated output: {result_mpc.output}")
    print(f"MPC proof: {result_mpc.proof.hex()[:32]}...")
    print(f"Verified: {mpc_exec.verify(result_mpc)}")

private_function_demo()
```

4.7. Encrypted Mempools

4.7.1. El Problema del Mempool Transparente

Definición 4.12 (Encrypted Mempool)**.** Mecanismo donde las transacciones permanecen encriptadas mientras están en el mempool, siendo descifradas solo después de su inclusión en un bloque:

Flujo:

1. Usuario encripta tx con clave threshold
2. Tx encriptada entra al mempool
3. Sequencer/Builder ordena sin ver contenido
4. Block producer incluye tx encriptada
5. Post-inclusión: threshold decryption
6. Ejecución con tx descifrada

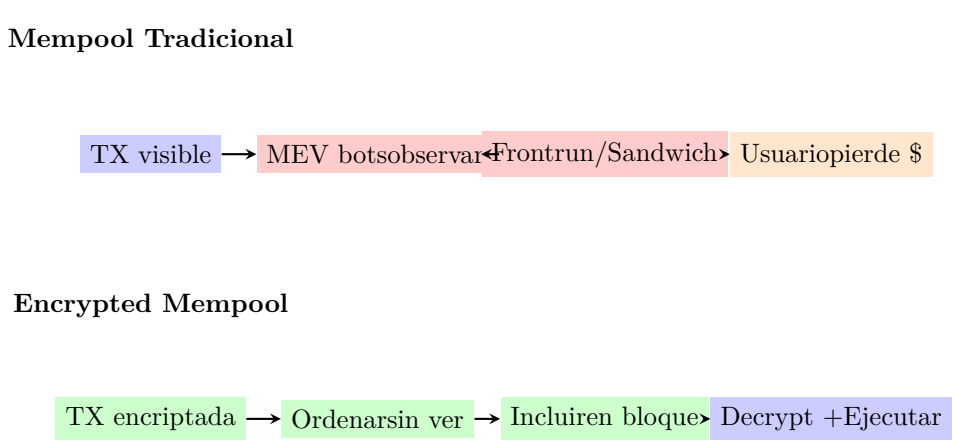


Figura 4.8: Encrypted mempool previene frontrunning

[Esquemas de Encriptación para Mempools] **1. Threshold Encryption:**

- Comité de n partes, threshold t
- t partes deben cooperar para descifrar
- Usado por: Shutter Network

2. Timelock Encryption:

- Descifrado automático después de tiempo T
- Basado en VDF o witness encryption
- Usado por: Penumbra (parcial)

3. Commit-Reveal:

- Fase 1: Commit hash de tx

- Fase 2: Reveal tx completa
- Más simple pero requiere 2 txs

4. Identity-Based Encryption (IBE):

- Encriptar a "bloque N"
- Clave para bloque N disponible después de finalizarlo
- Usado por: Flashbots SUAVE (propuesto)

Listing 4.10: Sistema de Encrypted Mempool

```
from dataclasses import dataclass
from datetime import datetime
from typing import Dict, List, Optional, Tuple
from enum import Enum
import hashlib
import secrets
from collections import deque

class EncryptionScheme(Enum):
    THRESHOLD = "threshold"
    TIMELOCK = "timelock"
    COMMIT_REVEAL = "commit_reveal"

@dataclass
class EncryptedTransaction:
    tx_id: str
    encrypted_data: bytes
    commitment: bytes  # Hash para verificaci n
    sender_pubkey: bytes
    timestamp: datetime
    target_block: Optional[int]  # Para timelock

@dataclass
class DecryptedTransaction:
    tx_id: str
    sender: str
    recipient: str
    value: int
    data: bytes
    gas_price: int

class EncryptedMempool:
    """
    Mempool con transacciones encriptadas.
    """

    def __init__(
        self,
        scheme: EncryptionScheme,
        threshold_parties: List[str] = None,
```

```
        threshold_k: int = None
    ):
        self.scheme = scheme
        self.pending_txs: Dict[str, EncryptedTransaction] = {}
        self.decryption_queue: deque = deque()

        # Para threshold encryption
        self.parties = threshold_parties or []
        self.threshold_k = threshold_k or 2
        self.partial_decryptions: Dict[str, List[bytes]] = {}

        # Para commit-reveal
        self.commits: Dict[bytes, str] = {}  # commitment -> tx_id
        self.reveals: Dict[str, DecryptedTransaction] = {}

        # Clave pública del comité (threshold)
        self.committee_pubkey = secrets.token_bytes(32)

    # ========== SUBMIT ENCRYPTED TX ==========

    def submit_encrypted_tx(
        self,
        encrypted_data: bytes,
        sender_pubkey: bytes,
        target_block: Optional[int] = None
    ) -> str:
        """
        Añadir transacción encriptada al mempool.
        """
        tx_id = secrets.token_hex(16)
        commitment = hashlib.sha256(encrypted_data).digest()

        enc_tx = EncryptedTransaction(
            tx_id=tx_id,
            encrypted_data=encrypted_data,
            commitment=commitment,
            sender_pubkey=sender_pubkey,
            timestamp=datetime.now(),
            target_block=target_block
        )

        self.pending_txs[tx_id] = enc_tx

        return tx_id

    def submit_commit(self, commitment: bytes) -> str:
        """
        Fase 1 de commit-reveal: submit commitment.
        """
        tx_id = secrets.token_hex(16)
        self.commits[commitment] = tx_id
```

```
        return tx_id

    def submit_reveal(
        self,
        commitment: bytes,
        tx_data: DecryptedTransaction
    ) -> bool:
        """
␣␣␣␣␣␣␣␣Fase␣2␣de␣commit-reveal:␣reveal␣transaction.
␣␣␣␣␣␣␣␣"""
        # Verificar commitment
        computed_commitment = self._compute_commitment(tx_data)
        if computed_commitment != commitment:
            return False

        tx_id = self.commits.get(commitment)
        if not tx_id:
            return False

        self.reveals[tx_id] = tx_data
        return True

    # ========== ORDENAMIENTO (sin ver contenido) ==========

    def order_transactions(self) -> List[str]:
        """
␣␣␣␣␣␣␣␣Ordenar␣transacciones␣sin␣ver␣contenido.
␣␣␣␣␣␣␣␣Solo␣podemos␣usar:␣timestamp,␣gas␣(si␣est␣␣en␣header),␣sender.
␣␣␣␣␣␣␣␣"""
        ordered = sorted(
            self.pending_txs.values(),
            key=lambda tx: tx.timestamp
        )

        return [tx.tx_id for tx in ordered]

    # ========== DECRYPTION ==========

    def submit_partial_decryption(
        self,
        tx_id: str,
        party: str,
        partial_decryption: bytes
    ):
        """
␣␣␣␣␣␣␣␣Una␣parte␣del␣comit␣␣submite␣su␣partial␣decryption.
␣␣␣␣␣␣␣␣"""
        if tx_id not in self.partial_decryptions:
            self.partial_decryptions[tx_id] = []

        self.partial_decryptions[tx_id].append(partial_decryption)
```

```
        # Verificar si tenemos threshold
        if len(self.partial_decryptions[tx_id]) >= self.threshold_k:
            self.decryption_queue.append(tx_id)

    def decrypt_transaction(self, tx_id: str) -> Optional[DecryptedTransaction]:
        """
␣␣␣␣␣␣␣␣Descifrar␣transacci n␣si␣tenemos␣suficientes␣partial␣decryptions.
␣␣␣␣␣␣␣␣"""
        if tx_id not in self.pending_txs:
            return None

        enc_tx = self.pending_txs[tx_id]

        if self.scheme == EncryptionScheme.THRESHOLD:
            return self._threshold_decrypt(enc_tx)
        elif self.scheme == EncryptionScheme.COMMIT_REVEAL:
            return self.reveals.get(tx_id)
        elif self.scheme == EncryptionScheme.TIMELOCK:
            return self._timelock_decrypt(enc_tx)

        return None

    def _threshold_decrypt(
        self,
        enc_tx: EncryptedTransaction
    ) -> Optional[DecryptedTransaction]:
        """
␣␣␣␣␣␣␣␣Descifrar␣usando␣threshold␣decryption.
␣␣␣␣␣␣␣␣"""
        partials = self.partial_decryptions.get(enc_tx.tx_id, [])

        if len(partials) < self.threshold_k:
            return None

        # Combinar partial decryptions (simplificado)
        combined_key = self._combine_partials(partials)

        # Descifrar
        plaintext = self._decrypt_with_key(enc_tx.encrypted_data, combined_key)

        # Parsear transacci n
        return self._parse_transaction(plaintext, enc_tx.tx_id)

    def _timelock_decrypt(
        self,
        enc_tx: EncryptedTransaction
    ) -> Optional[DecryptedTransaction]:
        """
␣␣␣␣␣␣␣␣Descifrar␣despu s␣de␣que␣el␣bloque␣target␣sea␣finalizado.
␣␣␣␣␣␣␣␣"""
```

```
        # Verificar que el bloque target fue alcanzado
        # (En producci n: verificar con el estado de la blockchain)

        # El descifrado timelock es autom tico despu s del tiempo
        key = self._derive_timelock_key(enc_tx.target_block)
        plaintext = self._decrypt_with_key(enc_tx.encrypted_data, key)

        return self._parse_transaction(plaintext, enc_tx.tx_id)

    # ========== HELPERS ==========

    def _compute_commitment(self, tx: DecryptedTransaction) -> bytes:
        """Calcular commitment de transacci n."""
        data = f"{tx.sender}:{tx.recipient}:{tx.value}:{tx.data.hex()}"
        return hashlib.sha256(data.encode()).digest()

    def _combine_partials(self, partials: List[bytes]) -> bytes:
        """Combinar partial decryptions (simplificado)."""
        # En producci n: Lagrange interpolation
        combined = bytes(32)
        for p in partials:
            combined = bytes(a ^ b for a, b in zip(combined, p))
        return combined

    def _decrypt_with_key(self, ciphertext: bytes, key: bytes) -> bytes:
        """Descifrar con clave sim trica."""
        # En producci n: AES-GCM
        return bytes(c ^ k for c, k in zip(ciphertext, key * 100))

    def _derive_timelock_key(self, block_number: int) -> bytes:
        """Derivar clave timelock del n mero de bloque."""
        return hashlib.sha256(f"timelock_{block_number}".encode()).digest()

    def _parse_transaction(
        self,
        plaintext: bytes,
        tx_id: str
    ) -> DecryptedTransaction:
        """Parsear transacci n desde bytes."""
        # Simplificado
        return DecryptedTransaction(
            tx_id=tx_id,
            sender="0xSender",
            recipient="0xRecipient",
            value=1000,
            data=plaintext[:32],
            gas_price=100
        )

    def get_mempool_stats(self) -> Dict:
        """Estad sticas del mempool."""
```

```
        return {
            'pending_encrypted': len(self.pending_txs),
            'ready_to_decrypt': len(self.decryption_queue),
            'commits_waiting_reveal': len(self.commits) - len(self.reveals),
            'scheme': self.scheme.value
        }


class ShutterIntegration:
    """
    Integración con Shutter Network para encrypted mempool.
    """

    def __init__(self, keypers: List[str], threshold: int):
        self.keypers = keypers  # Nodos que mantienen las claves
        self.threshold = threshold
        self.eon_keys: Dict[int, bytes] = {}  # eon -> public key

    def get_encryption_key(self, eon: int) -> bytes:
        """
        Obtener clave pública para un eon (período).
        Las transacciones se encriptan con esta clave.
        """
        if eon not in self.eon_keys:
            # En producción: query a los keypers
            self.eon_keys[eon] = secrets.token_bytes(32)

        return self.eon_keys[eon]

    def encrypt_for_eon(self, tx_data: bytes, eon: int) -> bytes:
        """
        Encriptar transacción para un eon específico.
        """
        eon_key = self.get_encryption_key(eon)

        # En producción: threshold encryption con eon_key
        nonce = secrets.token_bytes(12)
        ciphertext = bytes(t ^ k for t, k in zip(tx_data, eon_key * 100))

        return nonce + ciphertext


def encrypted_mempool_demo():
    print("=== Encrypted Mempool Demo ===\n")

    # Crear mempool con threshold encryption
    mempool = EncryptedMempool(
        scheme=EncryptionScheme.THRESHOLD,
        threshold_parties=["keyper_1", "keyper_2", "keyper_3"],
        threshold_k=2
    )
```

```
    # Usuario crea transacci n encriptada
    print("── User Creates Encrypted TX ──")

    tx_data = b"send 100 ETH to 0xBob"
    encrypted = bytes(d ^ 0x42 for d in tx_data)  # Encriptaci n simplificada

    tx_id = mempool.submit_encrypted_tx(
        encrypted_data=encrypted,
        sender_pubkey=b"alice_pubkey",
        target_block=None
    )

    print(f"Submitted encrypted TX: {tx_id}")
    print(f"MEV bots see: [encrypted blob]")
    print(f"Cannot extract: sender intent, recipient, amount")

    # Ordenamiento (sin ver contenido)
    print("\n── Block Builder Orders TXs ──")
    order = mempool.order_transactions()
    print(f"Ordered by timestamp only: {order}")

    # Block se produce
    print("\n── Block Produced ──")

    # Keypers submiten partial decryptions
    for i in range(2):
        mempool.submit_partial_decryption(
            tx_id=tx_id,
            party=f"keyper_{i+1}",
            partial_decryption=secrets.token_bytes(32)
        )
        print(f"Keyper {i+1} submitted partial decryption")

    # Descifrar
    print("\n── Decryption (post-inclusion) ──")
    decrypted = mempool.decrypt_transaction(tx_id)

    if decrypted:
        print(f"TX decrypted: {decrypted.tx_id}")
        print(f"Now executing: send {decrypted.value} to {decrypted.recipient}

    # Stats
    print("\n── Mempool Stats ──")
    stats = mempool.get_mempool_stats()
    for k, v in stats.items():
        print(f"  {k}: {v}")

encrypted_mempool_demo()
```

4.8. MEV Protection con Privacidad

4.8.1. El Problema del MEV

Definición 4.13 (MEV (Maximal Extractable Value))**.** Valor que puede ser extraído por block producers (miners/validators) o searchers mediante la manipulación del orden de transacciones:

Tipos de MEV:

- **Frontrunning:** Ejecutar antes de una tx observada
- **Backrunning:** Ejecutar inmediatamente después
- **Sandwich attacks:** Frontrun + backrun combinados
- **Liquidations:** Competir por liquidar posiciones
- **Arbitrage:** Explotar diferencias de precio

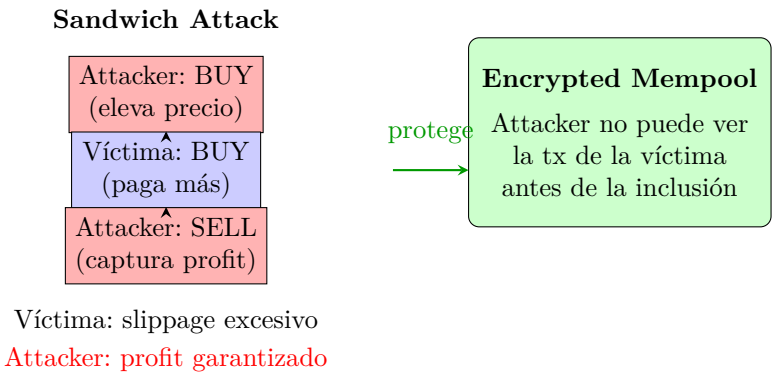


Figura 4.9: Sandwich attack y protección mediante encrypted mempool

4.8.2. Técnicas de Protección

Técnica	Descripción	Efectividad	Complejidad
Encrypted Mempools	TX encriptada hasta inclusión	Muy alta	Alta
Commit-Reveal	Hash primero, datos después	Alta	Media
Private Order Flow	Enviar directo a builder trusted	Media	Baja
Time-lock Puzzles	Descifrar requiere tiempo	Alta	Alta
Batch Auctions	Todas las órdenes al mismo precio	Alta	Media
Threshold Encryption	k-of-n para descifrar	Muy alta	Alta

Tabla 4.3: Técnicas de protección contra MEV

Listing 4.11: Sistema de protección MEV

```
from dataclasses import dataclass
from datetime import datetime, timedelta
from typing import Dict, List, Optional
from enum import Enum
```

```
import hashlib
import secrets

class MEVProtectionLevel(Enum):
    NONE = 0
    BASIC = 1       # Private order flow
    MEDIUM = 2      # Commit-reveal
    HIGH = 3        # Encrypted mempool
    MAXIMUM = 4     # Threshold + timelock

@dataclass
class ProtectedTransaction:
    tx_id: str
    protection_level: MEVProtectionLevel

    # Para commit-reveal
    commitment: Optional[bytes]
    reveal_deadline: Optional[datetime]
    revealed_data: Optional[bytes]

    # Para encryption
    encrypted_data: Optional[bytes]
    decryption_key: Optional[bytes]

    # Metadata
    submitted_at: datetime
    sender: str

class MEVProtectionService:
    """
    Servicio de protecci n contra MEV con m ltiples niveles.
    """

    def __init__(self):
        self.pending_commits: Dict[bytes, ProtectedTransaction] = {}
        self.protected_txs: Dict[str, ProtectedTransaction] = {}
        self.trusted_builders: List[str] = []

        # Threshold encryption committee
        self.threshold_committee: List[str] = []
        self.threshold_k = 2

    def configure_trusted_builders(self, builders: List[str]):
        """Configurar builders de confianza para private order flow."""
        self.trusted_builders = builders

    def configure_threshold_committee(
        self,
        members: List[str],
        threshold: int
    ):
```

```
        """Configurar comit para threshold encryption."""
        self.threshold_committee = members
        self.threshold_k = threshold

    # ========== NIVELES DE PROTECCIN ==========

    def submit_private_order_flow(
        self,
        tx_data: bytes,
        sender: str,
        preferred_builder: Optional[str] = None
    ) -> ProtectedTransaction:
        """
        Nivel BASIC: Enviar directamente a builder de confianza.
        No pasa por mempool p blico.
        """
        builder = preferred_builder or self.trusted_builders[0]

        tx = ProtectedTransaction(
            tx_id=secrets.token_hex(16),
            protection_level=MEVProtectionLevel.BASIC,
            commitment=None,
            reveal_deadline=None,
            revealed_data=tx_data,
            encrypted_data=None,
            decryption_key=None,
            submitted_at=datetime.now(),
            sender=sender
        )

        # Enviar directamente al builder (simulated)
        self._send_to_builder(builder, tx)

        return tx

    def submit_commit_reveal(
        self,
        tx_data: bytes,
        sender: str,
        reveal_delay_seconds: int = 12
    ) -> ProtectedTransaction:
        """
        Nivel MEDIUM: Commit-reveal en dos fases.
        """
        # Fase 1: Commit
        salt = secrets.token_bytes(32)
        commitment = hashlib.sha256(tx_data + salt).digest()

        tx = ProtectedTransaction(
            tx_id=secrets.token_hex(16),
            protection_level=MEVProtectionLevel.MEDIUM,
```

```
            commitment=commitment,
            reveal_deadline=datetime.now() + timedelta(seconds=reveal_delay_se
            revealed_data=None,
            encrypted_data=tx_data + salt,  # Para reveal posterior
            decryption_key=None,
            submitted_at=datetime.now(),
            sender=sender
        )

        self.pending_commits[commitment] = tx
        self.protected_txs[tx.tx_id] = tx

        return tx

    def reveal_transaction(
        self,
        commitment: bytes,
        tx_data: bytes,
        salt: bytes
    ) -> bool:
        """
␣␣␣␣␣␣␣␣Fase␣2␣de␣commit-reveal:␣revelar␣la␣transacci n.
␣␣␣␣␣␣␣␣"""
        if commitment not in self.pending_commits:
            return False

        tx = self.pending_commits[commitment]

        # Verificar que no pas  el deadline
        if datetime.now() > tx.reveal_deadline:
            return False

        # Verificar commitment
        expected = hashlib.sha256(tx_data + salt).digest()
        if expected != commitment:
            return False

        tx.revealed_data = tx_data
        return True

    def submit_encrypted(
        self,
        tx_data: bytes,
        sender: str,
        target_block: int
    ) -> ProtectedTransaction:
        """
␣␣␣␣␣␣␣␣Nivel␣HIGH:␣Transacci n␣encriptada␣con␣threshold␣encryption.
␣␣␣␣␣␣␣␣"""
        # Encriptar para el comit
        encryption_key = secrets.token_bytes(32)
```

```
        encrypted = self._encrypt(tx_data, encryption_key)

        # Distribuir key shares al comit
        key_shares = self._split_key(encryption_key)

        tx = ProtectedTransaction(
            tx_id=secrets.token_hex(16),
            protection_level=MEVProtectionLevel.HIGH,
            commitment=hashlib.sha256(encrypted).digest(),
            reveal_deadline=None,
            revealed_data=None,
            encrypted_data=encrypted,
            decryption_key=None,  # Key distribuida
            submitted_at=datetime.now(),
            sender=sender
        )

        self.protected_txs[tx.tx_id] = tx

        return tx

    def submit_maximum_protection(
        self,
        tx_data: bytes,
        sender: str,
        timelock_blocks: int = 2
    ) -> ProtectedTransaction:
        """
␣␣␣␣␣␣␣␣Nivel␣MAXIMUM:␣Threshold␣+␣timelock␣encryption.
␣␣␣␣␣␣␣␣Solo␣descifrable␣despu s␣de␣N␣bloques␣Y␣con␣threshold␣del␣comit .
␣␣␣␣␣␣␣␣"""
        # Encriptar con timelock
        timelock_key = self._generate_timelock_key(timelock_blocks)
        layer1_encrypted = self._encrypt(tx_data, timelock_key)

        # Encriptar con threshold
        threshold_key = secrets.token_bytes(32)
        layer2_encrypted = self._encrypt(layer1_encrypted, threshold_key)

        tx = ProtectedTransaction(
            tx_id=secrets.token_hex(16),
            protection_level=MEVProtectionLevel.MAXIMUM,
            commitment=hashlib.sha256(layer2_encrypted).digest(),
            reveal_deadline=None,
            revealed_data=None,
            encrypted_data=layer2_encrypted,
            decryption_key=None,
            submitted_at=datetime.now(),
            sender=sender
        )
```

```
        self.protected_txs[tx.tx_id] = tx

        return tx

    # ========= AN LISIS DE PROTECCIN =========

    def analyze_protection_effectiveness(
        self,
        tx: ProtectedTransaction
    ) -> Dict:
        """
␣␣␣␣␣␣␣␣Analizar␣qu ␣tan␣protegida␣est ␣una␣transacci n.
␣␣␣␣␣␣␣␣"""
        analysis = {
            'protection_level': tx.protection_level.name,
            'protected_from': [],
            'vulnerable_to': [],
            'recommendations': []
        }

        if tx.protection_level == MEVProtectionLevel.NONE:
            analysis['vulnerable_to'] = [
                'frontrunning', 'sandwich', 'backrunning'
            ]
            analysis['recommendations'].append(
                "Consider␣using␣at␣least␣private␣order␣flow"
            )

        elif tx.protection_level == MEVProtectionLevel.BASIC:
            analysis['protected_from'] = ['public␣mempool␣exposure']
            analysis['vulnerable_to'] = ['malicious␣builder']
            analysis['recommendations'].append(
                "Use␣commit-reveal␣for␣higher␣security"
            )

        elif tx.protection_level == MEVProtectionLevel.MEDIUM:
            analysis['protected_from'] = [
                'frontrunning␣(before␣reveal)',
                'instant␣sandwich'
            ]
            analysis['vulnerable_to'] = [
                'long-term␣correlation',
                'timing␣attacks␣on␣reveal'
            ]

        elif tx.protection_level == MEVProtectionLevel.HIGH:
            analysis['protected_from'] = [
                'all␣pre-inclusion␣MEV',
                'builder␣manipulation'
            ]
            analysis['vulnerable_to'] = [
```

```
                'committee collusion'
            ]

        elif tx.protection_level == MEVProtectionLevel.MAXIMUM:
            analysis['protected_from'] = [
                'all known MEV vectors',
                'timing attacks',
                'committee collusion (timelock)'
            ]
            analysis['vulnerable_to'] = [
                'quantum attacks (theoretical)'
            ]

        return analysis

    # ========== HELPERS ==========

    def _encrypt(self, data: bytes, key: bytes) -> bytes:
        """Encriptar datos."""
        return bytes(d ^ k for d, k in zip(data, key * 100))

    def _split_key(self, key: bytes) -> List[bytes]:
        """Dividir key en shares."""
        shares = []
        for i in range(len(self.threshold_committee)):
            share = bytes(k ^ i for k in key)
            shares.append(share)
        return shares

    def _generate_timelock_key(self, blocks: int) -> bytes:
        """Generar timelock key."""
        return hashlib.sha256(f"timelock_{blocks}".encode()).digest()

    def _send_to_builder(self, builder: str, tx: ProtectedTransaction):
        """Enviar tx a builder (simulado)."""
        pass


def mev_protection_demo():
    print("=== MEV Protection Demo ===\n")

    service = MEVProtectionService()

    # Configurar
    service.configure_trusted_builders([
        "flashbots_builder",
        "blocknative_builder"
    ])

    service.configure_threshold_committee(
        members=["keyper_1", "keyper_2", "keyper_3"],
```

```
        threshold=2
    )

    # Demostrar diferentes niveles
    tx_data = b"swap 1000 USDC for ETH"

    print("── Level: Private Order Flow ──")
    tx1 = service.submit_private_order_flow(tx_data, "user1")
    analysis1 = service.analyze_protection_effectiveness(tx1)
    print(f"Protected from: {analysis1['protected_from']}")
    print(f"Vulnerable to: {analysis1['vulnerable_to']}")

    print("\n── Level: Commit-Reveal ──")
    tx2 = service.submit_commit_reveal(tx_data, "user2", reveal_delay_seconds=
    analysis2 = service.analyze_protection_effectiveness(tx2)
    print(f"Protection: {analysis2['protection_level']}")
    print(f"Protected from: {analysis2['protected_from']}")

    print("\n── Level: Encrypted Mempool ──")
    tx3 = service.submit_encrypted(tx_data, "user3", target_block=1000)
    analysis3 = service.analyze_protection_effectiveness(tx3)
    print(f"Protected from: {analysis3['protected_from']}")

    print("\n── Level: Maximum (Threshold + Timelock) ──")
    tx4 = service.submit_maximum_protection(tx_data, "user4", timelock_blocks=
    analysis4 = service.analyze_protection_effectiveness(tx4)
    print(f"Protection: {analysis4['protection_level']}")
    print(f"Protected from: {analysis4['protected_from']}")
    print(f"Vulnerable to: {analysis4['vulnerable_to']}")

mev_protection_demo()
```

Resumen del Bloque

Idea Clave

Puntos clave de Private Smart Contracts (Bloque 17):
1. Aproximaciones principales:

- ZK (Aleo, Mina): Pura criptografía, sin trust en hardware
- TEE (Secret, Oasis): Hardware trust, baja latencia
- MPC: Trust distribuido, threshold de partes honestas
- FHE (Fhenix): Computación sobre encriptado, muy costoso

2. Secret Network:

- Estado encriptado con Intel SGX
- Viewing keys para acceso selectivo

- CosmWasm modificado (Rust)

3. Oasis Sapphire:

- EVM confidencial
- Compatible con herramientas Ethereum
- TEE para privacidad

4. Aleo:

- ZK-native, sin TEE
- Modelo de records (como UTXO)
- Lenguaje Leo

5. Mina:

- Blockchain de 22 KB (recursive SNARKs)
- zkApps con estado off-chain
- o1js para desarrollo

6. Encrypted Mempools:

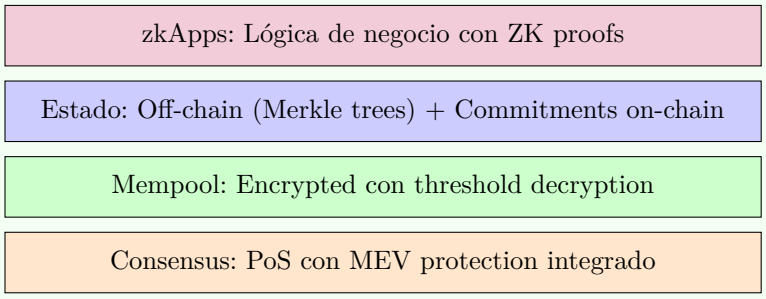
- Threshold encryption (Shutter)
- Timelock encryption
- Commit-reveal
- Previene frontrunning y MEV

7. MEV Protection:

- Private order flow (básico)
- Encrypted mempools (alto)
- Threshold + timelock (máximo)

Aplicación en Banco ZK

Private Smart Contracts en Banco ZK:



Casos de uso en Banco ZK:

1. **Pagos privados:**
   - Transferencias con montos ocultos (ZK)
   - Balances off-chain
2. **DeFi privado:**
   - Swaps sin frontrunning (encrypted mempool)
   - Lending sin revelar posiciones
3. **Votación privada:**
   - Votos confidenciales
   - Conteo verificable
4. **Subastas selladas:**
   - Pujas encriptadas
   - Revelación solo al cierre

Stack técnico recomendado:

- Core privacy: ZK proofs (Groth16/PLONK)
- Mempool protection: Threshold encryption
- Fallback TEE: Para operaciones legacy
- Future: FHE cuando sea práctico

Capítulo 5

Criptografía Avanzada para Privacidad

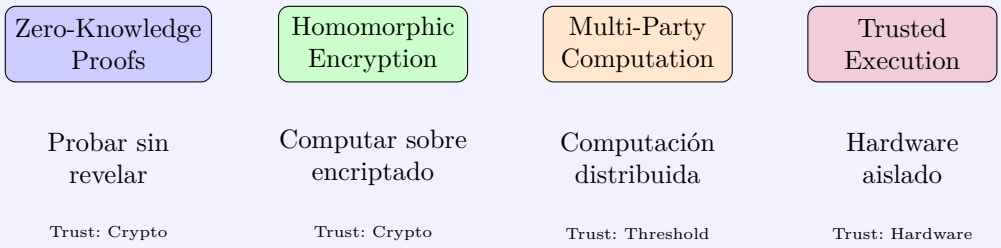
Bloque 18 de 20

Introducción

Más allá de las pruebas de conocimiento cero, existe un arsenal de técnicas criptográficas avanzadas que permiten computación privada. Este bloque explora las herramientas fundamentales que hacen posible la privacidad en sistemas distribuidos.

Idea Clave

Panorama de Criptografía para Privacidad:



Primitivas fundamentales:

- **Secret Sharing:** Dividir secretos entre múltiples partes
- **Oblivious Transfer:** Transferencia selectiva sin revelar elección
- **Commitments:** Comprometerse a un valor sin revelarlo
- **Threshold Cryptography:** Operaciones que requieren k-de-n partes

5.1. FHE (Fully Homomorphic Encryption)

5.1.1. Fundamentos Teóricos

Definición 5.1 (Fully Homomorphic Encryption)**.** Sistema de encriptación que permite realizar operaciones arbitrarias sobre datos cifrados:

Propiedades homomórficas:

$$\text{Dec}(\text{Enc}(a) \oplus \text{Enc}(b)) = a + b \quad \text{(Adición)} \tag{5.1}$$
$$\text{Dec}(\text{Enc}(a) \otimes \text{Enc}(b)) = a \cdot b \quad \text{(Multiplicación)} \tag{5.2}$$

Con suma y multiplicación, se puede computar **cualquier función** (circuito booleano/aritmético).

[Historia y Generaciones de FHE] **El Santo Grial de la criptografía:**
Propuesto conceptualmente por Rivest, Adleman, Dertouzos en 1978, pero sin construcción práctica hasta 2009.
Generación 1 (2009): Gentry

- Primera construcción teórica completa
- Basado en lattices ideales
- Bootstrapping para "refrescar" ruido
- Extremadamente lento ($> 10^6$ más lento que plaintext)

Generación 2 (2011-2014): BGV, BFV

- Brakerski-Gentry-Vaikuntanathan (BGV)
- Brakerski/Fan-Vercauteren (BFV)
- Operaciones sobre enteros/polinomios
- Leveled FHE (profundidad limitada sin bootstrap)
- $\sim 10^3 - 10^4$ más lento

Generación 3 (2016+): TFHE, CKKS

- TFHE: Operaciones bit-a-bit, bootstrap rápido (10ms)
- CKKS: Aritmética aproximada para ML/estadísticas
- Práctico para casos de uso específicos
- $\sim 10^2 - 10^3$ más lento

Generación 4 (Actual): Programable

- Compiladores FHE (HEIR, Concrete)
- Aceleradores hardware (GPU, FPGA, ASIC)
- Objetivo: $< 10\times$ overhead

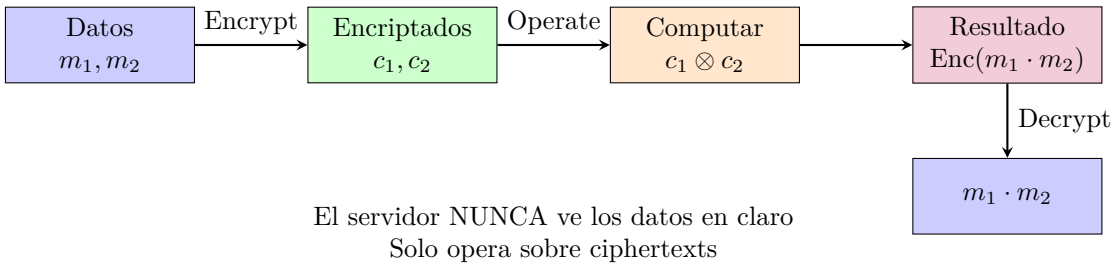


Figura 5.1: Flujo de computación con FHE

5.1.2. Matemáticas de FHE (Learning With Errors)

Definición 5.2 (Learning With Errors (LWE))**.** Problema computacional base de la mayoría de esquemas FHE modernos:

Problema: Dado $(A, b = As + e \mod q)$, encontrar s.

Donde:

- $A \in \mathbb{Z}_q^{m \times n}$: Matriz pública aleatoria
- $s \in \mathbb{Z}_q^n$: Vector secreto
- $e \in \mathbb{Z}_q^m$: Vector de "error" pequeño
- q: Módulo grande

Dificultad: Sin e, es álgebra lineal. Con e, se cree intratrable incluso para computadoras cuánticas.

[Esquema BFV Simplificado] **Parámetros:**

- n: Grado del polinomio (potencia de 2, ej: 4096)
- q: Módulo del ciphertext (grande, ej: 2^{109})
- t: Módulo del plaintext (pequeño, ej: 2^{16})
- $R = \mathbb{Z}[X]/(X^n + 1)$: Anillo de polinomios

KeyGen:

1. $sk \leftarrow$ polinomio con coeficientes pequeños
2. $a \leftarrow R_q$ aleatorio
3. $e \leftarrow$ error pequeño (distribución gaussiana)
4. $pk = (-a \cdot sk + e, a) = (b, a)$

Encrypt(m, pk):

1. $m \in R_t$ (mensaje como polinomio)
2. $u, e_1, e_2 \leftarrow$ errores pequeños
3. $\Delta = \lfloor q/t \rfloor$ (factor de escala)
4. $c_0 = b \cdot u + e_1 + \Delta \cdot m$
5. $c_1 = a \cdot u + e_2$

6. Ciphertext: (c_0, c_1)

Decrypt(c, sk):

1. $m' = c_0 + c_1 \cdot sk \mod q$
2. $m = \lfloor t \cdot m'/q \rceil \mod t$

Operaciones Homomórficas:
Suma: $(c_0, c_1) + (c'_0, c'_1) = (c_0 + c'_0, c_1 + c'_1)$
Multiplicación: Más compleja, genera ciphertext de 3 elementos, requiere "relinearization" para volver a 2 elementos.

Listing 5.1: Implementación conceptual de FHE (BFV simplificado)

```
import numpy as np
from typing import Tuple, List
from dataclasses import dataclass

@dataclass
class FHEParams:
    """Par metros del esquema FHE."""
    n: int  # Grado del polinomio
    q: int  # M dulo del ciphertext
    t: int  # M dulo del plaintext
    std_dev: float  # Desviaci n est ndar del error

class SimpleFHE:
    """
    Implementaci n simplificada de FHE estilo BFV.
    NOTA: Solo para demostraci n, NO usar en producci n.
    """

    def __init__(self, params: FHEParams):
        self.params = params
        self.n = params.n
        self.q = params.q
        self.t = params.t
        self.delta = self.q // self.t  # Factor de escala

        # Generar claves
        self.secret_key = None
        self.public_key = None
        self.relin_key = None

    def _sample_error(self, size: int = None) -> np.ndarray:
        """Muestrear error de distribuci n gaussiana discreta."""
        size = size or self.n
        return np.round(np.random.normal(0, self.params.std_dev, size)).astype

    def _sample_uniform(self, size: int = None) -> np.ndarray:
        """Muestrear uniformemente del anillo."""
        size = size or self.n
        return np.random.randint(0, self.q, size)
```

```
    def _sample_ternary(self, size: int = None) -> np.ndarray:
        """Muestrear de {-1, 0, 1}."""
        size = size or self.n
        return np.random.choice([-1, 0, 1], size)

    def _poly_mult(self, a: np.ndarray, b: np.ndarray) -> np.ndarray:
        """
        Multiplicaci n de polinomios en R_q = Z_q[X]/(X^n + 1).
        """
        # Convoluci n
        result = np.convolve(a, b)

        # Reducci n m dulo X^n + 1 (negacyclic)
        for i in range(self.n, len(result)):
            result[i - self.n] -= result[i]

        return result[:self.n] % self.q

    def keygen(self) -> Tuple[np.ndarray, Tuple[np.ndarray, np.ndarray]]:
        """
        Generar par de claves.

        Returns:
            (secret_key, public_key)
        """
        # Secret key: polinomio ternario
        sk = self._sample_ternary()

        # Public key
        a = self._sample_uniform()
        e = self._sample_error()

        # b = -a*sk + e (mod q)
        b = (-self._poly_mult(a, sk) + e) % self.q

        pk = (b, a)

        self.secret_key = sk
        self.public_key = pk

        # Generar relinearization key (simplificado)
        self._gen_relin_key()

        return sk, pk

    def _gen_relin_key(self):
        """Generar clave de relinearizaci n para multiplicaci n."""
        # Simplificado: en BFV real, es m s complejo
        sk_squared = self._poly_mult(self.secret_key, self.secret_key)
        a = self._sample_uniform()
```

```
        e = self._sample_error()

        b = (-self._poly_mult(a, self.secret_key) + e + sk_squared) % self.q
        self.relin_key = (b, a)

    def encrypt(self, message: int) -> Tuple[np.ndarray, np.ndarray]:
        """
␣␣␣␣␣␣␣␣Encriptar␣un␣mensaje␣entero.

␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣message:␣Entero␣en␣[0,␣t)

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣Ciphertext␣(c0,␣c1)
␣␣␣␣␣␣␣␣"""
        assert 0 <= message < self.t, f"Message must be in [0, {self.t})"

        # Codificar mensaje como polinomio (constante)
        m = np.zeros(self.n, dtype=int)
        m[0] = message

        # Sampling
        u = self._sample_ternary()
        e1 = self._sample_error()
        e2 = self._sample_error()

        b, a = self.public_key

        # c0 = b*u + e1 + delta*m
        c0 = (self._poly_mult(b, u) + e1 + self.delta * m) % self.q

        # c1 = a*u + e2
        c1 = (self._poly_mult(a, u) + e2) % self.q

        return (c0, c1)

    def decrypt(self, ciphertext: Tuple[np.ndarray, np.ndarray]) -> int:
        """
␣␣␣␣␣␣␣␣Descifrar␣ciphertext.

␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣ciphertext:␣(c0,␣c1)

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣Mensaje␣descifrado
␣␣␣␣␣␣␣␣"""
        c0, c1 = ciphertext

        # m' = c0 + c1*sk (mod q)
        m_scaled = (c0 + self._poly_mult(c1, self.secret_key)) % self.q
```

```
        # Centrar en [-q/2, q/2)
        m_scaled = np.where(m_scaled > self.q // 2, m_scaled - self.q, m_scaled)

        # Escalar de vuelta: m = round(t * m' / q)
        m = np.round(self.t * m_scaled / self.q) % self.t

        return int(m[0])

    def add(
        self,
        ct1: Tuple[np.ndarray, np.ndarray],
        ct2: Tuple[np.ndarray, np.ndarray]
    ) -> Tuple[np.ndarray, np.ndarray]:
        """
        Suma homomórfica de dos ciphertexts.

        Dec(add(Enc(a), Enc(b))) = a + b (mod t)
        """
        c0_1, c1_1 = ct1
        c0_2, c1_2 = ct2

        c0 = (c0_1 + c0_2) % self.q
        c1 = (c1_1 + c1_2) % self.q

        return (c0, c1)

    def multiply(
        self,
        ct1: Tuple[np.ndarray, np.ndarray],
        ct2: Tuple[np.ndarray, np.ndarray]
    ) -> Tuple[np.ndarray, np.ndarray]:
        """
        Multiplicación homomórfica de dos ciphertexts.

        Dec(multiply(Enc(a), Enc(b))) = a * b (mod t)
        """
        c0_1, c1_1 = ct1
        c0_2, c1_2 = ct2

        # Producto tensor (3 elementos)
        d0 = self._poly_mult(c0_1, c0_2)
        d1 = (self._poly_mult(c0_1, c1_2) + self._poly_mult(c1_1, c0_2)) % self.
        d2 = self._poly_mult(c1_1, c1_2)

        # Escalar por t/q para mantener el nivel de ruido
        scale = self.t / self.q
        d0 = np.round(d0 * scale) % self.q
        d1 = np.round(d1 * scale) % self.q
        d2 = np.round(d2 * scale) % self.q

        # Relinearización: reducir de 3 a 2 elementos
```

```
        c0, c1 = self._relinearize(d0.astype(int), d1.astype(int), d2.astype(i

        return (c0, c1)

    def _relinearize(
        self,
        d0: np.ndarray,
        d1: np.ndarray,
        d2: np.ndarray
    ) -> Tuple[np.ndarray, np.ndarray]:
        """
␣␣␣␣␣␣␣␣Relinearizaci n:␣convertir␣ciphertext␣de␣3␣elementos␣a␣2.
␣␣␣␣␣␣␣␣"""
        rlk_b, rlk_a = self.relin_key

        # c0 = d0 + d2 * rlk_b
        c0 = (d0 + self._poly_mult(d2, rlk_b)) % self.q

        # c1 = d1 + d2 * rlk_a
        c1 = (d1 + self._poly_mult(d2, rlk_a)) % self.q

        return (c0, c1)

    def add_plain(
        self,
        ct: Tuple[np.ndarray, np.ndarray],
        plaintext: int
    ) -> Tuple[np.ndarray, np.ndarray]:
        """Sumar␣plaintext␣a␣ciphertext."""
        c0, c1 = ct

        m = np.zeros(self.n, dtype=int)
        m[0] = plaintext * self.delta

        new_c0 = (c0 + m) % self.q

        return (new_c0, c1)

    def multiply_plain(
        self,
        ct: Tuple[np.ndarray, np.ndarray],
        plaintext: int
    ) -> Tuple[np.ndarray, np.ndarray]:
        """Multiplicar␣ciphertext␣por␣plaintext."""
        c0, c1 = ct

        new_c0 = (c0 * plaintext) % self.q
        new_c1 = (c1 * plaintext) % self.q

        return (new_c0, new_c1)
```

```
def fhe_demo():
    print("=== FHE Demo (BFV Simplificado) ===\n")

    # Par metros (peque os para demo, no seguros)
    params = FHEParams(
        n=64,              # Normalmente 4096+
        q=2**20,           # Normalmente 2^109+
        t=256,             # Plaintext modulus
        std_dev=3.2        # Error est ndar
    )

    fhe = SimpleFHE(params)

    # Generar claves
    print("--- Key Generation ---")
    sk, pk = fhe.keygen()
    print(f"Secret key: {sk[:5]}... (ternary polynomial)")
    print(f"Public key: 2 polynomials of degree {params.n}")

    # Encriptar valores
    print("\n--- Encryption ---")
    a, b = 15, 7

    ct_a = fhe.encrypt(a)
    ct_b = fhe.encrypt(b)

    print(f"Encrypted {a} and {b}")
    print(f"Ciphertext size: 2  {params.n} integers mod {params.q}")

    # Operaciones homom rficas
    print("\n--- Homomorphic Operations ---")

    # Suma
    ct_sum = fhe.add(ct_a, ct_b)
    result_sum = fhe.decrypt(ct_sum)
    print(f"Dec(Enc({a}) + Enc({b})) = {result_sum}")
    print(f"Expected: {(a + b) % params.t}")

    # Multiplicaci n
    ct_mult = fhe.multiply(ct_a, ct_b)
    result_mult = fhe.decrypt(ct_mult)
    print(f"Dec(Enc({a})   Enc({b})) = {result_mult}")
    print(f"Expected: {(a * b) % params.t}")

    # Operaciones mixtas
    print("\n--- Mixed Operations ---")
    ct_expr = fhe.add(fhe.multiply(ct_a, ct_b), ct_a)  # a*b + a
    result_expr = fhe.decrypt(ct_expr)
    print(f"Dec(Enc({a})   Enc({b}) + Enc({a})) = {result_expr}")
    print(f"Expected: {(a * b + a) % params.t}")
```

```
    print("\n--- Noise Growth ---")
    print("After each operation, noise grows.")
    print("Too many operations     decryption fails.")
    print("Bootstrapping 'refreshes' noise (expensive).")

fhe_demo()
```

5.1.3. Librerías y Aplicaciones

Librería	Esquema	Características
Microsoft SEAL	BFV, CKKS	C++, muy documentado, usado en industria
OpenFHE	BGV, BFV, CKKS, TFHE	C++, sucesor de PALISADE, más completo
TFHE-rs	TFHE	Rust, bootstrap rápido, Zama
Concrete	TFHE	Python/Rust, compilador FHE, Zama
HElib	BGV, CKKS	C++, IBM, muy optimizado
Lattigo	BFV, CKKS	Go, buen rendimiento

Tabla 5.1: Principales librerías FHE

5.2. MPC (Multi-Party Computation)

5.2.1. Fundamentos

Definición 5.3 (Secure Multi-Party Computation)**.** Protocolo que permite a n partes computar una función $f(x_1, ..., x_n)$ sobre sus inputs privados, tal que:

1. **Correctitud:** El output es correcto
2. **Privacidad:** Ninguna parte aprende más que su propio input y el output
3. **Independence of inputs:** Inputs elegidos independientemente

Ideal functionality: Como si existiera un tercero de confianza que recibe todos los inputs, computa f, y entrega outputs.

5.2.2. Protocolos MPC Principales

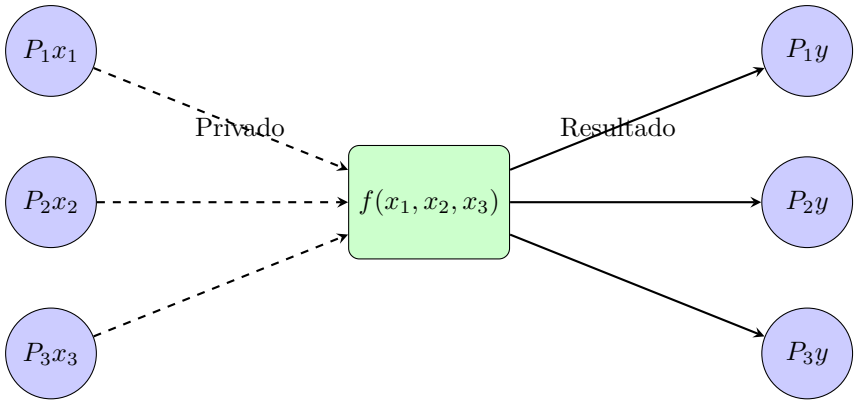
[Garbled Circuits (Yao, 1986)] Protocolo para 2 partes basado en "ofuscar" un circuito booleano:

Partes:

- Garbler: Construye el circuito ofuscado
- Evaluator: Evalúa el circuito

Proceso:

1. Garbler asigna 2 etiquetas aleatorias a cada wire: W_i^0, W_i^1



Ninguna parte ve los inputs de otras
Todas obtienen el mismo resultado correcto

Figura 5.2: Ideal functionality de MPC

2. Para cada gate, crea tabla ofuscada de 4 entradas (encriptada)
3. Evaluator recibe etiquetas de sus inputs via Oblivious Transfer
4. Evaluator evalúa gate por gate, descifrando la entrada correcta
5. Output: etiqueta final mapea a bit de salida

Seguridad: Evaluator solo ve una etiqueta por wire (no sabe si es 0 o 1 hasta el final).

Listing 5.2: Implementación de Garbled Circuits

```
from dataclasses import dataclass
from typing import Dict, List, Tuple, Optional
import secrets
import hashlib
from enum import Enum

class GateType(Enum):
    AND = "AND"
    OR = "OR"
    XOR = "XOR"
    NOT = "NOT"

@dataclass
class Wire:
    wire_id: int
    label_0: bytes  # Etiqueta para bit 0
    label_1: bytes  # Etiqueta para bit 1

@dataclass
class Gate:
    gate_id: int
    gate_type: GateType
    input_wires: List[int]
    output_wire: int
```

```
@dataclass
class GarbledGate:
    gate_id: int
    garbled_table: List[bytes]  # 4 entradas encriptadas

class GarbledCircuit:
    """
    Implementación de Garbled Circuits para MPC de 2 partes.
    """

    def __init__(self, label_size: int = 16):
        self.label_size = label_size
        self.wires: Dict[int, Wire] = {}
        self.gates: List[Gate] = []
        self.garbled_gates: List[GarbledGate] = []

        self.input_wires_garbler: List[int] = []
        self.input_wires_evaluator: List[int] = []
        self.output_wires: List[int] = []

    def _generate_label(self) -> bytes:
        """Generar etiqueta aleatoria."""
        return secrets.token_bytes(self.label_size)

    def _hash_labels(self, *labels: bytes, gate_id: int) -> bytes:
        """Hash de etiquetas para encriptar tabla."""
        data = b''.join(labels) + gate_id.to_bytes(4, 'big')
        return hashlib.sha256(data).digest()[:self.label_size]

    def add_wire(self, wire_id: int):
        """A adir wire al circuito."""
        self.wires[wire_id] = Wire(
            wire_id=wire_id,
            label_0=self._generate_label(),
            label_1=self._generate_label()
        )

    def add_gate(
        self,
        gate_id: int,
        gate_type: GateType,
        input_wires: List[int],
        output_wire: int
    ):
        """A adir gate al circuito."""
        # Crear wire de salida
        self.add_wire(output_wire)

        gate = Gate(
            gate_id=gate_id,
            gate_type=gate_type,
```

```
            input_wires=input_wires,
            output_wire=output_wire
        )
        self.gates.append(gate)

    def _evaluate_gate(
        self,
        gate_type: GateType,
        inputs: List[int]
    ) -> int:
        """Evaluar gate en plaintext."""
        if gate_type == GateType.AND:
            return inputs[0] & inputs[1]
        elif gate_type == GateType.OR:
            return inputs[0] | inputs[1]
        elif gate_type == GateType.XOR:
            return inputs[0] ^ inputs[1]
        elif gate_type == GateType.NOT:
            return 1 - inputs[0]
        return 0

    def garble(self) -> List[GarbledGate]:
        """
        Garbler: Ofuscar el circuito.
        """
        self.garbled_gates = []

        for gate in self.gates:
            garbled = self._garble_gate(gate)
            self.garbled_gates.append(garbled)

        return self.garbled_gates

    def _garble_gate(self, gate: Gate) -> GarbledGate:
        """Ofuscar una gate individual."""
        table = []

        # Para cada combinaci n de inputs
        for a in [0, 1]:
            for b in [0, 1] if len(gate.input_wires) > 1 else [0]:
                # Evaluar gate
                if len(gate.input_wires) > 1:
                    output_bit = self._evaluate_gate(gate.gate_type, [a, b])
                else:
                    output_bit = self._evaluate_gate(gate.gate_type, [a])

                # Obtener etiquetas de entrada
                wire_a = self.wires[gate.input_wires[0]]
                label_a = wire_a.label_1 if a else wire_a.label_0

                if len(gate.input_wires) > 1:
```

```
                    wire_b = self.wires[gate.input_wires[1]]
                    label_b = wire_b.label_1 if b else wire_b.label_0
                else:
                    label_b = b''

                # Obtener etiqueta de salida
                output_wire = self.wires[gate.output_wire]
                output_label = output_wire.label_1 if output_bit else output_w

                # Encriptar etiqueta de salida con etiquetas de entrada
                key = self._hash_labels(label_a, label_b, gate_id=gate.gate_id
                encrypted = bytes(o ^ k for o, k in zip(output_label, key))

                table.append(encrypted)

        # Permutar tabla aleatoriamente
        secrets.SystemRandom().shuffle(table)

        return GarbledGate(gate_id=gate.gate_id, garbled_table=table)

    def get_garbler_input_labels(
        self,
        garbler_inputs: List[int]
    ) -> List[bytes]:
        """
        Garbler obtiene las etiquetas para sus propios inputs.
        """
        labels = []
        for wire_id, bit in zip(self.input_wires_garbler, garbler_inputs):
            wire = self.wires[wire_id]
            label = wire.label_1 if bit else wire.label_0
            labels.append(label)
        return labels

    def get_evaluator_input_labels_via_ot(
        self,
        evaluator_inputs: List[int]
    ) -> List[bytes]:
        """
        Evaluator obtiene etiquetas via Oblivious Transfer.
        (Simplificado - en realidad ser a OT protocol)
        """
        labels = []
        for wire_id, bit in zip(self.input_wires_evaluator, evaluator_inputs):
            wire = self.wires[wire_id]
            label = wire.label_1 if bit else wire.label_0
            labels.append(label)
        return labels

    def evaluate(
        self,
```

```
        garbled_gates: List[GarbledGate],
        input_labels: Dict[int, bytes]
    ) -> Dict[int, bytes]:
        """
␣␣␣␣␣␣␣␣Evaluator: Evaluar el circuito ofuscado.
␣␣␣␣␣␣␣␣"""
        wire_labels = input_labels.copy()

        for garbled_gate in garbled_gates:
            gate = self.gates[garbled_gate.gate_id]

            # Obtener etiquetas de entrada
            label_a = wire_labels[gate.input_wires[0]]
            if len(gate.input_wires) > 1:
                label_b = wire_labels[gate.input_wires[1]]
            else:
                label_b = b''

            # Intentar descifrar cada entrada de la tabla
            output_label = None
            for encrypted in garbled_gate.garbled_table:
                key = self._hash_labels(label_a, label_b, gate_id=gate.gate_id)
                decrypted = bytes(e ^ k for e, k in zip(encrypted, key))

                # Verificar si es una etiqueta v lida
                # (En implementaci n real, se usa punto de verificaci n)
                output_wire = self.wires[gate.output_wire]
                if decrypted == output_wire.label_0 or decrypted == output_wire.
                    output_label = decrypted
                    break

            if output_label:
                wire_labels[gate.output_wire] = output_label

        return wire_labels

    def decode_output(
        self,
        output_labels: Dict[int, bytes]
    ) -> List[int]:
        """
␣␣␣␣␣␣␣␣Decodificar etiquetas de salida a bits.
␣␣␣␣␣␣␣␣"""
        outputs = []
        for wire_id in self.output_wires:
            wire = self.wires[wire_id]
            label = output_labels[wire_id]

            if label == wire.label_0:
                outputs.append(0)
            elif label == wire.label_1:
```

```
                outputs.append(1)
            else:
                raise ValueError(f"Invalid output label for wire {wire_id}")

        return outputs


class GMWProtocol:
    """
    Protocolo GMW para MPC basado en secret sharing.
    Soporta m ltiples partes.
    """

    def __init__(self, num_parties: int):
        self.num_parties = num_parties

    def share_value(self, value: int, num_bits: int = 1) -> List[int]:
        """
        Secret-share un valor entre todas las partes.
        XOR sharing: value = share_1 XOR share_2 XOR ... XOR share_n
        """
        shares = []
        running_xor = value

        for i in range(self.num_parties - 1):
            share = secrets.randbelow(2**num_bits)
            shares.append(share)
            running_xor ^= share

        #  ltima  share asegura que XOR de todas = value
        shares.append(running_xor)

        return shares

    def reconstruct_value(self, shares: List[int]) -> int:
        """Reconstruir valor desde shares."""
        result = 0
        for share in shares:
            result ^= share
        return result

    def xor_gate(
        self,
        shares_a: List[int],
        shares_b: List[int]
    ) -> List[int]:
        """
        XOR gate: cada parte XOREa sus shares localmente.
        """
        return [a ^ b for a, b in zip(shares_a, shares_b)]
```

```
    def and_gate(
        self,
        shares_a: List[int],
        shares_b: List[int]
    ) -> List[int]:
        """
␣␣␣␣␣␣␣␣AND␣gate:␣requiere␣comunicaci n␣(simplificado).
␣␣␣␣␣␣␣␣En␣GMW␣real,␣usa␣multiplication␣triples␣pre-computados.
␣␣␣␣␣␣␣␣"""
        # Simplificaci n: reconstruir, computar, re-compartir
        a = self.reconstruct_value(shares_a)
        b = self.reconstruct_value(shares_b)
        result = a & b
        return self.share_value(result)


def garbled_circuits_demo():
    print("===␣Garbled␣Circuits␣Demo␣===\n")

    gc = GarbledCircuit()

    # Construir circuito: (a AND b) XOR c
    # Inputs: a (wire 0), b (wire 1), c (wire 2)
    # Output: wire 4

    gc.add_wire(0)  # a
    gc.add_wire(1)  # b
    gc.add_wire(2)  # c

    gc.input_wires_garbler = [0, 1]  # Garbler tiene a, b
    gc.input_wires_evaluator = [2]    # Evaluator tiene c
    gc.output_wires = [4]

    # Gates
    gc.add_gate(0, GateType.AND, [0, 1], 3)  # wire 3 = a AND b
    gc.add_gate(1, GateType.XOR, [3, 2], 4)  # wire 4 = (a AND b) XOR c

    # Garbler garbles el circuito
    print("---␣Garbler␣Phase␣---")
    garbled_gates = gc.garble()
    print(f"Garbled␣{len(garbled_gates)}␣gates")

    # Inputs
    a, b = 1, 1  # Garbler inputs
    c = 0        # Evaluator input

    expected = (a & b) ^ c
    print(f"\nExpected␣result:␣({a}␣AND␣{b})␣XOR␣{c}␣=␣{expected}")

    # Garbler obtiene etiquetas de sus inputs
    garbler_labels = gc.get_garbler_input_labels([a, b])
```

```
    # Evaluator obtiene etiquetas via OT (simplificado)
    evaluator_labels = gc.get_evaluator_input_labels_via_ot([c])

    # Combinar etiquetas
    input_labels = {
        0: garbler_labels[0],
        1: garbler_labels[1],
        2: evaluator_labels[0]
    }

    # Evaluator eval a
    print("\n--- Evaluator Phase ---")
    output_labels = gc.evaluate(garbled_gates, input_labels)

    # Decodificar salida
    result = gc.decode_output(output_labels)
    print(f"Computed result: {result[0]}")
    print(f"Correct: {result[0] == expected}")

garbled_circuits_demo()
```

5.2.3. Protocolo SPDZ

Definición 5.4 (SPDZ (Speedz))**.** Protocolo MPC eficiente para múltiples partes con seguridad contra adversarios maliciosos:

Características:

- Basado en secret sharing aditivo sobre campo finito
- Fase offline (preprocessing) + fase online (evaluación)
- Online muy rápido (microsegundos por operación)
- Soporta número arbitrario de partes

Modelo de adversario:

- Corrompe hasta $n - 1$ partes (dishonest majority)
- Detecta trampas con alta probabilidad

[SPDZ Overview] **Secret Sharing:** Valor x compartido como:

$$\langle x \rangle = (x_1, ..., x_n) \text{ donde } x = \sum_{i=1}^{n} x_i$$

Operaciones:

- **Suma:** $\langle x \rangle + \langle y \rangle = \langle x + y \rangle$ (local)
- **Mult por constante:** $c \cdot \langle x \rangle = \langle c \cdot x \rangle$ (local)
- **Multiplicación:** Requiere "multiplication triples" pre-computados

Beaver Triples: Pre-computar (a, b, c) donde $c = a \cdot b$:

1. Para multiplicar $\langle x \rangle \cdot \langle y \rangle$:
2. Computar $\langle \epsilon \rangle = \langle x \rangle - \langle a \rangle$ y abrir ϵ
3. Computar $\langle \delta \rangle = \langle y \rangle - \langle b \rangle$ y abrir δ
4. $\langle xy \rangle = \langle c \rangle + \epsilon\langle b \rangle + \delta\langle a \rangle + \epsilon\delta$

Protocolo	**Partes**	**Modelo**	**Complejidad**	**Uso**
Garbled Circuits	2	Semi-honest	$O(\|C\|)$	General
GMW	n	Semi-honest	$O(\|C\| \cdot n^2)$	Booleano
SPDZ	n	Malicious	$O(\|C\|)$ online	Aritmético
BGW	n	Honest majority	$O(\|C\|)$	Aritmético

Tabla 5.2: Comparación de protocolos MPC

5.3. Secret Sharing (Shamir)

5.3.1. Fundamentos Matemáticos

Definición 5.5 (Shamir's Secret Sharing)**.** Esquema (k, n)-threshold que divide un secreto s en n partes (shares), tal que:

- Cualquier k o más shares pueden reconstruir s
- Menos de k shares no revelan información sobre s

Basado en: Interpolación de polinomios de Lagrange.

[Construcción de Shamir] **Setup:**

- Secreto: $s \in \mathbb{Z}_p$ (campo primo grande)
- Threshold: k (mínimo de shares para reconstruir)
- Total shares: n

Compartir (Split):

1. Elegir coeficientes aleatorios $a_1, ..., a_{k-1} \in \mathbb{Z}_p$
2. Construir polinomio de grado $k-1$:

$$f(x) = s + a_1 x + a_2 x^2 + ... + a_{k-1} x^{k-1}$$

3. Share i: $(i, f(i))$ para $i = 1, 2, ..., n$

Reconstruir:
Dados k puntos $(x_1, y_1), ..., (x_k, y_k)$, usar interpolación de Lagrange:

$$f(x) = \sum_{i=1}^{k} y_i \cdot L_i(x)$$

donde el polinomio base de Lagrange es:

$$L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

El secreto es $s = f(0)$.

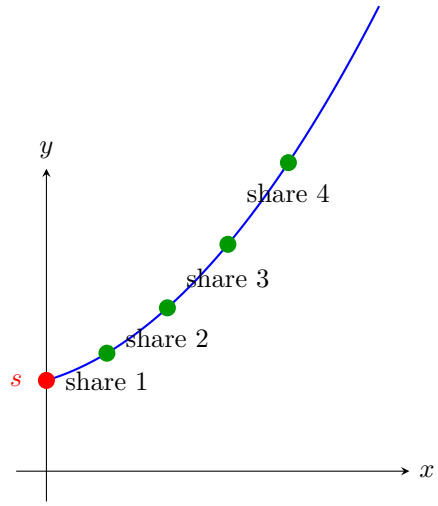


Figura 5.3: Shamir's Secret Sharing: el secreto es el término independiente del polinomio

Listing 5.3: Implementación completa de Shamir's Secret Sharing

```
from dataclasses import dataclass
from typing import List, Tuple
import secrets
from functools import reduce

@dataclass
class Share:
    """Un share del secreto."""
    x: int  # Punto de evaluaci n
    y: int  # Valor f(x)

class ShamirSecretSharing:
    """
    Implementaci n de Shamir's Secret Sharing sobre campo primo.
    """

    def __init__(self, prime: int = None):
        """
        Args:
            prime: Primo para el campo. Si None, usa primo de 256 bits.
        """
        # Primo de Mersenne para eficiencia
        self.prime = prime or (2**256 - 189)

    def _mod_inverse(self, a: int, p: int) -> int:
        """
        Calcular inverso modular usando extended Euclidean algorithm.
        """
        def extended_gcd(a: int, b: int) -> Tuple[int, int, int]:
            if a == 0:
                return b, 0, 1
            gcd, x1, y1 = extended_gcd(b % a, a)
```

```
            x = y1 - (b // a) * x1
            y = x1
            return gcd, x, y

        _, x, _ = extended_gcd(a % p, p)
        return (x % p + p) % p

    def split(
        self,
        secret: int,
        threshold: int,
        num_shares: int
    ) -> List[Share]:
        """
        Dividir secreto en shares.

        Args:
            secret: El secreto a compartir (entero)
            threshold: N mero m nimo de shares para reconstruir (k)
            num_shares: N mero total de shares a generar (n)

        Returns:
            Lista de n shares
        """
        assert threshold <= num_shares, "Threshold cannot exceed num_shares"
        assert 0 <= secret < self.prime, f"Secret must be in [0, {self.prime})"

        # Generar coeficientes aleatorios para el polinomio
        # f(x) = secret + a1*x + a2*x^2 + ... + a_{k-1}*x^{k-1}
        coefficients = [secret]
        for _ in range(threshold - 1):
            coefficients.append(secrets.randbelow(self.prime))

        # Evaluar polinomio en puntos 1, 2, ..., n
        shares = []
        for x in range(1, num_shares + 1):
            y = self._evaluate_polynomial(coefficients, x)
            shares.append(Share(x=x, y=y))

        return shares

    def _evaluate_polynomial(
        self,
        coefficients: List[int],
        x: int
    ) -> int:
        """
        Evaluar polinomio en punto x usando Horner's method.
        """
        result = 0
        for coef in reversed(coefficients):
```

```
            result = (result * x + coef) % self.prime
        return result

    def reconstruct(self, shares: List[Share]) -> int:
        """
␣␣␣␣␣␣␣␣Reconstruir␣secreto␣desde␣shares␣usando␣interpolaci n␣de␣Lagrange.

␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣shares:␣Lista␣de␣al␣menos␣k␣shares

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣El␣secreto␣original
␣␣␣␣␣␣␣␣"""
        if len(shares) < 2:
            raise ValueError("Need␣at␣least␣2␣shares")

        # Interpolaci n de Lagrange en x=0
        secret = 0

        for i, share_i in enumerate(shares):
            # Calcular t rmino de Lagrange L_i(0)
            numerator = 1
            denominator = 1

            for j, share_j in enumerate(shares):
                if i != j:
                    numerator = (numerator * (-share_j.x)) % self.prime
                    denominator = (denominator * (share_i.x - share_j.x)) % se

            # L_i(0) = numerator / denominator
            lagrange = (numerator * self._mod_inverse(denominator, self.prime)

            # A adir contribuci n: y_i * L_i(0)
            secret = (secret + share_i.y * lagrange) % self.prime

        return secret

    def verify_share(
        self,
        share: Share,
        commitment: List[int]
    ) -> bool:
        """
␣␣␣␣␣␣␣␣Verificar␣share␣usando␣Feldman's␣VSS␣commitment.

␣␣␣␣␣␣␣␣El␣dealer␣publica␣commitments␣g^{a_i}␣para␣cada␣coeficiente.
␣␣␣␣␣␣␣␣Verificador␣comprueba:␣g^{share.y}␣==␣prod(commitment[j]^{x^j})
␣␣␣␣␣␣␣␣"""
        # Simplificado: en producci n usar grupo de curva el ptica
        pass
```

```
    def add_shares(
        self,
        shares_a: List[Share],
        shares_b: List[Share]
    ) -> List[Share]:
        """
        Sumar dos secretos compartidos (localmente por cada parte).

        Si A = {(i, a_i)} y B = {(i, b_i)}, entonces
        A + B = {(i, a_i + b_i)} representa secret_a + secret_b
        """
        assert len(shares_a) == len(shares_b)

        result = []
        for sa, sb in zip(shares_a, shares_b):
            assert sa.x == sb.x, "Share indices must match"
            result.append(Share(x=sa.x, y=(sa.y + sb.y) % self.prime))

        return result

    def multiply_share_by_constant(
        self,
        shares: List[Share],
        constant: int
    ) -> List[Share]:
        """
        Multiplicar secreto compartido por constante (local).
        """
        return [Share(x=s.x, y=(s.y * constant) % self.prime) for s in shares]


class VerifiableSecretSharing(ShamirSecretSharing):
    """
    Feldman's Verifiable Secret Sharing.
    Permite verificar que los shares son consistentes.
    """

    def __init__(self, prime: int = None, generator: int = None):
        super().__init__(prime)
        # Generador del grupo multiplicativo Z_p^*
        self.g = generator or 2

    def split_with_commitments(
        self,
        secret: int,
        threshold: int,
        num_shares: int
    ) -> Tuple[List[Share], List[int]]:
        """
        Dividir secreto y generar commitments para verificaci n.
```

```
␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣(shares, commitments) donde commitments[i] = g^{a_i}
␣␣␣␣␣␣␣␣"""
        # Generar shares normalmente
        coefficients = [secret]
        for _ in range(threshold - 1):
            coefficients.append(secrets.randbelow(self.prime))

        shares = []
        for x in range(1, num_shares + 1):
            y = self._evaluate_polynomial(coefficients, x)
            shares.append(Share(x=x, y=y))

        # Generar commitments: C_j = g^{a_j} mod p
        commitments = [pow(self.g, coef, self.prime) for coef in coefficients]

        return shares, commitments

    def verify_share_with_commitment(
        self,
        share: Share,
        commitments: List[int]
    ) -> bool:
        """
␣␣␣␣␣␣␣␣Verificar que un share es consistente con los commitments.

␣␣␣␣␣␣␣␣Verifica: g^{y_i} == prod_{j=0}^{k-1} C_j^{i^j}
␣␣␣␣␣␣␣␣"""
        # LHS: g^{share.y}
        lhs = pow(self.g, share.y, self.prime)

        # RHS: prod(C_j^{x^j})
        rhs = 1
        x_power = 1
        for commitment in commitments:
            rhs = (rhs * pow(commitment, x_power, self.prime)) % self.prime
            x_power = (x_power * share.x) % self.prime

        return lhs == rhs


def secret_sharing_demo():
    print("=== Shamir's Secret Sharing Demo ===\n")

    sss = ShamirSecretSharing(prime=65537)  # Primo peque o para demo

    # Configuraci n
    secret = 12345
    threshold = 3  # k = 3
    num_shares = 5  # n = 5
```

```
print(f"Secret: {secret}")
print(f"Threshold (k): {threshold}")
print(f"Total shares (n): {num_shares}")

# Dividir secreto
print("\n--- Splitting Secret ---")
shares = sss.split(secret, threshold, num_shares)

for share in shares:
    print(f"  Share {share.x}: ({share.x}, {share.y})")

# Reconstruir con exactamente k shares
print("\n--- Reconstructing with k=3 shares ---")
subset = shares[:3]
reconstructed = sss.reconstruct(subset)
print(f"Using shares: {[s.x for s in subset]}")
print(f"Reconstructed: {reconstructed}")
print(f"Correct: {reconstructed == secret}")

# Reconstruir con m s de k shares
print("\n--- Reconstructing with k=4 shares ---")
subset = shares[:4]
reconstructed = sss.reconstruct(subset)
print(f"Using shares: {[s.x for s in subset]}")
print(f"Reconstructed: {reconstructed}")
print(f"Correct: {reconstructed == secret}")

# Reconstruir con menos de k shares falla
print("\n--- Reconstructing with k=2 shares (insufficient) ---")
subset = shares[:2]
wrong_result = sss.reconstruct(subset)
print(f"Using shares: {[s.x for s in subset]}")
print(f"Result: {wrong_result}")
print(f"Correct: {wrong_result == secret}")
print("(With insufficient shares, result is random)")

# Operaciones sobre shares
print("\n--- Operations on Shares ---")
secret_a = 100
secret_b = 200

shares_a = sss.split(secret_a, threshold, num_shares)
shares_b = sss.split(secret_b, threshold, num_shares)

# Suma
shares_sum = sss.add_shares(shares_a, shares_b)
result_sum = sss.reconstruct(shares_sum[:3])
print(f"Secret A: {secret_a}, Secret B: {secret_b}")
print(f"Sum of shares reconstructs to: {result_sum}")
print(f"Expected: {secret_a + secret_b}")
```

```
    # Verifiable Secret Sharing
    print("\n—— Verifiable Secret Sharing ——")
    vss = VerifiableSecretSharing(prime=65537)

    shares, commitments = vss.split_with_commitments(secret, threshold, num_sh
    print(f"Commitments: {commitments[:3]}...")

    # Verificar cada share
    for share in shares[:2]:
        valid = vss.verify_share_with_commitment(share, commitments)
        print(f"Share {share.x} valid: {valid}")

secret_sharing_demo()
```

5.4. Oblivious Transfer

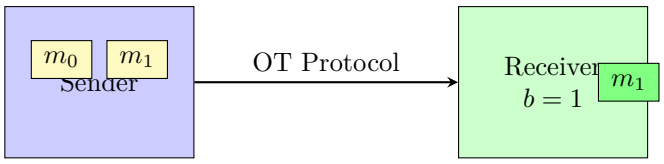
5.4.1. Definición y Variantes

Definición 5.6 (Oblivious Transfer (OT))**.** Protocolo criptográfico de dos partes donde:

- **Sender** tiene varios mensajes $(m_0, m_1, ..., m_{n-1})$
- **Receiver** tiene un índice de selección b
- Receiver obtiene m_b y nada más
- Sender no aprende qué índice eligió Receiver

Variantes:

- **1-out-of-2 OT:** Sender tiene 2 mensajes
- **1-out-of-n OT:** Sender tiene n mensajes
- **k-out-of-n OT:** Receiver obtiene k mensajes



Receiver obtiene m_b (solo m_1)
Sender no sabe que $b = 1$

Figura 5.4: 1-out-of-2 Oblivious Transfer

[OT basado en RSA (Simplificado)] **Setup:**

- Sender genera par RSA: (n, e) público, d secreto
- Sender tiene mensajes m_0, m_1

Protocolo:

1. Sender envía x_0, x_1 aleatorios

2. Receiver elige $b \in \{0, 1\}$, genera k aleatorio
3. Receiver envía $v = (x_b + k^e) \mod n$
4. Sender computa:
   - $k_0 = (v - x_0)^d \mod n$
   - $k_1 = (v - x_1)^d \mod n$
5. Sender envía $m'_0 = m_0 + k_0$, $m'_1 = m_1 + k_1$
6. Receiver computa $m_b = m'_b - k$ (solo puede computar uno)

Seguridad:

- Sender no sabe b (ambos k_i parecen aleatorios)
- Receiver solo puede descifrar m_b (no conoce el otro k)

Listing 5.4: Implementación de Oblivious Transfer

```
from dataclasses import dataclass
from typing import Tuple, List
import secrets
import hashlib

@dataclass
class OTSenderState:
    """Estado del sender durante OT."""
    messages: List[bytes]
    random_values: List[int]
    modulus: int
    public_exponent: int
    private_exponent: int

@dataclass
class OTReceiverState:
    """Estado del receiver durante OT."""
    choice: int
    random_k: int
    modulus: int

class ObliviousTransfer:
    """
    Implementación de 1-out-of-2 Oblivious Transfer.
    """

    def __init__(self, key_bits: int = 256):
        self.key_bits = key_bits

    def _generate_rsa_params(self) -> Tuple[int, int, int]:
        """
        Generar parámetros RSA simplificados.
        En producción usar librería criptográfica real.
        """
```

```
        # Primos peque os para demo (NO seguros)
        p = 65537
        q = 65521
        n = p * q
        phi = (p - 1) * (q - 1)
        e = 17
        d = pow(e, -1, phi)
        return n, e, d

    # ========== SENDER SIDE ==========

    def sender_init(
        self,
        m0: bytes,
        m1: bytes
    ) -> Tuple[OTSenderState, Tuple[int, int]]:
        """
␣␣␣␣␣␣␣␣Sender␣inicializa␣OT␣con␣sus␣dos␣mensajes.

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣(state,␣(x0,␣x1))␣-␣estado␣interno␣y␣valores␣p blicos
␣␣␣␣␣␣␣␣"""
        n, e, d = self._generate_rsa_params()

        # Generar valores aleatorios
        x0 = secrets.randbelow(n)
        x1 = secrets.randbelow(n)

        state = OTSenderState(
            messages=[m0, m1],
            random_values=[x0, x1],
            modulus=n,
            public_exponent=e,
            private_exponent=d
        )

        return state, (x0, x1)

    def sender_transfer(
        self,
        state: OTSenderState,
        v: int
    ) -> Tuple[bytes, bytes]:
        """
␣␣␣␣␣␣␣␣Sender␣responde␣al␣valor␣v␣del␣receiver.

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣(e0,␣e1)␣-␣mensajes␣encriptados
␣␣␣␣␣␣␣␣"""
        n = state.modulus
        d = state.private_exponent
```

```
        x0, x1 = state.random_values

        # Computar ambas posibles claves
        k0 = pow((v - x0) % n, d, n)
        k1 = pow((v - x1) % n, d, n)

        # Encriptar mensajes
        m0, m1 = state.messages

        # Usar hash de k como clave de encriptaci n
        key0 = hashlib.sha256(k0.to_bytes(32, 'big')).digest()
        key1 = hashlib.sha256(k1.to_bytes(32, 'big')).digest()

        # XOR para simplicidad (en producci n: AES-GCM)
        e0 = bytes(a ^ b for a, b in zip(m0.ljust(32, b'\x00'), key0))
        e1 = bytes(a ^ b for a, b in zip(m1.ljust(32, b'\x00'), key1))

        return e0, e1

    # ========== RECEIVER SIDE ==========

    def receiver_choose(
        self,
        choice: int,
        x0: int,
        x1: int,
        modulus: int,
        public_exp: int
    ) -> Tuple[OTReceiverState, int]:
        """
␣␣␣␣␣␣␣␣Receiver␣elige␣cu l␣mensaje␣quiere.

␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣choice:␣0␣o␣1
␣␣␣␣␣␣␣␣␣␣␣␣x0,␣x1:␣Valores␣p blicos␣del␣sender
␣␣␣␣␣␣␣␣␣␣␣␣modulus,␣public_exp:␣Par metros␣RSA

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣(state,␣v)␣-␣estado␣interno␣y␣valor␣para␣sender
␣␣␣␣␣␣␣␣"""
        n = modulus
        e = public_exp

        # Generar k aleatorio
        k = secrets.randbelow(n)

        # v = x_choice + k^e mod n
        x_choice = x0 if choice == 0 else x1
        v = (x_choice + pow(k, e, n)) % n

        state = OTReceiverState(
```

```
            choice=choice,
            random_k=k,
            modulus=n
        )

        return state, v

    def receiver_retrieve(
        self,
        state: OTReceiverState,
        e0: bytes,
        e1: bytes
    ) -> bytes:
        """
        Receiver obtiene el mensaje elegido.

        Returns:
            El mensaje m_choice
        """
        k = state.random_k
        choice = state.choice

        # Derivar clave
        key = hashlib.sha256(k.to_bytes(32, 'big')).digest()

        # Descifrar el mensaje elegido
        encrypted = e0 if choice == 0 else e1
        message = bytes(a ^ b for a, b in zip(encrypted, key))

        # Remover padding
        return message.rstrip(b'\x00')


class OTExtension:
    """
    OT Extension: generar muchos OTs desde pocos OTs base.
    T cnica clave para eficiencia en MPC.
    """

    def __init__(self, base_ot: ObliviousTransfer, security_param: int = 128):
        self.base_ot = base_ot
        self.kappa = security_param

    def extend(
        self,
        num_ots: int,
        sender_messages: List[Tuple[bytes, bytes]],
        receiver_choices: List[int]
    ) -> List[bytes]:
        """
        Ejecutar muchos OTs eficientemente.
```

```
        La idea: usar   OTs base para generar m OTs donde m >>   .
        Basado en IKNP03 OT extension.
        """
        # Simplificado: en implementaci n real, matriz de bits y PRG
        results = []

        for (m0, m1), choice in zip(sender_messages, receiver_choices):
            # Cada OT individual
            state_s, (x0, x1) = self.base_ot.sender_init(m0, m1)
            state_r, v = self.base_ot.receiver_choose(
                choice, x0, x1,
                state_s.modulus, state_s.public_exponent
            )
            e0, e1 = self.base_ot.sender_transfer(state_s, v)
            result = self.base_ot.receiver_retrieve(state_r, e0, e1)
            results.append(result)

        return results


def oblivious_transfer_demo():
    print("=== Oblivious Transfer Demo ===\n")

    ot = ObliviousTransfer()

    # Mensajes del sender
    m0 = b"Secret message ZERO"
    m1 = b"Secret message ONE"

    print(f"Sender has messages:")
    print(f"  m0: {m0}")
    print(f"  m1: {m1}")

    # Receiver quiere m1
    choice = 1
    print(f"\nReceiver wants message: m{choice}")

    # Protocolo
    print("\n--- OT Protocol ---")

    # Paso 1: Sender inicializa
    state_sender, (x0, x1) = ot.sender_init(m0, m1)
    print(f"Sender sends x0, x1: {x0}, {x1}")

    # Paso 2: Receiver elige y env a v
    state_receiver, v = ot.receiver_choose(
        choice, x0, x1,
        state_sender.modulus,
        state_sender.public_exponent
    )
```

```
    print(f"Receiver sends v: {v}")
    print("  (Sender cannot determine choice from v)")

    # Paso 3: Sender encripta y env a
    e0, e1 = ot.sender_transfer(state_sender, v)
    print(f"Sender sends e0, e1 (encrypted)")

    # Paso 4: Receiver descifra
    result = ot.receiver_retrieve(state_receiver, e0, e1)
    print(f"\nReceiver obtains: {result}")

    # Verificar propiedades
    print("\n--- Security Properties ---")
    print(f"   Receiver got m{choice}: {result == (m0 if choice == 0 else m1)}")
    print("   Sender does not know choice (v reveals nothing)")
    print("   Receiver cannot decrypt the other message")

oblivious_transfer_demo()
```

5.5. Private Set Intersection (PSI)

5.5.1. Definición y Aplicaciones

Definición 5.7 (Private Set Intersection)**.** Protocolo que permite a dos partes encontrar elementos comunes en sus conjuntos sin revelar el resto:

Inputs:

- Alice tiene conjunto $A = \{a_1, ..., a_n\}$
- Bob tiene conjunto $B = \{b_1, ..., b_m\}$

Output:

- Intersección $A \cap B$
- Ni Alice ni Bob aprenden elementos del otro que no estén en la intersección

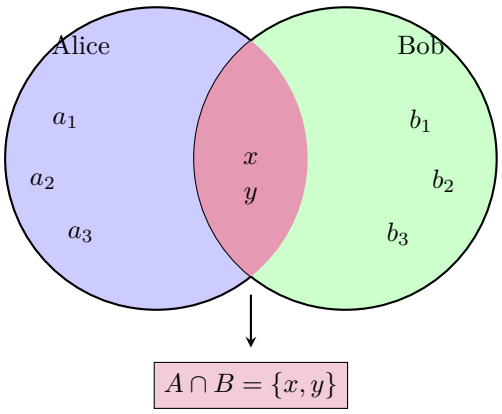


Figura 5.5: Private Set Intersection: solo se revela la intersección

[Aplicaciones de PSI] **1. Contact Discovery (Messaging apps):**

- Tu lista de contactos: A

- Usuarios del servicio: B
- Resultado: Quiénes de tus contactos usan la app
- Sin revelar tu lista completa al servidor

2. Password Breach Detection:

- Tus contraseñas (hasheadas): A
- Base de datos de leaks: B
- Resultado: Qué contraseñas están comprometidas
- Sin enviar tus contraseñas al servicio

3. Ad Conversion Matching:

- Usuarios que vieron ad: A
- Usuarios que compraron: B
- Resultado: Conversiones atribuidas
- Sin compartir bases de datos completas

4. Fraud Detection:

- Clientes del banco A: A
- Clientes del banco B: B
- Resultado: Clientes en común (potencial fraude)
- Sin revelar todos los clientes

Listing 5.5: Implementación de Private Set Intersection

```
from dataclasses import dataclass
from typing import Set, List, Tuple
import hashlib
import secrets
from functools import reduce

class NaivePSI:
    """
␣␣␣␣PSI␣usando␣hashing␣(inseguro␣pero␣ilustrativo).
␣␣␣␣"""

    def __init__(self, hash_function=hashlib.sha256):
        self.hash_fn = hash_function

    def hash_element(self, element: str, salt: bytes = b'') -> bytes:
        """Hash␣un␣elemento."""
        return self.hash_fn(salt + element.encode()).digest()

    def intersect(
        self,
```

```
        set_a: Set[str],
        set_b: Set[str]
    ) -> Set[str]:
        """
        Encontrar intersecci n usando hashes.
        NOTA: Vulnerable a ataques de diccionario.
        """
        salt = secrets.token_bytes(16)

        # Alice hashea su conjunto
        hashed_a = {self.hash_element(x, salt): x for x in set_a}

        # Bob hashea su conjunto
        hashed_b = {self.hash_element(x, salt) for x in set_b}

        # Intersecci n de hashes
        common_hashes = set(hashed_a.keys()) & hashed_b

        # Mapear de vuelta a elementos
        intersection = {hashed_a[h] for h in common_hashes}

        return intersection


class DiffieHellmanPSI:
    """
    PSI basado en Diffie-Hellman.
    M s seguro que hashing simple.
    """

    def __init__(self, prime: int = None, generator: int = 2):
        # Primo grande para DH
        self.p = prime or int(
            "FFFFFFFFFFFFFFFFC90FDAA22168C234C4C6628B80DC1CD1"
            "29024E088A67CC74020BBEA63B139B22514A08798E3404DD"
            "EF9519B3CD3A431B302B0A6DF25F14374FE1356D6D51C245"
            "E485B576625E7EC6F44C42E9A637ED6B0BFF5CB6F406B7ED"
            "EE386BFB5A899FA5AE9F24117C4B1FE649286651ECE65381"
            "FFFFFFFFFFFFFFFF", 16
        )
        self.g = generator

    def _hash_to_group(self, element: str) -> int:
        """Hash elemento a elemento del grupo."""
        h = int(hashlib.sha256(element.encode()).hexdigest(), 16)
        return pow(self.g, h, self.p)

    def alice_round1(self, set_a: Set[str]) -> Tuple[int, List[int]]:
        """
        Alice: Primera ronda.
```

```
␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣␣␣(secret_a, [H(a)^secret_a for a in A])
␣␣␣␣␣␣␣␣"""
        # Secreto de Alice
        secret_a = secrets.randbelow(self.p - 2) + 1

        # H(a)^secret_a para cada elemento
        encrypted_a = []
        for element in set_a:
            h = self._hash_to_group(element)
            encrypted = pow(h, secret_a, self.p)
            encrypted_a.append(encrypted)

        return secret_a, encrypted_a

    def bob_process(
        self,
        set_b: Set[str],
        alice_encrypted: List[int]
    ) -> Tuple[int, List[int], List[int]]:
        """
␣␣␣␣␣␣␣␣Bob: Procesar y encriptar.

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣␣␣(secret_b,
␣␣␣␣␣␣␣␣␣␣␣␣␣␣[H(a)^(secret_a * secret_b)], ␣␣# Alice's elements double-encrypted
␣␣␣␣␣␣␣␣␣␣␣␣␣␣[H(b)^secret_b]) ␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣# Bob's elements encrypted
␣␣␣␣␣␣␣␣"""
        # Secreto de Bob
        secret_b = secrets.randbelow(self.p - 2) + 1

        # Double-encrypt elementos de Alice
        double_encrypted_a = [
            pow(enc, secret_b, self.p) for enc in alice_encrypted
        ]

        # Encrypt elementos de Bob
        encrypted_b = []
        for element in set_b:
            h = self._hash_to_group(element)
            encrypted = pow(h, secret_b, self.p)
            encrypted_b.append(encrypted)

        return secret_b, double_encrypted_a, encrypted_b

    def alice_round2(
        self,
        secret_a: int,
        bob_encrypted: List[int]
    ) -> List[int]:
        """
```

```
        Alice: Segunda ronda - double-encrypt elementos de Bob.

        Returns:
            [H(b)^(secret_a * secret_b)]
        """
        double_encrypted_b = [
            pow(enc, secret_a, self.p) for enc in bob_encrypted
        ]
        return double_encrypted_b

    def find_intersection(
        self,
        double_encrypted_a: List[int],  # H(a)^(ab)
        double_encrypted_b: List[int],  # H(b)^(ab)
        original_elements_a: List[str]
    ) -> Set[str]:
        """
        Encontrar intersecci n comparando double-encrypted values.
        """
        # Crear mapping para elementos de Alice
        a_map = dict(zip(double_encrypted_a, original_elements_a))

        # Encontrar matches
        b_set = set(double_encrypted_b)
        intersection = set()

        for enc_a, element in a_map.items():
            if enc_a in b_set:
                intersection.add(element)

        return intersection


class BloomFilterPSI:
    """
    PSI usando Bloom filters para conjuntos grandes.
    Trade-off: puede tener falsos positivos.
    """

    def __init__(
        self,
        expected_elements: int,
        false_positive_rate: float = 0.01
    ):
        # Calcular tama o  ptimo
        import math
        self.size = int(-expected_elements * math.log(false_positive_rate) / (
        self.num_hashes = int(self.size / expected_elements * math.log(2))
        self.bit_array = [False] * self.size

    def _get_hash_values(self, element: str) -> List[int]:
```

```
        """Obtener ndices de hash para un elemento."""
        indices = []
        for i in range(self.num_hashes):
            h = hashlib.sha256(f"{i}:{element}".encode()).digest()
            index = int.from_bytes(h[:4], 'big') % self.size
            indices.append(index)
        return indices

    def add(self, element: str):
        """A adir elemento al filtro."""
        for index in self._get_hash_values(element):
            self.bit_array[index] = True

    def contains(self, element: str) -> bool:
        """Verificar si elemento posiblemente est en el filtro."""
        return all(self.bit_array[i] for i in self._get_hash_values(element))

    def intersect_with_set(self, other_set: Set[str]) -> Set[str]:
        """Encontrar posible intersecci n."""
        return {x for x in other_set if self.contains(x)}


def psi_demo():
    print("=== Private Set Intersection Demo ===\n")

    # Conjuntos de ejemplo
    alice_contacts = {"alice@email.com", "bob@email.com", "carol@email.com", "da
    app_users = {"bob@email.com", "eve@email.com", "carol@email.com", "frank@ema

    print("Alice's contacts:", alice_contacts)
    print("App users:", app_users)
    print("Expected intersection:", alice_contacts & app_users)

    # PSI con Diffie-Hellman
    print("\n--- Diffie-Hellman PSI ---")

    dh_psi = DiffieHellmanPSI(prime=65537)  # Primo peque o para demo

    # Ronda 1: Alice
    alice_list = list(alice_contacts)
    secret_a, encrypted_a = dh_psi.alice_round1(alice_contacts)
    print(f"Alice sends {len(encrypted_a)} encrypted elements")

    # Bob procesa
    secret_b, double_enc_a, encrypted_b = dh_psi.bob_process(
        app_users, encrypted_a
    )
    print(f"Bob sends back double-encrypted + his {len(encrypted_b)} elements")

    # Ronda 2: Alice
    double_enc_b = dh_psi.alice_round2(secret_a, encrypted_b)
```

```
    # Encontrar intersecci n
    intersection = dh_psi.find_intersection(
        double_enc_a, double_enc_b, alice_list
    )

    print(f"\nIntersection found: {intersection}")
    print(f"Correct: {intersection == (alice_contacts & app_users)}")

    # PSI con Bloom Filter
    print("\n--- Bloom Filter PSI (approximate) ---")

    bf_psi = BloomFilterPSI(expected_elements=100)

    # Bob a ade sus usuarios al filtro
    for user in app_users:
        bf_psi.add(user)

    # Alice consulta el filtro
    possible_intersection = bf_psi.intersect_with_set(alice_contacts)
    print(f"Possible intersection: {possible_intersection}")
    print("(May have false positives)")

psi_demo()
```

5.6. Secure Enclaves

5.6.1. Trusted Execution Environments (TEE)

Definición 5.8 (Trusted Execution Environment)**.** Hardware aislado que proporciona ejecución segura de código, garantizando:

1. **Confidencialidad:** Datos y código protegidos incluso del OS/hypervisor
2. **Integridad:** Código no puede ser modificado durante ejecución
3. **Attestation:** Prueba criptográfica de qué código se ejecuta

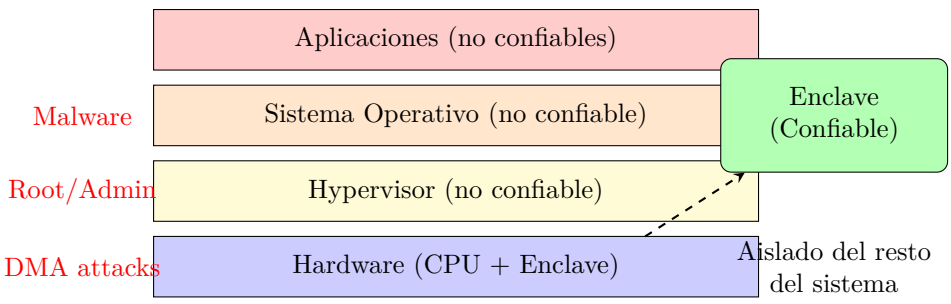


Figura 5.6: TEE aísla código incluso de software privilegiado

5.6.2. Intel SGX

Definición 5.9 (Intel SGX (Software Guard Extensions))**.** Extensiones de CPU que crean "enclaves" - regiones de memoria aisladas:

Características:

- Memoria encriptada en RAM (MEE)
- Código y datos protegidos del OS
- Attestation remota vía Intel
- Sealing para persistencia

Limitaciones:

- Descontinuado en CPUs consumer (desde 11th gen)
- Disponible en Xeon (servidor)
- Límite de memoria EPC (256MB típico)

Listing 5.6: Simulación conceptual de operaciones SGX

```
from dataclasses import dataclass, field
from typing import Dict, Optional, Tuple, List
import hashlib
import secrets
from enum import Enum

class EnclaveState(Enum):
    CREATED = "created"
    INITIALIZED = "initialized"
    RUNNING = "running"
    DESTROYED = "destroyed"

@dataclass
class EnclaveIdentity:
    """Identidad de un enclave para attestation."""
    mrenclave: bytes  # Hash del c digo del enclave
    mrsigner: bytes   # Hash de la clave del firmante
    product_id: int
    security_version: int

@dataclass
class SGXEnclave:
    """
    Simulaci n de un enclave SGX.
    """
    enclave_id: str
    identity: EnclaveIdentity
    state: EnclaveState = EnclaveState.CREATED

    # Memoria protegida (simulada)
    _protected_memory: Dict[str, bytes] = field(default_factory=dict)
```

```
    # Clave de sealing (derivada de CPU + enclave identity)
    _sealing_key: bytes = field(default_factory=lambda: secrets.token_bytes(32

    def initialize(self, code_hash: bytes):
        """Inicializar enclave con c digo."""
        self.identity.mrenclave = code_hash
        self.state = EnclaveState.INITIALIZED

    def ecall(self, function_name: str, params: bytes) -> bytes:
        """
        Llamada al enclave (ECALL).
        Ejecutar funci n dentro del enclave.
        """
        if self.state != EnclaveState.RUNNING:
            raise RuntimeError("Enclave not running")

        # Simular ejecuci n dentro del enclave
        # En SGX real, esto ejecuta c digo en modo protegido

        if function_name == "store_secret":
            key = hashlib.sha256(params[:32]).hexdigest()[:16]
            self._protected_memory[key] = params[32:]
            return b"OK"

        elif function_name == "compute_on_secret":
            key = hashlib.sha256(params[:32]).hexdigest()[:16]
            if key in self._protected_memory:
                secret = self._protected_memory[key]
                # Computar sobre el secreto
                result = hashlib.sha256(secret).digest()
                return result
            return b"NOT_FOUND"

        elif function_name == "get_report":
            return self._generate_report(params)

        return b"UNKNOWN_FUNCTION"

    def _generate_report(self, target_info: bytes) -> bytes:
        """
        Generar reporte para attestation.
        """
        report_data = (
            self.identity.mrenclave +
            self.identity.mrsigner +
            self.product_id.to_bytes(2, 'big') +
            self.security_version.to_bytes(2, 'big') +
            target_info
        )
```

```
        # En SGX real, esto es firmado por la CPU
        report_mac = hashlib.sha256(report_data + self._sealing_key).digest()

        return report_data + report_mac

    def seal_data(self, plaintext: bytes) -> bytes:
        """
␣␣␣␣␣␣␣␣Seal:␣Encriptar␣datos␣vinculados␣al␣enclave.
␣␣␣␣␣␣␣␣Solo␣este␣enclave␣(o␣uno␣id ntico)␣puede␣unseal.
␣␣␣␣␣␣␣␣"""
        nonce = secrets.token_bytes(12)

        # Derivar clave de sealing
        # En SGX real, derivada de CPU key + enclave identity
        seal_key = hashlib.sha256(
            self._sealing_key + self.identity.mrenclave
        ).digest()

        # Encriptar (simplificado)
        ciphertext = bytes(p ^ k for p, k in zip(
            plaintext.ljust(len(plaintext) + 16, b'\x00'),
            (seal_key * 10)[:len(plaintext) + 16]
        ))

        return nonce + ciphertext

    def unseal_data(self, sealed: bytes) -> bytes:
        """Unseal:␣Descifrar␣datos␣sellados."""
        nonce = sealed[:12]
        ciphertext = sealed[12:]

        seal_key = hashlib.sha256(
            self._sealing_key + self.identity.mrenclave
        ).digest()

        plaintext = bytes(c ^ k for c, k in zip(
            ciphertext,
            (seal_key * 10)[:len(ciphertext)]
        ))

        return plaintext.rstrip(b'\x00')


class RemoteAttestation:
    """
␣␣␣␣Simulaci n␣de␣attestation␣remota␣SGX.
␣␣␣␣"""

    def __init__(self):
        # "Intel Attestation Service" simulado
        self.ias_key = secrets.token_bytes(32)
```

```
        self.trusted_enclaves: Dict[bytes, bool] = {}

    def register_trusted_enclave(self, mrenclave: bytes):
        """Registrar un c digo de enclave como confiable."""
        self.trusted_enclaves[mrenclave] = True

    def verify_quote(
        self,
        quote: bytes,
        expected_mrenclave: bytes
    ) -> Tuple[bool, str]:
        """
        Verificar quote de attestation.

        Returns:
            (is_valid, message)
        """
        # Extraer datos del quote
        if len(quote) < 64:
            return False, "Invalid quote format"

        mrenclave = quote[:32]
        mrsigner = quote[32:64]

        # Verificar que el c digo es el esperado
        if mrenclave != expected_mrenclave:
            return False, "MRENCLAVE mismatch"

        # Verificar que est  en lista de confianza
        if mrenclave not in self.trusted_enclaves:
            return False, "Enclave not in trusted list"

        # Verificar firma (simplificado)
        # En producci n: verificar firma EPID/ECDSA de Intel

        return True, "Attestation successful"


def sgx_demo():
    print("=== Intel SGX Simulation Demo ===\n")

    # Crear enclave
    identity = EnclaveIdentity(
        mrenclave=b'\x00' * 32,
        mrsigner=hashlib.sha256(b"developer_key").digest(),
        product_id=1,
        security_version=1
    )

    enclave = SGXEnclave(
        enclave_id="enclave_001",
```

```
        identity=identity
    )

    # Inicializar con c digo
    code_hash = hashlib.sha256(b"my_enclave_code_v1").digest()
    enclave.initialize(code_hash)
    enclave.identity.mrenclave = code_hash
    enclave.state = EnclaveState.RUNNING

    print(f"Enclave created: {enclave.enclave_id}")
    print(f"MRENCLAVE: {enclave.identity.mrenclave.hex()[:32]}...")

    # Almacenar secreto en el enclave
    print("\n--- Storing Secret in Enclave ---")
    secret_key = b"my_secret_key_identifier_here!!!"
    secret_value = b"This is my super secret data that only the enclave can see"

    result = enclave.ecall("store_secret", secret_key + secret_value)
    print(f"Store result: {result}")

    # Computar sobre el secreto
    print("\n--- Computing on Secret ---")
    hash_result = enclave.ecall("compute_on_secret", secret_key)
    print(f"Hash of secret (computed inside enclave): {hash_result.hex()[:32]}..

    # Attestation
    print("\n--- Remote Attestation ---")
    ias = RemoteAttestation()
    ias.register_trusted_enclave(code_hash)

    target_info = b"verifier_identity"
    report = enclave.ecall("get_report", target_info)

    is_valid, message = ias.verify_quote(report, code_hash)
    print(f"Attestation valid: {is_valid}")
    print(f"Message: {message}")

    # Sealing
    print("\n--- Data Sealing ---")
    sensitive_data = b"Persistent secret that survives enclave restart"

    sealed = enclave.seal_data(sensitive_data)
    print(f"Sealed data: {sealed.hex()[:40]}...")

    unsealed = enclave.unseal_data(sealed)
    print(f"Unsealed: {unsealed}")
    print(f"Match: {unsealed == sensitive_data}")

sgx_demo()
```

5.6.3. Comparación de TEEs

Característica	Intel SGX	ARM TrustZone	AMD SEV	AWS Nitro
Granularidad	Proceso	Sistema	VM	VM
Aislamiento	Enclave	Mundo seguro	VM encriptada	Enclave dedicado
Attestation	Remota	Local	Remota	Remota
Memoria EPC	256MB	Configurable	Toda RAM	Variable
Side-channels	Vulnerable	Menos expuesto	Menos expuesto	Diseño diferente
Disponibilidad	Server Xeon	Móviles/IoT	EPYC	AWS Cloud

Tabla 5.3: Comparación de Trusted Execution Environments

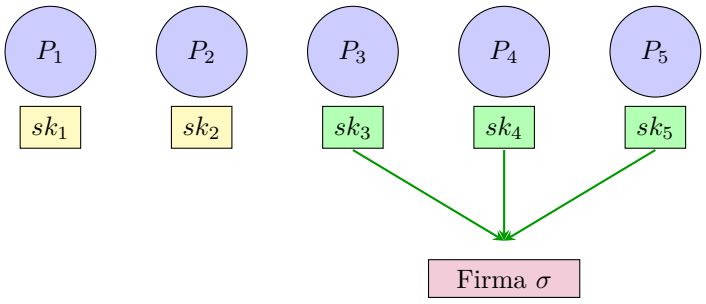
5.7. Threshold Signatures

5.7.1. Fundamentos

Definición 5.10 (Threshold Signature)**.** Esquema de firma donde k de n partes deben cooperar para firmar:

Propiedades:

- La clave privada **nunca existe completa**
- Cada parte tiene un share de la clave
- Cualquier subconjunto de k partes puede firmar
- Menos de k partes no pueden firmar ni recuperar la clave
- La firma resultante es indistinguible de firma normal



3-of-5: Cualquier 3 partes pueden firmar

Figura 5.7: Threshold signature: 3 de 5 partes cooperan para firmar

5.7.2. Threshold ECDSA

[Threshold ECDSA] **Desafío:** ECDSA no es "threshold-friendly" como Schnorr/BLS. La firma ECDSA es:

$$s = k^{-1}(z + r \cdot sk) \mod n$$

donde k es nonce aleatorio, r es coordenada x del punto $k \cdot G$.
Problema: Requiere multiplicación e inversión de secretos compartidos.
Soluciones:

- **GG18/GG20:** Protocolo de Gennaro-Goldfeder
- **CGGMP:** Canetti, Gennaro, Goldfeder, Makriyannis, Peled
- Usan MPC para multiplicación e inversión

5.7.3. FROST (Flexible Round-Optimized Schnorr Threshold)

Definición 5.11 (FROST)**.** Protocolo de threshold Schnorr signatures optimizado:
Ventajas sobre threshold ECDSA:

- Solo 2 rondas de comunicación
- Compatible con agregación de firmas
- Más eficiente computacionalmente
- Seguro contra adversario malicioso

Firma Schnorr:

$$\sigma = (R, s) \text{ donde } s = k + c \cdot sk$$

con $c = H(R||pk||m)$

Listing 5.7: Implementación de Threshold Schnorr (FROST simplificado)

```
from dataclasses import dataclass
from typing import List, Tuple, Dict
import secrets
import hashlib

# Par metros de curva secp256k1 (simplificados)
P = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F
N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
Gx = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
Gy = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8

@dataclass
class Point:
    x: int
    y: int

    def __add__(self, other):
        """Suma de puntos en curva el ptica."""
        if self.x == 0 and self.y == 0:
            return other
        if other.x == 0 and other.y == 0:
            return self

        if self.x == other.x and self.y != other.y:
            return Point(0, 0)  # Punto en infinito

        if self.x == other.x:
            # Point doubling
            lam = (3 * self.x * self.x * pow(2 * self.y, -1, P)) % P
```

```
        else:
            lam = ((other.y - self.y) * pow(other.x - self.x, -1, P)) % P

        x3 = (lam * lam - self.x - other.x) % P
        y3 = (lam * (self.x - x3) - self.y) % P

        return Point(x3, y3)

    def __mul__(self, scalar: int):
        """Multiplicaci n escalar."""
        result = Point(0, 0)
        addend = self

        while scalar:
            if scalar & 1:
                result = result + addend
            addend = addend + addend
            scalar >>= 1

        return result

G = Point(Gx, Gy)

@dataclass
class KeyShare:
    """Share de clave privada."""
    index: int
    value: int

@dataclass
class NonceShare:
    """Share de nonce para firma."""
    index: int
    d: int  # Hiding nonce
    e: int  # Binding nonce
    D: Point  # d*G
    E: Point  # e*G

class ThresholdSchnorr:
    """
    Implementaci n simplificada de Threshold Schnorr (estilo FROST).
    """

    def __init__(self, threshold: int, num_parties: int):
        self.t = threshold
        self.n = num_parties
        self.key_shares: Dict[int, KeyShare] = {}
        self.public_key: Point = None

    def _lagrange_coefficient(
        self,
```

```
        participant_index: int,
        participant_indices: List[int]
    ) -> int:
        """
␣␣␣␣␣␣␣␣Calcular␣coeficiente␣de␣Lagrange␣para␣interpolaci n.
␣␣␣␣␣␣␣␣"""
        numerator = 1
        denominator = 1

        for j in participant_indices:
            if j != participant_index:
                numerator = (numerator * (-j)) % N
                denominator = (denominator * (participant_index - j)) % N

        return (numerator * pow(denominator, -1, N)) % N

    def distributed_keygen(self) -> Point:
        """
␣␣␣␣␣␣␣␣Distributed␣Key␣Generation␣(DKG).
␣␣␣␣␣␣␣␣Cada␣parte␣genera␣polynomial␣y␣comparte␣evaluaciones.

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣Public␣key␣(punto␣en␣curva)
␣␣␣␣␣␣␣␣"""
        # Fase 1: Cada parte genera polynomial secreto
        polynomials = []
        commitments = []

        for i in range(1, self.n + 1):
            # Polynomial de grado t-1
            coeffs = [secrets.randbelow(N) for _ in range(self.t)]
            polynomials.append(coeffs)

            # Commitment: C_j = coeffs[j] * G
            party_commitments = [G * c for c in coeffs]
            commitments.append(party_commitments)

        # Fase 2: Cada parte calcula su share sumando evaluaciones
        for i in range(1, self.n + 1):
            share_value = 0
            for poly in polynomials:
                # Evaluar polynomial en punto i
                eval_i = sum(poly[j] * pow(i, j, N) for j in range(len(poly))) %
                share_value = (share_value + eval_i) % N

            self.key_shares[i] = KeyShare(index=i, value=share_value)

        # Public key: suma de los primeros coeficientes * G
        pk_x = 0
        pk_y = 0
        for party_commits in commitments:
```

```
            pk = party_commits[0]
            if pk_x == 0:
                pk_x, pk_y = pk.x, pk.y
            else:
                combined = Point(pk_x, pk_y) + pk
                pk_x, pk_y = combined.x, combined.y

        self.public_key = Point(pk_x, pk_y)
        return self.public_key

    def generate_nonce_share(self, party_index: int) -> NonceShare:
        """
␣␣␣␣␣␣␣␣Generar␣share␣de␣nonce␣para␣una␣firma.
␣␣␣␣␣␣␣␣Cada␣parte␣genera␣(d_i,␣e_i)␣y␣publica␣(D_i,␣E_i).
␣␣␣␣␣␣␣␣"""
        d = secrets.randbelow(N)
        e = secrets.randbelow(N)
        D = G * d
        E = G * e

        return NonceShare(index=party_index, d=d, e=e, D=D, E=E)

    def aggregate_nonces(
        self,
        nonce_shares: List[NonceShare],
        message: bytes,
        participant_indices: List[int]
    ) -> Tuple[Point, int]:
        """
␣␣␣␣␣␣␣␣Agregar␣nonces␣para␣formar␣R␣del␣grupo.

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣(R,␣rho)␣donde␣rho␣es␣el␣binding␣factor
␣␣␣␣␣␣␣␣"""
        # Calcular binding factor rho = H(participant_indices, message, {D_i,
        binding_data = (
            str(participant_indices).encode() +
            message +
            b''.join(
                ns.D.x.to_bytes(32, 'big') + ns.E.x.to_bytes(32, 'big')
                for ns in nonce_shares
            )
        )
        rho = int(hashlib.sha256(binding_data).hexdigest(), 16) % N

        # R = sum(D_i + rho * E_i)
        R = Point(0, 0)
        for ns in nonce_shares:
            Ri = ns.D + (ns.E * rho)
            R = R + Ri
```

```
        return R, rho

    def compute_partial_signature(
        self,
        party_index: int,
        nonce_share: NonceShare,
        message: bytes,
        R: Point,
        rho: int,
        participant_indices: List[int]
    ) -> int:
        """
        Calcular firma parcial de una parte.
        """
        # Challenge c = H(R || pk || message)
        c_data = (
            R.x.to_bytes(32, 'big') +
            self.public_key.x.to_bytes(32, 'big') +
            message
        )
        c = int(hashlib.sha256(c_data).hexdigest(), 16) % N

        # Coeficiente de Lagrange
        lambda_i = self._lagrange_coefficient(party_index, participant_indices)

        # z_i = d_i + rho * e_i + lambda_i * sk_i * c
        d_i = nonce_share.d
        e_i = nonce_share.e
        sk_i = self.key_shares[party_index].value

        z_i = (d_i + rho * e_i + lambda_i * sk_i * c) % N

        return z_i

    def aggregate_signature(
        self,
        R: Point,
        partial_signatures: List[int]
    ) -> Tuple[Point, int]:
        """
        Agregar firmas parciales en firma final.

        Returns:
            (R, s) - firma Schnorr
        """
        s = sum(partial_signatures) % N
        return (R, s)

    def verify(
        self,
        message: bytes,
```

```
        signature: Tuple[Point, int]
    ) -> bool:
        """
␣␣␣␣␣␣␣␣Verificar␣firma␣Schnorr.
␣␣␣␣␣␣␣␣"""
        R, s = signature

        # c = H(R || pk || message)
        c_data = (
            R.x.to_bytes(32, 'big') +
            self.public_key.x.to_bytes(32, 'big') +
            message
        )
        c = int(hashlib.sha256(c_data).hexdigest(), 16) % N

        # Verificar: s*G == R + c*pk
        lhs = G * s
        rhs = R + (self.public_key * c)

        return lhs.x == rhs.x and lhs.y == rhs.y


def threshold_schnorr_demo():
    print("===␣Threshold␣Schnorr␣(FROST-like)␣Demo␣===\n")

    threshold = 3
    num_parties = 5

    ts = ThresholdSchnorr(threshold, num_parties)

    # Distributed Key Generation
    print("---␣Distributed␣Key␣Generation␣---")
    pk = ts.distributed_keygen()
    print(f"Public␣key:␣({pk.x␣%␣(10**10)}...,␣{pk.y␣%␣(10**10)}...)")
    print(f"Key␣shares␣distributed␣to␣{num_parties}␣parties")

    # Signing
    message = b"Transfer␣100␣BTC␣to␣Alice"
    print(f"\n---␣Signing␣Message␣---")
    print(f"Message:␣{message}")

    # Participantes que firmar n (cualquier 3 de 5)
    signers = [1, 3, 5]
    print(f"Signing␣parties:␣{signers}")

    # Ronda 1: Generar nonce shares
    print("\n---␣Round␣1:␣Generate␣Nonce␣Shares␣---")
    nonce_shares = {}
    for i in signers:
        nonce_shares[i] = ts.generate_nonce_share(i)
    print(f"Each␣party␣generated␣(d_i,␣e_i)␣and␣published␣(D_i,␣E_i)")
```

```
    # Agregar nonces
    nonces_list = [nonce_shares[i] for i in signers]
    R, rho = ts.aggregate_nonces(nonces_list, message, signers)
    print(f"Aggregated R: ({R.x % (10**10)}...)")

    # Ronda 2: Firmas parciales
    print("\n--- Round 2: Partial Signatures ---")
    partial_sigs = []
    for i in signers:
        z_i = ts.compute_partial_signature(
            i, nonce_shares[i], message, R, rho, signers
        )
        partial_sigs.append(z_i)
        print(f"Party {i} generated z_{i} = {z_i % (10**10)}...")

    # Agregar firma
    print("\n--- Aggregate Signature ---")
    signature = ts.aggregate_signature(R, partial_sigs)
    R_final, s = signature
    print(f"Final signature: (R, s)")

    # Verificar
    print("\n--- Verification ---")
    valid = ts.verify(message, signature)
    print(f"Signature valid: {valid}")

threshold_schnorr_demo()
```

5.8. Distributed Key Generation (DKG)

5.8.1. Generación sin Trusted Dealer

Definición 5.12 (Distributed Key Generation)**.** Protocolo para generar clave compartida donde:

- No existe "dealer" que conoce la clave completa
- Cada parte contribuye aleatoriedad
- Nadie puede predecir o influenciar la clave final
- Output: $(pk, \{sk_1, ..., sk_n\})$ donde ninguno conoce sk

[Protocolo DKG (Feldman VSS)] **Fase 1 - Sharing:**

1. Cada parte P_i elige secreto $a_{i,0}$ y polynomial:

$$f_i(x) = a_{i,0} + a_{i,1}x + ... + a_{i,t-1}x^{t-1}$$

2. P_i publica commitments: $C_{i,j} = g^{a_{i,j}}$
3. P_i envía $s_{i,j} = f_i(j)$ a cada P_j

Fase 2 - Verificación:

1. P_j verifica share recibido de P_i:

$$g^{s_{i,j}} \stackrel{?}{=} \prod_{k=0}^{t-1} C_{i,k}^{j^k}$$

2. Si verificación falla, P_j protesta públicamente

Fase 3 - Cálculo de clave:

1. Clave privada (compartida): $sk = \sum_i a_{i,0}$
2. Share de P_j: $sk_j = \sum_i s_{i,j}$
3. Clave pública: $pk = \prod_i C_{i,0} = g^{sk}$

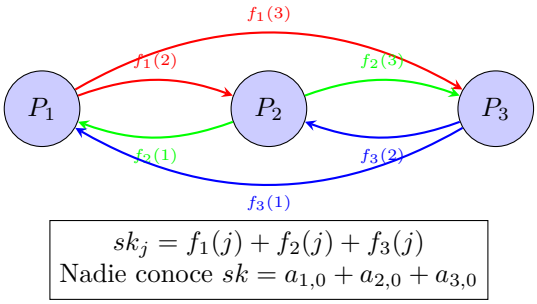


Figura 5.8: DKG: cada parte envía evaluaciones de su polynomial

5.8.2. Aplicaciones en Blockchain

Idea Clave

Uso de DKG + Threshold Signatures en Blockchain:

1. **Bridges cross-chain:**
   - Comité de validadores controla dirección bridge
   - k-of-n para autorizar transferencias
   - Sin single point of failure
2. **Custody institucional:**
   - Claves distribuidas entre múltiples ubicaciones
   - Requiere quorum para firmar
   - Resistente a compromiso de una ubicación
3. **Validator sets (PoS):**
   - BLS threshold signatures para consenso
   - Eficiente agregación de firmas
   - Usado en Ethereum 2.0
4. **Encrypted mempools:**
   - Comité de keypers para threshold decryption
   - Previene frontrunning

Resumen del Bloque

Idea Clave

Puntos clave de Criptografía Avanzada (Bloque 18):
1. FHE (Fully Homomorphic Encryption):

- Computar sobre datos encriptados
- Basado en LWE (post-quantum secure)
- Generaciones: BGV/BFV $\rightarrow$ TFHE/CKKS
- Todavía costoso (100-1000x overhead)

2. MPC (Multi-Party Computation):

- Garbled Circuits: 2 partes, propósito general
- GMW: n partes, basado en secret sharing
- SPDZ: Eficiente con preprocessing
- Base de muchos protocolos de privacidad

3. Secret Sharing (Shamir):

- (k, n)-threshold: k de n para reconstruir
- Basado en interpolación de Lagrange
- Operaciones sobre shares (suma, mult)
- VSS: Verifiable Secret Sharing

4. Oblivious Transfer:

- Transferencia sin revelar elección
- Base de Garbled Circuits
- OT Extension para eficiencia

5. PSI (Private Set Intersection):

- Encontrar elementos comunes sin revelar resto
- Basado en DH o hashing
- Aplicaciones: contact discovery, fraud detection

6. Secure Enclaves:

- SGX, TrustZone, SEV
- Hardware trust model
- Attestation remota
- Vulnerables a side-channels

7. Threshold Signatures:

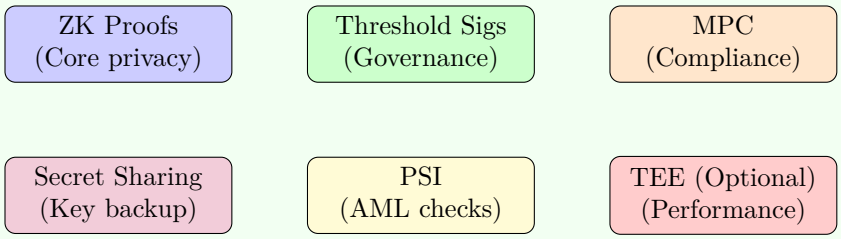
- k-of-n para firmar
- ECDSA: complejo (GG20)
- Schnorr/FROST: más eficiente
- Clave nunca existe completa

8. DKG (Distributed Key Generation):

- Generar clave sin trusted dealer
- Cada parte contribuye aleatoriedad
- Usado en bridges, custody, validators

Aplicación en Banco ZK

Criptografía Avanzada en Banco ZK:



Casos de uso:

1. **Transacciones privadas:** ZK proofs
2. **Multisig institucional:** Threshold ECDSA/Schnorr
3. **Compliance sin revelar datos:** MPC + PSI
4. **Key recovery:** Shamir secret sharing
5. **Bridge cross-chain:** DKG + threshold sigs

Stack recomendado:

- **Firmas:** FROST (Schnorr threshold)
- **Sharing:** Shamir con VSS
- **MPC:** SPDZ para compliance
- **FHE:** Experimental para casos específicos

Capítulo 6

Ataques y Defensa

Bloque 19 de 20

Introducción

La privacidad en blockchain es un juego constante entre atacantes y defensores. Este bloque analiza los principales vectores de ataque contra sistemas de privacidad y las defensas correspondientes.

Idea Clave

Taxonomía de Ataques a la Privacidad:

Network	Blockchain	Behavioral	Metadata
IP Correlation	Graph Analysis	Timing	Intersection
Eclipse	Taint Analysis	Patterns	Side Channel
Sybil	Dust	Long-term	Social Eng.

6.1. Deanonymization Attacks

Definición 6.1 (Deanonymization)**.** Proceso de vincular actividad anónima con identidad real: Deanon : $\text{Activity}_{\text{anon}} \to \text{Identity}_{\text{real}}$



Figura 6.1: Vectores de deanonymization

Listing 6.1: Analizador de vulnerabilidades de deanonymization

```
from dataclasses import dataclass, field
from datetime import datetime, timedelta
from typing import Dict, List, Set
from collections import defaultdict
```

```
from enum import Enum
import math
import random

class AttackVector(Enum):
    IP_CORRELATION = "ip_correlation"
    TIMING_ANALYSIS = "timing_analysis"
    AMOUNT_PATTERN = "amount_pattern"
    GRAPH_ANALYSIS = "graph_analysis"

@dataclass
class ObservedTransaction:
    tx_id: str
    first_seen_ip: str
    first_seen_time: datetime
    amount: float
    sender: str
    receiver: str

class DeanonymizationAnalyzer:
    """Analiza vulnerabilidades de deanonymization."""

    def __init__(self):
        self.tx_observations: List[ObservedTransaction] = []
        self.ip_to_addrs: Dict[str, Set[str]] = defaultdict(set)
        self.addr_to_ips: Dict[str, Set[str]] = defaultdict(set)
        self.addr_timing: Dict[str, List[datetime]] = defaultdict(list)
        self.addr_amounts: Dict[str, List[float]] = defaultdict(list)

    def observe(self, tx: ObservedTransaction):
        """Registrar observaci n del adversario."""
        self.tx_observations.append(tx)
        self.ip_to_addrs[tx.first_seen_ip].add(tx.sender)
        self.addr_to_ips[tx.sender].add(tx.first_seen_ip)
        self.addr_timing[tx.sender].append(tx.first_seen_time)
        self.addr_amounts[tx.sender].append(tx.amount)

    def analyze_ip_risk(self, address: str) -> float:
        """Risk score basado en unicidad de IP (0=seguro, 1=vulnerable)."""
        ips = self.addr_to_ips.get(address, set())
        if not ips:
            return 0.0
        if len(ips) == 1:
            return 0.9  # Siempre mismo IP
        return max(0, 1 - len(ips) / 10)

    def analyze_timing_risk(self, address: str) -> float:
        """Risk score basado en regularidad temporal."""
        times = self.addr_timing.get(address, [])
        if len(times) < 3:
            return 0.0
```

```
        sorted_times = sorted(times)
        intervals = [(sorted_times[i+1] - sorted_times[i]).total_seconds()
                     for i in range(len(sorted_times)-1)]

        if not intervals:
            return 0.0

        mean = sum(intervals) / len(intervals)
        if mean == 0:
            return 0.0

        variance = sum((i - mean)**2 for i in intervals) / len(intervals)
        cv = math.sqrt(variance) / mean

        return max(0, 1 - cv)  # CV bajo = regular = vulnerable

    def analyze_amount_risk(self, address: str) -> float:
        """Risk score basado en patrones de monto."""
        amounts = self.addr_amounts.get(address, [])
        if len(amounts) < 3:
            return 0.0

        round_count = sum(1 for a in amounts if a == round(a))
        return round_count / len(amounts) * 0.5

    def evaluate_overall_risk(self, address: str) -> Dict:
        """Evaluaci n completa de riesgo."""
        ip_risk = self.analyze_ip_risk(address)
        timing_risk = self.analyze_timing_risk(address)
        amount_risk = self.analyze_amount_risk(address)

        overall = 0.4 * ip_risk + 0.35 * timing_risk + 0.25 * amount_risk

        recommendations = []
        if ip_risk > 0.5:
            recommendations.append("Use Tor/VPN to vary IPs")
        if timing_risk > 0.5:
            recommendations.append("Randomize transaction timing")
        if amount_risk > 0.3:
            recommendations.append("Avoid round amounts")

        return {
            'overall_risk': overall,
            'ip_risk': ip_risk,
            'timing_risk': timing_risk,
            'amount_risk': amount_risk,
            'recommendations': recommendations
        }

    def correlate_addresses(self, addr1: str, addr2: str) -> float:
```

```
        """Probabilidad de que dos direcciones sean del mismo usuario."""
        scores = []

        ips1 = self.addr_to_ips.get(addr1, set())
        ips2 = self.addr_to_ips.get(addr2, set())
        if ips1 and ips2:
            overlap = len(ips1 & ips2) / len(ips1 | ips2)
            scores.append(overlap * 0.5)

        t1 = self.addr_timing.get(addr1, [])
        t2 = self.addr_timing.get(addr2, [])
        if t1 and t2:
            h1 = set(t.hour for t in t1)
            h2 = set(t.hour for t in t2)
            if h1 and h2:
                scores.append(len(h1 & h2) / len(h1 | h2) * 0.3)

        return sum(scores) if scores else 0.0


def deanon_demo():
    print("=== Deanonymization Analysis ===\n")

    analyzer = DeanonymizationAnalyzer()
    base = datetime(2024, 1, 15, 9, 0, 0)

    # Usuario predecible
    for i in range(15):
        analyzer.observe(ObservedTransaction(
            tx_id=f"tx_{i}",
            first_seen_ip="192.168.1.100",
            first_seen_time=base + timedelta(days=i, hours=random.randint(8,10
            amount=round(random.uniform(0.1, 1.0), 1),
            sender="0xPredictable",
            receiver=f"0xR{i}"
        ))

    # Usuario privado
    for i in range(15):
        analyzer.observe(ObservedTransaction(
            tx_id=f"tx_p_{i}",
            first_seen_ip=f"10.{random.randint(0,255)}.{random.randint(0,255)}
            first_seen_time=base + timedelta(days=random.randint(0,30), hours=
            amount=random.uniform(0.1, 1.0),
            sender="0xPrivate",
            receiver=f"0xR_p{i}"
        ))

    for addr, name in [("0xPredictable", "Predictable"), ("0xPrivate", "Privat
        r = analyzer.evaluate_overall_risk(addr)
        print(f"{name}: Overall={r['overall_risk']:.1%} IP={r['ip_risk']:.1%}
```

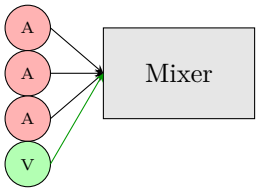
```
            f"Timing={r['timing_risk']:.1%}")
        for rec in r['recommendations'][:1]:
            print(f"  -> {rec}")

deanon_demo()
```

6.2. Sybil Attacks en Mixers

Definición 6.2 (Sybil Attack en Mixer)**.** Atacante crea múltiples depósitos propios para reducir anonymity set efectivo:

$$\text{AS}_{\text{real}} = n - k \quad \text{(donde } k = \text{depósitos del atacante)}$$



AS aparente=4, AS real=1

Figura 6.2: Sybil: atacante controla 3 de 4 depósitos

Listing 6.2: Simulador de Sybil attacks y defensas

```
from dataclasses import dataclass
from typing import List, Dict
from enum import Enum
import random
import math

class DepositType(Enum):
    HONEST = "honest"
    ATTACKER = "attacker"
    VICTIM = "victim"

@dataclass
class Deposit:
    id: str
    dtype: DepositType
    timestamp: int

class MixerSimulator:
    def __init__(self):
        self.deposits: List[Deposit] = []
        self.attacker_ids: List[str] = []
        self.victim_ids: List[str] = []

    def simulate_attack(self, total: int, attacker_frac: float) -> Dict:
        """Simular Sybil attack con fracci n de atacante dada."""
        att_count = int(total * attacker_frac)
```

```
        victim_count = 1
        honest_count = total - att_count - victim_count

        types = ([DepositType.ATTACKER] * att_count +
                 [DepositType.VICTIM] * victim_count +
                 [DepositType.HONEST] * honest_count)
        random.shuffle(types)

        self.deposits = []
        self.attacker_ids = []
        self.victim_ids = []

        for i, t in enumerate(types):
            d = Deposit(f"d{i}", t, i)
            self.deposits.append(d)
            if t == DepositType.ATTACKER:
                self.attacker_ids.append(d.id)
            elif t == DepositType.VICTIM:
                self.victim_ids.append(d.id)

        # Calcular AS para v ctima
        victim = next(d for d in self.deposits if d.dtype == DepositType.VICTI
        eligible = [d for d in self.deposits if d.timestamp <= victim.timestam
        apparent = len(eligible)
        real = sum(1 for d in eligible if d.dtype != DepositType.ATTACKER)

        return {
            'apparent_as': apparent,
            'real_as': real,
            'privacy_loss': (apparent - real) / apparent if apparent > 0 else
        }


class SybilDefense:
    def __init__(self, base_fee: float = 0.01):
        self.base_fee = base_fee

    def analyze_defense(self, budget: float, deposit_amt: float,
                        fee: float, stake: float, pop: bool) -> Dict:
        """Analizar␣efectividad␣de␣defensa␣contra␣presupuesto␣de␣atacante."""
        cost = deposit_amt + fee + stake
        max_by_budget = int(budget / cost) if cost > 0 else 1000
        max_effective = 1 if pop else max_by_budget

        return {
            'max_sybil_deposits': max_effective,
            'cost_per_deposit': cost,
            'effectiveness': 1 - max_effective / max(1, max_by_budget)
        }
```

```
def sybil_demo():
    print("=== Sybil Attack Analysis ===\n")

    sim = MixerSimulator()

    print(f"{'Attacker %':<10} {'Apparent':<10} {'Real':<8} {'Loss':<8}")
    print("-" * 36)
    for frac in [0, 0.25, 0.5, 0.75, 0.9]:
        r = sim.simulate_attack(100, frac)
        print(f"{frac*100:>8.0f}%  {r['apparent_as']:>8}  {r['real_as']:>6}  "
              f"{r['privacy_loss']*100:>6.1f}%")

    print("\n--- Defense Effectiveness ---")
    defense = SybilDefense()
    budget = 100.0

    scenarios = [
        ("Baseline", 0.01, 0, False),
        ("High fee", 0.5, 0, False),
        ("Staking", 0.01, 10, False),
        ("PoP", 0.01, 0, True),
    ]

    for name, fee, stake, pop in scenarios:
        r = defense.analyze_defense(budget, 1.0, fee, stake, pop)
        print(f"{name:<12} Max Sybil: {r['max_sybil_deposits']:<5} "
              f"Effectiveness: {r['effectiveness']*100:.0f}%")

sybil_demo()
```

6.3. Eclipse Attacks

Definición 6.3 (Eclipse Attack)**.** Aislar un nodo de la red honesta controlando todas sus conexiones:

- Atacante monopoliza peers de la víctima
- Puede alimentar información falsa (bloques, txs)
- Puede censurar transacciones de la víctima

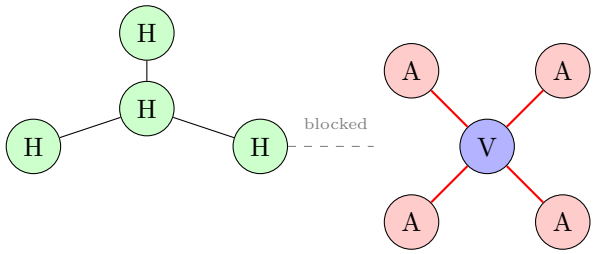


Figura 6.3: Eclipse attack: víctima solo conectada a atacantes

Listing 6.3: Simulador de Eclipse attacks

```
from dataclasses import dataclass, field
from typing import Dict, List, Set
import random

@dataclass
class NetworkNode:
    node_id: str
    is_honest: bool
    peers: Set[str] = field(default_factory=set)
    max_peers: int = 8

class EclipseSimulator:
    def __init__(self, honest_nodes: int, attacker_nodes: int):
        self.nodes: Dict[str, NetworkNode] = {}

        # Crear nodos honestos
        for i in range(honest_nodes):
            self.nodes[f"H{i}"] = NetworkNode(f"H{i}", True)

        # Crear nodos atacantes
        for i in range(attacker_nodes):
            self.nodes[f"A{i}"] = NetworkNode(f"A{i}", False)

        # V ctima
        self.nodes["V"] = NetworkNode("V", True)

    def connect_honest_network(self):
        """Conectar nodos honestos entre s ."""
        honest = [n for n in self.nodes.values() if n.is_honest and n.node_id

        for node in honest:
            available = [n for n in honest if n.node_id != node.node_id
                         and len(node.peers) < node.max_peers]
            connections = min(4, len(available))
            for peer in random.sample(available, connections):
                node.peers.add(peer.node_id)
                peer.peers.add(node.node_id)

    def execute_eclipse(self, victim_id: str = "V") -> Dict:
        """Ejecutar eclipse attack."""
        victim = self.nodes[victim_id]
        attackers = [n for n in self.nodes.values() if not n.is_honest]

        # Atacantes intentan llenar todos los slots de la v ctima
        for attacker in attackers:
            if len(victim.peers) < victim.max_peers:
                victim.peers.add(attacker.node_id)
                attacker.peers.add(victim.node_id)

        # Calcular resultado
```

```
        honest_peers = sum(1 for p in victim.peers if self.nodes[p].is_honest)
        attacker_peers = len(victim.peers) - honest_peers

        eclipsed = honest_peers == 0 and attacker_peers > 0

        return {
            'total_peers': len(victim.peers),
            'honest_peers': honest_peers,
            'attacker_peers': attacker_peers,
            'eclipsed': eclipsed,
            'eclipse_ratio': attacker_peers / len(victim.peers) if victim.peers
        }

    def apply_defense(self, victim_id: str = "V",
                      reserved_honest: int = 2) -> Dict:
        """Aplicar defensa: reservar slots para nodos conocidos."""
        victim = self.nodes[victim_id]

        # Primero conectar a nodos honestos conocidos
        honest = [n for n in self.nodes.values() if n.is_honest and n.node_id !=

        for i, node in enumerate(honest[:reserved_honest]):
            if len(victim.peers) < victim.max_peers:
                victim.peers.add(node.node_id)

        # Luego atacantes intentan llenar el resto
        attackers = [n for n in self.nodes.values() if not n.is_honest]
        for attacker in attackers:
            if len(victim.peers) < victim.max_peers:
                victim.peers.add(attacker.node_id)

        honest_peers = sum(1 for p in victim.peers if self.nodes[p].is_honest)

        return {
            'honest_peers': honest_peers,
            'eclipsed': honest_peers == 0,
            'defense_effective': honest_peers >= reserved_honest
        }


def eclipse_demo():
    print("=== Eclipse Attack Simulation ===\n")

    # Sin defensa
    sim = EclipseSimulator(honest_nodes=20, attacker_nodes=50)
    sim.connect_honest_network()

    result = sim.execute_eclipse()
    print("Without defense:")
    print(f"  Peers: {result['total_peers']} (Honest: {result['honest_peers'], 
          f"Attacker: {result['attacker_peers']})")
```

```
    print(f"  Eclipsed: {result['eclipsed']}")

    # Con defensa
    sim2 = EclipseSimulator(honest_nodes=20, attacker_nodes=50)
    sim2.connect_honest_network()

    result2 = sim2.apply_defense(reserved_honest=2)
    print("\nWith defense (2 reserved honest slots):")
    print(f"  Honest peers: {result2['honest_peers']}")
    print(f"  Eclipsed: {result2['eclipsed']}")
    print(f"  Defense effective: {result2['defense_effective']}")

eclipse_demo()
```

6.3.1. Defensas contra Eclipse

Defensa	Mecanismo	Efectividad
Peer diversity	Limitar peers por subnet/AS	Alta
Reserved slots	Slots para nodos conocidos/confiables	Alta
Checkpoints	Verificar contra estado conocido	Media
Multiple connections	Conexiones a diferentes redes	Alta
Proof of Work	Costo para crear identidades	Media

Tabla 6.1: Defensas contra Eclipse attacks

6.4. Dust Attacks

Definición 6.4 (Dust Attack)**.** Enviar cantidades mínimas ("dust") a múltiples direcciones para rastrear cuando las víctimas consoliden esos fondos con otros UTXOs, revelando qué direcciones controlan.

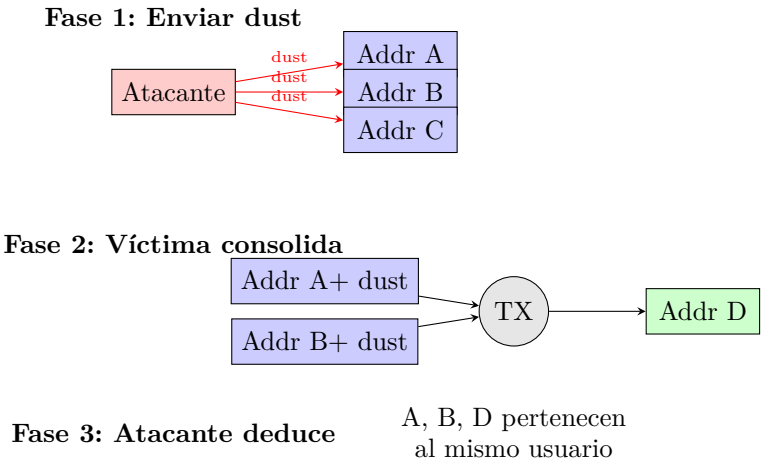


Figura 6.4: Dust attack: tracking mediante consolidación de UTXOs

Listing 6.4: Detector y defensa contra Dust attacks

```
from dataclasses import dataclass
from typing import Dict, List, Set, Tuple
from datetime import datetime
from collections import defaultdict

@dataclass
class UTXO:
    tx_id: str
    output_index: int
    address: str
    amount: float
    timestamp: datetime
    source_addresses: Set[str] = None  # Para tracking de inputs

    def __post_init__(self):
        if self.source_addresses is None:
            self.source_addresses = set()

@dataclass
class Transaction:
    tx_id: str
    inputs: List[UTXO]
    outputs: List[UTXO]
    timestamp: datetime

class DustAttackDetector:
    """
    Detecta dust attacks y proporciona recomendaciones.
    """

    def __init__(self, dust_threshold: float = 0.0001):
        self.dust_threshold = dust_threshold
        self.utxos: Dict[str, List[UTXO]] = defaultdict(list)  # address -> UTXO
        self.suspicious_dust: Dict[str, List[UTXO]] = defaultdict(list)
        self.known_attackers: Set[str] = set()

    def add_utxo(self, utxo: UTXO):
        """A adir UTXO y verificar si es dust sospechoso."""
        self.utxos[utxo.address].append(utxo)

        if self.is_dust(utxo):
            self.suspicious_dust[utxo.address].append(utxo)

    def is_dust(self, utxo: UTXO) -> bool:
        """Verificar si UTXO es dust."""
        return utxo.amount <= self.dust_threshold

    def analyze_incoming_dust(self, address: str) -> Dict:
        """Analizar dust recibido por una direcci n."""
        dust_utxos = self.suspicious_dust.get(address, [])
```

```
        if not dust_utxos:
            return {'has_dust': False, 'risk_level': 'none'}

        # Analizar fuentes del dust
        source_addresses = set()
        for utxo in dust_utxos:
            source_addresses.update(utxo.source_addresses)

        # Calcular riesgo
        risk_factors = {
            'dust_count': len(dust_utxos),
            'unique_sources': len(source_addresses),
            'total_dust_value': sum(u.amount for u in dust_utxos),
            'recent_dust': sum(1 for u in dust_utxos
                               if (datetime.now() - u.timestamp).days < 7)
        }

        # Determinar nivel de riesgo
        if risk_factors['dust_count'] > 5 or risk_factors['unique_sources'] ==
            risk_level = 'high'
        elif risk_factors['dust_count'] > 2:
            risk_level = 'medium'
        else:
            risk_level = 'low'

        return {
            'has_dust': True,
            'risk_level': risk_level,
            'dust_count': risk_factors['dust_count'],
            'total_value': risk_factors['total_dust_value'],
            'recommendations': self._get_recommendations(risk_level)
        }

    def _get_recommendations(self, risk_level: str) -> List[str]:
        """Obtener recomendaciones basadas en nivel de riesgo."""
        if risk_level == 'high':
            return [
                "DO NOT spend this dust with other UTXOs",
                "Consider this address compromised for privacy",
                "Use coin control to exclude dust from transactions",
                "Consider consolidating only dust separately"
            ]
        elif risk_level == 'medium':
            return [
                "Be cautious when spending from this address",
                "Use coin control if available",
                "Consider waiting before using these funds"
            ]
        else:
            return ["Monitor for additional suspicious deposits"]
```

```
    def simulate_consolidation_leak(
        self,
        addresses: List[str],
        include_dust: bool
    ) -> Dict:
        """
␣␣␣␣␣␣␣␣Simular␣qu ␣informaci n␣se␣filtra␣al␣consolidar.
␣␣␣␣␣␣␣␣"""
        # Recopilar todos los UTXOs
        all_utxos = []
        dust_utxos = []

        for addr in addresses:
            for utxo in self.utxos.get(addr, []):
                if self.is_dust(utxo):
                    dust_utxos.append(utxo)
                    if include_dust:
                        all_utxos.append(utxo)
                else:
                    all_utxos.append(utxo)

        # Informaci n filtrada
        if include_dust:
            linked_addresses = set(addresses)
            dust_sources = set()
            for utxo in dust_utxos:
                dust_sources.update(utxo.source_addresses)

            return {
                'addresses_linked': len(linked_addresses),
                'linked_addresses': linked_addresses,
                'dust_trackers_alerted': len(dust_sources),
                'privacy_leak': 'HIGH␣-␣All␣addresses␣now␣linked'
            }
        else:
            return {
                'addresses_linked': len(addresses),
                'dust_excluded': len(dust_utxos),
                'privacy_leak': 'LOW␣-␣Dust␣not␣included'
            }


class CoinControl:
    """
␣␣␣␣Sistema␣de␣coin␣control␣para␣gesti n␣segura␣de␣UTXOs.
␣␣␣␣"""

    def __init__(self):
        self.utxos: List[UTXO] = []
        self.frozen_utxos: Set[Tuple[str, int]] = set()  # (tx_id, output_index)
        self.labels: Dict[Tuple[str, int], str] = {}
```

```
    def add_utxo(self, utxo: UTXO, label: str = None):
        """A adir UTXO con etiqueta opcional."""
        self.utxos.append(utxo)
        if label:
            self.labels[(utxo.tx_id, utxo.output_index)] = label

    def freeze_utxo(self, tx_id: str, output_index: int):
        """Congelar UTXO para que no se use autom ticamente."""
        self.frozen_utxos.add((tx_id, output_index))

    def is_frozen(self, utxo: UTXO) -> bool:
        return (utxo.tx_id, utxo.output_index) in self.frozen_utxos

    def auto_freeze_dust(self, threshold: float = 0.0001):
        """Congelar autom ticamente todo el dust."""
        for utxo in self.utxos:
            if utxo.amount <= threshold:
                self.freeze_utxo(utxo.tx_id, utxo.output_index)

    def select_utxos_for_amount(
        self,
        target_amount: float,
        exclude_dust: bool = True,
        same_source_only: bool = False
    ) -> List[UTXO]:
        """
        Seleccionar UTXOs para una transacci n.
        """
        available = [
            u for u in self.utxos
            if not self.is_frozen(u)
            and (not exclude_dust or u.amount > 0.0001)
        ]

        if same_source_only and available:
            # Solo usar UTXOs del mismo origen para evitar linking
            first_source = available[0].source_addresses
            available = [u for u in available
                         if u.source_addresses == first_source]

        # Selecci n simple: ordenar por cantidad y tomar los necesarios
        available.sort(key=lambda u: u.amount, reverse=True)

        selected = []
        total = 0.0

        for utxo in available:
            if total >= target_amount:
                break
            selected.append(utxo)
```

```
            total += utxo.amount

        return selected if total >= target_amount else []

    def get_privacy_score(self) -> Dict:
        """Evaluar privacidad del conjunto de UTXOs."""
        total_utxos = len(self.utxos)
        dust_count = sum(1 for u in self.utxos if u.amount <= 0.0001)
        frozen_dust = sum(1 for u in self.utxos
                          if u.amount <= 0.0001 and self.is_frozen(u))

        unique_sources = len(set(
            frozenset(u.source_addresses) for u in self.utxos
        ))

        # Score de 0 a 100
        dust_risk = (dust_count - frozen_dust) / max(1, total_utxos) * 50
        source_diversity = min(50, unique_sources * 10)

        score = 100 - dust_risk + source_diversity / 2

        return {
            'score': min(100, max(0, score)),
            'total_utxos': total_utxos,
            'dust_utxos': dust_count,
            'frozen_dust': frozen_dust,
            'unfrozen_dust': dust_count - frozen_dust,
            'recommendation': 'Freeze remaining dust' if dust_count > frozen_dus
        }


def dust_attack_demo():
    print("=== Dust Attack Detection Demo ===\n")

    detector = DustAttackDetector(dust_threshold=0.0001)

    # Simular recepci n de dust
    victim_address = "bc1qvictim..."

    # UTXOs normales
    detector.add_utxo(UTXO("tx1", 0, victim_address, 0.5, datetime.now(), {"exch
    detector.add_utxo(UTXO("tx2", 0, victim_address, 0.3, datetime.now(), {"frien

    # Dust attacks
    for i in range(5):
        detector.add_utxo(UTXO(
            f"dust_tx_{i}", 0, victim_address,
            0.00001,  # Dust
            datetime.now(),
            {"attacker_wallet"}
        ))
```

```
    # Analizar
    analysis = detector.analyze_incoming_dust(victim_address)

    print(f"Address: {victim_address}")
    print(f"Has dust: {analysis['has_dust']}")
    print(f"Risk level: {analysis['risk_level']}")
    print(f"Dust count: {analysis['dust_count']}")
    print(f"Total dust value: {analysis['total_value']:.8f}")
    print("\nRecommendations:")
    for rec in analysis['recommendations']:
        print(f"  - {rec}")

    # Simular consolidaci n
    print("\n--- Consolidation Simulation ---")

    # Con dust
    leak_with = detector.simulate_consolidation_leak([victim_address], include_
    print(f"With dust: {leak_with['privacy_leak']}")

    # Sin dust
    leak_without = detector.simulate_consolidation_leak([victim_address], inclu
    print(f"Without dust: {leak_without['privacy_leak']}")

dust_attack_demo()
```

6.5. Graph Analysis (Clustering Heuristics)

Definición 6.5 (Address Clustering)**.** Agrupar direcciones que probablemente pertenecen al mismo usuario usando heurísticas:

- **Common Input Ownership:** Inputs de la misma TX = mismo dueño
- **Change Detection:** Output de cambio = mismo dueño que inputs
- **Address Reuse:** Misma dirección = mismo usuario

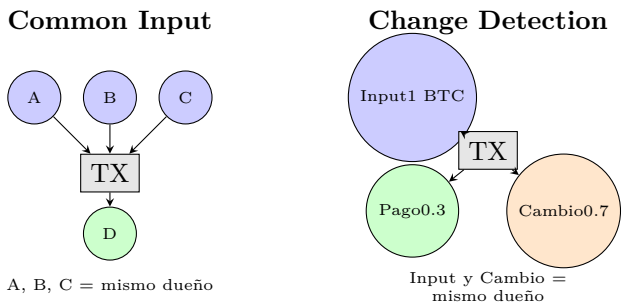


Figura 6.5: Heurísticas de clustering: common input y change detection

Listing 6.5: Sistema de Graph Analysis y Clustering

```
from dataclasses import dataclass, field
```

```
from typing import Dict, List, Set, Tuple
from collections import defaultdict

@dataclass
class TxInput:
    address: str
    amount: float
    tx_id: str
    output_index: int

@dataclass
class TxOutput:
    address: str
    amount: float
    output_index: int

@dataclass
class Transaction:
    tx_id: str
    inputs: List[TxInput]
    outputs: List[TxOutput]
    fee: float = 0.0

class AddressClusterer:
    """
    Clustering de direcciones usando heur sticas on-chain.
    """

    def __init__(self):
        self.transactions: Dict[str, Transaction] = {}
        self.address_to_txs: Dict[str, List[str]] = defaultdict(list)

        # Union-Find para clustering
        self.parent: Dict[str, str] = {}
        self.rank: Dict[str, int] = {}

    def _find(self, addr: str) -> str:
        """Find con path compression."""
        if addr not in self.parent:
            self.parent[addr] = addr
            self.rank[addr] = 0

        if self.parent[addr] != addr:
            self.parent[addr] = self._find(self.parent[addr])

        return self.parent[addr]

    def _union(self, addr1: str, addr2: str):
        """Union by rank."""
        root1 = self._find(addr1)
        root2 = self._find(addr2)
```

```
        if root1 == root2:
            return

        if self.rank[root1] < self.rank[root2]:
            self.parent[root1] = root2
        elif self.rank[root1] > self.rank[root2]:
            self.parent[root2] = root1
        else:
            self.parent[root2] = root1
            self.rank[root1] += 1

    def add_transaction(self, tx: Transaction):
        """A adir transacci n y aplicar heur sticas."""
        self.transactions[tx.tx_id] = tx

        # Registrar direcciones
        for inp in tx.inputs:
            self.address_to_txs[inp.address].append(tx.tx_id)
        for out in tx.outputs:
            self.address_to_txs[out.address].append(tx.tx_id)

        # Aplicar heur sticas
        self._apply_common_input_heuristic(tx)
        self._apply_change_heuristic(tx)

    def _apply_common_input_heuristic(self, tx: Transaction):
        """
        Common Input Ownership Heuristic:
        Si m ltiples direcciones son inputs de la misma TX,
        probablemente pertenecen al mismo usuario.
        """
        if len(tx.inputs) <= 1:
            return

        input_addresses = [inp.address for inp in tx.inputs]

        # Unir todas las direcciones de input
        first = input_addresses[0]
        for addr in input_addresses[1:]:
            self._union(first, addr)

    def _apply_change_heuristic(self, tx: Transaction):
        """
        Change Detection Heuristic:
        Detectar output de cambio y vincularlo con inputs.
        """
        if len(tx.outputs) != 2 or len(tx.inputs) == 0:
            return

        # Heur sticas para detectar cambio:
```

```
        # 1. Output que es address reusada (ya vista antes)
        # 2. Output con monto "raro" (no redondo)
        # 3. Output m s peque o (el pago suele ser espec fico)

        input_total = sum(inp.amount for inp in tx.inputs)
        output_amounts = [(out.address, out.amount) for out in tx.outputs]
        output_amounts.sort(key=lambda x: x[1])

        # Heur stica simple: el output m s peque o es el pago,
        # el m s grande es el cambio
        larger_output = output_amounts[1]

        # Verificar que no es pago redondo (m s probable que sea cambio)
        if larger_output[1] != round(larger_output[1], 2):
            # Probablemente es cambio - vincular con input
            change_addr = larger_output[0]
            input_addr = tx.inputs[0].address
            self._union(change_addr, input_addr)

    def get_cluster(self, address: str) -> Set[str]:
        """Obtener todas las direcciones en el mismo cluster."""
        if address not in self.parent:
            return {address}

        root = self._find(address)
        return {addr for addr in self.parent if self._find(addr) == root}

    def get_all_clusters(self) -> List[Set[str]]:
        """Obtener todos los clusters."""
        clusters = defaultdict(set)

        for addr in self.parent:
            root = self._find(addr)
            clusters[root].add(addr)

        return list(clusters.values())

    def analyze_cluster(self, address: str) -> Dict:
        """Analizar un cluster espec fico."""
        cluster = self.get_cluster(address)

        if not cluster:
            return {'size': 0}

        # Calcular estad sticas
        total_txs = set()
        total_volume_in = 0.0
        total_volume_out = 0.0

        for addr in cluster:
            for tx_id in self.address_to_txs.get(addr, []):
```

```
                total_txs.add(tx_id)
                tx = self.transactions.get(tx_id)
                if tx:
                    for inp in tx.inputs:
                        if inp.address == addr:
                            total_volume_out += inp.amount
                    for out in tx.outputs:
                        if out.address == addr:
                            total_volume_in += out.amount

        return {
            'cluster_size': len(cluster),
            'addresses': list(cluster)[:5],  # Primeras 5
            'transaction_count': len(total_txs),
            'total_volume_in': total_volume_in,
            'total_volume_out': total_volume_out,
            'net_flow': total_volume_in - total_volume_out
        }


def clustering_demo():
    print("=== Address Clustering Demo ===\n")

    clusterer = AddressClusterer()

    # Transacci n 1: Multi-input (common input heuristic)
    tx1 = Transaction(
        tx_id="tx1",
        inputs=[
            TxInput("addr_A", 0.5, "prev1", 0),
            TxInput("addr_B", 0.3, "prev2", 0),
            TxInput("addr_C", 0.2, "prev3", 0),
        ],
        outputs=[
            TxOutput("addr_merchant", 0.9, 0),
            TxOutput("addr_D", 0.09, 1),  # Cambio
        ],
        fee=0.01
    )
    clusterer.add_transaction(tx1)

    # Transacci n 2: Pago simple con cambio
    tx2 = Transaction(
        tx_id="tx2",
        inputs=[TxInput("addr_E", 1.0, "prev4", 0)],
        outputs=[
            TxOutput("addr_service", 0.3, 0),  # Pago redondo
            TxOutput("addr_F", 0.699, 1),  # Cambio (no redondo)
        ],
        fee=0.001
    )
```

```
    clusterer.add_transaction(tx2)

    # Analizar clusters
    print("── Clusters Found ──")
    clusters = clusterer.get_all_clusters()

    for i, cluster in enumerate(clusters):
        if len(cluster) > 1:
            print(f"\nCluster {i+1}: {len(cluster)} addresses")
            for addr in list(cluster)[:5]:
                print(f"  - {addr}")

    # Analizar cluster espec fico
    print("\n── Cluster Analysis (addr_A) ──")
    analysis = clusterer.analyze_cluster("addr_A")
    print(f"Size: {analysis['cluster_size']} addresses")
    print(f"Transactions: {analysis['transaction_count']}")
    print(f"Volume in: {analysis['total_volume_in']:.4f}")
    print(f"Volume out: {analysis['total_volume_out']:.4f}")

clustering_demo()
```

6.6. Taint Analysis

Definición 6.6 (Taint Analysis)**.** Rastrear "contaminación" de fondos a través del grafo de transacciones:

- **Poison:** Cualquier contacto con tainted = tainted
- **Haircut:** Proporcional al monto mezclado
- **FIFO/LIFO:** Orden de inputs determina taint

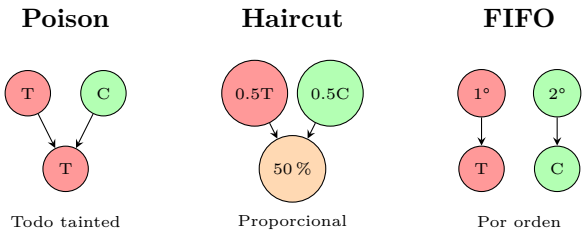


Figura 6.6: Métodos de taint analysis: Poison, Haircut, FIFO

Listing 6.6: Sistema de Taint Analysis

```
from dataclasses import dataclass
from typing import Dict, List, Set, Optional
from enum import Enum
from collections import defaultdict

class TaintMethod(Enum):
    POISON = "poison"       # Cualquier contacto = tainted
```

```
    HAIRCUT = "haircut"     # Proporcional
    FIFO = "fifo"           # First-in-first-out
    LIFO = "lifo"           # Last-in-first-out

@dataclass
class TaintedFund:
    amount: float
    taint_ratio: float  # 0.0 = limpio, 1.0 = completamente tainted
    source: str         # Origen del taint
    hops: int           # Distancia del origen

@dataclass
class TxForTaint:
    tx_id: str
    inputs: List[tuple]   # (address, amount)
    outputs: List[tuple]  # (address, amount)

class TaintAnalyzer:
    """
    Analiza taint/contaminaci n de fondos.
    """

    def __init__(self, method: TaintMethod = TaintMethod.HAIRCUT):
        self.method = method
        self.address_taint: Dict[str, TaintedFund] = {}
        self.tainted_sources: Set[str] = set()
        self.transactions: List[TxForTaint] = []

    def mark_tainted(self, address: str, source: str = "illicit"):
        """Marcar una direcci n como fuente de taint."""
        self.tainted_sources.add(address)
        self.address_taint[address] = TaintedFund(
            amount=float('inf'),  # Cualquier cantidad
            taint_ratio=1.0,
            source=source,
            hops=0
        )

    def process_transaction(self, tx: TxForTaint):
        """Procesar transacci n y propagar taint."""
        self.transactions.append(tx)

        # Calcular taint de inputs
        input_taint = self._calculate_input_taint(tx.inputs)

        # Propagar a outputs seg n m todo
        if self.method == TaintMethod.POISON:
            self._propagate_poison(tx, input_taint)
        elif self.method == TaintMethod.HAIRCUT:
            self._propagate_haircut(tx, input_taint)
        elif self.method in [TaintMethod.FIFO, TaintMethod.LIFO]:
```

```
        self._propagate_fifo_lifo(tx, input_taint)

    def _calculate_input_taint(self, inputs: List[tuple]) -> Dict:
        """Calcular taint total de los inputs."""
        total_amount = sum(amt for _, amt in inputs)
        total_taint = 0.0
        max_hops = 0
        sources = set()

        for addr, amt in inputs:
            if addr in self.address_taint:
                taint = self.address_taint[addr]
                total_taint += taint.taint_ratio * amt
                max_hops = max(max_hops, taint.hops)
                sources.add(taint.source)

        return {
            'total_amount': total_amount,
            'taint_amount': total_taint,
            'taint_ratio': total_taint / total_amount if total_amount > 0 else 0
            'max_hops': max_hops,
            'sources': sources
        }

    def _propagate_poison(self, tx: TxForTaint, input_taint: Dict):
        """M todo Poison: cualquier contacto = 100% tainted."""
        if input_taint['taint_ratio'] > 0:
            for addr, amt in tx.outputs:
                self.address_taint[addr] = TaintedFund(
                    amount=amt,
                    taint_ratio=1.0,
                    source=list(input_taint['sources'])[0] if input_taint['sourc
                    hops=input_taint['max_hops'] + 1
                )

    def _propagate_haircut(self, tx: TxForTaint, input_taint: Dict):
        """M todo Haircut: taint proporcional."""
        if input_taint['taint_ratio'] > 0:
            for addr, amt in tx.outputs:
                self.address_taint[addr] = TaintedFund(
                    amount=amt,
                    taint_ratio=input_taint['taint_ratio'],
                    source=list(input_taint['sources'])[0] if input_taint['sourc
                    hops=input_taint['max_hops'] + 1
                )

    def _propagate_fifo_lifo(self, tx: TxForTaint, input_taint: Dict):
        """M todo FIFO/LIFO: asignar taint por orden."""
        # Ordenar inputs por taint (tainted primero para FIFO)
        sorted_inputs = []
        for addr, amt in tx.inputs:
```

```
            taint = self.address_taint.get(addr)
            ratio = taint.taint_ratio if taint else 0
            sorted_inputs.append((addr, amt, ratio))

        if self.method == TaintMethod.FIFO:
            sorted_inputs.sort(key=lambda x: -x[2])  # Tainted primero
        else:  # LIFO
            sorted_inputs.sort(key=lambda x: x[2])   # Clean primero

        # Asignar a outputs en orden
        remaining_taint = sum(amt * ratio for _, amt, ratio in sorted_inputs)

        for addr, amt in tx.outputs:
            if remaining_taint >= amt:
                self.address_taint[addr] = TaintedFund(amt, 1.0, "fifo", 1)
                remaining_taint -= amt
            elif remaining_taint > 0:
                ratio = remaining_taint / amt
                self.address_taint[addr] = TaintedFund(amt, ratio, "fifo", 1)
                remaining_taint = 0
            # Else: output stays clean

    def get_taint_score(self, address: str) -> Dict:
        """Obtener score de taint para una direcci n."""
        if address not in self.address_taint:
            return {
                'address': address,
                'taint_ratio': 0.0,
                'status': 'clean',
                'risk_level': 'none'
            }

        taint = self.address_taint[address]

        if taint.taint_ratio >= 0.9:
            risk = 'critical'
        elif taint.taint_ratio >= 0.5:
            risk = 'high'
        elif taint.taint_ratio >= 0.1:
            risk = 'medium'
        else:
            risk = 'low'

        return {
            'address': address,
            'taint_ratio': taint.taint_ratio,
            'hops_from_source': taint.hops,
            'source': taint.source,
            'risk_level': risk
        }
```

```
    def trace_funds(self, address: str, max_hops: int = 10) -> List[Dict]:
        """Rastrear origen de fondos."""
        # Simplified: return taint info
        taint = self.address_taint.get(address)
        if not taint:
            return [{'address': address, 'status': 'clean'}]

        return [{
            'address': address,
            'taint_ratio': taint.taint_ratio,
            'source': taint.source,
            'hops': taint.hops
        }]


def taint_demo():
    print("=== Taint Analysis Demo ===\n")

    # Comparar m todos
    for method in [TaintMethod.POISON, TaintMethod.HAIRCUT]:
        print(f"--- Method: {method.value} ---")

        analyzer = TaintAnalyzer(method=method)

        # Marcar fuente tainted
        analyzer.mark_tainted("hacker_wallet", "ransomware")

        # Transacci n: mezclar tainted con clean
        tx = TxForTaint(
            tx_id="mix_tx",
            inputs=[
                ("hacker_wallet", 5.0),  # Tainted
                ("honest_user", 5.0),    # Clean
            ],
            outputs=[
                ("exchange", 9.9),
                ("fee_addr", 0.1),
            ]
        )
        analyzer.process_transaction(tx)

        # Verificar taint
        result = analyzer.get_taint_score("exchange")
        print(f"  Exchange taint: {result['taint_ratio']:.1%}")
        print(f"  Risk level: {result['risk_level']}")
        print()

taint_demo()
```

6.7. Intersection Attacks

Definición 6.7 (Intersection Attack)**.** Correlacionar múltiples observaciones para reducir progresivamente el anonymity set:

Proceso:

1. Observar anonymity set en tiempo T_1: $AS_1 = \{u_1, ..., u_n\}$
2. Observar en T_2: $AS_2 = \{v_1, ..., v_m\}$
3. Intersección: $AS_{1,2} = AS_1 \cap AS_2$
4. Con suficientes observaciones: $|AS| \to 1$

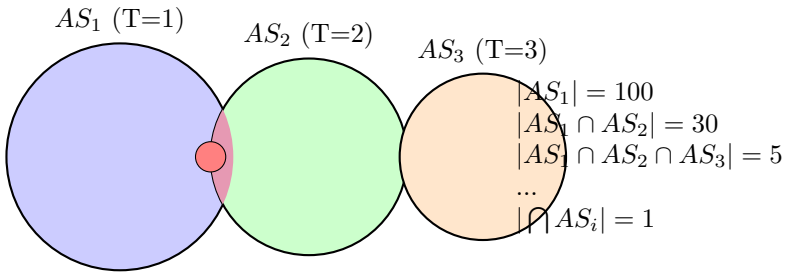


Figura 6.7: Intersection attack: reducción progresiva del anonymity set

[Análisis Probabilístico] **Modelo:**

- N: Población total de usuarios
- n_i: Tamaño del anonymity set en observación i
- k: Número de observaciones

Tamaño esperado de intersección: Si los sets son independientes:

$$E[|\bigcap_{i=1}^{k} AS_i|] \approx N \cdot \prod_{i=1}^{k} \frac{n_i}{N}$$

Observaciones necesarias para deanon:

$$k^* \approx \frac{\log N}{\log(N/\bar{n})}$$

donde $\bar{n}$ es el tamaño promedio del anonymity set.

Listing 6.7: Simulador de Intersection Attacks

```
from dataclasses import dataclass
from typing import List, Set, Dict
import random
import math

@dataclass
class Observation:
    time: int
    anonymity_set: Set[str]
```

```
    context: str  # Qu  actividad se observ

class IntersectionAttackSimulator:
    """
␣␣␣␣Simula␣intersection␣attacks␣sobre␣anonymity␣sets.
␣␣␣␣"""

    def __init__(self, population_size: int):
        self.population = {f"user_{i}" for i in range(population_size)}
        self.target: str = random.choice(list(self.population))
        self.observations: List[Observation] = []

    def generate_observation(
        self,
        time: int,
        anonymity_set_size: int,
        context: str = ""
    ) -> Observation:
        """
␣␣␣␣␣␣␣␣Generar␣observaci n␣donde␣el␣target␣est ␣en␣el␣anonymity␣set.
␣␣␣␣␣␣␣␣"""
        # El target siempre est  en el set
        others = self.population - {self.target}
        sample_size = min(anonymity_set_size - 1, len(others))
        selected = random.sample(list(others), sample_size)

        anonymity_set = set(selected) | {self.target}

        obs = Observation(time, anonymity_set, context)
        self.observations.append(obs)

        return obs

    def compute_intersection(self) -> Set[str]:
        """Calcular␣intersecci n␣de␣todas␣las␣observaciones."""
        if not self.observations:
            return self.population

        result = self.observations[0].anonymity_set.copy()
        for obs in self.observations[1:]:
            result &= obs.anonymity_set

        return result

    def analyze_attack_progress(self) -> List[Dict]:
        """Analizar␣progreso␣del␣ataque␣con␣cada␣observaci n."""
        if not self.observations:
            return []

        results = []
        current_intersection = self.observations[0].anonymity_set.copy()
```

```
        for i, obs in enumerate(self.observations):
            if i > 0:
                current_intersection &= obs.anonymity_set

            results.append({
                'observation': i + 1,
                'individual_set_size': len(obs.anonymity_set),
                'intersection_size': len(current_intersection),
                'target_in_set': self.target in current_intersection,
                'uniquely_identified': len(current_intersection) == 1
            })

        return results

    def estimate_observations_needed(
        self,
        avg_set_size: int,
        target_set_size: int = 1
    ) -> float:
        """
        Estimar observaciones necesarias para reducir a target_set_size.
        """
        N = len(self.population)
        n = avg_set_size

        if n >= N:
            return float('inf')

        # Aproximaci n: cada observaci n reduce por factor n/N
        reduction_factor = n / N

        # Queremos N * (n/N)^k <= target
        # k >= log(target/N) / log(n/N)

        k = math.log(target_set_size / N) / math.log(reduction_factor)

        return max(1, k)

    def simulate_full_attack(
        self,
        avg_set_size: int,
        max_observations: int = 100
    ) -> Dict:
        """
        Simular ataque completo hasta identificar al target.
        """
        self.observations = []

        for i in range(max_observations):
            # Variar tama o del set ligeramente
```

```
            actual_size = avg_set_size + random.randint(-5, 5)
            actual_size = max(10, min(actual_size, len(self.population)))

            self.generate_observation(i, actual_size, f"obs_{i}")

            intersection = self.compute_intersection()

            if len(intersection) == 1:
                return {
                    'success': True,
                    'observations_needed': i + 1,
                    'target_identified': self.target in intersection,
                    'final_set': intersection
                }

        final_intersection = self.compute_intersection()
        return {
            'success': False,
            'observations_made': max_observations,
            'final_set_size': len(final_intersection),
            'target_in_set': self.target in final_intersection
        }


class IntersectionDefense:
    """
    Defensas contra intersection attacks.
    """

    def __init__(self):
        self.dummy_activity_rate = 0.1
        self.rotation_frequency = 10  # Rotar identidad cada N actividades

    def add_noise_to_observations(
        self,
        real_activity: bool,
        noise_probability: float = 0.2
    ) -> bool:
        """
        A adir actividad dummy para confundir observaciones.
        """
        if real_activity:
            return True  # Actividad real siempre ocurre

        # Con probabilidad noise_probability, generar actividad dummy
        return random.random() < noise_probability

    def calculate_effective_anonymity(
        self,
        base_anonymity_set: int,
        noise_rate: float,
```

```
        observations: int
    ) -> float:
        """
␣␣␣␣␣␣␣␣Calcular␣anonymity␣set␣efectivo␣considerando␣noise.

␣␣␣␣␣␣␣␣El␣noise␣hace␣que␣las␣intersecciones␣sean␣menos␣efectivas.
␣␣␣␣␣␣␣␣"""
        # Con noise, cada observaci n tiene falsos positivos
        # Esto aumenta el tama o efectivo de la intersecci n

        noise_factor = 1 + noise_rate
        effective_set = base_anonymity_set * (noise_factor ** observations)

        return min(effective_set, base_anonymity_set * 2)  # Cap

    def recommend_behavior(self, current_observations: int) -> List[str]:
        """Recomendar␣comportamiento␣para␣minimizar␣intersection␣risk."""
        recommendations = []

        if current_observations > 5:
            recommendations.append("Consider␣rotating␣pseudonym/address")

        if current_observations > 10:
            recommendations.append("Add␣dummy␣transactions␣to␣increase␣noise")

        if current_observations > 20:
            recommendations.append("Critical:␣Change␣identity␣completely")

        recommendations.append(f"Estimated␣observations␣until␣deanon:␣~{30␣-␣c

        return recommendations


def intersection_demo():
    print("===␣Intersection␣Attack␣Simulation␣===\n")

    simulator = IntersectionAttackSimulator(population_size=1000)

    print(f"Target␣user:␣{simulator.target}")
    print(f"Population␣size:␣{len(simulator.population)}")

    # Generar observaciones
    for i in range(15):
        simulator.generate_observation(i, anonymity_set_size=100)

    # Analizar progreso
    print("\n--␣Attack␣Progress␣--")
    progress = simulator.analyze_attack_progress()

    print(f"{'Obs':<5}␣{'Set␣Size':<10}␣{'Intersection':<12}␣{'Identified'}")
    print("-" * 40)
```

```
    for p in progress:
        print(f"{p['observation']:<5} {p['individual_set_size']:<10} "
              f"{p['intersection_size']:<12} {p['uniquely_identified']}")

    # Estimaci n te rica
    print("\n-- Theoretical Analysis --")
    estimated = simulator.estimate_observations_needed(avg_set_size=100)
    print(f"Estimated observations needed: {estimated:.1f}")

    # Ataque completo
    print("\n-- Full Attack Simulation --")
    simulator2 = IntersectionAttackSimulator(population_size=1000)
    result = simulator2.simulate_full_attack(avg_set_size=100)

    if result['success']:
        print(f"Target identified after {result['observations_needed']} observat
    else:
        print(f"Not identified after {result['observations_made']} observations"
        print(f"Final set size: {result['final_set_size']}")

intersection_demo()
```

6.8. Long-term Correlation

Definición 6.8 (Long-term Correlation Attack)**.** Acumular datos sobre un período extendido para crear fingerprint conductual único:

Señales utilizadas:

- Patrones de horario de actividad
- Distribución de montos
- Frecuencia de transacciones
- Contrapartes frecuentes
- Patrones de fee/gas

Listing 6.8: Detector de patrones para long-term correlation

```
from dataclasses import dataclass
from datetime import datetime, timedelta
from typing import Dict, List, Tuple
from collections import defaultdict
import math
import random

@dataclass
class ActivityRecord:
    timestamp: datetime
    amount: float
    counterparty: str
```

```
    fee: float
    tx_type: str  # send, receive, contract

class BehavioralFingerprint:
    """
    Crea fingerprint conductual a partir de historial de actividad.
    """

    def __init__(self, address: str):
        self.address = address
        self.activities: List[ActivityRecord] = []

    def add_activity(self, activity: ActivityRecord):
        self.activities.append(activity)

    def extract_features(self) -> Dict:
        """Extraer features para fingerprinting."""
        if len(self.activities) < 5:
            return {'insufficient_data': True}

        features = {}

        # 1. Distribuci n horaria
        hours = [a.timestamp.hour for a in self.activities]
        hour_dist = [0] * 24
        for h in hours:
            hour_dist[h] += 1
        total = sum(hour_dist)
        features['hour_distribution'] = [h/total for h in hour_dist]

        # 2. Distribuci n de d as de semana
        days = [a.timestamp.weekday() for a in self.activities]
        day_dist = [0] * 7
        for d in days:
            day_dist[d] += 1
        features['day_distribution'] = [d/len(days) for d in day_dist]

        # 3. Estad sticas de montos
        amounts = [a.amount for a in self.activities]
        features['amount_mean'] = sum(amounts) / len(amounts)
        features['amount_std'] = math.sqrt(
            sum((a - features['amount_mean'])**2 for a in amounts) / len(amoun
        )
        features['round_amount_ratio'] = sum(1 for a in amounts if a == round(

        # 4. Frecuencia de actividad
        sorted_acts = sorted(self.activities, key=lambda x: x.timestamp)
        if len(sorted_acts) > 1:
            intervals = [
                (sorted_acts[i+1].timestamp - sorted_acts[i].timestamp).total_
                for i in range(len(sorted_acts) - 1)
```

```
            ]
            features['avg_interval_hours'] = sum(intervals) / len(intervals) / 3

        # 5. Contrapartes frecuentes
        counterparties = defaultdict(int)
        for a in self.activities:
            counterparties[a.counterparty] += 1
        features['unique_counterparties'] = len(counterparties)
        features['top_counterparty_ratio'] = max(counterparties.values()) / len(

        # 6. Fee behavior
        fees = [a.fee for a in self.activities]
        features['avg_fee'] = sum(fees) / len(fees)

        return features

    def calculate_similarity(self, other: 'BehavioralFingerprint') -> float:
        """
␣␣␣␣␣␣␣␣Calcular␣similitud␣con␣otro␣fingerprint.
␣␣␣␣␣␣␣␣Retorna␣0-1␣(1␣=␣id ntico).
␣␣␣␣␣␣␣␣"""
        f1 = self.extract_features()
        f2 = other.extract_features()

        if f1.get('insufficient_data') or f2.get('insufficient_data'):
            return 0.0

        similarities = []

        # Similitud de distribuci n horaria (cosine)
        h1 = f1['hour_distribution']
        h2 = f2['hour_distribution']
        dot = sum(a*b for a, b in zip(h1, h2))
        norm1 = math.sqrt(sum(a*a for a in h1))
        norm2 = math.sqrt(sum(b*b for b in h2))
        if norm1 > 0 and norm2 > 0:
            similarities.append(dot / (norm1 * norm2))

        # Similitud de estad sticas de monto
        if f1['amount_std'] > 0 and f2['amount_std'] > 0:
            mean_diff = abs(f1['amount_mean'] - f2['amount_mean']) / max(f1['amo
            similarities.append(1 - min(1, mean_diff))

        # Similitud de round ratio
        similarities.append(1 - abs(f1['round_amount_ratio'] - f2['round_amount_

        # Similitud de frecuencia
        if 'avg_interval_hours' in f1 and 'avg_interval_hours' in f2:
            int1, int2 = f1['avg_interval_hours'], f2['avg_interval_hours']
            if max(int1, int2) > 0:
                similarities.append(1 - abs(int1 - int2) / max(int1, int2))
```

```
        return sum(similarities) / len(similarities) if similarities else 0.0


class LongTermCorrelationDefense:
    """
    Defensas contra long-term correlation.
    """

    def __init__(self):
        self.behavior_randomization = True

    def randomize_timing(
        self,
        intended_time: datetime,
        max_delay_hours: float = 6
    ) -> datetime:
        """A adir delay aleatorio al timing."""
        delay = random.expovariate(1 / (max_delay_hours / 2))
        delay = min(delay, max_delay_hours)
        return intended_time + timedelta(hours=delay)

    def randomize_amount(
        self,
        intended_amount: float,
        noise_percent: float = 5
    ) -> float:
        """A adir ruido al monto."""
        noise = random.uniform(-noise_percent, noise_percent) / 100
        return intended_amount * (1 + noise)

    def recommend_countermeasures(
        self,
        fingerprint: BehavioralFingerprint
    ) -> List[str]:
        """Recomendar contramedidas basadas en fingerprint actual."""
        features = fingerprint.extract_features()
        recommendations = []

        if features.get('insufficient_data'):
            return ["Continue building activity history for analysis"]

        # Verificar patrones identificables
        hour_dist = features['hour_distribution']
        max_hour_ratio = max(hour_dist)
        if max_hour_ratio > 0.3:
            recommendations.append(
                f"High activity concentration in certain hours ({max_hour_rati
                "Spread transactions across more hours."
            )
```

```
        if features['round_amount_ratio'] > 0.5:
            recommendations.append(
                f"Too many round amounts ({features['round_amount_ratio']:.0%}). "
                "Use precise amounts with decimals."
            )

        if features['top_counterparty_ratio'] > 0.3:
            recommendations.append(
                "High concentration on few counterparties. "
                "Use intermediate addresses or mixers."
            )

        if features.get('avg_interval_hours', 0) < 24:
            recommendations.append(
                "Very frequent activity. Consider batching transactions."
            )

        return recommendations


def long_term_demo():
    print("=== Long-term Correlation Analysis ===\n")

    # Crear dos usuarios con patrones diferentes
    user_a = BehavioralFingerprint("user_A")
    user_b = BehavioralFingerprint("user_B")
    user_a_clone = BehavioralFingerprint("user_A_clone")  # Mismo usuario, otra

    base = datetime(2024, 1, 1, 9, 0)

    # User A: Patrón de oficina, 9-5, montos redondos
    for i in range(50):
        user_a.add_activity(ActivityRecord(
            timestamp=base + timedelta(days=i, hours=random.randint(0, 8)),
            amount=round(random.uniform(0.1, 1.0), 1),
            counterparty=f"merchant_{i % 5}",
            fee=0.001,
            tx_type="send"
        ))

    # User A clone: Mismo patrón
    for i in range(50):
        user_a_clone.add_activity(ActivityRecord(
            timestamp=base + timedelta(days=i, hours=random.randint(0, 8)),
            amount=round(random.uniform(0.1, 1.0), 1),
            counterparty=f"other_{i % 5}",
            fee=0.001,
            tx_type="send"
        ))

    # User B: Patrón nocturno, montos precisos
```

```
    for i in range(50):
        user_b.add_activity(ActivityRecord(
            timestamp=base + timedelta(days=i, hours=random.randint(18, 23)),
            amount=random.uniform(0.1, 1.0),
            counterparty=f"vendor_{i % 10}",
            fee=0.002,
            tx_type="send"
        ))

    # Calcular similitudes
    print("— Fingerprint Similarity —")
    sim_aa = user_a.calculate_similarity(user_a_clone)
    sim_ab = user_a.calculate_similarity(user_b)

    print(f"User A vs A_clone (same person): {sim_aa:.2%}")
    print(f"User A vs User B (different): {sim_ab:.2%}")

    # Recomendaciones
    print("\n— Defense Recommendations for User A —")
    defense = LongTermCorrelationDefense()
    recs = defense.recommend_countermeasures(user_a)
    for rec in recs[:3]:
        print(f"    {rec}")

long_term_demo()
```

Resumen del Bloque

Idea Clave

Puntos clave de Ataques y Defensa (Bloque 19):
1. Deanonymization Attacks:

- IP correlation (first-seen heuristic)
- Timing analysis (patrones de actividad)
- Amount patterns (montos característicos)

2. Sybil Attacks:

- Atacante inunda mixer con depósitos propios
- Reduce anonymity set efectivo
- Defensas: fees, staking, proof-of-personhood

3. Eclipse Attacks:

- Aislar nodo de red honesta
- Defensas: peer diversity, reserved slots

4. Dust Attacks:

- Enviar dust para tracking
- Defensas: coin control, freeze dust

5. Graph Analysis:

- Common input ownership
- Change detection
- Address clustering

6. Taint Analysis:

- Poison: todo contacto = tainted
- Haircut: proporcional
- FIFO/LIFO: por orden

7. Intersection Attacks:

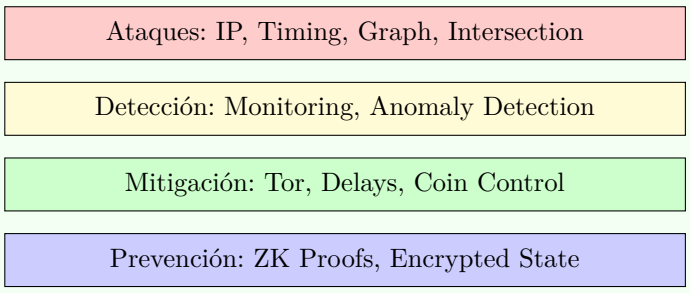
- Múltiples observaciones reducen anonymity set
- k observaciones: $AS \to 1$

8. Long-term Correlation:

- Behavioral fingerprinting
- Timing, amounts, counterparties

Aplicación en Banco ZK

Defensa Integral en Banco ZK:



Implementación en capas:

1. **Network:** Tor obligatorio, IP rotation
2. **Timing:** Random delays, cover traffic
3. **Amounts:** Non-round amounts, splitting
4. **Graph:** ZK proofs eliminan linking
5. **Behavioral:** Address rotation, identity management

Métricas de seguridad:

- Anonymity set efectivo > 100
- IP exposure $= 0$
- Timing regularity $< 0{,}3$
- Graph linkability $< 5\,\%$

Parte I

Herramientas y Ecosistema

Capítulo 7

Herramientas y Ecosistema

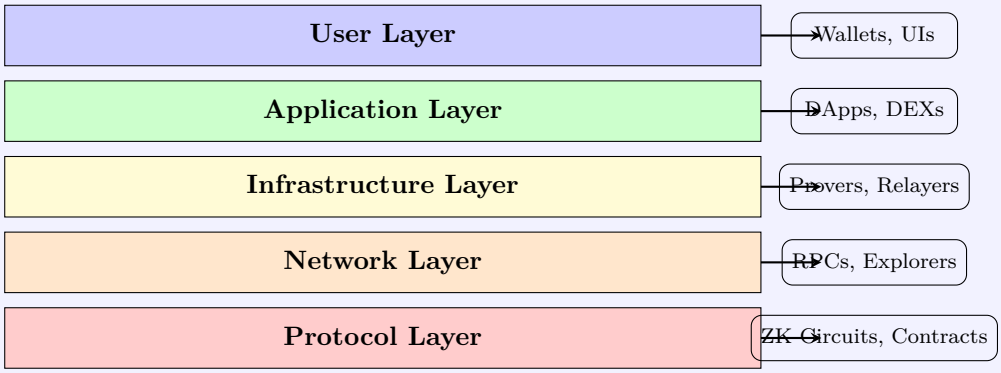
Bloque 20 de 20 — Bloque Final

Introducción

Este bloque final presenta las herramientas prácticas y el ecosistema necesario para implementar sistemas de privacidad en blockchain. Cubrimos desde exploradores privacy-aware hasta infraestructura de proving, proporcionando una guía completa para construir y operar aplicaciones privadas.

Idea Clave

Taxonomía del Ecosistema de Privacidad:



Principio fundamental: La privacidad end-to-end requiere que cada capa del stack preserve la privacidad del usuario.

7.1. Exploradores Privacy-Aware

Definición 7.1 (Privacy-Aware Explorer)**.** Un explorador de blockchain diseñado para permitir consultas sin comprometer la privacidad del usuario que consulta. Características:

- **No tracking:** Sin cookies, analytics, fingerprinting
- **Acceso anónimo:** Soporte Tor (.onion), no requiere cuenta
- **Logging mínimo:** No guardar IPs ni queries
- **Open source:** Código auditable

7.1.1. Arquitectura de Explorador Privado

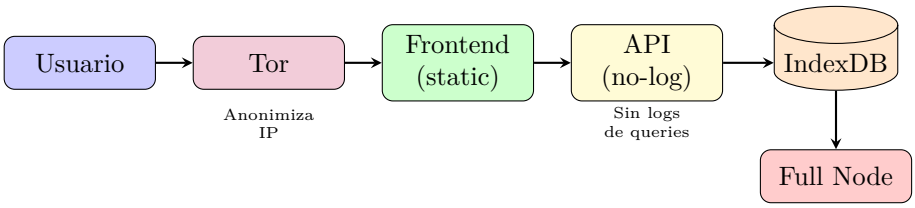


Figura 7.1: Arquitectura de explorador privacy-aware

7.1.2. Análisis Comparativo de Exploradores

Explorador	Tor	No Track	Open Source	Self-Host	Score
Blockchair	Parcial	No	No	No	1/4
Etherscan	No	No	No	No	0/4
Blockscout	No	Sí	Sí	Sí	3/4
Self-hosted	Sí	Sí	Sí	Sí	4/4

Tabla 7.1: Comparativa de exploradores por privacidad

7.1.3. Implementación de Explorador Privado

```python
"""
Privacy-Aware Blockchain Explorer
Diseñado para preservar privacidad de queries
"""

from dataclasses import dataclass, field
from typing import Optional, Dict, List, Set, Any
from datetime import datetime, timedelta
from abc import ABC, abstractmethod
import hashlib
import json
from enum import Enum

class PrivacyLevel(Enum):
    """Niveles de privacidad del explorador"""
    MINIMAL = "minimal"      # Solo funcionalidad basica
    STANDARD = "standard"    # Balance privacidad/features
    MAXIMUM = "maximum"      # Maxima privacidad

@dataclass
class QueryConfig:
    """Configuracion de privacidad para queries"""
    privacy_level: PrivacyLevel = PrivacyLevel.STANDARD
    use_tor: bool = True
    cache_locally: bool = True          # Cache en cliente, no servidor
    batch_queries: bool = True          # Agrupar queries
    add_dummy_queries: bool = True      # Queries falsas para confusion
    delay_random: bool = True           # Delays aleatorios
```

```
29
30      def get_dummy_count(self) -> int:
31          """Numero de queries dummy segun nivel"""
32          if self.privacy_level == PrivacyLevel.MAXIMUM:
33              return 5
34          elif self.privacy_level == PrivacyLevel.STANDARD:
35              return 2
36          return 0
37
38  @dataclass
39  class ExplorerQuery:
40      """Query individual al explorador"""
41      query_type: str           # 'address', 'tx', 'block'
42      target: str               # Hash o direccion
43      timestamp: datetime = field(default_factory=datetime.now)
44      is_dummy: bool = False    # Es query de confusion?
45
46      def anonymize(self) -> 'ExplorerQuery':
47          """Crear version anonimizada (sin timestamp exacto)"""
48          # Redondear timestamp a hora
49          rounded = self.timestamp.replace(
50              minute=0, second=0, microsecond=0
51          )
52          return ExplorerQuery(
53              query_type=self.query_type,
54              target=self.target,
55              timestamp=rounded,
56              is_dummy=self.is_dummy
57          )
58
59  @dataclass
60  class CachedResult:
61      """Resultado cacheado localmente"""
62      query: ExplorerQuery
63      result: Dict[str, Any]
64      cached_at: datetime
65      expires_at: datetime
66
67      def is_expired(self) -> bool:
68          return datetime.now() > self.expires_at
69
70  class PrivacyExplorer:
71      """
72      Explorador de blockchain con privacidad mejorada.
73
74      Estrategias de privacidad:
75      1. Queries batch para ocultar patron individual
76      2. Dummy queries para confusion
77      3. Cache local para reducir queries
78      4. Delays aleatorios para evitar timing analysis
79      """
80
81      def __init__(self, config: QueryConfig):
82          self.config = config
83          self.local_cache: Dict[str, CachedResult] = {}
84          self.pending_queries: List[ExplorerQuery] = []
85          self.query_history: List[ExplorerQuery] = []  # Solo local
86
```

```
87      def query_address(self, address: str) -> Dict[str, Any]:
88          """Query de direccion con privacidad"""
89          query = ExplorerQuery(
90              query_type="address",
91              target=address
92          )
93          return self._execute_query(query)
94
95      def query_transaction(self, tx_hash: str) -> Dict[str, Any]:
96          """Query de transaccion con privacidad"""
97          query = ExplorerQuery(
98              query_type="tx",
99              target=tx_hash
100         )
101         return self._execute_query(query)
102
103     def query_block(self, block_id: str) -> Dict[str, Any]:
104         """Query de bloque con privacidad"""
105         query = ExplorerQuery(
106             query_type="block",
107             target=block_id
108         )
109         return self._execute_query(query)
110
111     def _execute_query(self, query: ExplorerQuery) -> Dict[str,
        Any]:
112         """Ejecutar query con protecciones de privacidad"""
113         # 1. Verificar cache local
114         cache_key = f"{query.query_type}:{query.target}"
115         if self.config.cache_locally and cache_key in
            self.local_cache:
116             cached = self.local_cache[cache_key]
117             if not cached.is_expired():
118                 return cached.result
119
120         # 2. Preparar batch con dummies si corresponde
121         batch = self._prepare_batch(query)
122
123         # 3. Ejecutar batch
124         results = self._execute_batch(batch)
125
126         # 4. Cachear resultado real
127         if query.target in results:
128             self._cache_result(query, results[query.target])
129             return results[query.target]
130
131         return {}
132
133     def _prepare_batch(self, real_query: ExplorerQuery) ->
        List[ExplorerQuery]:
134         """Preparar batch con queries reales y dummy"""
135         batch = [real_query]
136
137         if self.config.add_dummy_queries:
138             dummy_count = self.config.get_dummy_count()
139             for _ in range(dummy_count):
140                 dummy =
                        self._generate_dummy_query(real_query.query_type)
```

```
                batch.append(dummy)

        # Mezclar orden
        import random
        random.shuffle(batch)

        return batch

    def _generate_dummy_query(self, query_type: str) ->
        ExplorerQuery:
        """Generar query dummy realista"""
        import random

        if query_type == "address":
            # Direccion aleatoria
            dummy_target = "0x" + "".join(
                random.choices("0123456789abcdef", k=40)
            )
        elif query_type == "tx":
            # Hash aleatorio
            dummy_target = "0x" + "".join(
                random.choices("0123456789abcdef", k=64)
            )
        else:
            # Bloque aleatorio reciente
            dummy_target = str(random.randint(1000000, 2000000))

        return ExplorerQuery(
            query_type=query_type,
            target=dummy_target,
            is_dummy=True
        )

    def _execute_batch(
        self,
        batch: List[ExplorerQuery]
    ) -> Dict[str, Dict[str, Any]]:
        """Ejecutar batch de queries"""
        import random
        import time

        results = {}

        for query in batch:
            # Delay aleatorio entre queries
            if self.config.delay_random:
                delay = random.uniform(0.1, 0.5)
                time.sleep(delay)

            # Simular llamada a API
            result = self._mock_api_call(query)
            if not query.is_dummy:
                results[query.target] = result

            # Guardar en historial local (solo queries reales)
            if not query.is_dummy:
                self.query_history.append(query.anonymize())

```

```
198         return results
199
200     def _mock_api_call(self, query: ExplorerQuery) -> Dict[str,
            Any]:
201         """Simular llamada a API (reemplazar con implementacion
                real)"""
202         # En produccion, esto llamaria al backend real
203         return {
204             "target": query.target,
205             "type": query.query_type,
206             "found": True,
207             "data": {"balance": "1.5 ETH"}  # Datos simulados
208         }
209
210     def _cache_result(self, query: ExplorerQuery, result:
            Dict[str, Any]):
211         """Cachear resultado localmente"""
212         cache_key = f"{query.query_type}:{query.target}"
213
214         # TTL segun tipo de query
215         if query.query_type == "block":
216             ttl = timedelta(hours=24)  # Bloques no cambian
217         elif query.query_type == "tx":
218             ttl = timedelta(hours=1)   # TXs confirmadas no cambian
219         else:
220             ttl = timedelta(minutes=5) # Balances cambian
221
222         cached = CachedResult(
223             query=query,
224             result=result,
225             cached_at=datetime.now(),
226             expires_at=datetime.now() + ttl
227         )
228
229         self.local_cache[cache_key] = cached
230
231     def get_privacy_stats(self) -> Dict[str, Any]:
232         """Estadisticas de privacidad de la sesion"""
233         real_queries = [q for q in self.query_history if not
                q.is_dummy]
234
235         return {
236             "total_queries": len(self.query_history),
237             "real_queries": len(real_queries),
238             "cache_hits": len(self.local_cache),
239             "privacy_level": self.config.privacy_level.value,
240             "tor_enabled": self.config.use_tor,
241             "dummy_ratio": 1 - (len(real_queries) / max(1,
                    len(self.query_history)))
242         }
243
244     def clear_local_data(self):
245         """Limpiar todos los datos locales"""
246         self.local_cache.clear()
247         self.query_history.clear()
248         self.pending_queries.clear()
249
250
```

```python
class ExplorerBackend:
    """
    Backend de explorador sin logging de usuarios.

    Politicas:
    - No guardar IPs
    - No guardar queries individuales
    - Solo estadisticas agregadas
    """

    def __init__(self, full_node_url: str):
        self.full_node_url = full_node_url
        self.aggregate_stats = {
            "total_queries": 0,
            "queries_by_type": {"address": 0, "tx": 0, "block": 0}
        }

    def handle_query(
        self,
        query_type: str,
        target: str,
        client_ip: str  # NO lo usamos
    ) -> Dict[str, Any]:
        """
        Manejar query sin logging de cliente.

        IMPORTANTE: client_ip se ignora intencionalmente.
        """
        # NO logueamos: client_ip, query especifica, timestamp exacto

        # Solo actualizamos estadisticas agregadas
        self.aggregate_stats["total_queries"] += 1
        self.aggregate_stats["queries_by_type"][query_type] += 1

        # Obtener datos del full node
        result = self._query_full_node(query_type, target)

        return result

    def _query_full_node(
        self,
        query_type: str,
        target: str
    ) -> Dict[str, Any]:
        """Query al full node (sin logging)"""
        # Implementacion real llamaria al nodo
        return {"type": query_type, "target": target, "data": {}}

    def get_aggregate_stats(self) -> Dict[str, Any]:
        """Solo estadisticas agregadas, sin datos individuales"""
        return self.aggregate_stats.copy()


# Demo
def privacy_explorer_demo():
    print("=== Privacy-Aware Explorer Demo ===\n")

```

```
    # Configuracion maxima privacidad
    config = QueryConfig(
        privacy_level=PrivacyLevel.MAXIMUM,
        use_tor=True,
        add_dummy_queries=True,
        delay_random=True
    )

    explorer = PrivacyExplorer(config)

    # Queries de ejemplo
    print("Ejecutando queries con proteccion de privacidad...")

    result1 = explorer.query_address(
        "0x1234567890abcdef1234567890abcdef12345678"
    )
    print(f"  Query 1: Address lookup")

    result2 = explorer.query_transaction(
        "0xabcdef1234567890abcdef1234567890abcdef1234567890abcdef1234567890ab"
    )
    print(f"  Query 2: Transaction lookup")

    # Segunda query a misma address (cache hit)
    result3 = explorer.query_address(
        "0x1234567890abcdef1234567890abcdef12345678"
    )
    print(f"  Query 3: Address lookup (cached)")

    # Estadisticas
    stats = explorer.get_privacy_stats()
    print(f"\nEstadisticas de privacidad:")
    print(f"  Nivel: {stats['privacy_level']}")
    print(f"  Queries reales: {stats['real_queries']}")
    print(f"  Cache hits: {stats['cache_hits']}")
    print(f"  Dummy ratio: {stats['dummy_ratio']:.1%}")
    print(f"  Tor enabled: {stats['tor_enabled']}")

privacy_explorer_demo()
```

Listing 7.1: Explorador de blockchain privacy-aware

7.2. Wallets con Soporte ZK

Definición 7.2 (ZK-Enabled Wallet)**.** Wallet que soporta operaciones de privacidad basadas en zero-knowledge proofs:

- **Shielded balances:** Fondos ocultos criptográficamente
- **Private transfers:** Transferencias sin revelar montos/destinos
- **Proof generation:** Capacidad de generar ZK proofs
- **Note management:** Gestión de notas/commitments privados

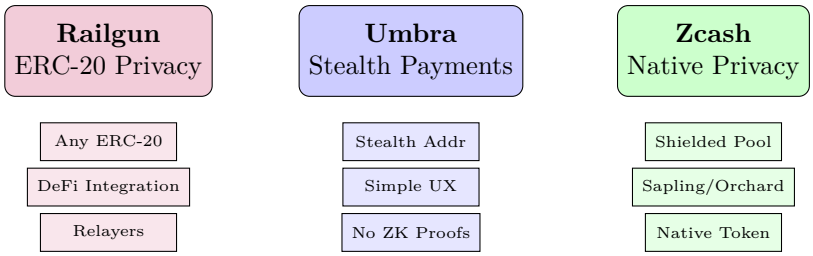


Figura 7.2: Ecosistema de wallets con soporte de privacidad

Wallet	Chain	ZK	Mobile	DeFi	AS	Complejidad
Railgun	EVM	Sí	Parcial	Sí	Alto	Alta
Umbra	EVM	No	Sí	No	Bajo	Baja
Zashi	Zcash	Sí	Sí	No	Alto	Media
YWallet	Zcash	Sí	Sí	No	Alto	Media
Nighthawk	Zcash	Sí	Sí	No	Alto	Media

Tabla 7.2: Comparativa de wallets ZK (AS = Anonymity Set)

7.2.1. Comparativa de Wallets ZK

7.2.2. Arquitectura de Wallet ZK

```python
"""
ZK-Enabled Wallet Architecture
Soporte para operaciones privadas con zero-knowledge proofs
"""

from dataclasses import dataclass, field
from typing import Optional, Dict, List, Tuple, Any
from datetime import datetime
from abc import ABC, abstractmethod
from enum import Enum
import hashlib
import secrets
import json

class WalletType(Enum):
    """Tipos de wallet soportados"""
    RAILGUN = "railgun"
    UMBRA = "umbra"
    ZCASH = "zcash"
    GENERIC_ZK = "generic_zk"

@dataclass
class PrivateNote:
    """
    Representacion de un deposito privado.

    Una Note contiene:
    - commitment: hash publico en el contrato
    - nullifier_secret: secreto para gastar
    - amount: cantidad (encriptada on-chain)
    - asset: token/moneda
    """
    commitment: str
```

```
34      nullifier_secret: bytes
35      amount: int
36      asset: str
37      created_at: datetime
38      spent: bool = False
39      spending_tx: Optional[str] = None
40
41      def compute_nullifier(self) -> str:
42          """Calcular nullifier para gastar la note"""
43          data = self.nullifier_secret + self.commitment.encode()
44          return hashlib.sha256(data).hexdigest()
45
46      def to_encrypted(self, encryption_key: bytes) -> bytes:
47          """Serializar note encriptada para backup"""
48          # Simplificado - en produccion usar AES-GCM
49          note_data = json.dumps({
50              "commitment": self.commitment,
51              "nullifier_secret": self.nullifier_secret.hex(),
52              "amount": self.amount,
53              "asset": self.asset,
54              "created_at": self.created_at.isoformat(),
55              "spent": self.spent
56          }).encode()
57
58          # XOR simple para demo (usar crypto real en produccion)
59          key_extended = (encryption_key * (len(note_data) //
                len(encryption_key) + 1))
60          encrypted = bytes(a ^ b for a, b in zip(note_data,
                key_extended))
61
62          return encrypted
63
64      @classmethod
65      def from_encrypted(
66          cls,
67          encrypted: bytes,
68          encryption_key: bytes
69      ) -> 'PrivateNote':
70          """Deserializar note desde backup encriptado"""
71          key_extended = (encryption_key * (len(encrypted) //
                len(encryption_key) + 1))
72          decrypted = bytes(a ^ b for a, b in zip(encrypted,
                key_extended))
73
74          data = json.loads(decrypted.decode())
75          return cls(
76              commitment=data["commitment"],
77              nullifier_secret=bytes.fromhex(data["nullifier_secret"]),
78              amount=data["amount"],
79              asset=data["asset"],
80              created_at=datetime.fromisoformat(data["created_at"]),
81              spent=data["spent"]
82          )
83
84  @dataclass
85  class WalletState:
86      """Estado completo del wallet"""
87      notes: List[PrivateNote] = field(default_factory=list)
```

```
    spent_nullifiers: set = field(default_factory=set)
    pending_deposits: List[str] = field(default_factory=list)
    pending_withdrawals: List[str] = field(default_factory=list)
    last_sync_block: int = 0

    def get_balance(self, asset: str) -> int:
        """Balance disponible de un asset"""
        return sum(
            note.amount
            for note in self.notes
            if note.asset == asset and not note.spent
        )

    def get_all_balances(self) -> Dict[str, int]:
        """Todos los balances por asset"""
        balances: Dict[str, int] = {}
        for note in self.notes:
            if not note.spent:
                balances[note.asset] = balances.get(note.asset, 0)
                    + note.amount
        return balances

    def get_spendable_notes(
        self,
        asset: str,
        amount: int
    ) -> List[PrivateNote]:
        """Seleccionar notes para gastar un monto"""
        available = [
            n for n in self.notes
            if n.asset == asset and not n.spent
        ]
        available.sort(key=lambda n: n.amount, reverse=True)

        selected = []
        total = 0
        for note in available:
            selected.append(note)
            total += note.amount
            if total >= amount:
                break

        if total < amount:
            raise ValueError(f"Insufficient balance: {total} <
                {amount}")

        return selected


class ZKProver(ABC):
    """Interfaz abstracta para generacion de pruebas"""

    @abstractmethod
    def generate_deposit_proof(
        self,
        amount: int,
        commitment: str,
        blinding: bytes
```

```
    ) -> bytes:
        """Generar prueba de deposito"""
        pass

    @abstractmethod
    def generate_transfer_proof(
        self,
        input_notes: List[PrivateNote],
        output_amounts: List[int],
        output_recipients: List[str]
    ) -> bytes:
        """Generar prueba de transferencia"""
        pass

    @abstractmethod
    def generate_withdrawal_proof(
        self,
        note: PrivateNote,
        recipient: str
    ) -> bytes:
        """Generar prueba de retiro"""
        pass


class ClientSideProver(ZKProver):
    """Prover que genera pruebas en el dispositivo del usuario"""

    def __init__(self, circuit_path: str):
        self.circuit_path = circuit_path
        self.proving_key: Optional[bytes] = None

    def load_proving_key(self, key_path: str):
        """Cargar proving key (puede ser pesado)"""
        # En produccion, cargar archivo real
        self.proving_key = b"mock_proving_key"

    def generate_deposit_proof(
        self,
        amount: int,
        commitment: str,
        blinding: bytes
    ) -> bytes:
        """Generar prueba localmente"""
        # Simular generacion de prueba
        witness = {
            "amount": amount,
            "commitment": commitment,
            "blinding": blinding.hex()
        }

        # En produccion: llamar a snarkjs/circom/etc
        proof_data = hashlib.sha256(
            json.dumps(witness).encode()
        ).digest()

        return proof_data

    def generate_transfer_proof(
```

```
        self,
        input_notes: List[PrivateNote],
        output_amounts: List[int],
        output_recipients: List[str]
    ) -> bytes:
        """Generar prueba de transferencia localmente"""
        witness = {
            "inputs": [n.commitment for n in input_notes],
            "outputs": list(zip(output_amounts, output_recipients))
        }

        return
            hashlib.sha256(json.dumps(witness).encode()).digest()

    def generate_withdrawal_proof(
        self,
        note: PrivateNote,
        recipient: str
    ) -> bytes:
        """Generar prueba de retiro localmente"""
        witness = {
            "nullifier": note.compute_nullifier(),
            "recipient": recipient,
            "amount": note.amount
        }

        return
            hashlib.sha256(json.dumps(witness).encode()).digest()


class ZKWallet:
    """
    Wallet con soporte completo de ZK.

    Operaciones:
    - deposit(): Shield fondos (transparente -> privado)
    - transfer(): Transferencia privada
    - withdraw(): Unshield fondos (privado -> transparente)
    """

    def __init__(
        self,
        wallet_type: WalletType,
        prover: ZKProver,
        encryption_key: Optional[bytes] = None
    ):
        self.wallet_type = wallet_type
        self.prover = prover
        self.encryption_key = encryption_key or
            secrets.token_bytes(32)
        self.state = WalletState()

    def deposit(
        self,
        amount: int,
        asset: str
    ) -> Tuple[str, bytes]:
        """
```

```
257         Depositar fondos en pool privado.
258
259         Returns:
260             (commitment, proof) para enviar a contrato
261         """
262         # Generar secretos
263         nullifier_secret = secrets.token_bytes(32)
264         blinding = secrets.token_bytes(32)
265
266         # Calcular commitment
267         preimage = (
268             amount.to_bytes(32, 'big') +
269             asset.encode().ljust(32, b'\x00') +
270             nullifier_secret +
271             blinding
272         )
273         commitment = hashlib.sha256(preimage).hexdigest()
274
275         # Generar prueba
276         proof = self.prover.generate_deposit_proof(
277             amount=amount,
278             commitment=commitment,
279             blinding=blinding
280         )
281
282         # Crear note (guardada localmente)
283         note = PrivateNote(
284             commitment=commitment,
285             nullifier_secret=nullifier_secret,
286             amount=amount,
287             asset=asset,
288             created_at=datetime.now()
289         )
290
291         self.state.notes.append(note)
292         self.state.pending_deposits.append(commitment)
293
294         return commitment, proof
295
296     def transfer(
297         self,
298         amount: int,
299         asset: str,
300         recipient_viewing_key: str
301     ) -> Tuple[str, bytes]:
302         """
303         Transferencia privada.
304
305         Args:
306             amount: Monto a transferir
307             asset: Token/moneda
308             recipient_viewing_key: Clave publica del destinatario
309
310         Returns:
311             (output_commitment, proof)
312         """
313         # Seleccionar notes a gastar
314         input_notes = self.state.get_spendable_notes(asset, amount)
```

```
315         total_input = sum(n.amount for n in input_notes)
316         change = total_input - amount
317 
318         # Outputs: [amount para recipient, change para self]
319         output_amounts = [amount]
320         output_recipients = [recipient_viewing_key]
321 
322         if change > 0:
323             output_amounts.append(change)
324             output_recipients.append("self")  # Nuestra propia
                clave
325 
326         # Generar prueba
327         proof = self.prover.generate_transfer_proof(
328             input_notes=input_notes,
329             output_amounts=output_amounts,
330             output_recipients=output_recipients
331         )
332 
333         # Marcar inputs como gastados
334         for note in input_notes:
335             note.spent = True
336             self.state.spent_nullifiers.add(note.compute_nullifier())
337 
338         # Crear note de cambio si hay
339         if change > 0:
340             change_note = PrivateNote(
341                 commitment=hashlib.sha256(
342                     secrets.token_bytes(32)
343                 ).hexdigest(),
344                 nullifier_secret=secrets.token_bytes(32),
345                 amount=change,
346                 asset=asset,
347                 created_at=datetime.now()
348             )
349             self.state.notes.append(change_note)
350 
351         # Output commitment para recipient (simplificado)
352         output_commitment = hashlib.sha256(
353             f"{amount}:{recipient_viewing_key}".encode()
354         ).hexdigest()
355 
356         return output_commitment, proof
357 
358     def withdraw(
359         self,
360         amount: int,
361         asset: str,
362         recipient_address: str
363     ) -> Tuple[str, bytes]:
364         """
365         Retirar fondos a direccion transparente.
366 
367         Returns:
368             (nullifier, proof)
369         """
370         # Seleccionar note exacta o con suficiente balance
371         notes = self.state.get_spendable_notes(asset, amount)
```

```
372
373         if len(notes) == 1 and notes[0].amount == amount:
374             # Retiro exacto
375             note = notes[0]
376         else:
377             # Necesitamos consolidar primero (simplificado)
378             raise ValueError("Complex withdrawal not implemented")
379
380         # Generar prueba
381         proof = self.prover.generate_withdrawal_proof(
382             note=note,
383             recipient=recipient_address
384         )
385
386         # Marcar como gastada
387         nullifier = note.compute_nullifier()
388         note.spent = True
389         note.spending_tx = "pending"
390         self.state.spent_nullifiers.add(nullifier)
391
392         return nullifier, proof
393
394     def backup(self) -> bytes:
395         """Crear backup encriptado del wallet"""
396         backup_data = {
397             "wallet_type": self.wallet_type.value,
398             "notes": [
399                 note.to_encrypted(self.encryption_key).hex()
400                 for note in self.state.notes
401             ],
402             "last_sync": self.state.last_sync_block
403         }
404
405         # Encriptar todo el backup
406         backup_json = json.dumps(backup_data).encode()
407         encrypted = bytes(
408             a ^ b
409             for a, b in zip(
410                 backup_json,
411                 self.encryption_key * (len(backup_json) // 32 + 1)
412             )
413         )
414
415         return encrypted
416
417     def get_status(self) -> Dict[str, Any]:
418         """Estado actual del wallet"""
419         return {
420             "wallet_type": self.wallet_type.value,
421             "balances": self.state.get_all_balances(),
422             "total_notes": len(self.state.notes),
423             "spent_notes": len(self.state.spent_nullifiers),
424             "pending_deposits": len(self.state.pending_deposits),
425             "pending_withdrawals":
                    len(self.state.pending_withdrawals),
426             "last_sync_block": self.state.last_sync_block
427         }
428
```

```python

# Demo
def zk_wallet_demo():
    print("=== ZK Wallet Demo ===\n")

    # Crear prover y wallet
    prover = ClientSideProver("/path/to/circuits")
    wallet = ZKWallet(
        wallet_type=WalletType.GENERIC_ZK,
        prover=prover
    )

    # 1. Depositar
    print("1. Depositando 100 ETH en pool privado...")
    commitment, proof = wallet.deposit(
        amount=100_000_000_000,  # 100 ETH en wei simplificado
        asset="ETH"
    )
    print(f"   Commitment: {commitment[:20]}...")
    print(f"   Proof size: {len(proof)} bytes")

    # 2. Segundo deposito
    print("\n2. Depositando 50 ETH adicionales...")
    wallet.deposit(amount=50_000_000_000, asset="ETH")

    # 3. Estado
    status = wallet.get_status()
    print(f"\n3. Estado del wallet:")
    print(f"   Balance ETH: {status['balances'].get('ETH', 0) /
       1e9:.0f}")
    print(f"   Notes totales: {status['total_notes']}")

    # 4. Transferencia privada
    print("\n4. Transfiriendo 30 ETH a otro usuario...")
    recipient_key = "0xrecipient_viewing_key_abc123"
    output, proof = wallet.transfer(
        amount=30_000_000_000,
        asset="ETH",
        recipient_viewing_key=recipient_key
    )
    print(f"   Output commitment: {output[:20]}...")

    # 5. Estado actualizado
    status = wallet.get_status()
    print(f"\n5. Balance despues de transferencia:")
    print(f"   Balance ETH: {status['balances'].get('ETH', 0) /
       1e9:.0f}")
    print(f"   Notes gastadas: {status['spent_notes']}")

    # 6. Backup
    print("\n6. Creando backup encriptado...")
    backup = wallet.backup()
    print(f"   Backup size: {len(backup)} bytes")

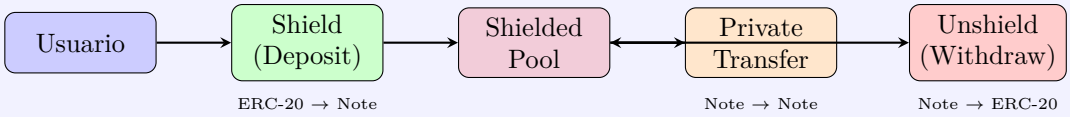
zk_wallet_demo()
```

Listing 7.2: Arquitectura de wallet con soporte ZK

7.2.3. Railgun: Privacy para ERC-20

Idea Clave

Arquitectura Railgun:



Características clave:

- Cualquier token ERC-20 puede ser "shielded"
- Integración con DeFi (swaps, lending) sin revelar identidad
- Red de relayers para privacidad de gas
- Proving del lado del cliente (WASM)

7.2.4. Umbra: Stealth Payments Simplificados

Definición 7.3 (Stealth Address)**.** Dirección única generada para cada transacción usando:

1. Sender genera secreto efímero r
2. Publica $R = r \cdot G$ (punto en curva elíptica)
3. Stealth address: $P_{stealth} = P_{recipient} + H(r \cdot P_{recipient}) \cdot G$
4. Solo recipient puede derivar private key correspondiente

```python
"""
Stealth Address Implementation (Umbra-style)
Direcciones unicas por transaccion
"""

from dataclasses import dataclass
from typing import Tuple
import hashlib
import secrets

# Simulacion de operaciones de curva eliptica
# En produccion usar libreria como py_ecc

@dataclass
class Point:
    """Punto en curva eliptica (simplificado)"""
    x: int
    y: int

    def __add__(self, other: 'Point') -> 'Point':
        # Simplificado - en realidad es suma de puntos EC
        return Point(
            (self.x + other.x) % (2**256),
            (self.y + other.y) % (2**256)
        )
```

```
26
27      def __mul__(self, scalar: int) -> 'Point':
28          # Simplificado - multiplicacion escalar
29          return Point(
30              (self.x * scalar) % (2**256),
31              (self.y * scalar) % (2**256)
32          )
33
34      def to_address(self) -> str:
35          """Convertir punto a direccion Ethereum"""
36          data = self.x.to_bytes(32, 'big') + self.y.to_bytes(32,
                'big')
37          hash_bytes = hashlib.keccak_256(data).digest()
38          return "0x" + hash_bytes[-20:].hex()
39
40  # Generador de curva (simplificado)
41  G = Point(
42      0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798,
43      0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8
44  )
45
46  @dataclass
47  class StealthKeyPair:
48      """Par de claves para stealth payments"""
49      spending_key: int       # Clave privada de gasto
50      viewing_key: int        # Clave privada de visualizacion
51      spending_pubkey: Point  # Clave publica de gasto
52      viewing_pubkey: Point   # Clave publica de visualizacion
53
54      @classmethod
55      def generate(cls) -> 'StealthKeyPair':
56          """Generar nuevo par de claves"""
57          spending_key = secrets.randbelow(2**256)
58          viewing_key = secrets.randbelow(2**256)
59
60          return cls(
61              spending_key=spending_key,
62              viewing_key=viewing_key,
63              spending_pubkey=G * spending_key,
64              viewing_pubkey=G * viewing_key
65          )
66
67      def get_meta_address(self) -> str:
68          """Direccion meta (spending || viewing pubkeys)"""
69          # Formato: st:eth:0x<spending_pubkey><viewing_pubkey>
70          spending_hex = hex(self.spending_pubkey.x)[2:].zfill(64)
71          viewing_hex = hex(self.viewing_pubkey.x)[2:].zfill(64)
72          return f"st:eth:0x{spending_hex}{viewing_hex}"
73
74
75  class StealthAddressGenerator:
76      """Generador de stealth addresses (lado sender)"""
77
78      @staticmethod
79      def generate_stealth_address(
80          recipient_spending_pubkey: Point,
81          recipient_viewing_pubkey: Point
82      ) -> Tuple[str, Point, int]:
```

```
83          """
84          Generar stealth address para recipient.
85
86          Returns:
87              (stealth_address, ephemeral_pubkey, shared_secret)
88          """
89          # 1. Generar secreto efimero
90          ephemeral_secret = secrets.randbelow(2**256)
91          ephemeral_pubkey = G * ephemeral_secret
92
93          # 2. Calcular shared secret: r * viewing_pubkey
94          shared_point = recipient_viewing_pubkey * ephemeral_secret
95          shared_secret = int(hashlib.sha256(
96              shared_point.x.to_bytes(32, 'big')
97          ).hexdigest(), 16)
98
99          # 3. Calcular stealth address
100         # P_stealth = P_spending + hash(shared_secret) * G
101         stealth_point = recipient_spending_pubkey + (G *
                shared_secret)
102         stealth_address = stealth_point.to_address()
103
104         return stealth_address, ephemeral_pubkey, shared_secret
105
106
107 class StealthAddressScanner:
108     """Scanner de stealth payments (lado recipient)"""
109
110     def __init__(self, keypair: StealthKeyPair):
111         self.keypair = keypair
112
113     def check_payment(
114         self,
115         ephemeral_pubkey: Point,
116         payment_address: str
117     ) -> Tuple[bool, int]:
118         """
119         Verificar si un pago es para nosotros.
120
121         Returns:
122             (is_ours, spending_key) si es nuestro
123         """
124         # 1. Calcular shared secret usando viewing key
125         # shared_point = viewing_key * ephemeral_pubkey
126         shared_point = ephemeral_pubkey * self.keypair.viewing_key
127         shared_secret = int(hashlib.sha256(
128             shared_point.x.to_bytes(32, 'big')
129         ).hexdigest(), 16)
130
131         # 2. Calcular stealth address esperada
132         expected_point = self.keypair.spending_pubkey + (G *
                shared_secret)
133         expected_address = expected_point.to_address()
134
135         # 3. Comparar
136         if expected_address.lower() == payment_address.lower():
137             # 4. Calcular spending key para esta address
138             stealth_spending_key = (
```

```
                self.keypair.spending_key + shared_secret
            ) % (2**256)
            return True, stealth_spending_key

        return False, 0

    def scan_announcements(
        self,
        announcements: list
    ) -> list:
        """
        Escanear lista de anuncios buscando pagos nuestros.

        Cada anuncio: (ephemeral_pubkey, payment_address, amount)
        """
        our_payments = []

        for ephemeral, address, amount in announcements:
            is_ours, spending_key = self.check_payment(ephemeral,
                address)
            if is_ours:
                our_payments.append({
                    "address": address,
                    "amount": amount,
                    "spending_key": spending_key
                })

        return our_payments


# Demo
def stealth_demo():
    print("=== Stealth Address Demo (Umbra-style) ===\n")

    # 1. Recipient genera keypair
    print("1. Recipient genera stealth keypair...")
    recipient = StealthKeyPair.generate()
    meta_address = recipient.get_meta_address()
    print(f"   Meta-address: {meta_address[:50]}...")

    # 2. Sender genera stealth address
    print("\n2. Sender genera stealth address para pago...")
    stealth_addr, ephemeral_pub, _ =
        StealthAddressGenerator.generate_stealth_address(
        recipient.spending_pubkey,
        recipient.viewing_pubkey
    )
    print(f"   Stealth address: {stealth_addr}")
    print(f"   Ephemeral pubkey: ({ephemeral_pub.x % 10000}...)")

    # 3. Sender envia ETH a stealth address (on-chain)
    print("\n3. Sender envia 1 ETH a stealth address")
    print("   [TX enviada on-chain]")
    print(f"   Announcement publicado: ephemeral_pubkey")

    # 4. Recipient escanea announcements
    print("\n4. Recipient escanea announcements...")
    scanner = StealthAddressScanner(recipient)
```

```
195
196     # Simular varios announcements
197     announcements = [
198         (Point(123, 456), "0xdecoy1address", 0.5),
199         (ephemeral_pub, stealth_addr, 1.0),  # El nuestro
200         (Point(789, 012), "0xdecoy2address", 2.0),
201     ]
202
203     our_payments = scanner.scan_announcements(announcements)
204
205     if our_payments:
206         print(f"   Encontrado pago:")
207         print(f"     Address: {our_payments[0]['address']}")
208         print(f"     Amount: {our_payments[0]['amount']} ETH")
209         print(f"     Spending key derivado:
                {our_payments[0]['spending_key'] % 10000}...")
210
211     # 5. Recipient puede gastar
212     print("\n5. Recipient puede gastar fondos con spending key
            derivado")
213
214 stealth_demo()
```

Listing 7.3: Implementación simplificada de Stealth Addresses

7.2.5. Zcash Wallets: Privacidad Nativa

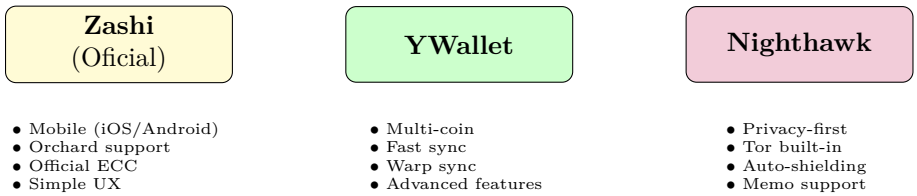


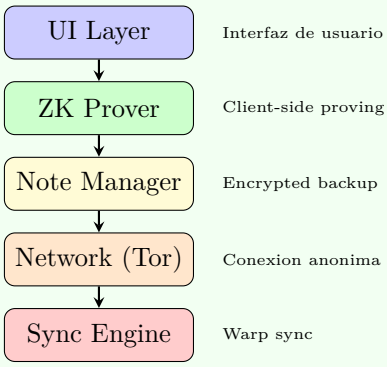
Figura 7.3: Ecosistema de wallets Zcash

Característica	Zashi	YWallet	Nighthawk
Plataformas	iOS, Android	iOS, Android, Desktop	iOS, Android
Shielded pools	Sapling, Orchard	Sapling, Orchard	Sapling
Sync method	Linear	Warp	Linear
Tor integrado	No	No	Sí
Auto-shield	No	Configurable	Sí
Memos	Sí	Sí	Sí
Multi-cuenta	Básico	Avanzado	Básico

Tabla 7.3: Comparativa detallada de wallets Zcash

Aplicación en Banco ZK

Integración de Wallet en Banco ZK:
El wallet de Banco ZK combina las mejores características:



Características del wallet Banco ZK:

1. Client-side ZK proving (WASM optimizado)
2. Backup encriptado con passphrase
3. Tor obligatorio para conexiones
4. Warp sync para sincronización rápida
5. Stealth addresses para pagos entrantes

7.3. RPC Privados

Definición 7.4 (Private RPC)**.** Endpoint RPC que protege la privacidad del usuario al enviar transacciones:

- **No mempool pública:** Transacciones van directo a builders
- **Protección MEV:** Evita frontrunning y sandwich attacks
- **Reverts privados:** Transacciones fallidas no se publican
- **Sin logging de usuarios:** No se guardan IPs ni actividad

7.3.1. Arquitectura de Transacción Privada

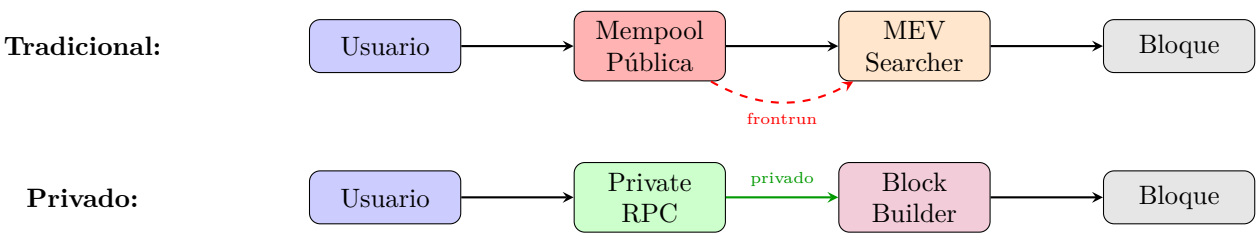


Figura 7.4: Comparación de flujo de transacciones tradicional vs privado

7.3.2. Flashbots Protect

```
"""
Flashbots Protect RPC Client
```

```
Envio de transacciones con proteccion MEV
"""

from dataclasses import dataclass, field
from typing import Optional, Dict, List, Any, Tuple
from enum import Enum
from datetime import datetime, timedelta
import hashlib
import json
import secrets

class HintPreference(Enum):
    """Preferencias de hints para builders"""
    NONE = "none"                    # Sin hints (maxima
       privacidad)
    CALLDATA = "calldata"            # Revelar calldata
    CONTRACT_ADDRESS = "contract"    # Revelar contrato destino
    FUNCTION_SELECTOR = "selector"   # Revelar funcion llamada
    LOGS = "logs"                    # Revelar logs
    HASH = "hash"                    # Solo hash de tx

class RefundPreference(Enum):
    """Preferencia de refund de MEV"""
    NO_REFUND = 0        # Sin refund
    PARTIAL_REFUND = 50  # 50% refund
    FULL_REFUND = 90     # 90% refund (maximo posible)

@dataclass
class ProtectTransaction:
    """Transaccion para enviar via Flashbots Protect"""
    to: str
    value: int
    data: bytes
    gas_limit: int
    max_fee_per_gas: int
    max_priority_fee: int
    nonce: int
    chain_id: int = 1

    # Preferencias de privacidad
    hints: List[HintPreference] = field(
        default_factory=lambda: [HintPreference.HASH]
    )
    refund_percent: int = 90

    # Metadata
    created_at: datetime = field(default_factory=datetime.now)

    def to_raw(self) -> str:
        """Serializar a formato raw (simplificado)"""
        tx_data = {
            "to": self.to,
            "value": hex(self.value),
            "data": self.data.hex(),
            "gas": hex(self.gas_limit),
            "maxFeePerGas": hex(self.max_fee_per_gas),
            "maxPriorityFeePerGas": hex(self.max_priority_fee),
            "nonce": hex(self.nonce),
```

```
            "chainId": hex(self.chain_id)
        }
        return json.dumps(tx_data)

    def get_hash(self) -> str:
        """Hash de la transaccion"""
        return hashlib.sha256(self.to_raw().encode()).hexdigest()


@dataclass
class ProtectResult:
    """Resultado de envio via Protect"""
    tx_hash: str
    status: str           # 'pending', 'included', 'failed',
        'cancelled'
    block_number: Optional[int] = None
    refund_received: int = 0
    hints_used: List[str] = field(default_factory=list)
    simulation_result: Optional[Dict] = None

    def was_frontrun(self) -> bool:
        """Detectar si hubo intento de frontrun"""
        # En Protect, frontrunning deberia ser imposible
        return False


class FlashbotsProtectClient:
    """
    Cliente para Flashbots Protect RPC.

    Caracteristicas:
    - Transacciones no van a mempool publica
    - Simulacion pre-envio
    - Refund de MEV capturado
    - Reverts no se publican
    """

    MAINNET_RPC = "https://rpc.flashbots.net"
    GOERLI_RPC = "https://rpc-goerli.flashbots.net"

    def __init__(
        self,
        private_key: str,
        rpc_url: str = MAINNET_RPC,
        default_hints: List[HintPreference] = None
    ):
        self.private_key = private_key
        self.rpc_url = rpc_url
        self.default_hints = default_hints or [HintPreference.HASH]
        self.pending_txs: Dict[str, ProtectTransaction] = {}
        self.history: List[ProtectResult] = []

    def send_transaction(
        self,
        tx: ProtectTransaction,
        simulate_first: bool = True
    ) -> ProtectResult:
        """
```

```
117         Enviar transaccion via Flashbots Protect.
118
119         Args:
120             tx: Transaccion a enviar
121             simulate_first: Simular antes de enviar
122
123         Returns:
124             Resultado del envio
125         """
126         # 1. Simular si se solicita
127         simulation = None
128         if simulate_first:
129             simulation = self._simulate_transaction(tx)
130             if simulation.get("error"):
131                 return ProtectResult(
132                     tx_hash=tx.get_hash(),
133                     status="simulation_failed",
134                     simulation_result=simulation
135                 )
136
137         # 2. Preparar request con preferencias
138         request = self._build_request(tx)
139
140         # 3. Enviar a Protect RPC
141         response = self._send_to_protect(request)
142
143         # 4. Crear resultado
144         result = ProtectResult(
145             tx_hash=response.get("hash", tx.get_hash()),
146             status="pending",
147             hints_used=[h.value for h in tx.hints],
148             simulation_result=simulation
149         )
150
151         # 5. Guardar en pending
152         self.pending_txs[result.tx_hash] = tx
153
154         return result
155
156     def _build_request(self, tx: ProtectTransaction) -> Dict:
157         """Construir request para Protect RPC"""
158         # Headers con preferencias
159         preferences = {
160             "fast": True,  # Inclusion rapida
161             "privacy": {
162                 "hints": [h.value for h in tx.hints],
163                 "builders": ["flashbots", "all"]  # Builders
                        permitidos
164             },
165             "validity": {
166                 "refund": [{
167                     "address": self._get_address(),
168                     "percent": tx.refund_percent
169                 }]
170             }
171         }
172
173         return {
```

```
            "jsonrpc": "2.0",
            "method": "eth_sendPrivateTransaction",
            "params": [{
                "tx": tx.to_raw(),
                "preferences": preferences
            }],
            "id": 1
        }

    def _simulate_transaction(self, tx: ProtectTransaction) ->
        Dict:
        """Simular transaccion antes de enviar"""
        # En produccion, llamar a eth_callBundle o similar
        return {
            "success": True,
            "gasUsed": tx.gas_limit * 0.8,
            "logs": [],
            "revert": None
        }

    def _send_to_protect(self, request: Dict) -> Dict:
        """Enviar request a Protect RPC"""
        # En produccion, hacer HTTP POST real
        return {
            "hash": hashlib.sha256(
                json.dumps(request).encode()
            ).hexdigest()[:64],
            "status": "pending"
        }

    def _get_address(self) -> str:
        """Obtener direccion del usuario"""
        # Derivar de private key (simplificado)
        return "0x" + hashlib.sha256(
            self.private_key.encode()
        ).hexdigest()[-40:]

    def check_status(self, tx_hash: str) -> ProtectResult:
        """Verificar estado de transaccion"""
        # En produccion, consultar RPC
        if tx_hash in self.pending_txs:
            return ProtectResult(
                tx_hash=tx_hash,
                status="pending"
            )
        return ProtectResult(tx_hash=tx_hash, status="unknown")

    def cancel_transaction(self, tx_hash: str) -> bool:
        """
        Cancelar transaccion pendiente.

        Solo posible si aun no fue incluida en bloque.
        """
        if tx_hash not in self.pending_txs:
            return False

        # En produccion, enviar eth_cancelPrivateTransaction
        del self.pending_txs[tx_hash]
```

```
231         return True
232
233     def get_stats(self) -> Dict[str, Any]:
234         """Estadisticas de uso"""
235         total_refunds = sum(r.refund_received for r in
                self.history)
236         successful = sum(1 for r in self.history if r.status ==
                "included")
237
238         return {
239             "total_transactions": len(self.history),
240             "successful": successful,
241             "total_refunds_wei": total_refunds,
242             "pending": len(self.pending_txs),
243             "frontrun_attempts": 0  # Protect previene esto
244         }
245
246
247 class MEVBlockerClient:
248     """
249     Cliente para MEV Blocker RPC.
250
251     Diferencias vs Flashbots Protect:
252     - Coalition de builders
253     - OFA (Order Flow Auction)
254     - Refunds mas altos potencialmente
255     """
256
257     RPC_URL = "https://rpc.mevblocker.io"
258
259     def __init__(self, private_key: str):
260         self.private_key = private_key
261         self.total_refunds = 0
262
263     def send_transaction(
264         self,
265         tx: ProtectTransaction,
266         auction_deadline_blocks: int = 25
267     ) -> ProtectResult:
268         """
269         Enviar via MEV Blocker con OFA.
270
271         El Order Flow Auction permite que builders
272         compitan por incluir la transaccion,
273         maximizando refund al usuario.
274         """
275         # OFA preferences
276         ofa_config = {
277             "deadlineBlocks": auction_deadline_blocks,
278             "minRefundPercent": 80,
279             "allowedBuilders": "all"
280         }
281
282         # Simular envio
283         result = ProtectResult(
284             tx_hash=tx.get_hash(),
285             status="pending",
286             hints_used=["ofa_enabled"]
```

```
287         )
288
289         return result
290
291     def estimate_refund(
292         self,
293         tx: ProtectTransaction,
294         mev_opportunity_wei: int
295     ) -> int:
296         """Estimar refund potencial de OFA"""
297         # Tipicamente 80-90% del MEV capturado
298         return int(mev_opportunity_wei * 0.85)
299
300
301 # Demo
302 def private_rpc_demo():
303     print("=== Private RPC Demo ===\n")
304
305     # Crear cliente Flashbots Protect
306     client = FlashbotsProtectClient(
307         private_key="0xdemo_private_key",
308         default_hints=[HintPreference.HASH]
309     )
310
311     # Crear transaccion
312     tx = ProtectTransaction(
313         to="0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D",  #
            Uniswap Router
314         value=0,
315         data=bytes.fromhex("38ed1739"),  # swap
316         gas_limit=250000,
317         max_fee_per_gas=50_000_000_000,  # 50 gwei
318         max_priority_fee=2_000_000_000,   # 2 gwei
319         nonce=42,
320         hints=[HintPreference.HASH],      # Solo revelar hash
321         refund_percent=90
322     )
323
324     print("1. Enviando transaccion via Flashbots Protect...")
325     print(f"   Destino: {tx.to[:20]}... (Uniswap)")
326     print(f"   Hints: {[h.value for h in tx.hints]}")
327     print(f"   Refund: {tx.refund_percent}%")
328
329     # Enviar
330     result = client.send_transaction(tx, simulate_first=True)
331
332     print(f"\n2. Resultado:")
333     print(f"   TX Hash: {result.tx_hash[:20]}...")
334     print(f"   Status: {result.status}")
335     print(f"   Simulacion: {'OK' if result.simulation_result else
            'N/A'}")
336
337     # Comparar con MEV Blocker
338     print("\n3. Comparando con MEV Blocker...")
339     mev_client = MEVBlockerClient("0xdemo_key")
340     estimated_refund = mev_client.estimate_refund(tx,
            1_000_000_000_000)
341     print(f"   Refund estimado: {estimated_refund / 1e18:.6f} ETH")
```

```
342
343     print("\n4. Beneficios de Private RPC:")
344     print("   - Sin frontrunning: TX no visible en mempool")
345     print("   - Sin sandwich: Nadie puede ver tu swap")
346     print("   - Reverts privados: Fallos no consumen gas
            publicamente")
347     print("   - MEV rebate: Parte del MEV vuelve al usuario")
348
349 private_rpc_demo()
```

Listing 7.4: Cliente para Flashbots Protect RPC

Servicio	Builders	Refund	Hints	Cancel
Flashbots Protect	Multi	90 %	Configurable	Sí
MEV Blocker	Coalition	80-90 %	OFA	Sí
Blocknative	Multi	Variable	Básicos	Sí
RPC público	N/A	0 %	N/A	No

Tabla 7.4: Comparativa de servicios Private RPC

7.4. Relayer Networks

Definición 7.5 (Relayer)**.** Servicio que envía transacciones en nombre del usuario, pagando el gas y ocultando la identidad real del remitente:

$$\text{Usuario} \xrightarrow{\text{firma}} \text{Relayer} \xrightarrow{\text{TX + gas}} \text{Blockchain}$$

7.4.1. Arquitectura de Red de Relayers

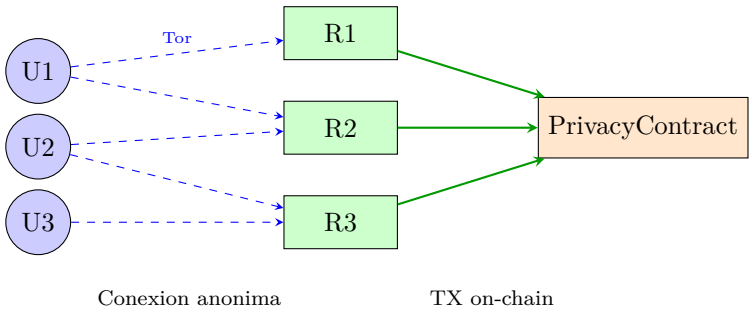


Figura 7.5: Arquitectura de red de relayers descentralizada

7.4.2. Implementación de Sistema de Relayers

```
1 """
2 Decentralized Relayer Network
3 Sistema de relayers para transacciones privadas
4 """
5
6 from dataclasses import dataclass, field
7 from typing import Optional, Dict, List, Any, Callable
8 from enum import Enum
```

```python
from datetime import datetime, timedelta
import hashlib
import json
import secrets
import heapq

class RelayerStatus(Enum):
    """Estado del relayer"""
    ACTIVE = "active"
    PAUSED = "paused"
    OFFLINE = "offline"
    SLASHED = "slashed"

@dataclass
class RelayerInfo:
    """Informacion de un relayer"""
    address: str
    endpoint: str               # URL del servicio
    stake: int                  # Stake en wei
    fee_percent: float          # Fee cobrado (e.g., 0.3%)
    min_amount: int             # Minimo por transaccion
    max_amount: int             # Maximo por transaccion
    supported_tokens: List[str]
    status: RelayerStatus = RelayerStatus.ACTIVE

    # Metricas
    total_relayed: int = 0
    successful_txs: int = 0
    failed_txs: int = 0
    average_latency_ms: float = 0
    uptime_percent: float = 100.0
    last_seen: datetime = field(default_factory=datetime.now)

    def get_score(self) -> float:
        """Score para seleccion (mayor = mejor)"""
        # Factores: fee, uptime, success rate, stake
        success_rate = (
            self.successful_txs / max(1, self.successful_txs +
                self.failed_txs)
        )

        # Normalizar fee (invertido - menor fee = mayor score)
        fee_score = max(0, 1 - self.fee_percent / 2)  # 0% = 1, 2%
            = 0

        # Stake score (logaritmico)
        import math
        stake_score = min(1, math.log10(max(1, self.stake / 1e18))
            / 3)

        # Uptime
        uptime_score = self.uptime_percent / 100

        # Latency score (menor = mejor)
        latency_score = max(0, 1 - self.average_latency_ms / 5000)

        return (
            fee_score * 0.3 +
```

```
            success_rate * 0.25 +
            stake_score * 0.2 +
            uptime_score * 0.15 +
            latency_score * 0.1
        )

    def calculate_fee(self, amount: int) -> int:
        """Calcular fee para un monto"""
        return int(amount * self.fee_percent / 100)


@dataclass
class RelayRequest:
    """Request para relayer"""
    proof: bytes                    # ZK proof
    public_inputs: List[int]        # Inputs publicos
    recipient: str                  # Destinatario (puede ser
        contract)
    amount: int                     # Monto a transferir
    token: str                      # Token address
    fee: int                        # Fee para relayer
    deadline: datetime              # Deadline para inclusion
    refund_address: str             # Donde enviar refund si falla

    def get_hash(self) -> str:
        """Hash del request para firma"""
        data = json.dumps({
            "proof": self.proof.hex(),
            "public_inputs": self.public_inputs,
            "recipient": self.recipient,
            "amount": self.amount,
            "token": self.token,
            "fee": self.fee,
            "deadline": self.deadline.isoformat()
        }, sort_keys=True)
        return hashlib.sha256(data.encode()).hexdigest()


@dataclass
class RelayResult:
    """Resultado de relay"""
    request_hash: str
    status: str           # 'pending', 'submitted', 'confirmed',
        'failed'
    tx_hash: Optional[str] = None
    relayer: Optional[str] = None
    fee_paid: int = 0
    gas_used: int = 0
    block_number: Optional[int] = None
    error: Optional[str] = None
    latency_ms: float = 0


class RelayerRegistry:
    """
    Registro descentralizado de relayers.

    Caracteristicas:
```

```
    - Registro abierto con stake
    - Slashing por mal comportamiento
    - Metricas on-chain verificables
    """

    def __init__(self, min_stake: int = 10**18):  # 1 ETH minimo
        self.min_stake = min_stake
        self.relayers: Dict[str, RelayerInfo] = {}
        self.slashed: set = set()

    def register(
        self,
        address: str,
        endpoint: str,
        stake: int,
        fee_percent: float,
        supported_tokens: List[str]
    ) -> bool:
        """Registrar nuevo relayer"""
        if stake < self.min_stake:
            return False

        if address in self.slashed:
            return False

        self.relayers[address] = RelayerInfo(
            address=address,
            endpoint=endpoint,
            stake=stake,
            fee_percent=fee_percent,
            min_amount=10**15,       # 0.001 ETH
            max_amount=100 * 10**18, # 100 ETH
            supported_tokens=supported_tokens
        )

        return True

    def get_active_relayers(
        self,
        token: Optional[str] = None,
        amount: Optional[int] = None
    ) -> List[RelayerInfo]:
        """Obtener relayers activos filtrados"""
        active = [
            r for r in self.relayers.values()
            if r.status == RelayerStatus.ACTIVE
        ]

        if token:
            active = [r for r in active if token in
                r.supported_tokens]

        if amount:
            active = [
                r for r in active
                if r.min_amount <= amount <= r.max_amount
            ]

```

```
        return active

    def update_metrics(
        self,
        address: str,
        success: bool,
        latency_ms: float
    ):
        """Actualizar metricas de relayer"""
        if address not in self.relayers:
            return

        r = self.relayers[address]
        r.total_relayed += 1
        r.last_seen = datetime.now()

        if success:
            r.successful_txs += 1
        else:
            r.failed_txs += 1

        # Moving average de latencia
        n = r.successful_txs + r.failed_txs
        r.average_latency_ms = (
            (r.average_latency_ms * (n - 1) + latency_ms) / n
        )

    def slash(self, address: str, reason: str) -> int:
        """Slashear relayer por mal comportamiento"""
        if address not in self.relayers:
            return 0

        r = self.relayers[address]
        slash_amount = r.stake // 2  # 50% del stake

        r.stake -= slash_amount
        r.status = RelayerStatus.SLASHED
        self.slashed.add(address)

        return slash_amount


class RelayerSelector:
    """
    Selector inteligente de relayers.

    Estrategias:
    - Best score: Mejor relayer segun metricas
    - Random weighted: Aleatorio ponderado por score
    - Round robin: Rotacion para descentralizacion
    - Cheapest: Menor fee
    """

    def __init__(self, registry: RelayerRegistry):
        self.registry = registry
        self.last_used: Dict[str, datetime] = {}

    def select_best(
```

```
235         self,
236         token: str,
237         amount: int,
238         count: int = 3
239     ) -> List[RelayerInfo]:
240         """Seleccionar mejores N relayers"""
241         candidates = self.registry.get_active_relayers(token,
                amount)
242 
243         # Ordenar por score
244         scored = [(r.get_score(), r) for r in candidates]
245         scored.sort(reverse=True)
246 
247         return [r for _, r in scored[:count]]
248 
249     def select_random_weighted(
250         self,
251         token: str,
252         amount: int
253     ) -> Optional[RelayerInfo]:
254         """Seleccion aleatoria ponderada por score"""
255         import random
256 
257         candidates = self.registry.get_active_relayers(token,
                amount)
258         if not candidates:
259             return None
260 
261         # Pesos basados en score
262         weights = [r.get_score() for r in candidates]
263         total = sum(weights)
264 
265         if total == 0:
266             return random.choice(candidates)
267 
268         # Seleccion ponderada
269         r = random.random() * total
270         cumulative = 0
271         for i, w in enumerate(weights):
272             cumulative += w
273             if r <= cumulative:
274                 return candidates[i]
275 
276         return candidates[-1]
277 
278     def select_cheapest(
279         self,
280         token: str,
281         amount: int
282     ) -> Optional[RelayerInfo]:
283         """Seleccionar relayer mas barato"""
284         candidates = self.registry.get_active_relayers(token,
                amount)
285         if not candidates:
286             return None
287 
288         return min(candidates, key=lambda r:
                r.calculate_fee(amount))
```

```
289
290
291 class RelayerClient:
292     """
293     Cliente para interactuar con red de relayers.
294
295     Flujo:
296     1. Seleccionar relayer(s)
297     2. Enviar request
298     3. Monitorear estado
299     4. Reintentar si falla
300     """
301
302     def __init__(
303         self,
304         registry: RelayerRegistry,
305         max_retries: int = 3
306     ):
307         self.registry = registry
308         self.selector = RelayerSelector(registry)
309         self.max_retries = max_retries
310         self.pending: Dict[str, RelayRequest] = {}
311
312     def submit_relay(
313         self,
314         request: RelayRequest,
315         strategy: str = "best"
316     ) -> RelayResult:
317         """
318         Enviar request a la red de relayers.
319
320         Args:
321             request: Request a enviar
322             strategy: 'best', 'random', 'cheapest'
323         """
324         # Seleccionar relayer segun estrategia
325         if strategy == "best":
326             relayers = self.selector.select_best(
327                 request.token, request.amount
328             )
329             if not relayers:
330                 return RelayResult(
331                     request_hash=request.get_hash(),
332                     status="no_relayers",
333                     error="No relayers available"
334                 )
335             relayer = relayers[0]
336         elif strategy == "random":
337             relayer = self.selector.select_random_weighted(
338                 request.token, request.amount
339             )
340         else:  # cheapest
341             relayer = self.selector.select_cheapest(
342                 request.token, request.amount
343             )
344
345         if not relayer:
346             return RelayResult(
```

```
                request_hash=request.get_hash(),
                status="no_relayers"
            )

        # Verificar fee
        min_fee = relayer.calculate_fee(request.amount)
        if request.fee < min_fee:
            return RelayResult(
                request_hash=request.get_hash(),
                status="fee_too_low",
                error=f"Min fee: {min_fee}, provided:
                    {request.fee}"
            )

        # Enviar request
        result = self._send_to_relayer(relayer, request)

        if result.status == "pending":
            self.pending[request.get_hash()] = request

        return result

    def _send_to_relayer(
        self,
        relayer: RelayerInfo,
        request: RelayRequest
    ) -> RelayResult:
        """Enviar request a relayer especifico"""
        import time
        start = time.time()

        # Simular llamada HTTP
        # En produccion: POST a relayer.endpoint

        latency = (time.time() - start) * 1000

        # Simular respuesta exitosa
        tx_hash = hashlib.sha256(
            (request.get_hash() + relayer.address).encode()
        ).hexdigest()

        # Actualizar metricas
        self.registry.update_metrics(
            relayer.address,
            success=True,
            latency_ms=latency
        )

        return RelayResult(
            request_hash=request.get_hash(),
            status="pending",
            tx_hash=tx_hash,
            relayer=relayer.address,
            fee_paid=request.fee,
            latency_ms=latency
        )

    def check_status(self, request_hash: str) -> RelayResult:
```

```
404         """Verificar estado de request"""
405         if request_hash not in self.pending:
406             return RelayResult(
407                 request_hash=request_hash,
408                 status="unknown"
409             )
410 
411         # Simular verificacion on-chain
412         return RelayResult(
413             request_hash=request_hash,
414             status="confirmed",
415             block_number=12345678
416         )
417 
418 
419 # Demo
420 def relayer_demo():
421     print("=== Relayer Network Demo ===\n")
422 
423     # Crear registro
424     registry = RelayerRegistry(min_stake=10**18)
425 
426     # Registrar relayers
427     print("1. Registrando relayers en la red...")
428 
429     relayers_data = [
430         ("0xRelayer1", "https://relay1.example.com", 5 * 10**18,
            0.3, 98.5),
431         ("0xRelayer2", "https://relay2.example.com", 10 * 10**18,
            0.2, 99.2),
432         ("0xRelayer3", "https://relay3.example.com", 2 * 10**18,
            0.5, 95.0),
433     ]
434 
435     for addr, endpoint, stake, fee, uptime in relayers_data:
436         registry.register(
437             address=addr,
438             endpoint=endpoint,
439             stake=stake,
440             fee_percent=fee,
441             supported_tokens=["ETH", "USDC", "DAI"]
442         )
443         r = registry.relayers[addr]
444         r.uptime_percent = uptime
445         r.successful_txs = 100
446         print(f"   {addr}: stake={stake/1e18:.0f} ETH, fee={fee}%")
447 
448     # Crear cliente
449     client = RelayerClient(registry)
450 
451     # Crear request
452     print("\n2. Creando relay request...")
453     request = RelayRequest(
454         proof=secrets.token_bytes(256),
455         public_inputs=[1, 2, 3],
456         recipient="0xRecipientAddress",
457         amount=10**18,  # 1 ETH
458         token="ETH",
```

```
459             fee=3 * 10**15,  # 0.003 ETH
460             deadline=datetime.now() + timedelta(minutes=10),
461             refund_address="0xRefundAddress"
462         )
463         print(f"   Amount: {request.amount / 1e18:.2f} ETH")
464         print(f"   Fee: {request.fee / 1e18:.4f} ETH")
465
466         # Seleccionar y enviar
467         print("\n3. Seleccionando mejor relayer...")
468         selector = RelayerSelector(registry)
469         best = selector.select_best("ETH", request.amount, count=3)
470
471         for i, r in enumerate(best):
472             print(f"   #{i+1}: {r.address}, score={r.get_score():.3f},
                    fee={r.fee_percent}%")
473
474         # Enviar request
475         print("\n4. Enviando request...")
476         result = client.submit_relay(request, strategy="best")
477
478         print(f"   Status: {result.status}")
479         print(f"   TX Hash: {result.tx_hash[:20]}...")
480         print(f"   Relayer: {result.relayer}")
481         print(f"   Latency: {result.latency_ms:.2f} ms")
482
483 relayer_demo()
```

Listing 7.5: Sistema de relayers descentralizado

7.5. Note Management

Definición 7.6 (Note)**.** Representación criptográfica de un depósito privado que contiene:

- **Commitment:** Hash público en el contrato ($C = H(v, r, k)$)
- **Nullifier secret:** Secreto para gastar (k)
- **Amount:** Valor del depósito (v)
- **Blinding:** Factor de randomización (r)

7.5.1. Ciclo de Vida de una Note

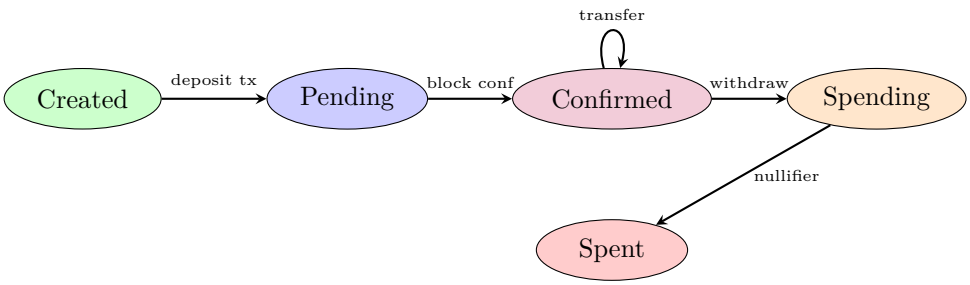


Figura 7.6: Ciclo de vida de una note en sistema de privacidad

7.5.2. Sistema de Gestión de Notes

```python
"""
Secure Note Management System
Gestion completa del ciclo de vida de notes privadas
"""

from dataclasses import dataclass, field
from typing import Optional, Dict, List, Tuple, Any, Callable
from datetime import datetime, timedelta
from enum import Enum
import hashlib
import json
import secrets
import os
from abc import ABC, abstractmethod

class NoteStatus(Enum):
    """Estado de la note"""
    CREATED = "created"          # Creada localmente
    PENDING = "pending"          # TX enviada, esperando
        confirmacion
    CONFIRMED = "confirmed"      # Depositada y lista para usar
    SPENDING = "spending"        # En proceso de gasto
    SPENT = "spent"              # Gastada (nullifier revelado)
    FAILED = "failed"            # Deposito fallido

@dataclass
class Note:
    """
    Representacion completa de una note privada.

    CRITICAL: Esta informacion es sensible.
    Perder nullifier_secret = perder fondos.
    """
    # Identificadores
    note_id: str                    # ID local unico
    commitment: str                 # Hash publico on-chain

    # Secretos (NUNCA revelar)
    nullifier_secret: bytes         # Para generar nullifier
    blinding_factor: bytes          # Factor de randomizacion

    # Datos
    amount: int                     # Monto en wei
    asset: str                      # Token address o "ETH"

    # Metadata
    status: NoteStatus = NoteStatus.CREATED
    created_at: datetime = field(default_factory=datetime.now)
    confirmed_at: Optional[datetime] = None
    spent_at: Optional[datetime] = None

    # Referencias on-chain
    deposit_tx: Optional[str] = None
    deposit_block: Optional[int] = None
    spending_tx: Optional[str] = None

```

```
    # Index en el arbol Merkle
    leaf_index: Optional[int] = None

    def compute_nullifier(self) -> str:
        """Calcular nullifier hash"""
        preimage = (
            self.nullifier_secret +
            self.commitment.encode() +
            self.amount.to_bytes(32, 'big')
        )
        return hashlib.sha256(preimage).hexdigest()

    def to_dict(self, include_secrets: bool = False) -> Dict:
        """Serializar note (sin secretos por defecto)"""
        data = {
            "note_id": self.note_id,
            "commitment": self.commitment,
            "amount": self.amount,
            "asset": self.asset,
            "status": self.status.value,
            "created_at": self.created_at.isoformat(),
            "deposit_tx": self.deposit_tx,
            "leaf_index": self.leaf_index
        }

        if include_secrets:
            data["nullifier_secret"] = self.nullifier_secret.hex()
            data["blinding_factor"] = self.blinding_factor.hex()

        return data

    @classmethod
    def from_dict(cls, data: Dict) -> 'Note':
        """Deserializar note"""
        return cls(
            note_id=data["note_id"],
            commitment=data["commitment"],
            nullifier_secret=bytes.fromhex(data.get("nullifier_secret",
                "00" * 32)),
            blinding_factor=bytes.fromhex(data.get("blinding_factor",
                "00" * 32)),
            amount=data["amount"],
            asset=data["asset"],
            status=NoteStatus(data.get("status", "created")),
            created_at=datetime.fromisoformat(data["created_at"]),
            deposit_tx=data.get("deposit_tx"),
            leaf_index=data.get("leaf_index")
        )


class EncryptionProvider(ABC):
    """Interfaz abstracta para encriptacion"""

    @abstractmethod
    def encrypt(self, data: bytes) -> bytes:
        pass

    @abstractmethod
```

```
112     def decrypt(self, encrypted: bytes) -> bytes:
113         pass
114
115
116 class AESEncryption(EncryptionProvider):
117     """
118     Encriptacion AES-256-GCM.
119
120     En produccion, usar libreria como cryptography.
121     """
122
123     def __init__(self, key: bytes):
124         if len(key) != 32:
125             raise ValueError("Key must be 32 bytes")
126         self.key = key
127
128     def encrypt(self, data: bytes) -> bytes:
129         """Encriptar con AES-GCM (simplificado)"""
130         # En produccion: usar AES-GCM real
131         nonce = secrets.token_bytes(12)
132
133         # XOR simple para demo (NO usar en produccion)
134         key_stream = self._expand_key(len(data))
135         ciphertext = bytes(a ^ b for a, b in zip(data, key_stream))
136
137         # MAC simplificado
138         mac = hashlib.sha256(self.key + nonce +
                ciphertext).digest()[:16]
139
140         return nonce + ciphertext + mac
141
142     def decrypt(self, encrypted: bytes) -> bytes:
143         """Desencriptar"""
144         nonce = encrypted[:12]
145         mac = encrypted[-16:]
146         ciphertext = encrypted[12:-16]
147
148         # Verificar MAC
149         expected_mac = hashlib.sha256(
150             self.key + nonce + ciphertext
151         ).digest()[:16]
152         if mac != expected_mac:
153             raise ValueError("MAC verification failed")
154
155         # Desencriptar
156         key_stream = self._expand_key(len(ciphertext))
157         return bytes(a ^ b for a, b in zip(ciphertext, key_stream))
158
159     def _expand_key(self, length: int) -> bytes:
160         """Expandir key para keystream"""
161         expanded = b""
162         counter = 0
163         while len(expanded) < length:
164             expanded += hashlib.sha256(
165                 self.key + counter.to_bytes(4, 'big')
166             ).digest()
167             counter += 1
168         return expanded[:length]
```

```python


class NoteManager:
    """
    Gestor completo de notes privadas.

    Responsabilidades:
    - Crear notes seguras
    - Almacenamiento encriptado
    - Backup y recovery
    - Sincronizacion multi-device
    """

    def __init__(
        self,
        encryption: EncryptionProvider,
        storage_path: Optional[str] = None
    ):
        self.encryption = encryption
        self.storage_path = storage_path
        self.notes: Dict[str, Note] = {}
        self.nullifiers: Dict[str, str] = {}  # nullifier ->
            note_id

    def create_note(
        self,
        amount: int,
        asset: str
    ) -> Note:
        """
        Crear nueva note para deposito.

        Genera secretos criptograficamente seguros.
        """
        # Generar secretos
        nullifier_secret = secrets.token_bytes(32)
        blinding_factor = secrets.token_bytes(32)

        # Calcular commitment
        preimage = (
            amount.to_bytes(32, 'big') +
            asset.encode().ljust(32, b'\x00') +
            nullifier_secret +
            blinding_factor
        )
        commitment = hashlib.sha256(preimage).hexdigest()

        # Crear note
        note = Note(
            note_id=secrets.token_hex(16),
            commitment=commitment,
            nullifier_secret=nullifier_secret,
            blinding_factor=blinding_factor,
            amount=amount,
            asset=asset,
            status=NoteStatus.CREATED
        )

```

```
226         # Guardar
227         self.notes[note.note_id] = note
228         nullifier = note.compute_nullifier()
229         self.nullifiers[nullifier] = note.note_id
230 
231         return note
232 
233     def mark_pending(self, note_id: str, tx_hash: str):
234         """Marcar note como pendiente (TX enviada)"""
235         if note_id not in self.notes:
236             raise ValueError(f"Note not found: {note_id}")
237 
238         note = self.notes[note_id]
239         note.status = NoteStatus.PENDING
240         note.deposit_tx = tx_hash
241 
242     def mark_confirmed(
243         self,
244         note_id: str,
245         block_number: int,
246         leaf_index: int
247     ):
248         """Confirmar deposito de note"""
249         if note_id not in self.notes:
250             raise ValueError(f"Note not found: {note_id}")
251 
252         note = self.notes[note_id]
253         note.status = NoteStatus.CONFIRMED
254         note.confirmed_at = datetime.now()
255         note.deposit_block = block_number
256         note.leaf_index = leaf_index
257 
258     def mark_spent(self, note_id: str, tx_hash: str):
259         """Marcar note como gastada"""
260         if note_id not in self.notes:
261             raise ValueError(f"Note not found: {note_id}")
262 
263         note = self.notes[note_id]
264         note.status = NoteStatus.SPENT
265         note.spent_at = datetime.now()
266         note.spending_tx = tx_hash
267 
268     def get_spendable_notes(
269         self,
270         asset: str,
271         min_confirmations: int = 10
272     ) -> List[Note]:
273         """Obtener notes disponibles para gastar"""
274         return [
275             note for note in self.notes.values()
276             if note.status == NoteStatus.CONFIRMED
277             and note.asset == asset
278         ]
279 
280     def get_balance(self, asset: str) -> int:
281         """Balance total de un asset"""
282         return sum(
283             note.amount
```

```
            for note in self.get_spendable_notes(asset)
        )

    def export_backup(self) -> bytes:
        """
        Exportar backup encriptado de todas las notes.

        CRITICAL: Este backup permite recuperar fondos.
        """
        # Serializar todas las notes con secretos
        backup_data = {
            "version": 1,
            "created_at": datetime.now().isoformat(),
            "notes": [
                note.to_dict(include_secrets=True)
                for note in self.notes.values()
            ],
            "nullifiers": dict(self.nullifiers)
        }

        # Encriptar
        json_bytes = json.dumps(backup_data).encode()
        encrypted = self.encryption.encrypt(json_bytes)

        return encrypted

    def import_backup(self, encrypted_backup: bytes):
        """Restaurar notes desde backup encriptado"""
        # Desencriptar
        json_bytes = self.encryption.decrypt(encrypted_backup)
        backup_data = json.loads(json_bytes.decode())

        # Restaurar notes
        for note_data in backup_data["notes"]:
            note = Note.from_dict(note_data)
            self.notes[note.note_id] = note

        # Restaurar nullifiers
        self.nullifiers = backup_data.get("nullifiers", {})

    def check_nullifier_spent(self, nullifier: str) -> bool:
        """Verificar si nullifier ya fue usado on-chain"""
        # En produccion: consultar contrato
        return nullifier in self.nullifiers

    def scan_chain_for_notes(
        self,
        viewing_key: bytes,
        from_block: int,
        to_block: int
    ) -> List[Note]:
        """
        Escanear blockchain buscando notes nuestras.

        Usado para recovery o sync.
        """
        # En produccion: iterar eventos de deposito
        # y probar desencriptar con viewing_key
```

```
342         found_notes = []
343         return found_notes
344
345     def get_stats(self) -> Dict[str, Any]:
346         """Estadisticas del note manager"""
347         by_status = {}
348         by_asset = {}
349
350         for note in self.notes.values():
351             status = note.status.value
352             by_status[status] = by_status.get(status, 0) + 1
353
354             if note.status == NoteStatus.CONFIRMED:
355                 asset = note.asset
356                 by_asset[asset] = by_asset.get(asset, 0) +
                        note.amount
357
358         return {
359             "total_notes": len(self.notes),
360             "by_status": by_status,
361             "balances": by_asset,
362             "nullifiers_tracked": len(self.nullifiers)
363         }
364
365
366 class SecureBackupManager:
367     """
368     Gestion segura de backups.
369
370     Estrategias:
371     - Split backup (Shamir Secret Sharing)
372     - Cloud sync encriptado
373     - Hardware backup
374     """
375
376     def __init__(self, note_manager: NoteManager):
377         self.note_manager = note_manager
378         self.backup_history: List[Dict] = []
379
380     def create_split_backup(
381         self,
382         threshold: int = 2,
383         total_shares: int = 3
384     ) -> List[bytes]:
385         """
386         Crear backup dividido con Shamir Secret Sharing.
387
388         Args:
389             threshold: Shares minimos para reconstruir
390             total_shares: Total de shares a generar
391         """
392         # Obtener backup completo
393         full_backup = self.note_manager.export_backup()
394
395         # Simplificado - en produccion usar Shamir real
396         # Por ahora, generamos "shares" simulados
397         shares = []
398         for i in range(total_shares):
```

```
            share_data = {
                "share_index": i,
                "threshold": threshold,
                "total": total_shares,
                "data": full_backup.hex()  # Simplificado
            }
            shares.append(json.dumps(share_data).encode())

        self.backup_history.append({
            "type": "split",
            "threshold": threshold,
            "total_shares": total_shares,
            "created_at": datetime.now().isoformat()
        })

        return shares

    def recover_from_shares(self, shares: List[bytes]) -> bool:
        """Reconstruir backup desde shares"""
        # Verificar threshold
        share_data = json.loads(shares[0].decode())
        threshold = share_data["threshold"]

        if len(shares) < threshold:
            raise ValueError(
                f"Need at least {threshold} shares, got
                    {len(shares)}"
            )

        # Reconstruir (simplificado)
        encrypted_backup = bytes.fromhex(share_data["data"])
        self.note_manager.import_backup(encrypted_backup)

        return True

    def create_mnemonic_backup(self) -> List[str]:
        """
        Crear backup como mnemonic phrase.

        La phrase codifica el master secret del cual
        se derivan todas las keys.
        """
        # Generar entropia
        entropy = secrets.token_bytes(32)

        # Convertir a palabras (simplificado)
        # En produccion usar BIP39
        wordlist = [
            "abandon", "ability", "able", "about", "above",
            "absent", "absorb", "abstract", "absurd", "abuse",
            "access", "accident", "account", "accuse", "achieve",
            "acid", "acoustic", "acquire", "across", "act"
        ]  # Solo ejemplo

        words = []
        for i in range(0, 32, 2):
            index = int.from_bytes(entropy[i:i+2], 'big') %
                len(wordlist)
```

```
            words.append(wordlist[index])

        return words


# Demo
def note_management_demo():
    print("=== Note Management Demo ===\n")

    # Setup encriptacion
    master_key = secrets.token_bytes(32)
    encryption = AESEncryption(master_key)

    # Crear note manager
    manager = NoteManager(encryption)

    # 1. Crear notes
    print("1. Creando notes...")
    note1 = manager.create_note(amount=10**18, asset="ETH")
    print(f"   Note 1: {note1.amount / 1e18:.2f} ETH")
    print(f"   Commitment: {note1.commitment[:20]}...")

    note2 = manager.create_note(amount=5 * 10**18, asset="ETH")
    print(f"   Note 2: {note2.amount / 1e18:.2f} ETH")

    # 2. Simular deposito
    print("\n2. Simulando depositos...")
    manager.mark_pending(note1.note_id, "0xdeposit_tx_1")
    manager.mark_confirmed(note1.note_id, block_number=12345,
        leaf_index=100)
    print(f"   Note 1 confirmada en bloque 12345")

    manager.mark_pending(note2.note_id, "0xdeposit_tx_2")
    manager.mark_confirmed(note2.note_id, block_number=12350,
        leaf_index=101)
    print(f"   Note 2 confirmada en bloque 12350")

    # 3. Balance
    balance = manager.get_balance("ETH")
    print(f"\n3. Balance total: {balance / 1e18:.2f} ETH")

    # 4. Crear backup
    print("\n4. Creando backup encriptado...")
    backup = manager.export_backup()
    print(f"   Backup size: {len(backup)} bytes")

    # 5. Split backup
    print("\n5. Creando split backup (2-of-3)...")
    backup_manager = SecureBackupManager(manager)
    shares = backup_manager.create_split_backup(threshold=2,
        total_shares=3)
    print(f"   Generados {len(shares)} shares")
    print(f"   Threshold: 2 shares para recuperar")

    # 6. Stats
    stats = manager.get_stats()
    print(f"\n6. Estadisticas:")
    print(f"   Total notes: {stats['total_notes']}")
```

```
    print(f"   Por status: {stats['by_status']}")
    print(f"   Balance ETH: {stats['balances'].get('ETH', 0) /
        1e18:.2f}")

    # 7. Verificar nullifier
    print("\n7. Verificando nullifier...")
    nullifier = note1.compute_nullifier()
    print(f"   Nullifier: {nullifier[:20]}...")
    print(f"   Registrado:
        {manager.check_nullifier_spent(nullifier)}")

note_management_demo()
```

Listing 7.6: Sistema completo de gestión de notes

Idea Clave

Mejores Prácticas para Note Management:

1. **Backup inmediato:** Crear backup al generar note, ANTES de depositar
2. **Encriptación fuerte:** AES-256-GCM con key derivada de passphrase
3. **Split backup:** Shamir 2-of-3 para redundancia sin single point of failure
4. **Verificación:** Probar recovery ANTES de depositar fondos significativos
5. **Rotación:** Rotar keys periódicamente sin perder acceso a notes antiguas

Qué NO hacer:

- Almacenar notes sin encriptar
- Backup único sin redundancia
- Compartir nullifier_secret
- Depositar sin verificar backup funciona

7.6. Proving Infrastructure

Definición 7.7 (Proving Infrastructure)**.** Conjunto de servicios y herramientas para la generación de zero-knowledge proofs:

- **Provers:** Sistemas que ejecutan la generación de pruebas
- **Verification:** Contratos/sistemas que verifican pruebas
- **Key management:** Gestión de proving/verification keys
- **Witness generation:** Cálculo de inputs privados

7.6.1. Arquitecturas de Proving

7.6.2. Sistema de Proving Completo

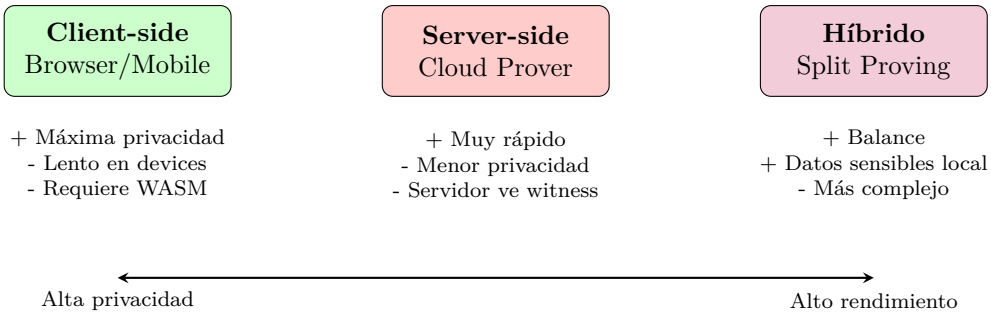


Figura 7.7: Trade-offs en arquitecturas de proving

```python
"""
ZK Proving Infrastructure
Sistema completo para generacion y verificacion de pruebas
"""

from dataclasses import dataclass, field
from typing import Optional, Dict, List, Any, Tuple, Callable
from datetime import datetime, timedelta
from enum import Enum
from abc import ABC, abstractmethod
import hashlib
import json
import time
import secrets

class ProvingMode(Enum):
    """Modos de generacion de pruebas"""
    CLIENT = "client"       # Completamente en cliente
    SERVER = "server"       # Completamente en servidor
    HYBRID = "hybrid"       # Division client/server
    DELEGATED = "delegated" # Delegado con MPC

class ProofSystem(Enum):
    """Sistemas de pruebas soportados"""
    GROTH16 = "groth16"     # Trusted setup, pequeñas pruebas
    PLONK = "plonk"         # Universal setup
    STARK = "stark"         # Sin trusted setup, pruebas grandes
    HALO2 = "halo2"         # Recursive, sin trusted setup

@dataclass
class ProvingKey:
    """Clave de proving para un circuito"""
    circuit_id: str
    proof_system: ProofSystem
    key_data: bytes
    size_bytes: int
    created_at: datetime

    def get_hash(self) -> str:
        return hashlib.sha256(self.key_data).hexdigest()

@dataclass
class VerificationKey:
    """Clave de verificacion (publica)"""
```

```
45      circuit_id: str
46      proof_system: ProofSystem
47      key_data: bytes
48      commitment: str  # Commitment on-chain
49
50      def verify_proof(self, proof: bytes, public_inputs: List[int])
            -> bool:
51          """Verificar prueba (simplificado)"""
52          # En produccion: llamar a verificador real
53          return len(proof) > 0 and len(public_inputs) > 0
54
55  @dataclass
56  class Witness:
57      """Witness (inputs privados) para generar prueba"""
58      circuit_id: str
59      private_inputs: Dict[str, Any]
60      public_inputs: List[int]
61      created_at: datetime = field(default_factory=datetime.now)
62
63      def is_sensitive(self) -> bool:
64          """Verificar si contiene datos sensibles"""
65          sensitive_fields = ["nullifier", "secret", "private_key",
                "amount"]
66          return any(
67              field in self.private_inputs
68              for field in sensitive_fields
69          )
70
71      def get_public_hash(self) -> str:
72          """Hash de inputs publicos (seguro de compartir)"""
73          return hashlib.sha256(
74              json.dumps(self.public_inputs).encode()
75          ).hexdigest()
76
77  @dataclass
78  class Proof:
79      """Prueba ZK generada"""
80      circuit_id: str
81      proof_system: ProofSystem
82      proof_data: bytes
83      public_inputs: List[int]
84
85      # Metadata
86      proving_time_ms: float
87      prover_mode: ProvingMode
88      created_at: datetime = field(default_factory=datetime.now)
89
90      def size_bytes(self) -> int:
91          return len(self.proof_data)
92
93      def to_calldata(self) -> bytes:
94          """Serializar para llamada a contrato"""
95          # Formato: proof || public_inputs
96          inputs_bytes = b"".join(
97              i.to_bytes(32, 'big') for i in self.public_inputs
98          )
99          return self.proof_data + inputs_bytes
100
```

```
101
102 class Prover(ABC):
103     """Interfaz abstracta para provers"""
104
105     @abstractmethod
106     def generate_proof(
107         self,
108         proving_key: ProvingKey,
109         witness: Witness
110     ) -> Proof:
111         pass
112
113     @abstractmethod
114     def get_mode(self) -> ProvingMode:
115         pass
116
117
118 class ClientProver(Prover):
119     """
120     Prover del lado del cliente.
121
122     Genera pruebas completamente en el dispositivo del usuario.
123     Maxima privacidad pero puede ser lento.
124     """
125
126     def __init__(self, wasm_path: Optional[str] = None):
127         self.wasm_path = wasm_path
128         self.loaded_circuits: Dict[str, bool] = {}
129
130     def load_circuit(self, circuit_id: str):
131         """Cargar circuito WASM"""
132         # En produccion: cargar archivo WASM real
133         self.loaded_circuits[circuit_id] = True
134
135     def generate_proof(
136         self,
137         proving_key: ProvingKey,
138         witness: Witness
139     ) -> Proof:
140         """Generar prueba localmente"""
141         start_time = time.time()
142
143         if proving_key.circuit_id not in self.loaded_circuits:
144             self.load_circuit(proving_key.circuit_id)
145
146         # Simular generacion de prueba
147         # En produccion: llamar a snarkjs/circom WASM
148         proof_data = self._generate_groth16_proof(
149             proving_key, witness
150         )
151
152         proving_time = (time.time() - start_time) * 1000
153
154         return Proof(
155             circuit_id=proving_key.circuit_id,
156             proof_system=proving_key.proof_system,
157             proof_data=proof_data,
158             public_inputs=witness.public_inputs,
```

```
            proving_time_ms=proving_time,
            prover_mode=ProvingMode.CLIENT
        )

    def _generate_groth16_proof(
        self,
        proving_key: ProvingKey,
        witness: Witness
    ) -> bytes:
        """Generar prueba Groth16 (simulado)"""
        # En produccion: llamar a prover real
        # Groth16 proof: 3 puntos G1 + 1 punto G2     256 bytes
        proof_hash = hashlib.sha256(
            proving_key.get_hash().encode() +
            json.dumps(witness.private_inputs).encode()
        ).digest()

        # Simular tamaño de prueba Groth16
        return proof_hash * 8  # ~256 bytes

    def get_mode(self) -> ProvingMode:
        return ProvingMode.CLIENT

    def estimate_time(self, circuit_id: str) -> float:
        """Estimar tiempo de proving en ms"""
        # Estimaciones tipicas para diferentes circuitos
        estimates = {
            "deposit": 5000,      # 5 segundos
            "transfer": 15000,    # 15 segundos
            "withdrawal": 8000,   # 8 segundos
        }
        return estimates.get(circuit_id, 10000)


class ServerProver(Prover):
    """
    Prover del lado del servidor.

    Rapido pero requiere enviar witness al servidor.
    El servidor ve los datos privados.
    """

    def __init__(self, server_url: str, api_key: str):
        self.server_url = server_url
        self.api_key = api_key
        self.request_count = 0

    def generate_proof(
        self,
        proving_key: ProvingKey,
        witness: Witness
    ) -> Proof:
        """Enviar witness al servidor para proving"""
        start_time = time.time()

        # WARNING: El servidor ve el witness completo
        if witness.is_sensitive():
            print("WARNING: Sending sensitive data to server!")
```

```
217
218         # Simular request al servidor
219         proof_data = self._request_proof(proving_key, witness)
220
221         proving_time = (time.time() - start_time) * 1000
222         self.request_count += 1
223
224         return Proof(
225             circuit_id=proving_key.circuit_id,
226             proof_system=proving_key.proof_system,
227             proof_data=proof_data,
228             public_inputs=witness.public_inputs,
229             proving_time_ms=proving_time,
230             prover_mode=ProvingMode.SERVER
231         )
232
233     def _request_proof(
234         self,
235         proving_key: ProvingKey,
236         witness: Witness
237     ) -> bytes:
238         """Request al servidor de proving"""
239         # En produccion: HTTP POST a servidor
240         # Simular respuesta
241         return secrets.token_bytes(256)
242
243     def get_mode(self) -> ProvingMode:
244         return ProvingMode.SERVER
245
246
247 class HybridProver(Prover):
248     """
249     Prover hibrido.
250
251     Divide el circuito en partes:
252     - Partes sensibles: client-side
253     - Partes computacionalmente pesadas: server-side
254
255     Usa folding schemes o split proving.
256     """
257
258     def __init__(
259         self,
260         client_prover: ClientProver,
261         server_prover: ServerProver
262     ):
263         self.client = client_prover
264         self.server = server_prover
265
266     def generate_proof(
267         self,
268         proving_key: ProvingKey,
269         witness: Witness
270     ) -> Proof:
271         """
272         Generar prueba en modo hibrido.
273
274         1. Client genera commitment a datos sensibles
```

```
275         2. Server genera prueba de parte publica
276         3. Client combina ambas partes
277         """
278         start_time = time.time()
279 
280         # Separar witness en sensible y no-sensible
281         sensitive_inputs = {}
282         public_computation_inputs = {}
283 
284         for key, value in witness.private_inputs.items():
285             if key in ["nullifier_secret", "private_key",
                    "amount"]:
286                 sensitive_inputs[key] = value
287             else:
288                 public_computation_inputs[key] = value
289 
290         # 1. Client-side: commitment a datos sensibles
291         sensitive_commitment =
                self._client_commit(sensitive_inputs)
292 
293         # 2. Server-side: proving de parte publica
294         # El servidor NO ve los datos sensibles, solo el commitment
295         server_witness = Witness(
296             circuit_id=witness.circuit_id + "_public",
297             private_inputs={
298                 **public_computation_inputs,
299                 "sensitive_commitment": sensitive_commitment
300             },
301             public_inputs=witness.public_inputs
302         )
303 
304         # Simulamos que el server hace parte del trabajo
305         # pero sin ver datos sensibles
306 
307         proving_time = (time.time() - start_time) * 1000
308 
309         # Combinar pruebas
310         combined_proof = self._combine_proofs(
311             sensitive_commitment,
312             witness
313         )
314 
315         return Proof(
316             circuit_id=proving_key.circuit_id,
317             proof_system=proving_key.proof_system,
318             proof_data=combined_proof,
319             public_inputs=witness.public_inputs,
320             proving_time_ms=proving_time,
321             prover_mode=ProvingMode.HYBRID
322         )
323 
324     def _client_commit(self, sensitive_inputs: Dict) -> str:
325         """Generar commitment a datos sensibles"""
326         data = json.dumps(sensitive_inputs, sort_keys=True)
327         return hashlib.sha256(data.encode()).hexdigest()
328 
329     def _combine_proofs(
330         self,
```

```
        commitment: str,
        witness: Witness
    ) -> bytes:
        """Combinar partes de prueba"""
        return hashlib.sha256(
            commitment.encode() +
            json.dumps(witness.public_inputs).encode()
        ).digest() * 8

    def get_mode(self) -> ProvingMode:
        return ProvingMode.HYBRID


class ProvingService:
    """
    Servicio completo de proving.

    Caracteristicas:
    - Seleccion automatica de modo
    - Cache de proving keys
    - Benchmarking
    - Rate limiting
    """

    def __init__(self):
        self.provers: Dict[ProvingMode, Prover] = {}
        self.proving_keys: Dict[str, ProvingKey] = {}
        self.verification_keys: Dict[str, VerificationKey] = {}
        self.cache: Dict[str, Proof] = {}  # Cache de pruebas
        self.benchmarks: Dict[str, List[float]] = {}

    def register_prover(self, prover: Prover):
        """Registrar prover disponible"""
        self.provers[prover.get_mode()] = prover

    def load_circuit(
        self,
        circuit_id: str,
        proving_key: ProvingKey,
        verification_key: VerificationKey
    ):
        """Cargar circuito con sus claves"""
        self.proving_keys[circuit_id] = proving_key
        self.verification_keys[circuit_id] = verification_key

    def generate_proof(
        self,
        circuit_id: str,
        witness: Witness,
        mode: Optional[ProvingMode] = None
    ) -> Proof:
        """
        Generar prueba con modo optimo.

        Si no se especifica modo, selecciona automaticamente.
        """
        if circuit_id not in self.proving_keys:
            raise ValueError(f"Circuit not loaded: {circuit_id}")
```

```
389
390         # Seleccionar modo si no se especifica
391         if mode is None:
392             mode = self._select_optimal_mode(witness)
393
394         if mode not in self.provers:
395             raise ValueError(f"Prover not available for mode:
                {mode}")
396
397         prover = self.provers[mode]
398         proving_key = self.proving_keys[circuit_id]
399
400         # Generar prueba
401         proof = prover.generate_proof(proving_key, witness)
402
403         # Actualizar benchmarks
404         if circuit_id not in self.benchmarks:
405             self.benchmarks[circuit_id] = []
406         self.benchmarks[circuit_id].append(proof.proving_time_ms)
407
408         return proof
409
410     def _select_optimal_mode(self, witness: Witness) ->
        ProvingMode:
411         """Seleccionar modo optimo basado en witness"""
412         # Si tiene datos muy sensibles, preferir client
413         if witness.is_sensitive():
414             if ProvingMode.CLIENT in self.provers:
415                 return ProvingMode.CLIENT
416             elif ProvingMode.HYBRID in self.provers:
417                 return ProvingMode.HYBRID
418
419         # Si no hay datos sensibles, server es mas rapido
420         if ProvingMode.SERVER in self.provers:
421             return ProvingMode.SERVER
422
423         # Default a client
424         return ProvingMode.CLIENT
425
426     def verify_proof(
427         self,
428         circuit_id: str,
429         proof: Proof
430     ) -> bool:
431         """Verificar prueba"""
432         if circuit_id not in self.verification_keys:
433             raise ValueError(f"Verification key not found:
                {circuit_id}")
434
435         vk = self.verification_keys[circuit_id]
436         return vk.verify_proof(proof.proof_data,
            proof.public_inputs)
437
438     def get_benchmarks(self) -> Dict[str, Dict]:
439         """Obtener estadisticas de benchmarks"""
440         stats = {}
441         for circuit_id, times in self.benchmarks.items():
442             if times:
```

```
                stats[circuit_id] = {
                    "count": len(times),
                    "avg_ms": sum(times) / len(times),
                    "min_ms": min(times),
                    "max_ms": max(times)
                }
        return stats


# Servicios de proving externos
@dataclass
class ExternalProvingService:
    """Servicio externo de proving"""
    name: str
    url: str
    supported_systems: List[ProofSystem]
    pricing_per_proof: float  # USD
    avg_latency_ms: float
    trusted: bool  # Si el servicio ve datos privados

PROVING_SERVICES = [
    ExternalProvingService(
        name="Bonsai (RISC Zero)",
        url="https://bonsai.risc0.dev",
        supported_systems=[ProofSystem.STARK],
        pricing_per_proof=0.10,
        avg_latency_ms=5000,
        trusted=True  # Execution encapsulada
    ),
    ExternalProvingService(
        name="Sindri",
        url="https://sindri.app",
        supported_systems=[ProofSystem.GROTH16, ProofSystem.PLONK],
        pricing_per_proof=0.05,
        avg_latency_ms=3000,
        trusted=False  # Ve witness
    ),
    ExternalProvingService(
        name="Succinct",
        url="https://succinct.xyz",
        supported_systems=[ProofSystem.PLONK, ProofSystem.HALO2],
        pricing_per_proof=0.08,
        avg_latency_ms=4000,
        trusted=False
    ),
]


# Demo
def proving_demo():
    print("=== Proving Infrastructure Demo ===\n")

    # Crear servicio de proving
    service = ProvingService()

    # Registrar provers
    client_prover = ClientProver()
    server_prover = ServerProver(
```

```
        server_url="https://prover.example.com",
        api_key="demo_key"
    )
    hybrid_prover = HybridProver(client_prover, server_prover)

    service.register_prover(client_prover)
    service.register_prover(server_prover)
    service.register_prover(hybrid_prover)

    # Cargar circuito
    proving_key = ProvingKey(
        circuit_id="withdrawal",
        proof_system=ProofSystem.GROTH16,
        key_data=secrets.token_bytes(1024),
        size_bytes=1024,
        created_at=datetime.now()
    )

    verification_key = VerificationKey(
        circuit_id="withdrawal",
        proof_system=ProofSystem.GROTH16,
        key_data=secrets.token_bytes(256),
        commitment="0xvk_commitment"
    )

    service.load_circuit("withdrawal", proving_key,
        verification_key)
    print("1. Circuito 'withdrawal' cargado")

    # Crear witness
    witness = Witness(
        circuit_id="withdrawal",
        private_inputs={
            "nullifier_secret": "0x" + secrets.token_hex(32),
            "amount": 1000000000000000000,
            "merkle_path": [1, 2, 3, 4, 5]
        },
        public_inputs=[
            int("0x1234", 16),  # root
            int("0xabcd", 16),  # nullifier
            int("0x9999", 16),  # recipient
        ]
    )
    print(f"\n2. Witness creado (sensible:
        {witness.is_sensitive()})")

    # Generar pruebas con diferentes modos
    modes = [ProvingMode.CLIENT, ProvingMode.SERVER,
        ProvingMode.HYBRID]

    print("\n3. Generando pruebas en diferentes modos:")
    for mode in modes:
        proof = service.generate_proof("withdrawal", witness, mode)
        print(f"\n   {mode.value.upper()}:")
        print(f"     Tiempo: {proof.proving_time_ms:.2f} ms")
        print(f"     Tamaño: {proof.size_bytes()} bytes")
        print(f"     Sistema: {proof.proof_system.value}")

```

```
    # Verificar
    proof = service.generate_proof("withdrawal", witness,
        ProvingMode.CLIENT)
    is_valid = service.verify_proof("withdrawal", proof)
    print(f"\n4. Verificacion: {'VALID' if is_valid else
        'INVALID'}")

    # Benchmarks
    print("\n5. Benchmarks:")
    stats = service.get_benchmarks()
    for circuit, data in stats.items():
        print(f"   {circuit}: avg={data['avg_ms']:.2f}ms,
            count={data['count']}")

    # Servicios externos
    print("\n6. Servicios de proving externos:")
    for svc in PROVING_SERVICES:
        trust = "trusted" if svc.trusted else "untrusted"
        print(f"   {svc.name}: ${svc.pricing_per_proof}/proof,
            {svc.avg_latency_ms}ms ({trust})")

proving_demo()
```

Listing 7.7: Infraestructura de proving ZK

Modo	Privacidad	Velocidad	Costo	Complejidad
Client-side	Máxima	Lenta	Gratis	Baja
Server-side	Baja	Rápida	$/proof	Baja
Híbrido	Alta	Media	Parcial	Alta
MPC Delegated	Alta	Media	Alto	Muy alta

Tabla 7.5: Comparativa de modos de proving

7.7. Hardware Wallets y ZK

Definición 7.8 (Hardware Wallet ZK)**.** Dispositivo seguro que almacena claves privadas y potencialmente participa en operaciones ZK:

- **Almacenamiento seguro:** Keys nunca salen del dispositivo
- **Firma segura:** Firmas generadas internamente
- **ZK limitado:** Capacidad de cómputo restringida

7.7.1. Limitaciones Actuales

7.7.2. Estrategias de Integración

```
"""
Hardware Wallet ZK Integration
Estrategias para usar hardware wallets con sistemas ZK
"""

from dataclasses import dataclass, field
```

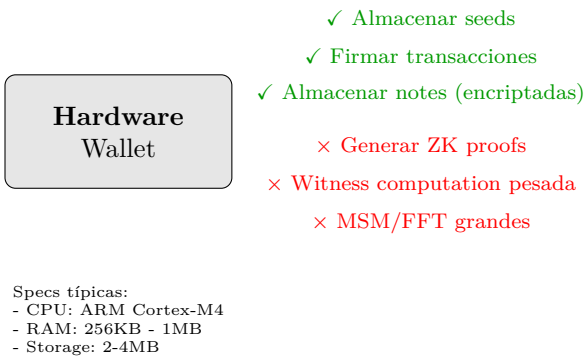


Figura 7.8: Capacidades y limitaciones de hardware wallets actuales

```python
from typing import Optional, Dict, List, Any, Tuple
from enum import Enum
from abc import ABC, abstractmethod
import hashlib
import secrets

class HardwareCapability(Enum):
    """Capacidades de hardware wallet"""
    SIGN_TX = "sign_tx"           # Firmar transacciones
    SIGN_MESSAGE = "sign_message" # Firmar mensajes
    DERIVE_KEY = "derive_key"     # Derivar claves
    STORE_DATA = "store_data"     # Almacenar datos
    COMPUTE_HASH = "compute_hash" # Calcular hashes
    BASIC_CRYPTO = "basic_crypto" # Operaciones crypto basicas

@dataclass
class HardwareWalletSpec:
    """Especificaciones de hardware wallet"""
    model: str
    cpu: str
    ram_kb: int
    storage_kb: int
    capabilities: List[HardwareCapability]
    max_operation_ms: int  # Timeout maximo

    def can_generate_proof(self, circuit_constraints: int) -> bool:
        """Verificar si puede generar prueba para circuito"""
        # Estimacion: ~1KB RAM por 100 constraints
        required_ram = circuit_constraints * 10  # bytes
        return required_ram < self.ram_kb * 1024

# Dispositivos comunes
LEDGER_NANO_X = HardwareWalletSpec(
    model="Ledger Nano X",
    cpu="ST33J2M0",
    ram_kb=256,
    storage_kb=2048,
    capabilities=[
        HardwareCapability.SIGN_TX,
        HardwareCapability.SIGN_MESSAGE,
        HardwareCapability.DERIVE_KEY,
        HardwareCapability.COMPUTE_HASH
    ],
```

```
    max_operation_ms=30000
)

TREZOR_MODEL_T = HardwareWalletSpec(
    model="Trezor Model T",
    cpu="ARM Cortex-M4",
    ram_kb=256,
    storage_kb=4096,
    capabilities=[
        HardwareCapability.SIGN_TX,
        HardwareCapability.SIGN_MESSAGE,
        HardwareCapability.DERIVE_KEY,
        HardwareCapability.STORE_DATA
    ],
    max_operation_ms=60000
)


class HardwareWallet(ABC):
    """Interfaz abstracta para hardware wallet"""

    @abstractmethod
    def sign_transaction(self, tx_hash: bytes) -> bytes:
        pass

    @abstractmethod
    def derive_key(self, path: str) -> bytes:
        pass

    @abstractmethod
    def get_public_key(self, path: str) -> bytes:
        pass


class SimulatedHardwareWallet(HardwareWallet):
    """Hardware wallet simulado para desarrollo"""

    def __init__(self, spec: HardwareWalletSpec, seed: bytes):
        self.spec = spec
        self.seed = seed
        self.derived_keys: Dict[str, bytes] = {}

    def sign_transaction(self, tx_hash: bytes) -> bytes:
        """Firmar hash de transaccion"""
        # Simular firma ECDSA
        signature = hashlib.sha256(
            self.seed + tx_hash
        ).digest() + hashlib.sha256(
            tx_hash + self.seed
        ).digest()
        return signature[:64]  # 64 bytes signature

    def derive_key(self, path: str) -> bytes:
        """Derivar clave desde path BIP32"""
        if path not in self.derived_keys:
            key_material = hashlib.sha256(
                self.seed + path.encode()
            ).digest()
```

```
            self.derived_keys[path] = key_material
        return self.derived_keys[path]

    def get_public_key(self, path: str) -> bytes:
        """Obtener clave publica"""
        private = self.derive_key(path)
        # Simular derivacion de pubkey
        return hashlib.sha256(private).digest()


@dataclass
class ZKDelegationRequest:
    """Request para delegacion segura de proving"""
    nullifier_commitment: str      # H(nullifier_secret)
    amount_commitment: str         # H(amount, blinding)
    merkle_path_commitment: str    # H(path)
    public_inputs: List[int]

    # NO incluye secretos reales

@dataclass
class ZKDelegationProof:
    """Prueba de delegacion"""
    proof_data: bytes
    hw_signature: bytes  # Firma del hardware wallet
    delegation_commitment: str


class SecureZKDelegation:
    """
    Sistema de delegacion segura de proving.

    Permite que un servidor genere pruebas sin ver
    los secretos del usuario.

    Protocolo:
    1. HW genera commitments a secretos
    2. HW firma los commitments
    3. Servidor genera prueba sobre commitments
    4. HW verifica y autoriza
    """

    def __init__(
        self,
        hardware_wallet: HardwareWallet,
        server_url: str
    ):
        self.hw = hardware_wallet
        self.server_url = server_url

    def create_delegation_request(
        self,
        nullifier_secret: bytes,
        amount: int,
        merkle_path: List[int]
    ) -> ZKDelegationRequest:
        """
        Crear request de delegacion.
```

```
166
167         IMPORTANTE: Los secretos nunca salen del cliente.
168         Solo se envian commitments.
169         """
170         # Generar commitments
171         nullifier_commitment =
               hashlib.sha256(nullifier_secret).hexdigest()
172
173         blinding = secrets.token_bytes(32)
174         amount_commitment = hashlib.sha256(
175             amount.to_bytes(32, 'big') + blinding
176         ).hexdigest()
177
178         merkle_path_commitment = hashlib.sha256(
179             b"".join(p.to_bytes(32, 'big') for p in merkle_path)
180         ).hexdigest()
181
182         # Calcular public inputs
183         # (estos si se revelan)
184         nullifier_hash = hashlib.sha256(
185             nullifier_secret + b"nullifier"
186         ).digest()
187
188         public_inputs = [
189             int.from_bytes(nullifier_hash[:16], 'big'),  #
                   nullifier
190             # merkle root, recipient, etc (simplificado)
191         ]
192
193         return ZKDelegationRequest(
194             nullifier_commitment=nullifier_commitment,
195             amount_commitment=amount_commitment,
196             merkle_path_commitment=merkle_path_commitment,
197             public_inputs=public_inputs
198         )
199
200     def authorize_delegation(
201         self,
202         request: ZKDelegationRequest
203     ) -> bytes:
204         """
205         Hardware wallet autoriza la delegacion.
206
207         Firma los commitments para que el servidor
208         sepa que son validos.
209         """
210         # Crear mensaje a firmar
211         message = (
212             request.nullifier_commitment +
213             request.amount_commitment +
214             request.merkle_path_commitment
215         ).encode()
216
217         message_hash = hashlib.sha256(message).digest()
218
219         # Firmar con hardware wallet
220         signature = self.hw.sign_transaction(message_hash)
221
```

```
222         return signature
223
224     def request_proof(
225         self,
226         request: ZKDelegationRequest,
227         hw_signature: bytes
228     ) -> ZKDelegationProof:
229         """
230         Solicitar prueba al servidor.
231
232         El servidor genera la prueba usando los commitments
233         sin conocer los valores reales.
234         """
235         # En produccion: HTTP request al servidor
236
237         # Simular respuesta
238         proof_data = secrets.token_bytes(256)
239
240         return ZKDelegationProof(
241             proof_data=proof_data,
242             hw_signature=hw_signature,
243             delegation_commitment=request.nullifier_commitment
244         )
245
246     def verify_and_submit(
247         self,
248         proof: ZKDelegationProof,
249         original_request: ZKDelegationRequest
250     ) -> bool:
251         """
252         Verificar prueba y preparar para submit.
253
254         El hardware wallet verifica que la prueba
255         corresponde a los commitments originales.
256         """
257         # Verificar que la prueba usa nuestros commitments
258         if proof.delegation_commitment !=
               original_request.nullifier_commitment:
259             return False
260
261         # Verificar firma (ya la tenemos)
262         # En produccion: verificar criptograficamente
263
264         return True
265
266
267 class FutureHardwareZK:
268     """
269     Especulacion sobre futuro hardware ZK.
270
271     Hardware especializado para ZK proving.
272     """
273
274     @staticmethod
275     def projected_specs() -> Dict[str, Any]:
276         return {
277             "zk_accelerator": {
278                 "description": "ASIC/FPGA para MSM/NTT",
```

```
                "speedup": "100-1000x vs CPU",
                "timeline": "2025-2026"
            },
            "secure_element_zk": {
                "description": "Secure element con soporte ZK basico",
                "capabilities": ["small_circuits", "commitment_generation"],
                "timeline": "2024-2025"
            },
            "tee_proving": {
                "description": "TEE (SGX/TrustZone) para proving",
                "benefit": "Privacidad server-side",
                "timeline": "Available now (limited)"
            },
            "mpc_hardware": {
                "description": "Hardware optimizado para MPC",
                "use_case": "Distributed proving",
                "timeline": "2025+"
            }
        }


# Demo
def hardware_zk_demo():
    print("=== Hardware Wallet ZK Demo ===\n")

    # Mostrar specs de dispositivos
    print("1. Especificaciones de hardware wallets:")
    for spec in [LEDGER_NANO_X, TREZOR_MODEL_T]:
        print(f"\n   {spec.model}:")
        print(f"     CPU: {spec.cpu}")
        print(f"     RAM: {spec.ram_kb}KB")
        print(f"     Storage: {spec.storage_kb}KB")

        # Verificar capacidad ZK
        can_small = spec.can_generate_proof(1000)
        can_large = spec.can_generate_proof(100000)
        print(f"     ZK (1K constraints): {'Si' if can_small else 'No'}")
        print(f"     ZK (100K constraints): {'Si' if can_large else 'No'}")

    # Crear hardware wallet simulado
    print("\n2. Simulando delegacion segura de proving...")

    hw = SimulatedHardwareWallet(
        spec=LEDGER_NANO_X,
        seed=secrets.token_bytes(32)
    )

    delegation = SecureZKDelegation(
        hardware_wallet=hw,
        server_url="https://prover.example.com"
    )

    # Crear request (secretos se quedan en cliente)
    nullifier_secret = secrets.token_bytes(32)
```

```
    request = delegation.create_delegation_request(
        nullifier_secret=nullifier_secret,
        amount=10**18,
        merkle_path=[1, 2, 3, 4, 5]
    )

    print(f"   Nullifier commitment:
       {request.nullifier_commitment[:20]}...")
    print(f"   Amount commitment:
       {request.amount_commitment[:20]}...")
    print("   (Secretos nunca salen del cliente)")

    # Autorizar con HW
    print("\n3. Hardware wallet autoriza delegacion...")
    signature = delegation.authorize_delegation(request)
    print(f"   Firma: {signature[:20].hex()}...")

    # Request proof
    print("\n4. Servidor genera prueba (sin ver secretos)...")
    proof = delegation.request_proof(request, signature)
    print(f"   Proof size: {len(proof.proof_data)} bytes")

    # Verificar
    print("\n5. Verificando prueba...")
    is_valid = delegation.verify_and_submit(proof, request)
    print(f"   Valida: {'Si' if is_valid else 'No'}")

    # Futuro
    print("\n6. Futuro del hardware ZK:")
    future = FutureHardwareZK.projected_specs()
    for name, info in future.items():
        print(f"\n   {name}:")
        print(f"     {info['description']}")
        print(f"     Timeline: {info['timeline']}")

hardware_zk_demo()
```

Listing 7.8: Integración de hardware wallet con ZK

Resumen del Bloque

Idea Clave

Ecosistema de Privacidad - Componentes Esenciales:
1. Exploradores Privacy-Aware:

- Sin tracking (cookies, analytics, fingerprinting)
- Acceso via Tor (.onion services)
- Dummy queries para confusión estadística
- Self-hosting recomendado para máxima privacidad
- Índices locales sin dependencias externas

2. Wallets ZK:

- Railgun: ERC-20 privacy, DeFi integration, shielded pools
- Umbra: Stealth payments simples, bajo overhead
- Zcash: Privacidad nativa (Zashi, YWallet, Nighthawk)
- Gestión segura de viewing keys y spending keys
- Derivación determinística compatible con hardware wallets

3. RPC Privados:

- Flashbots Protect: No mempool pública, bundle submission
- MEV Blocker: OFA (Order Flow Auctions) con refunds
- Protección contra frontrunning, sandwich attacks, y liquidation sniping
- Private mempools con commit-reveal schemes
- Encrypted transaction propagation

4. Relayer Networks:

- Descentralización para censorship resistance
- Competencia económica reduce fees
- Staking y slashing para accountability
- Rotación automática para unlinkability
- Múltiples jurisdicciones para resiliencia legal

5. Note Management:

- Backup ANTES de depositar (regla de oro)
- Encriptación AES-256-GCM con key derivation segura
- Split backup con Shamir Secret Sharing (2-of-3 mínimo)
- Sincronización segura entre dispositivos
- Recovery testing periódico obligatorio

6. Proving Infrastructure:

- Client-side (WASM/native): Máxima privacidad, witness nunca sale
- Server-side: Máxima velocidad, requiere confianza
- Híbrido: Balance óptimo con witness encryption
- GPU acceleration para proving intensivo
- Proof caching y batching para eficiencia

7. Hardware Wallets:

- Limitaciones actuales: memoria insuficiente para ZK proving

- Delegación segura con commit-reveal posible
- Signing de transacciones blindadas
- Futuro: Hardware ZK especializado (FPGAs, ASICs)
- Integración con secure elements para key storage

Aplicación en Banco ZK

Stack Completo de Banco ZK:

UI Layer: Web App + Mobile (PWA) + Desktop	React/Vue, Capacitor
Wallet Layer: Note Manager + Backup System + Key Derivation	IndexedDB, Encrypted
Proving Layer: Client WASM + Hybrid Delegation + GPU Accel	snarkjs, rapidsnark
Privacy Layer: Tor + Mixnet + Dummy Traffic	Arti, I2P optional
Network Layer: Relayer Network + P2P Gossip	libp2p, stake-weighted
RPC Layer: Flashbots Protect + Private Mempool	MEV protection
Chain Layer: ZK Contracts + Merkle Trees + Nullifier Sets	Solidity, Circom

Principios de Diseño:

1. **Privacy by default:** Toda operación es privada, transparencia es opt-in
2. **Redundancia:** Múltiples relayers, backups distribuidos, failover automático
3. **Verificabilidad:** Todo open source, builds reproducibles, auditable on-chain
4. **Usabilidad:** Complejidad criptográfica oculta al usuario final
5. **Descentralización:** Sin single points of failure, governance distribuida
6. **Composabilidad:** Integración con DeFi existente vía adapters
7. **Upgradeability:** Migración segura de circuitos y contratos

Métricas Objetivo:

Métrica	Objetivo	Crítico
Tiempo de depósito	< 30s	< 60s
Tiempo de retiro	< 45s	< 90s
Anonymity set	$> 10,000$	$> 1,000$
Uptime relayers	$> 99{,}9\,\%$	$> 99\,\%$
Recovery success	$100\,\%$	$100\,\%$
MEV leakage	$0\,\%$	$< 1\,\%$
False positive rate	$< 0{,}1\,\%$	$< 1\,\%$
Gas overhead vs EOA	$< 5x$	$< 10x$

Recomendaciones de Implementación:

1. Comenzar con client-side proving (privacidad máxima)
2. Integrar Flashbots Protect desde día 1
3. Implementar backup obligatorio antes de depósitos
4. Usar Tor para todas las conexiones por defecto
5. Auditar código antes de mainnet (mínimo 2 auditores)
6. Programa de bug bounty activo desde testnet
7. Monitoring de anonymity set en tiempo real
8. Alertas automáticas para comportamiento anómalo
9. Documentación completa para usuarios y desarrolladores
10. Proceso de incident response documentado

Roadmap Sugerido:

1. **Fase 1 (MVP):** Depósitos/retiros básicos, single relayer, backup manual
2. **Fase 2 (Beta):** Multi-relayer, backup automático, mobile app
3. **Fase 3 (Mainnet):** DeFi integrations, hardware wallet support
4. **Fase 4 (Mature):** Cross-chain bridges, institutional features

Idea Clave

Conclusión de la Serie Banco ZK:
A lo largo de estos 20 bloques, hemos construido el conocimiento necesario para entender y desarrollar sistemas de privacidad en blockchain:

1. **Fundamentos Matemáticos:**
   - Teoría de grupos y campos finitos
   - Curvas elípticas (secp256k1, BN254, BLS12-381)
   - Funciones hash criptográficas (Poseidon, Pedersen)
   - Compromisos y esquemas de firma
2. **Zero-Knowledge Proofs:**
   - SNARKs: Groth16, PLONK, Marlin
   - STARKs: Transparencia y post-quantum security
   - Circuitos aritméticos y R1CS
   - Sistemas de prueba interactivos y no interactivos
3. **Protocolos de Privacidad:**
   - Mixers y coin joins

- Commitment schemes y nullifiers
- Ring signatures y stealth addresses
- Confidential transactions

4. **Vectores de Ataque:**
   - Análisis de grafos y clustering
   - Timing y traffic analysis
   - Sybil attacks en anonymity sets
   - Side-channel attacks en proving
5. **Infraestructura Práctica:**
   - Wallets y key management
   - Relayer networks descentralizados
   - Proving infrastructure escalable
   - Integración con ecosistema existente

El futuro de la privacidad en blockchain depende de:

- **Eficiencia:** Reducción de proving time de minutos a segundos
- **Hardware:** GPUs, FPGAs, y ASICs especializados para ZK
- **Estándares:** Interoperabilidad entre sistemas de privacidad
- **Educación:** Comprensión pública de la importancia de privacidad
- **Regulación:** Balance entre compliance y derechos fundamentales
- **Usabilidad:** Privacidad invisible para el usuario promedio
- **Investigación:** Nuevos sistemas de prueba más eficientes

Desafíos Pendientes:

- Escalabilidad de anonymity sets a millones de usuarios
- Composabilidad completa con DeFi sin leakage
- Resistencia a computación cuántica (transición a STARKs)
- Compliance selectivo sin comprometer privacidad
- UX comparable a aplicaciones tradicionales

"Privacy is not about having something to hide.
Privacy is about having the right to choose what to reveal."
— Anónimo

"Arguing that you don't care about privacy because you have nothing to hide
is no different than saying you don't care about free speech
because you have nothing to say."
— Edward Snowden

Conclusión

Este volumen ha cubierto los **Bloques 14 a 20** de la serie Banco ZK, enfocándose en aplicaciones avanzadas, seguridad, y el ecosistema de herramientas necesario para implementar sistemas de privacidad en blockchain.

Resumen por Bloque

Bloque 14: Identidad y Credenciales Anónimas

Exploramos cómo demostrar propiedades sobre uno mismo sin revelar información innecesaria:

- Anonymous credentials y selective disclosure
- Pruebas de membresía en grupos (group signatures)
- Verificación de edad sin revelar fecha de nacimiento
- DIDs (Decentralized Identifiers) y VCs (Verifiable Credentials)
- Soulbound tokens con privacidad
- KYC preservando privacidad mediante ZK proofs

Bloque 15: Compliance vs Privacidad

Analizamos el delicado balance entre requisitos regulatorios y privacidad del usuario:

- Selective disclosure para auditorías
- Viewing keys y su gestión segura
- Travel Rule compliance con ZK
- Sanctions screening sin revelar identidad
- Arquitecturas de compliance privado
- Casos de estudio: Zcash, Tornado Cash, Railgun

Bloque 16: Metadata y Network Privacy

Profundizamos en la protección contra análisis de metadatos:

- Timing analysis y sus mitigaciones
- Traffic analysis en redes P2P
- Tor, I2P, y mixnets para privacidad de red

- Dummy transactions y cover traffic
- Fingerprinting de wallets y protecciones
- Análisis de grafos y técnicas de clustering

Bloque 17: Private Smart Contracts

Cubrimos la ejecución privada de lógica arbitraria:

- zkEVM y sus variantes (Type 1-4)
- Private state en smart contracts
- Aztec, Aleo, y Mina: comparativa
- Composabilidad entre contratos privados
- Oráculos privados y feeds de datos
- Private DeFi: AMMs, lending, derivatives

Bloque 18: Criptografía Avanzada

Exploramos primitivas criptográficas de frontera:

- Fully Homomorphic Encryption (FHE)
- Multi-Party Computation (MPC)
- Oblivious Transfer y Private Set Intersection
- Witness Encryption
- Post-quantum cryptography
- Combinación de técnicas para máxima seguridad

Bloque 19: Ataques y Defensa

Analizamos vectores de ataque y estrategias defensivas:

- Ataques al anonymity set (Sybil, poisoning)
- Deanonymization mediante análisis heurístico
- Side-channel attacks en proving
- Smart contract vulnerabilities específicas de privacidad
- Incident response y recovery
- Diseño de sistemas resilientes

Bloque 20: Herramientas y Ecosistema

Presentamos el stack completo para implementación:

- Exploradores privacy-aware
- Wallets con soporte ZK (Railgun, Umbra, Zcash)
- RPCs privados (Flashbots, MEV Blocker)
- Redes de relayers descentralizados
- Gestión de notas y backups
- Infraestructura de proving
- Integración con hardware wallets

Conexiones con Volumen 1

El Volumen 1 (Bloques 1-13) estableció los fundamentos:

- **Bloques 1-3:** Criptografía básica, curvas elípticas, hashing
- **Bloques 4-6:** Estructuras de datos (Merkle trees), compromisos
- **Bloques 7-9:** Firmas digitales, ring signatures, stealth addresses
- **Bloques 10-13:** Zero-knowledge proofs (SNARKs, STARKs, circuitos)

Este Volumen 2 construye sobre esos fundamentos para aplicaciones prácticas.

Implementación Recomendada

Para implementar un sistema como Banco ZK, sugerimos:

1. **Fase de Diseño (2-3 meses):**
   - Definir modelo de amenazas
   - Seleccionar primitivas criptográficas
   - Diseñar arquitectura de circuitos
   - Especificar interfaces de usuario
2. **Fase de Desarrollo (4-6 meses):**
   - Implementar contratos core
   - Desarrollar circuitos ZK
   - Construir infraestructura de relayers
   - Crear aplicaciones cliente
3. **Fase de Testing (2-3 meses):**
   - Auditorías de seguridad (mínimo 2)
   - Testnet público
   - Bug bounty program

- Stress testing

4. **Fase de Lanzamiento:**
   - Mainnet con límites iniciales
   - Monitoreo 24/7
   - Proceso de escalado gradual
   - Documentación y soporte

Recursos Adicionales

Para profundizar en los temas cubiertos:
Documentación Técnica:

- Zcash Protocol Specification
- Tornado Cash documentation
- Railgun technical papers
- Aztec protocol documentation

Papers Académicos:

- "Zerocash: Decentralized Anonymous Payments" (Sasson et al.)
- "PLONK: Permutations over Lagrange-bases" (Gabizon et al.)
- "STARKs" (Ben-Sasson et al.)
- "Bulletproofs" (Bünz et al.)

Herramientas de Desarrollo:

- Circom + snarkjs
- Noir (Aztec)
- Leo (Aleo)
- Cairo (StarkWare)

Glosario

A

AES Advanced Encryption Standard. Algoritmo de cifrado simétrico estándar.

AMM Automated Market Maker. Protocolo DeFi para intercambio sin order book.

Anonymity Set Conjunto de usuarios indistinguibles entre sí para un observador.

ASIC Application-Specific Integrated Circuit. Hardware especializado.

B

BFT Byzantine Fault Tolerance. Tolerancia a fallos bizantinos en consenso.

BLS Boneh-Lynn-Shacham. Esquema de firmas con agregación eficiente.

BN254 Barreto-Naehrig curve. Curva elíptica pairing-friendly de 254 bits.

Bulletproofs Sistema de range proofs sin trusted setup.

C

CRS Common Reference String. String de referencia para SNARKs con trusted setup.

Commitment Compromiso criptográfico que oculta un valor hasta su revelación.

Cover Traffic Tráfico ficticio para ocultar patrones reales.

Circom Lenguaje DSL para definir circuitos aritméticos ZK.

D

DA Data Availability. Disponibilidad de datos en sistemas L2.

DID Decentralized Identifier. Identificador descentralizado W3C.

DKG Distributed Key Generation. Generación distribuida de claves.

DSL Domain-Specific Language. Lenguaje especializado para un dominio.

Dummy Query Consulta ficticia para ocultar consultas reales.

E

ECDLP Elliptic Curve Discrete Log Problem. Problema base de seguridad ECC.

ECDSA Elliptic Curve Digital Signature Algorithm. Firma estándar en Bitcoin/Ethereum.

EdDSA Edwards-curve Digital Signature Algorithm. Firma en curvas Edwards.

EIP Ethereum Improvement Proposal. Propuesta de mejora de Ethereum.

EVM Ethereum Virtual Machine. Máquina virtual de Ethereum.

F

FHE Fully Homomorphic Encryption. Cifrado que permite cómputo sobre datos cifrados.

FFT Fast Fourier Transform. Algoritmo para evaluación rápida de polinomios.

FPGA Field-Programmable Gate Array. Hardware reconfigurable.

FRI Fast Reed-Solomon IOP. Protocolo de proximidad usado en STARKs.

Frontrunning Ataque que ejecuta transacciones antes de otras conocidas.

G

Gas Unidad de cómputo en Ethereum.

Groth16 Sistema SNARK eficiente con trusted setup por circuito.

Group Signature Firma que prueba membresía sin revelar identidad.

H

HMAC Hash-based Message Authentication Code.

HSM Hardware Security Module. Módulo de seguridad hardware.

Hybrid Proving Combinación de proving local y delegado.

I

IVC Incrementally Verifiable Computation. Computación verificable incremental.

I2P Invisible Internet Project. Red de anonimato alternativa a Tor.

K

KDF Key Derivation Function. Función para derivar claves.

KZG Kate-Zaverucha-Goldberg. Esquema de commitment polinomial.

KYC Know Your Customer. Requisitos de identificación de clientes.

L

L1 Layer 1. Blockchain base (ej: Ethereum mainnet).

L2 Layer 2. Solución de escalabilidad sobre L1.

Lookup Table Tabla precalculada para optimizar circuitos.

M

Merkle Tree Árbol hash para compromisos eficientes de conjuntos.

MEV Maximal Extractable Value. Valor extraíble por reordenación de transacciones.

Mixnet Red de mezcla para anonimato de comunicaciones.

MPC Multi-Party Computation. Cómputo seguro entre múltiples partes.

MSM Multi-Scalar Multiplication. Operación crítica en SNARKs.

N

NIZK Non-Interactive Zero-Knowledge. ZK sin interacción.

Note Representación de valor en sistemas UTXO privados.

Nullifier Identificador único que previene doble gasto sin revelar origen.

NTT Number Theoretic Transform. FFT sobre campos finitos.

O

OFA Order Flow Auction. Subasta de flujo de órdenes para MEV.

Onion Routing Enrutamiento en capas (usado por Tor).

OT Oblivious Transfer. Transferencia inconsciente en MPC.

Oracle Servicio que proporciona datos externos a smart contracts.

P

Pairing Operación bilineal en curvas elípticas.

Pedersen Hash Hash algebraico eficiente en circuitos.

PLONK Permutations over Lagrange-bases. Sistema SNARK universal.

Poseidon Función hash optimizada para circuitos ZK.

Prover Entidad que genera pruebas ZK.

PSI Private Set Intersection. Intersección privada de conjuntos.

PWA Progressive Web App. Aplicación web instalable.

R

R1CS Rank-1 Constraint System. Sistema de restricciones para SNARKs.

Range Proof Prueba de que un valor está en un rango sin revelarlo.

Relayer Intermediario que envía transacciones por cuenta de usuarios.

Ring Signature Firma que oculta al firmante real entre un grupo.

Rollup Solución L2 que publica datos en L1.

S

Sandwich Attack Ataque MEV que rodea una transacción víctima.

SBT Soulbound Token. Token no transferible.

Schnorr Esquema de firma con agregación simple.

Shamir Shamir Secret Sharing. División de secretos con threshold.

Shielded Pool Pool de fondos privados en sistemas como Zcash.

SNARK Succinct Non-interactive ARgument of Knowledge.

Spending Key Clave que autoriza gastos en sistemas ZK.

STARK Scalable Transparent ARgument of Knowledge.

Stealth Address Dirección de un solo uso para privacidad.

T

TEE Trusted Execution Environment. Entorno de ejecución aislado.

Tor The Onion Router. Red de anonimato más popular.

Trusted Setup Ceremonia para generar CRS en algunos SNARKs.

TWAP Time-Weighted Average Price. Precio promedio ponderado por tiempo.

U

Umbra Protocolo de stealth addresses para Ethereum.

UTXO Unspent Transaction Output. Modelo de Bitcoin/Zcash.

V

VC Verifiable Credential. Credencial verificable W3C.

Verifier Entidad que verifica pruebas ZK.

Viewing Key Clave que permite ver transacciones sin gastar.

VRF Verifiable Random Function. Función aleatoria verificable.

W

WASM WebAssembly. Formato binario para ejecución en navegadores.

Witness Datos privados usados para generar pruebas ZK.

Witness Encryption Cifrado que requiere prueba ZK para descifrar.

Z

Zcash Criptomoneda pionera en privacidad con zk-SNARKs.

zkEVM Zero-Knowledge EVM. EVM compatible con pruebas ZK.

zkML Zero-Knowledge Machine Learning. ML con privacidad ZK.

zkSNARK Zero-Knowledge Succinct Non-interactive ARgument of Knowledge.

zkSTARK Zero-Knowledge STARK.

zkVM Zero-Knowledge Virtual Machine. VM genérica con ZK.

ZKP Zero-Knowledge Proof. Prueba de conocimiento cero.

Índice de figuras

1.1. Modelo de anonymous credentials: el holder obtiene una credencial del issuer y la presenta al verifier sin que puedan coludir. . . . 20
1.2. Selective disclosure: diferentes verificadores ven diferentes subconjuntos de atributos . . . 27
1.3. Flujo del protocolo Semaphore . . . 34
1.4. SBTs públicos vs privados . . . 45
1.5. Flujo de Verifiable Credentials . . . 51
1.6. Flujo de verificación de edad con ZK . . . 62
1.7. KYC tradicional vs KYC con ZK . . . 71

2.1. Espectro de privacidad-compliance y la zona óptima habilitada por ZK . . . 81
2.2. Proof-of-Innocence temporal: probar que el depósito ocurrió antes de un evento contaminante . . . 83
2.3. Threshold decryption: 3 de 5 autoridades revelan la identidad . . . 95
2.4. Jerarquía de claves: spending, viewing, audit . . . 104
2.5. Arquitectura de mixer con compliance integrado . . . 113
2.6. Técnicas de clustering y taint analysis . . . 125
2.7. Privacy Pools: múltiples association sets con diferentes niveles de restricción . 132
2.8. Casos de uso de opt-in transparency . . . 143
2.9. ZK permite superar el trade-off tradicional entre privacidad y compliance . . 155

3.1. Capas de privacidad: ZK protege la aplicación pero la red sigue expuesta . . . 157
3.2. Vectores de IP leakage en interacción con blockchain . . . 158
3.3. Onion routing en Tor: cada relay solo conoce el anterior y siguiente . . . 165
3.4. Timing correlation: pequeños Δt revelan vinculación . . . 167
3.5. Diferentes actividades generan patrones de tráfico distintivos . . . 175
3.6. Onion routing vs Mixnet: el mixing rompe la correlación temporal . . . 183
3.7. Trade-off entre latencia y nivel de anonimato . . . 190
3.8. Packet padding uniformiza el tamaño visible de los paquetes . . . 191
3.9. Cover traffic oculta los patrones del tráfico real . . . 197
3.10. Mapeo de amenazas de red a protecciones correspondientes . . . 210

4.1. Cuatro aproximaciones principales a smart contracts privados . . . 212
4.2. Arquitectura de Secret Network: estado encriptado, procesamiento en enclave SGX . . . 213
4.3. Arquitectura de Oasis: consenso separado de ejecución . . . 224
4.4. Arquitectura de Aleo: ejecución off-chain, verificación on-chain . . . 232
4.5. Mina comprime toda la blockchain en una prueba de 22 KB . . . 239
4.6. FHE permite computar sobre datos encriptados . . . 244
4.7. Cuatro modelos de ejecución privada de funciones . . . 249
4.8. Encrypted mempool previene frontrunning . . . 256
4.9. Sandwich attack y protección mediante encrypted mempool . . . 264

5.1. Flujo de computación con FHE 276
5.2. Ideal functionality de MPC 284
5.3. Shamir's Secret Sharing: el secreto es el término independiente del polinomio 293
5.4. 1-out-of-2 Oblivious Transfer 299
5.5. Private Set Intersection: solo se revela la intersección 305
5.6. TEE aísla código incluso de software privilegiado 311
5.7. Threshold signature: 3 de 5 partes cooperan para firmar 317
5.8. DKG: cada parte envía evaluaciones de su polynomial 325

6.1. Vectores de deanonymization 328
6.2. Sybil: atacante controla 3 de 4 depósitos 332
6.3. Eclipse attack: víctima solo conectada a atacantes 334
6.4. Dust attack: tracking mediante consolidación de UTXOs 337
6.5. Heurísticas de clustering: common input y change detection 343
6.6. Métodos de taint analysis: Poison, Haircut, FIFO 348
6.7. Intersection attack: reducción progresiva del anonymity set 353

7.1. Arquitectura de explorador privacy-aware 369
7.2. Ecosistema de wallets con soporte de privacidad 376
7.3. Ecosistema de wallets Zcash 389
7.4. Comparación de flujo de transacciones tradicional vs privado 390
7.5. Arquitectura de red de relayers descentralizada 397
7.6. Ciclo de vida de una note en sistema de privacidad 406
7.7. Trade-offs en arquitecturas de proving 417
7.8. Capacidades y limitaciones de hardware wallets actuales 428

Índice de tablas

1.1. Comparación de sistemas de anonymous credentials 26
1.2. Comparación de sistemas de Proof of Humanity 61
1.3. Resumen de tecnologías de identidad ZK 80

2.1. Casos de uso para diferentes tipos de viewing keys 112
2.2. Contramedidas contra análisis de blockchain 131
2.3. Trade-offs de diferentes association sets 142
2.4. Resumen de técnicas de balance compliance-privacidad 155

3.1. Defensas contra timing correlation 174
3.2. Comparación entre Onion Routing y Mixnets 184
3.3. Comparación de soluciones de privacidad de red 189
3.4. Comparación de privacidad entre libp2p y Waku 208

4.1. Comparación entre Secret Network y Oasis Sapphire 231
4.2. Comparación de plataformas de smart contracts privados 248
4.3. Técnicas de protección contra MEV 264

5.1. Principales librerías FHE 283
5.2. Comparación de protocolos MPC 292
5.3. Comparación de Trusted Execution Environments 317

6.1. Defensas contra Eclipse attacks 337

7.1. Comparativa de exploradores por privacidad 369
7.2. Comparativa de wallets ZK (AS = Anonymity Set) 376
7.3. Comparativa detallada de wallets Zcash 389
7.4. Comparativa de servicios Private RPC 397
7.5. Comparativa de modos de proving 427

Referencias y Recursos

Papers Fundamentales

1. Sasson, E. B., et al. (2014). *Zerocash: Decentralized Anonymous Payments from Bitcoin.*

2. Groth, J. (2016). *On the Size of Pairing-based Non-interactive Arguments.*

3. Gabizon, A., Williamson, Z., & Ciobotaru, O. (2019). *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge.*

4. Ben-Sasson, E., et al. (2018). *Scalable, transparent, and post-quantum secure computational integrity.*

5. Bünz, B., et al. (2018). *Bulletproofs: Short Proofs for Confidential Transactions and More.*

6. Maller, M., et al. (2019). *Sonic: Zero-Knowledge SNARKs from Linear-Size Universal and Updateable Structured Reference Strings.*

Documentación de Protocolos

- **Zcash:** https://zips.z.cash/
- **Tornado Cash:** https://docs.tornado.cash/
- **Railgun:** https://docs.railgun.org/
- **Aztec:** https://docs.aztec.network/
- **Aleo:** https://developer.aleo.org/

Herramientas de Desarrollo

- **Circom:** https://docs.circom.io/
- **snarkjs:** https://github.com/iden3/snarkjs
- **Noir:** https://noir-lang.org/
- **Cairo:** https://www.cairo-lang.org/
- **Halo2:** https://zcash.github.io/halo2/

Comunidades y Foros

- **ZK Research:** https://zkresear.ch/
- **Ethereum Research:** https://ethresear.ch/
- **Zero Knowledge Podcast:** https://zeroknowledge.fm/